



Teacher guide

Phaser 4 - a guide for OO programmers

Teacher notes, worked solutions and slides for every chapter

Contents

1 Introduction: your first Phaser 4 game

2 A three-scene game

3 Preloading

4 The life of a scene

5 Audio

6 Scoring

7 Animations

8 Collisions

9 2D physics

10 Tilemaps

11 Tilemaps with Tiled

12 Higher or lower

13 Memory match

14 Catch, avoid and shoot

15 Platformer

Teacher materials

Everything a teacher needs to run each chapter as a class - and nothing a student should see before attempting the challenges. The chapter folders in `chapters/` contain no answers, so they can be handed out on their own; keep this folder back.

Each chapter has a folder here, named like the chapter:

File	What it is
<code>teacher_notes.md</code>	overview, prerequisites, learning outcomes, a timed session plan, key points, common errors (with the messages students will actually see), discussion questions, extension and assessment ideas
<code>solutions.md</code>	a worked solution to each of the chapter's six challenges: the approach, the key code, and what to look for when marking
<code>slides.md</code>	slides for the chapter, in MARP format
<code>solutions/</code>	one complete, runnable Celbridge project per challenge. Every change from the chapter project is marked with a <code>// CHALLENGE n</code> comment - search for <code>CHALLENGE</code> to find them

Using the slides

The slides are Markdown written for [MARP](#). Three ways to show them:

- **VS Code** - install the "Marp for VS Code" extension, open `slides.md`, and use the preview; the extension can also export to PDF, HTML or PowerPoint
- **the command line** - `npx @marp-team/marp-cli slides.md --pdf` (needs Node), or the MARP CLI's standalone download
- **as plain Markdown** - every slide is readable as it is; `---` separates slides

Images in the slides point into the chapter folders (`../../chapters/...`), so keep the folder structure as it is when presenting.

Chapters

Chapter	Notes	Solutions	Slides
1 - Introduction	notes	solutions	slides
2 - A three-scene game	notes	solutions	slides
3 - Preloading	notes	solutions	slides
4 - The life of a scene	notes	solutions	slides
5 - Audio	notes	solutions	slides
6 - Scoring	notes	solutions	slides
7 - Animations	notes	solutions	slides
8 - Collisions	notes	solutions	slides
9 - 2D physics	notes	solutions	slides
10 - Tilemaps	notes	solutions	slides
11 - Tilemaps with Tiled	notes	solutions	slides
12 - Higher or lower	notes	solutions	slides
13 - Memory match	notes	solutions	slides
14 - Catch, avoid and shoot	notes	solutions	slides
15 - Platformer	notes	solutions	slides



Introduction: your first Phaser 4 game

Overview

The first contact with Phaser, Deno and TypeScript. By the end, students have built and run two projects, can find their way round a project folder, and can read (and make small changes to) a config, a scene, and a game object class. Nothing here is hard; the risk in this chapter is **tooling friction** and **TypeScript surprises** for Java programmers, so leave time for both.

Prerequisites

- comfortable writing classes, constructors, fields and methods in Java (or C#)
 - has seen a little JavaScript or HTML - not essential
 - Deno 2.4 or later installed (`deno --version`); Celbridge installed, or any editor plus a terminal
 - a network connection for the **first** build of the first project (Phaser is downloaded once, into Deno's cache, and shared by every project after that)
-

Learning outcomes

Students can:

1. build and serve a Phaser project, and explain why it has to be served rather than opened as a file
2. identify the config, the scenes and the game objects in a project, and say what each is for
3. read and write the TypeScript used in the chapter: typed fields and parameters, `const / let` , classes with `extends / super / override` , `this.` , arrow functions, template strings, `import / export`
4. place game objects using the coordinate system and `setOrigin`

5. describe the game loop, and use `delta` to move something at a speed in pixels per second

Suggested session plan (2 x 1 hour, or one 2 hour lab)

Time	Activity
0:00 - 0:15	Slides 1-6: what Phaser is, the tools, the build pipeline. Live demo: open <code>ch01_hello_phaser</code> in Celbridge, build, serve, open the webview
0:15 - 0:35	Students do the same. Circulate - this is where tooling problems surface (see below)
0:35 - 0:55	Slides 7-11: TypeScript for Java programmers; config, scene, game objects. Walk through <code>HelloScene.ts</code> line by line
0:55 - 1:10	Challenges 1-3
1:10 - 1:25	Slides 12-15: coordinates and origins, the game loop, <code>delta</code> . Live demo: change <code>speedX</code> in <code>ch01_moving_ball</code> ; then change the movement to "4 pixels a frame" and discuss why that is wrong
1:25 - 1:55	Challenges 4-6 (6 is a stretch; fine to set as homework)
1:55 - 2:00	Recap: the three parts of a game; the loop

Key points to stress

- **The game is always 800 x 600 inside**, whatever size it appears. Students who make the browser window small and then try to "fix" coordinates have misunderstood the scale manager
- **You never draw.** You add game objects and change them; Phaser draws them every frame. Java students who have used `paint(Graphics g)` expect to redraw things themselves
- `this.` **is compulsory** inside a class. Forgetting it is the single most common error this week

- **Speeds are per second, and `delta` is in milliseconds.** Get this right now and Chapter 4 and Chapter 8 are much easier

Common problems and errors

What students see	Cause	Fix
Blank page, or the page loads with no game	opened <code>dist/index.html</code> as a file, or the server is not running	run <code>deno task serve</code> , and open http://127.0.0.1:8000/
<code>address already in use</code> when serving	another project's server is still running (often in another terminal tab)	stop it with Ctrl+C, or just use it - it serves whichever <code>dist/</code> it was started in, so stop it before switching project
Old version of the game after changing the code	forgot to rebuild, or the browser cached the page	build, then refresh (Ctrl+Shift+R forces it). Suggest <code>deno task dev</code>
Cannot find name 'message'	wrote <code>message</code> instead of <code>this.message</code>	fields always need <code>this.</code>
This member must have an 'override' modifier	wrote <code>update(...)</code> in a scene without <code>override</code>	add <code>override</code> . Worth explaining: the compiler is checking that the method really replaces one
This member cannot have an 'override' modifier because it is not declared in the base class 'Image'. Did you mean 'update'?	copied <code>override</code> onto <code>Ball.preUpdate</code>	<code>Image</code> has no <code>preUpdate()</code> ; remove <code>override</code> (Chapter 4 explains when it is needed)
Property 'message' has no initializer and is not definitely assigned in the constructor	left the <code>!</code> off a field set in <code>create()</code>	add the <code>!</code> , and explain what it promises
Module not found <code>"file:///.../HelloScene"</code>	import without <code>.ts</code>	Deno needs the full file name
A picture shows as a green-and-black box, and the console says <code>Failed to process file: image "logo"</code>	wrong path in <code>this.load.image</code> , or the file was not copied into <code>public/assets/</code>	paths are relative to <code>index.html</code> ; check <code>public/assets/images/</code>

What students see	Cause	Fix
Keys do nothing	the game does not have keyboard focus	click the game first

Discussion questions

- Why does the build still produce a game when there are type errors? When is that helpful, and when is it dangerous?
- The ball moves itself in `preUpdate()` ; the scene could move it in `update()` instead. What is gained by putting the code in `Ball` ? (Encapsulation; many balls for free - challenge 4.)
- What would happen to a game written as "move 4 pixels a frame" on a 144 Hz monitor? On a slow school PC that manages 30 frames a second?

Extension ideas

- Look at `game.loop.actualFps` in the Moving Ball project with the browser's developer tools performance throttling turned on: the ball's speed should not change
- Read `build.ts` together - it is short, and shows how Deno runs other commands
- Try `type: Phaser.CANVAS` in the config and compare; discuss why WebGL is the default

Assessment ideas

- A short practical: "make a scene with your name centred at the top, a picture in the bottom right corner using `setOrigin` , and a key that changes the text"
- Code reading: give students `Ball.ts` with `this.` removed in three places and the `!` missing from a field, and ask them to predict the compiler's messages
- Explain, in two or three sentences, why speeds are multiplied by `delta / 1000`

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 1 - Introduction: your first Phaser 4 game - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment, so a search for `CHALLENGE` finds them all.

1. Make it yours

Project: `solutions/ch01_challenge_1_make_it_yours` (from `ch01_hello Phaser`)

Goal: get comfortable finding and changing values: the config, the page, the text style, an array.

`src/main.ts`

```
title: "Star Painter",
...
backgroundColor: "#264653",
```

`src/scenes/HelloScene.ts`

```
const COLOURS = ["#1d2433", "#3a0ca3", "#2a9d8f", "#9d0208", "#264653",
"#6a4c93", "#ff006e", "#fb8500"];
...
this.message = this.add.text(centreX, 380, "Press SPACE", {
  fontFamily: "Arial",
  fontSize: "48px",
  color: "#ffd166",
});
```

and `<title>Star Painter</title>` in `public/index.html`.

Look for: the title changed in **both** places (the config's title and the page's `<title>` are separate things). `fontSize` accepts either a number (pixels) or a CSS string such as `"48px"`; the guide uses strings because the style object follows CSS - either is correct.

2. Corner labels

Project: `solutions/ch01_challenge_2_corner_labels` (from `ch01_hello_phaser`)

Goal: use origins to line things up against edges without measuring text.

`src/scenes/HelloScene.ts`

```
const w = this.scale.width;
const h = this.scale.height;
this.addLabel(36, 8, "(0, 0)", 0, 0);
this.addLabel(w - 36, 8, `(${w}, 0)`, 1, 0);
this.addLabel(36, h - 8, `(0, ${h})`, 0, 1);
this.addLabel(w - 36, h - 8, `(${w}, ${h})`, 1, 1);
```

```
private addLabel(x: number, y: number, label: string, originX: number,
originY: number): void {
    const text = this.add.text(x, y, label, {
        fontFamily: "Arial",
        fontSize: "16px",
        color: "#ffd166",
    });
    text.setOrigin(originX, originY);
}
```

The right-hand labels use origin `x = 1`, so `x` is their **right** edge and they grow leftwards from it; the bottom labels use origin `y = 1`. The labels are 36 pixels in from the corner so they clear the 32 pixel stars.

Look for: no `text.width` arithmetic; the corner values taken from `this.scale` rather than typed in as 800 and 600; a helper method rather than four copies of the style object (not required, but worth praising). Weaker answers hard-code `x = 720` and so on - it works, until the text changes.

3. Reset key

Project: `solutions/ch01_challenge_3_reset_key` (from `ch01_hello_phaser`)

Goal: a second event listener, and keeping "the starting state" in one place.

```
const START_COLOUR = "#1d2433";  
const START_MESSAGE = "Press SPACE";
```

```
this.input.keyboard!.on("keydown-R", () => {  
  this.reset();  
});
```

```
private reset(): void {  
  this.presses = 0;  
  this.cameras.main.setBackgroundColor(START_COLOUR);  
  this.message.setText(START_MESSAGE);  
}
```

The same message is now used in `create()` and in `reset()`, so it became a constant.

Look for: constants for the starting values (the "same thing written twice" smell); the reset in a named method. A common mistake is setting `this.presses = 0` but not the text, so the screen still says "Presses: 7" until the next press.

4. More balls

Project: `solutions/ch01_challenge_4_more_balls` (from `ch01_moving_ball`)

Goal: see the pay-off of a class that looks after itself: many balls, no extra moving code.

`src/scenes/GameScene.ts`

```

const BALL_COUNT = 5;
const MAX_SPEED = 300;
...
for (let i = 0; i < BALL_COUNT; i++) {
  const x = Phaser.Math.Between(32, this.scale.width - 32);
  const y = Phaser.Math.Between(32, this.scale.height - 32);
  const speedX = Phaser.Math.Between(-MAX_SPEED, MAX_SPEED);
  const speedY = Phaser.Math.Between(-MAX_SPEED, MAX_SPEED);
  new Ball(this, x, y, speedX, speedY);
}

```

`Ball.ts` does not change at all - which is the point to draw out.

Look for: start positions that keep the whole ball on screen (a ball that starts half off the edge gets stuck flipping its speed every frame until `Clamp` rescues it - a nice bug to discuss if someone hits it). A speed of 0 in one direction is possible and harmless.

5. Bounce counter

Project: `solutions/ch01_challenge_5_bounce_counter` (from `ch01_moving_ball`)

Goal: an object that keeps its own state private, and a scene that asks for it.

`src/objects/Ball.ts`

```

private bounces = 0;
...
if (this.x < radius || this.x > right) {
  this.speedX = -this.speedX;
  this.x = Phaser.Math.Clamp(this.x, radius, right);
  this.bounces = this.bounces + 1;
}
...
public getBounces(): number {
  return this.bounces;
}

```

`src/scenes/GameScene.ts`

```
private ball!: Ball;
private bounceText!: Phaser.GameObjects.Text;
...
this.ball = new Ball(this, 400, 300, 240, 180);
...
override update(time: number, delta: number): void {
    this.bounceText.setText(`Bounces: ${this.ball.getBounces()}`);
}
```

The scene now keeps the ball in a field - it needs to ask it something.

Look for: the count kept **in the ball** (it is the ball's fact) and read with a getter, rather than a public field the scene could change. Hitting a corner counts as two bounces in this solution (one each way) - accept either answer if the student can explain their choice. An alternative some students find: the ball emits an event when it bounces (Chapter 6 covers events properly).

6. Steer the ball

Project: `solutions/ch01_challenge_6_steer_the_ball` (from `ch01_moving_ball`)

Goal: polling keys every frame, and applying a change **per second** using `delta` - a force, in effect.

`src/objects/Ball.ts`

```
const TOP_SPEED = 600;
...
public push(amountX: number, amountY: number): void {
    this.speedX = Phaser.Math.Clamp(this.speedX + amountX, -TOP_SPEED,
TOP_SPEED);
    this.speedY = Phaser.Math.Clamp(this.speedY + amountY, -TOP_SPEED,
TOP_SPEED);
}

public getSpeed(): number {
    return Math.sqrt(this.speedX * this.speedX + this.speedY * this.speedY);
}
```

src/scenes/GameScene.ts

```
const PUSH_STRENGTH = 400;
...
this.cursors = this.input.keyboard!.createCursorKeys();
...
const push = PUSH_STRENGTH * (delta / 1000);
let pushX = 0;
let pushY = 0;
if (this.cursors.left.isDown) pushX = pushX - push;
if (this.cursors.right.isDown) pushX = pushX + push;
if (this.cursors.up.isDown) pushY = pushY - push;
if (this.cursors.down.isDown) pushY = pushY + push;
this.ball.push(pushX, pushY);
```

Holding RIGHT for one second from rest gives a speed of about 400 across - tested with the harness (399.3 after 1000 ms).

Look for:

- the push scaled by `delta`. Without it, the ball speeds up faster on faster screens - exactly the mistake the chapter warned about, one level up (this is **acceleration**, so it needs time as much as movement does)
- `createCursorKeys()` and `isDown` (polling) rather than `keydown` events, which fire once per press (and then repeat at the operating system's key-repeat rate - an interesting bug when tried)
- the limit applied in the ball, which owns the speed
- the overall speed shown with Pythagoras is a nice touch but not required; showing `speedX` and `speedY` is fine

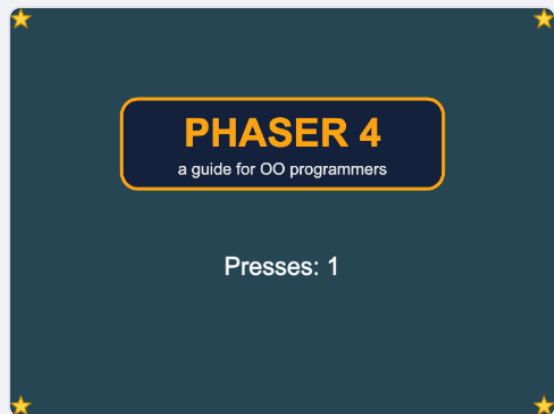
Stronger students may notice this is what Arcade Physics' `setAcceleration` and `setMaxVelocity` do (Chapter 9).

Chapter 1 - Introduction: your first Phaser 4 game - Slides

Chapter 1

Introduction: your first Phaser 4 game

Phaser 4 2D games, in TypeScript, with Deno



1 / 19

Today

- what Phaser is - and what changed in Phaser 4
- building and running a project with Deno and Celbridge
- TypeScript, for Java programmers
- config, scenes, game objects
- coordinates and origins
- the game loop, and pixels per **second**

2 / 19

What is Phaser?

- a free, open source framework for **2D games in the browser**
- you write the rules; Phaser draws, loops, loads, reads input, plays sound, runs physics
- **Phaser 4** (2025): a new drawing engine; the API you write is almost the same as Phaser 3
- most tutorials online are Phaser 3 - nearly always still right
- ships with TypeScript types: the compiler knows every Phaser class and method

3 / 19

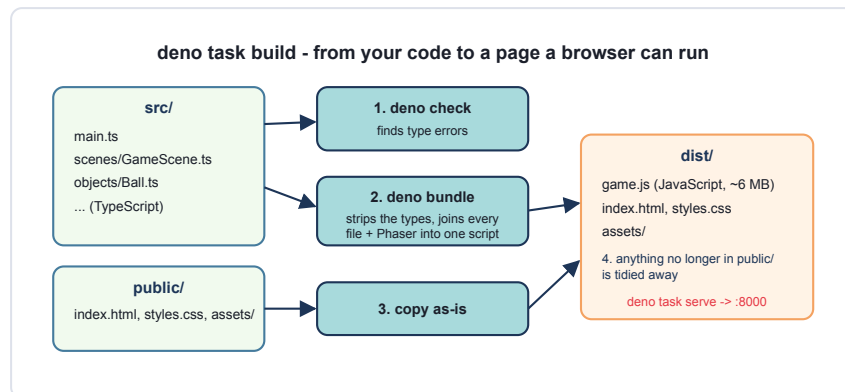
The tools

- **Deno** - runs TypeScript, type checks, bundles, serves. Nothing else to install
- the only package: **Phaser** (downloaded once, on the first build)
- **Celbridge** - `terminal.console` (buttons for each command) and `game.webview`

```
deno task build    type check + bundle into dist/  
deno task serve    http://127.0.0.1:8000/  
deno task dev      rebuild on every save
```

4 / 19

What "build" does



The game must be **served**: browsers will not load pictures for a page opened from a file.

5 / 19

TypeScript for Java programmers (1)

```
const SPEED = 200;           // never changes
let score = 0;                // changes - type worked out: number
let name: string = "Ada";     // type AFTER the name

7 / 2                         // 3.5 - there is only `number`
a === b                        // always ===, never ==

`Presses: ${this.pressures}` // template string
```

6 / 19

TypeScript for Java programmers (2)

```
export class GameScene extends Phaser.Scene {  
  private score: number;  
  
  constructor() {  
    super("GameScene");           // before using `this`  
    this.score = 0;               // ALWAYS `this.`  
  }  
  
  override update(_time: number, delta: number): void {  
    this.score = this.score + delta / 1000;  
  }  
}
```

`override` is compulsory here. `_time` = "not used".

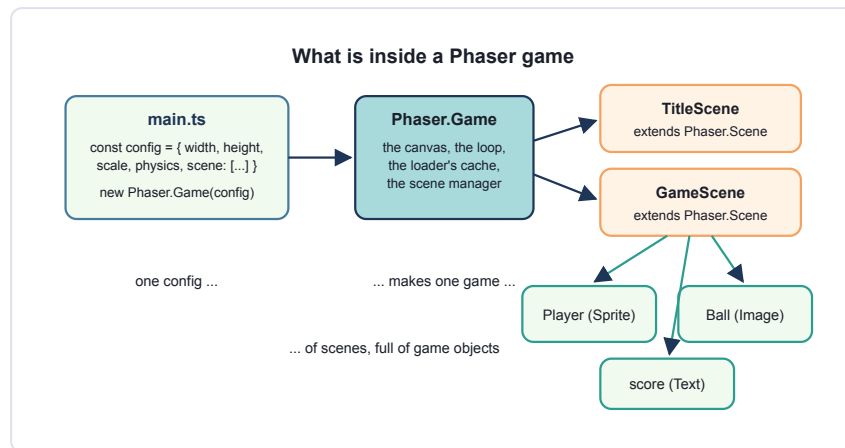
7 / 19

TypeScript for Java programmers (3)

```
private message!: Phaser.GameObjects.Text; // "set later, in create()"  
  
this.input.keyboard!.on("keydown-SPACE", () => { // arrow function:  
  this.changeColour();                          // `this` is still the scene  
});  
  
import { HelloScene } from "../scenes/HelloScene.ts"; // .ts on your own files
```

8 / 19

Inside a Phaser game



9 / 19

1. The config - main.ts

```
const config: Phaser.Types.Core.GameConfig = {
  type: Phaser.AUTO,
  parent: "game",
  backgroundColor: "#1d2433",
  scale: { width: 800, height: 600,
    mode: Phaser.Scale.FIT, autoCenter: Phaser.Scale.CENTER_BOTH },
  scene: [HelloScene],
};
new Phaser.Game(config);
```

Always **800 x 600 inside**, scaled to fit the page.

10 / 19

2. Scenes

Method	Called	For
<code>constructor()</code>	once	<code>super("HelloScene")</code> - its key
<code>preload()</code>	scene starts	load pictures and sounds
<code>create()</code>	all loaded	make game objects
<code>update(time, delta)</code>	every frame	the rules

11 / 19

3. Game objects

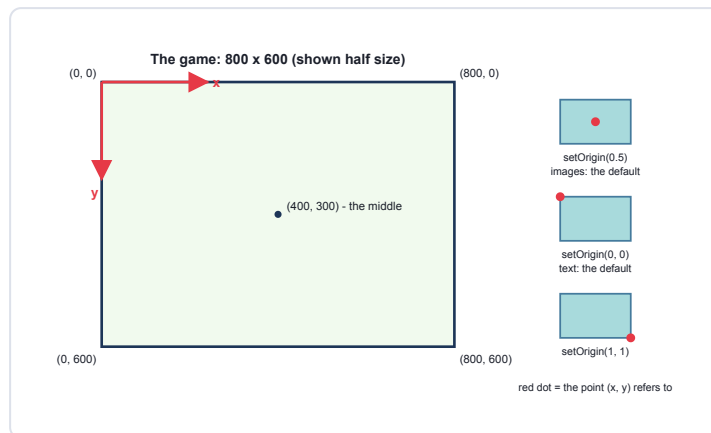
```
preload(): void {
    this.load.image("logo", "assets/images/logo.png");
}

create(): void {
    this.add.image(400, 200, "logo");
    this.message = this.add.text(400, 380, "Press SPACE", { ... });
    this.message.setOrigin(0.5);
}
```

You never draw. Change a game object; Phaser draws it next frame.

12 / 19

Coordinates and origins



13 / 19

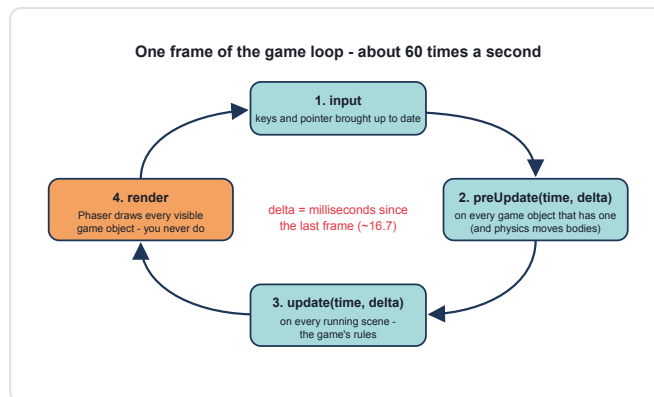
Try it: Hello Phaser

- open `ch01_hello_phaser` , build, serve, open `game.webview`
- click the game, press SPACE
- find the line that picks the colour - and the one that counts

```
this.input.keyboard!.on("keydown-SPACE", () => {  
    this.changeColour();  
});
```

14 / 19

The game loop



15 / 19

A game object that moves itself

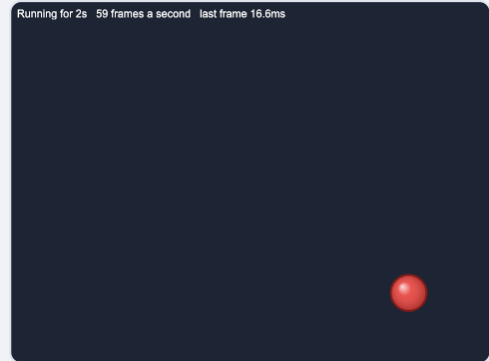
```
export class Ball extends Phaser.GameObjects.Image {
  constructor(scene: Phaser.Scene, x: number, y: number,
    speedX: number, speedY: number) {
    super(scene, x, y, BALL_KEY);
    ...
    scene.add.existing(this); // put it in the scene
  }

  preUpdate(_time: number, delta: number): void {
    const seconds = delta / 1000;
    this.x = this.x + this.speedX * seconds;
    this.y = this.y + this.speedY * seconds;
  }
}
```

16 / 19

Why pixels per second?

- `delta` = milliseconds since the last frame (~16.7 at 60 fps)
- "4 pixels a frame" = 240/s at 60 Hz ... and **576/s** at 144 Hz
- speed x time = distance: the same game on every screen



17 / 19

Summary

- `config` -> `game` -> `scenes` -> `game objects`
- Phaser calls `preload`, `create`, `update`; you never draw
- `setOrigin` says which point (x, y) means; y grows downwards
- events (`keyboard.on(...)`) for one-off actions
- the loop runs ~60 times a second; move by `speed * delta / 1000`

18 / 19

Challenges

1. **Make it yours** - title, colours, text style
2. **Corner labels** - coordinates in each corner, using origins
3. **Reset key** - R puts everything back
4. **More balls** - five random balls, one loop
5. **Bounce counter** - the ball counts, the scene shows
6. **Steer the ball** - arrows push the ball, up to a top speed

Next: **Chapter 2 - a three-scene game**



A three-scene game

Overview

Students build a complete (tiny) game with a beginning, a middle and an end. The Phaser ideas are **scenes and the scene manager**, **passing data between scenes**, and **pointer input on game objects**; the TypeScript ideas are **interfaces**, **object literals**, **union types** and the **spread operator**. The one conceptual trap - scene objects are reused, so fields keep old values - is worth a whole discussion: it is the most common "my game works once" bug students will write this term.

Prerequisites

- Chapter 1: building and serving, config/scene/game object, `preUpdate` and `delta`
- Java interfaces (for the analogy with TypeScript interfaces)

Learning outcomes

Students can:

1. structure a game as several scenes, and move between them with `this.scene.start(key, data)`
2. pass data between scenes through `init(data)`, described by an `interface`
3. explain why a restarted scene keeps its field values, and reset them in `init()`
4. make a game object respond to the pointer with `setInteractive()` and `"pointerdown"`, and detect clicks on nothing with `this.input.on("pointerdown", ...)`
5. choose between `on` and `once` for an event
6. share a value between scenes with `this.registry`

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Play <code>ch02_click_the_ball</code> together. Ask: "how many screens? what moves you between them?" Draw the scene-flow diagram on the board before showing it
0:10 - 0:30	Slides 3-8: scenes, keys, <code>scene.start</code> , <code>on</code> vs <code>once</code> , loading once. Walk through <code>StartScene.ts</code>
0:30 - 0:45	Slides 9-11: <code>setInteractive</code> , <code>pointerdown</code> , the clock. Walk through <code>PlayScene.ts</code>
0:45 - 1:00	Slides 12-13: passing data, interfaces. Challenges 1-3
1:00 - 1:15	Live demo of the trap: in the plus version, comment out the body of <code>init()</code> in <code>PlayScene</code> , win once, play again - the first click wins. Ask students to explain before revealing slide 14
1:15 - 1:30	Slides 15-17: the plus version: misses, encapsulation, sound, registry, spread
1:30 - 2:00	Challenges 4-6

Key points to stress

- `scene.start` **shuts down** the current scene: its game objects are destroyed and its listeners removed. Students sometimes expect the old scene to be "underneath" still
- scenes talk to each other through **data passed to** `start` (one-off hand-over) or the **registry** (shared, long-lived). Resist letting students reach into other scenes with `this.scene.get(...)` to read their fields - it couples scenes tightly (Chapter 6 shows events, which are better still)
- the object is reused: `init()` **is where a scene resets itself**
- the `Ball` class did not change to become clickable - the **scene** decided. A good moment to talk about keeping classes focused

Common problems and errors

What students see	Cause	Fix
Pressing SPACE does nothing; no error	scene key misspelled as a string ("Playscene")	use the constants in <code>keys.ts</code>
Second round starts with the old score/hits, or ends instantly	fields reset only where declared	reset in <code>init()</code>
Property 'seconds' does not exist on type 'object'	<code>init(data: object)</code> or untyped <code>data</code>	type the parameter with the interface
Object literal may only specify known properties, and 'secs' does not exist in type 'WinData'	misspelled property in the data	this is the interface doing its job
Clicking the ball does nothing	forgot <code>setInteractive()</code>	game objects ignore the pointer until made interactive
Every click counts as a miss, including hits	the scene-wide listener ignores the <code>over</code> parameter	check <code>over.length === 0</code>
No sound in the plus version	the browser blocks audio until the page has been clicked or a key pressed	click the game first; mention Chapter 5
'best' is possibly 'undefined'	using <code>best.toFixed()</code> without checking	this is the union type <code>`number`</code>

Discussion questions

- Why keep scene keys in one file? What other strings in a game deserve the same treatment? (Asset keys, event names, registry names.)
- `WinScene` needs the time. Name three ways it could get it. (Data passed to `start`, the registry, reading `PlayScene`'s field.) Which is best, and why?
- In the plus version the ball's speed is private and the scene calls `speedUp()`. What would be lost if the scene set `ball.speedX` directly?

- Is "Too slow!" (challenge 4) a win scene any more? When is it right to reuse a scene with a flag, and when should it be a new scene?
-

Extension ideas

- add a pause: `this.scene.pause()` and `this.scene.resume()` , and a "PAUSED" text
 - launch a second scene **on top** of the play scene (`this.scene.launch(...)`) for a HUD - a preview of Chapter 6
 - make the ball a different random colour each round with `setTint(...)`
-

Assessment ideas

- Practical: "add a fourth scene - an instructions screen between start and play - with its own key, reached by SPACE, left by SPACE"
 - Code reading: show `PlayScene` without `init()` , and a transcript of two rounds; ask for the bug and the fix
 - Short answer: the difference between `this.registry` and the data passed to `this.scene.start`
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 2 - A three-scene game - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Restyle

Project: `solutions/ch02_challenge_1_restyle` (from `ch02_click_the_ball`)

Goal: find where each thing is decided - the picture, the speed, the words - and change it.

`src/objects/Ball.ts`

```
export const BALL_KEY = "ball";  
export const BALL_FILE = "assets/images/ball_blue.png";
```

`src/scenes/PlayScene.ts`

```
const BALL_SPEED = 520;
```

plus new wording and colours in `StartScene.create()` and `WinScene.create()`, and `ball_blue.png` copied from `assets/images/` into the project's `public/assets/images/`.

Look for: the file copied into `public/` (the most common failure: the path is right but the file is not there, and Phaser shows a green-and-black "missing texture" square). Changing `BALL_FILE` but keeping the key `"ball"` is fine - the key is just a name.

2. Enter works too

Project: `solutions/ch02_challenge_2_enter_works_too` (from `ch02_click_the_ball`)

Goal: two listeners, one action - and no duplicated code.

`src/scenes/StartScene.ts` (and the same pattern in `WinScene.ts`)

```
this.input.keyboard!.once("keydown-SPACE", () => {
  this.next();
});
this.input.keyboard!.once("keydown-ENTER", () => {
  this.next();
});
}

private next(): void {
  this.scene.start(PLAY_SCENE);
}
```

Look for: the action in one method rather than written twice; the on-screen messages updated to match. Some students look for a single event for "SPACE or ENTER" - there is not one, and two listeners is the normal answer. A student who uses `this.input.keyboard!.on("keydown", (event) => ...)` and checks `event.code` has found a valid alternative.

3. Give up

Project: `solutions/ch02_challenge_3_give_up` (from `ch02_click_the_ball`)

Goal: a new way out of a scene, and confidence that restarting a scene rebuilds it.

`src/scenes/PlayScene.ts`

```
this.input.keyboard!.once("keydown-ESC", () => {
  this.scene.start(START_SCENE);
});
```

The start scene "works exactly as it did the first time" because its `create()` runs again, making fresh text and a fresh SPACE listener. Nothing needs resetting: `StartScene` has no fields.

Look for: the import of `START_SCENE`. Tested with the harness: SPACE, ESC, SPACE goes Start -> Play -> Start -> Play.

4. Too slow!

Project: `solutions/ch02_challenge_4_too_slow` (from `ch02_click_the_ball`)

Goal: a timer event, and a scene that shows two different outcomes from its data.

`src/scenes/PlayScene.ts`

```
const TIME_LIMIT = 5000;
...
this.time.delayedCall(TIME_LIMIT, () => {
  this.lose();
});
...
private win(): void {
  const seconds = (this.time.now - this.startTime) / 1000;
  const data: WinData = { won: true, seconds: seconds };
  this.scene.start(WIN_SCENE, data);
}

private lose(): void {
  const data: WinData = { won: false, seconds: TIME_LIMIT / 1000 };
  this.scene.start(WIN_SCENE, data);
}
```

`src/scenes/WinScene.ts`

```
export interface WinData {
  won: boolean;
  seconds: number;
}
...
const heading = this.won ? "You got it!" : "Too slow!";
const detail = this.won ? `in ${this.seconds.toFixed(2)} seconds` : `You had
${this.seconds} seconds`;
```

The timer belongs to the play scene's clock, so when the player wins and the scene shuts down, the timer is thrown away with it - it can never fire after a win. Worth asking students *why* it does not.

Look for: adding `won` to the interface - the compiler then insists both `win()` and `lose()` say which it is. Checking elapsed time in `update()` instead of `delayedCall` also works; accept it, but point out the timer is simpler and cannot be forgotten.

5. The decoy

Project: `solutions/ch02_challenge_5_the_decoy` (from `ch02_click_the_ball_plus`)

Goal: reuse a class with a parameter; understand how the "missed everything" listener sees the world.

`src/objects/Ball.ts`

```
export const DECOY_KEY = "decoy";
export const DECOY_FILE = "assets/images/ball_blue.png";
...
constructor(scene: Phaser.Scene, x: number, y: number, speedX: number,
speedY: number, key: string = BALL_KEY) {
  super(scene, x, y, key);
```

`src/scenes/PlayScene.ts`

```
const decoy = new Ball(this, Phaser.Math.Between(100, 700),
Phaser.Math.Between(100, 500),
  Math.cos(decoyAngle) * DECOY_SPEED, Math.sin(decoyAngle) * DECOY_SPEED,
DECOY_KEY);
decoy.setInteractive({ useHandCursor: true });
decoy.on("pointerdown", () => {
  for (let i = 0; i < DECOY_MISSES; i++) {
    this.miss();
  }
});
```

The decoy is interactive, so a click on it puts it in the `over` list - the scene-wide listener does **not** count it as a miss, and the decoy's own listener counts two. Tested with the harness: decoy, ball, empty space gives 1 hit and 3 misses.

Look for:

- a **default parameter** (`key: string = BALL_KEY`) so the existing call still works - or a new subclass `Decoy` extends `Ball` , which is also a good answer
- the blue picture loaded in `StartScene`
- a common wrong answer: making the decoy *not* interactive and relying on the "missed everything" listener - that counts it as **one** miss, not two

6. Top five

Project: `solutions/ch02_challenge_6_top_five` (from `ch02_click_the_ball_plus`)

Goal: keep a structured value (an array) in the registry; basic array work.

`src/scenes/WinScene.ts`

```
const times: number[] = this.registry.get(BEST_TIMES) ?? [];
times.push(total);
times.sort((a, b) => a - b);
const top = times.slice(0, TABLE_SIZE);
this.registry.set(BEST_TIMES, top);

const place = top.indexOf(total) + 1;
```

`src/scenes/StartScene.ts`

```
const times: number[] = this.registry.get(BEST_TIMES) ?? [];
let table = "BEST TIMES\n";
if (times.length === 0) {
  table = table + "none yet";
}
times.forEach((time, index) => {
  table = table + `${index + 1}.  ${time.toFixed(2)} s\n`;
});
```

- `??` is the **nullish coalescing** operator: "this, unless it is `null` or `undefined` , in which case that"
- `sort()` on its own sorts numbers **as text** (so 10 comes before 9) - hence the compare function. This catches out nearly everyone; a great discussion point

- `indexOf` gives `-1` when the value is not there, so `place` is `0` when the time missed the table

Look for: the compare function in `sort` ; `slice` so the table never grows past five; a sensible message when the time does not make the table. Sharp students notice `indexOf(total)` finds the *first* equal time if two totals tie exactly - fine to accept.

Chapter 2 - A three-scene game - Slides

Chapter 2

A three-scene game

Start - play - win

Time: 2.3



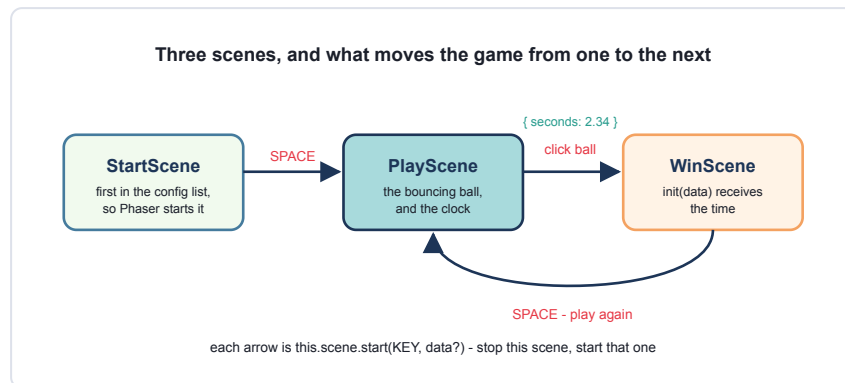
1 / 18

Today

- splitting a game into **scenes**
- moving between scenes; passing data along
- the trap: scenes are **reused**
- clicking on game objects
- `on` and `once`
- the **registry**: values every scene can see

2 / 18

Three scenes



```
scene: [StartScene, PlayScene, WinScene], // Phaser starts the FIRST
```

3 / 18

Scene keys - as constants

```
// keys.ts
export const START_SCENE = "StartScene";
export const PLAY_SCENE = "PlayScene";
export const WIN_SCENE = "WinScene";

// PlayScene.ts
constructor() {
  super(PLAY_SCENE);
}
```

`start("Playscene")` fails **silently**. `start(PLAYSCENE)` fails the **build**.

4 / 18

Starting a scene

```
this.input.keyboard!.once("keydown-SPACE", () => {  
  this.scene.start(PLAY_SCENE);  
});
```

What this.scene.start(PLAY_SCENE, data) does



The scene OBJECT is not new: Phaser made it once, when the game began, and starts it again.
Its fields keep their old values - which is why init() puts them back.

5 / 18

on or once?

- `on(event, fn)` - every time
- `once(event, fn)` - the first time, then stop listening
- a scene's listeners are removed when it shuts down
- use `once` when something should happen once: it says what you mean

6 / 18

Load once, use everywhere

```
// StartScene - the first scene
preload(): void {
    this.load.image(LOGO_KEY, LOGO_FILE);
    this.load.image(BALL_KEY, BALL_FILE);
}
```

Loaded files belong to the **game**. `PlayScene` has no `preload()` .

7 / 18

Clicking a game object

```
const ball = new Ball(this, 400, 300, speedX, speedY);

ball.setInteractive({ useHandCursor: true });

ball.on("pointerdown", () => {
    this.win();
});
```

- ignored by the pointer until `setInteractive()`
- **pointer** = mouse *or* finger
- `Ball` does not know it can be clicked - the scene decides

8 / 18

Timing

```
create(): void {  
    this.startTime = this.time.now;  
}  
  
override update(time: number, _delta: number): void {  
    const seconds = (time - this.startTime) / 1000;  
    this.timeText.setText(`Time: ${seconds.toFixed(1)}`);  
}
```

`this.time` - the scene's clock, in milliseconds

9 / 18

Try it: Click the Ball

- build, serve, play `ch02_click_the_ball`
- find the three `this.scene.start(...)` calls
- which scene loads the ball's picture? Which uses it?

PHASER 4

a guide for OO programmers

Click the bouncing ball as fast as you can

Press SPACE to start

10 / 18

Passing data

```
// PlayScene
const data: WinData = { seconds: seconds };
this.scene.start(WIN_SCENE, data);

// WinScene
export interface WinData {
  seconds: number;
}

init(data: WinData): void {
  this.seconds = data.seconds;
}
```

`{ seconds: 2.3 }` - an **object literal**. `interface` - its shape, checked.

11 / 18

The trap

```
private hits = 0; // runs ONCE - when the object is made
```

- Phaser makes **one** `PlayScene` , and starts the **same object** again
- game objects: new every time (`create()`)
- fields: **whatever the last round left**

```
init(): void { // runs EVERY time the scene starts
  this.hits = 0;
  this.misses = 0;
}
```

12 / 18

The plus version

- 5 hits to win; faster each time
- misses cost a second
- sounds
- a best time



13 / 18

Clicks on nothing

```
this.input.on("pointerdown",
  (_pointer: Phaser.Input.Pointer, over: Phaser.GameObjects.GameObject[]) => {
    if (over.length === 0) {
      this.miss();
    }
  });
```

`over` = the interactive game objects under the pointer

14 / 18

Ask, don't reach in

```
// Ball
public speedUp(factor: number): void {
    this.speedX = this.speedX * factor;
    this.speedY = this.speedY * factor;
}

// PlayScene
this.ball.speedUp(SPEED_UP);
this.ball.teleport();
```

15 / 18

The registry

```
// WinScene
const best: number | undefined = this.registry.get(BEST_TIME);
if (best === undefined || total < best) {
    this.registry.set(BEST_TIME, total);
}
```

- shared by **every scene**, for as long as the game runs
- `number | undefined` - a **union type**
- `{ ...style, fontSize: "64px" }` - **spread**: a copy, with changes

16 / 18

Summary

- `scene: [...]` - Phaser starts the first
- `this.scene.start(KEY, data)` -> the new scene's `init(data)`
- keys as constants; data described by an `interface`
- scenes are reused: reset in `init()`
- `setInteractive()` + `"pointerdown"` ; `this.input.on(...)` for every click
- `this.registry` - shared values

17 / 18

Challenges

1. **Restyle** - blue ball, faster, new words
2. **Enter works too** - ENTER does what SPACE does
3. **Give up** - ESC back to the title
4. **Too slow!** - 5 seconds, then "Too slow!" - no fourth scene
5. **The decoy** - a blue ball worth two misses
6. **Top five** - a best-times table in the registry

Next: **Chapter 3 - Preloading**

18 / 18



Preloading

Overview

Students have called `this.load.image` since Chapter 1 without thinking about it. This chapter opens the loader up: `load...` calls only **queue** files; the loader fetches them after `preload()`; loaded files live in **game-wide caches** under **keys**. On that foundation students build the standard **Boot -> Preload -> Menu** structure with a progress bar (their first use of `Graphics`), load every common kind of file (including JSON described by an interface, and an asset pack), learn what a missing file looks like, and load files in the middle of a game with `this.load.start()`.

The practical difficulty of teaching it: **local files load too fast to see**. The loading screen project pads its queue with 150 copies of small pictures, but the honest demonstration is the browser's network throttling - rehearse it before the session.

Prerequisites

- Chapter 1: building and serving, config/scene/game object, `preload()` / `create()`
- Chapter 2: several scenes, `this.scene.start(key, data)`, `init(data)`, interfaces, `on / once`, spread
- JSON as a format (most students have met it; a two-minute reminder is enough)

Learning outcomes

Students can:

1. explain what `this.load.image(key, file)` does and does not do, and when `create()` runs
2. say where loaded files are kept, and use keys to read them back from `this.textures` and `this.cache.*`
3. structure a game as Boot -> Preload -> Menu, and explain why the boot scene exists
4. draw a progress bar with `Graphics`, driven by the loader's `"progress"` event

5. load images, sprite sheets, audio, JSON (typed with an interface), text and asset packs
6. recognise and diagnose a missing file (the missing texture, `"loadererror"`, the console's 404)
7. load files after `preload()` with `this.load.start()` and `"complete"`, checking the caches first

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Ask: "what happens between pressing refresh and seeing the title screen?" Collect answers; then show the Network tab of the developer tools while <code>ch02_click_the_ball_plus</code> loads
0:10 - 0:25	Slides 3-6: the queue, keys and caches, paths, <code>setPath</code> . Draw the queue -> loader -> caches diagram
0:25 - 0:45	Slides 7-11: Boot -> Preload -> Menu, <code>Graphics</code> , loader events. Walk through <code>PreloadScene.ts</code> . Live demo: run it normally (a flash), then with throttling on "Fast 4G"
0:45 - 1:00	Challenges 1 and 2
1:00 - 1:20	Slides 12-16: every kind of file, JSON and interfaces, asset packs, missing files. Live demo: rename a picture in <code>public/</code> , rebuild, find it with the console
1:20 - 1:30	Slides 17-18: loading later. Play <code>ch03_load_on_demand</code> throttled; press U and go back in
1:30 - 2:00	Challenges 3-6

For a 1 hour lecture + 2 hour lab: lecture slides 1-18 with the three demos; lab challenges 1-6.

Key points to stress

- `preload()` **only makes a list.** Nothing is loaded until it returns. This one sentence explains most loading bugs students write

- **The caches are the game's, not the scene's.** A file loaded anywhere can be used everywhere, and a key that is already loaded is skipped. This is what makes both "load everything at the start" and "load on demand, but only once" work
- **Keys are names; files are addresses.** After `preload()` the file name should never appear again. Keep keys as constants, as scene keys were in Chapter 2
- **Paths are relative to the page** (`dist/index.html`), not to the `.ts` file - and everything in `public/` is copied to `dist/`, so "not in `public/` " means "not there"
- **The boot scene exists because a loading screen cannot show something it is still loading**
- **Outside `preload()` , you start the loader yourself.** No `this.load.start()` , no loading, no error
- `cache.json.get` returns `any` ; the interface is a promise *about* the file, checked against the code, never against the file. Good discussion of what static typing can and cannot do

Common problems and errors

What students see	Cause	Fix
A black square with a green outline and a green diagonal	the missing texture: no picture with that key	check the console for a 404; check the file is in <code>public/</code> , and the name, folder and capitals; check the key matches
Works at home, missing pictures once uploaded (or on a lab machine)	capitals: <code>Sky.png</code> vs <code>sky.png</code> - macOS and Windows usually do not mind, web servers do	make the names match exactly; use lower case for every file
Error: Audio key "shoot" not found in cache in the console, and the scene stops	<code>this.sound.play(key)</code> for a sound that is not loaded (misspelt key, missing file, or loaded in a scene that has not run yet)	load it first; check the key
Error: TextureManager.SpriteSheet: Invalid frameWidth given. and an empty screen	<code>this.load.spritesheet(key, file)</code> with no <code>{ frameWidth, frameHeight }</code>	give the frame size (the asset library README lists them)
A sprite sheet's frames are cut in the wrong places, or show parts of two pictures	wrong <code>frameWidth / frameHeight</code>	check the frame size
Parameter 'value' implicitly has an 'any' type. on a loader listener	the listener's parameter has no type; <code>this.load.on</code> accepts any function, so TypeScript cannot work it out	write the type: <code>(value: number)</code> , <code>(file: Phaser.Loader.File)</code>
Property 'gem' does not exist on type 'LevelData'. Did you mean 'gems'?	a typo reading JSON data - the interface catching it	fix the name
Property 'hearts' does not exist on type 'LevelData'.	the JSON file gained a field; the interface did not	add it to the interface
Cannot read properties of undefined (reading 'length') in the browser	the interface has a field the JSON file does not (misspelt in the file, or a missing comma broke the file)	fix the file; check it is valid JSON

What students see	Cause	Fix
Loading in <code>create()</code> does nothing at all	forgot <code>this.load.start()</code>	call it after queueing
A new picture under an old key does not appear; the old one stays	the loader skips keys already in the cache	<code>this.textures.remove(key)</code> first, or use a new key
Loading screen shows the missing texture instead of the logo	the logo was loaded in <code>PreloadScene</code> , not <code>BootScene</code>	load what the loading screen shows in the boot scene
The bar never appears / the loading screen flashes past	local files are fast	throttle the network; that is the real test
A path starting with <code>/</code> works with <code>deno task serve</code> , then breaks elsewhere	<code>/</code> means the top of the web server, not the page	leave the leading <code>/</code> off

A subtle one for the curious: in `preload()`, `this.time.now` can be out of date - the scene's clock is only updated once the scene is running frames, and `preload()` comes before its first frame. The first time a scene runs it holds the time the game started (close enough, for a scene that starts straight away); when the scene is started again it holds the time the scene last stopped. That is why `PreloadScene` times the load with `performance.now()`. Chapter 4 explains the order in which Phaser does things.

Suggested demos

- **The queue is not the load.** In `BootScene.preload()`, add `this.add.image(400, 300, LOGO_KEY)` straight after `this.load.image(LOGO_KEY, ...)`. The missing texture appears (briefly - the scene moves on). Ask why, before explaining
- **Throttling.** Run `ch03_loading_screen` normally, then with the Network tab's "Disable cache" and "Fast 4G". Show the waterfall of requests: several at once, big files taking longest. Then set `COPIES` to 0 and run it again throttled - the bar still moves, because the music is big
- **Loading again.** Press R in the menu: "Loaded 0 files". Ask what the loader did with 170 keys that were already loaded

- **A missing file.** Rename `sky.png` in `public/assets/images/`, rebuild, refresh. Find the 404 in the console; notice the rest of the game still works. Then misspell a sound's key and watch the scene stop with an error instead
 - **Unloading.** In `ch03_load_on_demand`, visit Space, press U, visit Space again with throttling on: it loads again. Discuss what a 40-level game would do
-

Discussion questions

- A loading screen is a scene that shows pictures while loading pictures. How does the boot scene get round this? What else would you put in a boot scene?
 - Why does the loader skip a key that is already loaded? When is that helpful, and when is it a trap?
 - JSON level files, or levels written in TypeScript? What does each make easier? Who else might edit a JSON file?
 - An asset pack moves the list of files out of the code. The code still needs the keys. What has been gained?
 - Load everything up front, or on demand? Think of a game where each is clearly right. (A small arcade game; an open world, or a game with many levels.)
 - `this.textures.remove(key)` while a game object is still showing that picture - what would you expect? Why is it safe in the rooms project?
-

Extension ideas

- load a **bitmap font** or a **web font** (`this.load.font`) and use it on the menu
- a loading screen whose tips change: load `tips.txt` in the boot scene, split it into lines with `split("\n")`, and show a random one under the bar
- `filecomplete-json-level`: load a JSON file that *lists* other files, and queue those from inside that event while the loader is still running (the progress bar goes backwards - why?)
- preload the *next* room in the background while the player is in the current one
- count bytes rather than files: add up `file.bytesTotal` for a truer progress bar

Assessment ideas

- Practical: "add a fifth room with its own background and three pictures; nothing should need to change except `rooms.ts` and `public/` "
- Code reading: show a `create()` that queues two files and adds images of them straight away, with no `start()` and no `"complete"` . What does the player see? Fix it
- Short answer: explain the difference between a key and a file name, and why the rest of the game should only use keys
- Short answer: what is the missing texture, and name three different causes of it

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/` .

Chapter 3 - Preloading - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (JSON files cannot have comments, so changes to them are described here instead).

To see a loading screen solution working, slow the network down (Chrome or Edge: F12, **Network, Disable cache, Fast 4G**) - or it is over before you can look at it.

1. Restyle the bar

Project: `solutions/ch03_challenge_1_restyle_the_bar` (from `ch03_loading_screen`)

Goal: find where each part of the loading screen is decided, and learn a little more of the `Graphics` API.

`src/scenes/PreloadScene.ts`

```
// CHALLENGE 1: a longer, taller bar, in a rounded box with an outline
const BAR_WIDTH = 500;
const BAR_HEIGHT = 36;
const BAR_Y = 350;
const BOX_COLOUR = 0x0b0e14;
const BOX_OUTLINE = 0xf1faee;
const BOX_PADDING = 8;           // the gap between the box and the bar
inside it
const BOX_RADIUS = 12;
```

```
box.fillStyle(BOX_COLOUR);
box.fillRoundedRect(boxX, boxY, boxWidth, boxHeight, BOX_RADIUS);
box.strokeStyle(3, BOX_OUTLINE);
box.strokeRoundedRect(boxX, boxY, boxWidth, boxHeight, BOX_RADIUS);
```

```
bar.fillStyle(this.barColour(value));    // CHALLENGE 1
```

```
// CHALLENGE 1: red, then orange, then green, as the bar fills
private barColour(value: number): number {
    if (value < 1 / 3) {
        return EARLY_COLOUR;
    }
    if (value < 2 / 3) {
        return MIDDLE_COLOUR;
    }
    return LATE_COLOUR;
}
```

plus new wording and colours for the two lines of text. Because the whole bar is cleared and redrawn on every "progress" event, choosing its colour from `value` each time is all it takes - the bar turns wholly orange at a third, and wholly green at two thirds.

Look for: the sizes kept as named constants, and the box worked out *from* them (so changing `BAR_WIDTH` still centres everything); `LineStyle` before `strokeRoundedRect` (the stroke uses whatever line style was set last - forget it and there is no outline); the colour choice in a small method rather than inline in the listener. A student who makes the colour **blend** from red to green (`Phaser.Display.Color.Interpolate.ColorWithColor`) has gone further than asked - welcome it.

2. Percentage

Project: `solutions/ch03_challenge_2_percentage` (from `ch03_loading_screen`)

Goal: turn the "progress" value into something a person reads; think about rounding.

`src/scenes/PreloadScene.ts`

```
// CHALLENGE 2: the percentage, in the middle of the bar. Made AFTER the
// bar, so it is drawn
// on top of it
const percentText = this.add.text(centreX, BAR_Y + BAR_HEIGHT / 2, "0%", {
  fontFamily: "Arial",
  fontSize: "20px",
  color: "#ffffff",
  stroke: "#1b1f2a",
  strokeThickness: 4,
}).setOrigin(0.5);
```

```
// CHALLENGE 2: value is 0 to 1; floor, so it only says 100% when it really
// is finished
percentText.setText(`${Math.floor(value * 100)}%`);
```

- **order of creation is order of drawing:** text made before the bar `Graphics` would be hidden behind the bar as it grows. A good moment to say so (Chapter 4 covers draw order and depth)
- `Math.round(0.996 * 100)` is 100 - the screen would say 100% with a file still to come. `Math.floor` never over-promises
- the stroke makes white text readable over both the yellow bar and the dark box

Look for: the update inside the existing `"progress"` listener (a second listener is fine too); a whole number (students who write `value * 100` get `36.470588...%`); the text drawn on top of the bar. Showing "12 of 170 files" as well, with `this.load.totalComplete` and `this.load.totalToLoad`, is a good extension.

3. Hearts in the level

Project: `solutions/ch03_challenge_3_hearts_in_the_level` (from `ch03_asset_types`)

Goal: change a data file *and* its TypeScript description together.

`public/assets/data/level.json` - a new list, after `"gems"` (note the comma that now has to follow the `]` of `"gems"`):

```
"hearts": [
  { "x": 30, "y": 40 },
  { "x": 160, "y": 130 },
  { "x": 225, "y": 45 }
]
```

src/LevelData.ts

```
hearts: Point[];    // CHALLENGE 3: the new list in level.json
```

src/scenes/AssetTypesScene.ts

```
// CHALLENGE 3: say how many hearts too
this.add.text(x + 10, y + 34, `${level.name} - ${level.gems.length} gems,
${level.hearts.length} hearts`, SMALL_STYLE);
```

```
// CHALLENGE 3: the hearts - heart.png is already loaded, by the asset pack
level.hearts.forEach((heart) => {
  this.add.image(originX + heart.x, originY + heart.y, HEART_KEY);
});
```

No new file needs loading: `heart.png` comes in with the asset pack, and textures are shared, so the JSON panel can use it.

Look for: the interface updated. A student who skips it gets `Property 'hearts' does not exist on type 'LevelData'`. - which is the point of the interface. The opposite mistake - interface updated, JSON not (or the JSON broken by a missing comma) - builds cleanly and then fails in the browser: with no `"hearts"` in the file, `level.hearts.length` throws `Cannot read properties of undefined`. Ask students which of the two mistakes they would rather make. Sharp students make `hearts` optional (`hearts?: Point[]`) so older level files still work, and then have to check for `undefined` - a fine answer.

4. Keep it moving

Project: `solutions/ch03_challenge_4_keep_it_moving` (from `ch03_loading_screen`)

Goal: understand what runs while a scene is loading - and which scene must load what.

src/assets.ts

```
// CHALLENGE 4: the spinner is on the loading screen, so the boot scene must load it too  
export const SPINNER_KEY = "spinner";  
export const SPINNER_FILE = "assets/images/star.png";
```

src/scenes/BootScene.ts

```
this.load.image(SPINNER_KEY, SPINNER_FILE);    // CHALLENGE 4
```

src/scenes/PreloadScene.ts

```
const spinner = this.add.image(barX + BAR_WIDTH + 45, BAR_Y + BAR_HEIGHT /  
2, SPINNER_KEY);  
this.tweens.add({  
  targets: spinner,  
  angle: 360,  
  duration: SPIN_TIME,  
  repeat: -1,  
});
```

While a scene is loading, Phaser still runs its **systems** every frame - its clock, its tweens, its loader - but it does not call the scene's own `update()` until `create()` has run. So code in `update()` that turns the star does nothing until the loading is over; a tween turns it throughout. (Tested with the harness on a throttled network: the star's `angle` changes between two readings taken while `load.progress` is still below 1.)

`this.time.addEvent({ delay, loop: true, callback })` that changes the angle a step at a time also works, for the same reason.

Look for: the picture loaded in `BootScene`. Students who load it in `PreloadScene` see the missing-texture square spinning - worth letting them find that out and explain it; it is the whole reason for the boot scene. Students who try `update()` see a star that sits still until the bar is full.

5. When files fail

Project: `solutions/ch03_challenge_5_when_files_fail` (from `ch03_loading_screen`)

Goal: listen for failures, pass them to a new scene, and reset state for a scene that runs again.

`src/scenes/PreloadScene.ts`

```
private failed: string[] = [];    // CHALLENGE 5: every file that could not
be loaded
```

```
// CHALLENGE 5: the scene object is reused if the player tries again - start
with an empty list
init(): void {
    this.failed = [];
}
```

```
// CHALLENGE 5: "loaderror": one file could not be loaded - remember which,
and from where
this.load.on("loaderror", (file: Phaser.Loader.File) => {
    this.failed.push(`${file.key} - ${file.src}`);
});
```

```
// CHALLENGE 5: if anything failed, the error screen - it can still go on to
the menu
if (this.failed.length > 0) {
    const errorData: ErrorData = { failed: this.failed, menu: data };
    this.scene.start(ERROR_SCENE, errorData);
} else {
    this.scene.start(MENU_SCENE, data);
}
```

`src/scenes/ErrorScene.ts` (new)

```
export interface ErrorData {
  failed: string[];    // "key - url" for each file that failed
  menu: MenuData;     // what to pass on to the menu, if the player
  carries on
}
```

```
this.input.keyboard!.once("keydown-R", () => {
  this.scene.start(BOOT_SCENE);
});
this.input.keyboard!.once("keydown-SPACE", () => {
  this.scene.start(MENU_SCENE, this.error.menu);
});
```

To test it, the solution asks for `strr.png` (a misspelling) when `TEST_A_MISSING_FILE` is `true`. Tested with the harness: the error screen lists `broken - assets/images/strr.png`; R runs the loading screen again and comes back to the error screen with **0** files loaded (everything that worked is already in its cache, so only the broken file is tried again); SPACE reaches the menu.

Look for:

- `init()` resetting the list - without it, each retry adds the same failure again, and the list grows (the Chapter 2 trap, in a new place)
- the decision made in `create()` (or on `"complete"`), after all files have been tried - not in the `"loaderror"` listener, which would leave while other files were still loading
- `file.src` shown, not just the key: the address is what tells you *why* it failed
- the answer to the hint's question: a retry only fetches what failed, because the rest is cached

6. Rooms from a pack

Project: `solutions/ch03_challenge_6_rooms_from_a_pack` (from `ch03_load_on_demand`)

Goal: separate *which files* (data, in a pack) from *what to do with them* (code); discover that a pack is itself a cached file.

public/assets/rooms_pack.json (new) - one section per room:

```
"space": {
  "path": "assets/",
  "files": [
    { "type": "image", "key": "space", "url": "images/space.png" },
    { "type": "image", "key": "player_ship", "url": "images/player_ship.png" },
  ],
  { "type": "image", "key": "enemy_ship", "url": "images/enemy_ship.png" },
  { "type": "image", "key": "rock", "url": "images/rock.png" },
  { "type": "audio", "key": "shoot", "url": "audio/shoot.wav" }
]
```

src/rooms.ts

```
export interface Room {
  name: string;
  section: string;           // CHALLENGE 6: this room's section in
rooms_pack.json
  background: string;       // CHALLENGE 6: keys, not files - the pack says
which file each key is
  props: RoomProp[];
  sound: string;            // played as you walk in
}
```

```
// CHALLENGE 6: the key the pack file itself is kept under, once loaded for
this room
export function packKey(room: Room): string {
  return `pack_${room.section}`;
}
```

src/scenes/MenuScene.ts

```
private queueRoom(room: Room): void {
  this.load.pack(packKey(room), ROOMS_PACK_FILE, room.section);
}
```



```
// CHALLENGE 6: the pack file itself is kept in the JSON cache, under its  
key. Left there,  
// the loader would skip it next time - and so load none of the room's files  
this.cache.json.remove(packKey(room));
```

`isLoading()` now checks the background, every prop's key, and the sound key - the keys the room actually uses.

The trap, and the reason for the last hint: a loaded pack is stored in `this.cache.json` under the key it was loaded with. After U removes the pictures and sounds, loading the room again with the same pack key is **skipped** - the key is in the cache - so no files are queued, the loader completes at once, and the room starts with nothing loaded. Tested with the harness: with the `this.cache.json.remove` line taken out, going back into an unloaded room draws every picture as the missing texture and stops with `Error: Audio key "shoot" not found in cache`. With it, U and going back in works every time.

Look for:

- a different key per room for `load.pack` (one key for all four means only the first room ever loads - the others' packs are skipped)
- the pack's `"player"` entry in two sections - the loader skips it the second time, as before
- `rooms.ts` no longer containing any file names
- an alternative that is just as good: load `rooms_pack.json` once with `this.load.json`, and give `this.load.pack` one section of it as an object instead of a file name -
`this.load.pack({ key: packKey(room), url: { [room.section]:
packData[room.section] } })`

Chapter 3 - Preloading - Slides

Chapter 3

Preloading

Queues, caches, loading screens - and files that go missing



1 / 20

Today

- what `this.load.image(...)` really does
- keys, and the game's **caches**
- **Boot -> Preload -> Menu**, with a progress bar
- every kind of file: image, sprite sheet, audio, JSON, text, pack
- when a file is missing
- loading in the middle of a game

2 / 20

preload() only makes a list

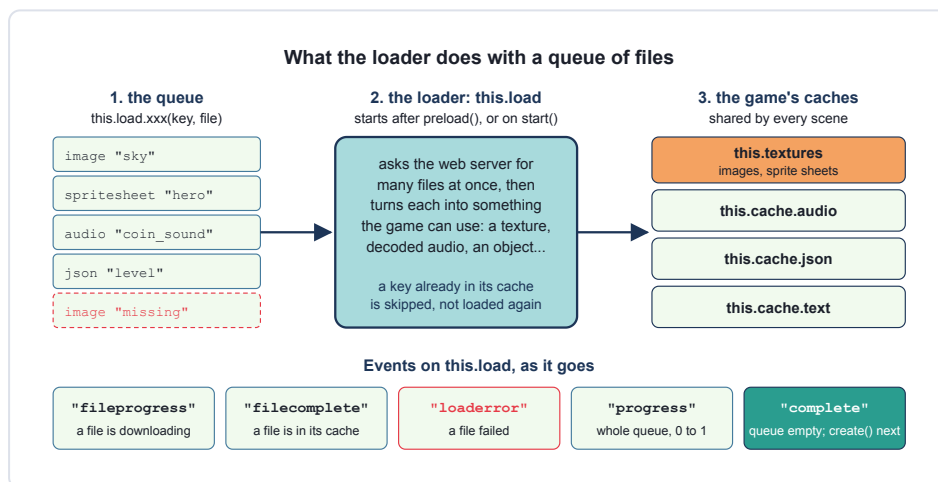
```
// preload() - queued, NOT loaded
this.load.image(CRATE_KEY, CRATE_FILE);

// create() (in showImages) - loaded by now
this.add.image(x + 70, y + 120, CRATE_KEY).setScale(2);
```

1. `preload()` queues files
2. Phaser starts the loader when `preload()` returns
3. `create()` runs when the queue is empty

3 / 20

Queue -> loader -> caches



4 / 20

Keys and caches

Loaded with	Kept in
image , spritesheet	this.textures
audio	this.cache.audio
json	this.cache.json
text	this.cache.text

- the caches belong to the **game** - every scene can use them
- a key is unique **within its kind**
- a key that is already loaded is **skipped**

5 / 20

Paths

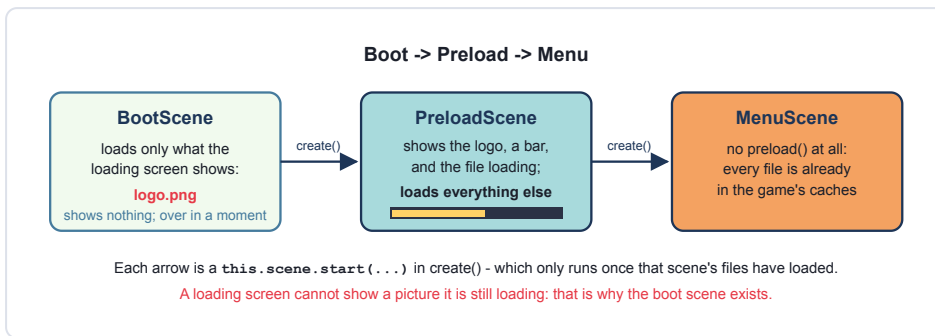
```
this.load.setPath("assets/images/");
this.load.image(SKY_KEY, "sky.png");
this.load.image(SPACE_KEY, "space.png");

this.load.setPath("assets/audio/");
this.load.audio(COIN_SOUND_KEY, "coin.wav");
```

- address = **base URL + path + file name**
- relative to the **page**: public/ is copied to dist/
- no leading / ; capitals matter on a web server

6 / 20

Boot -> Preload -> Menu



A loading screen cannot show a picture it is still loading.

7 / 20

The boot scene

```
preload(): void {  
    this.load.image(LOGO_KEY, LOGO_FILE);  
}  
  
// create() only runs once everything queued in preload() has loaded  
create(): void {  
    this.scene.start(PRELOAD_SCENE);  
}
```

Small, fast, shows nothing.

8 / 20

Drawing the bar: Graphics

```
const box = this.add.graphics();
box.fillStyle(BOX_COLOUR);
box.fillRect(barX - BOX_PADDING, BAR_Y - BOX_PADDING,
  BAR_WIDTH + BOX_PADDING * 2, BAR_HEIGHT + BOX_PADDING * 2);

const bar = this.add.graphics();
```

- `fillStyle(0x2b3245)` - colours as hex numbers
- `fillRect(x, y, width, height)` - from the **top-left**
- `clear()` - rub it all out

9 / 20

Listening to the loader

```
this.load.on("progress", (value: number) => {
  bar.clear();
  bar.fillStyle(BAR_COLOUR);
  bar.fillRect(barX, BAR_Y, BAR_WIDTH * value, BAR_HEIGHT);
});

this.load.on("fileprogress", (file: Phaser.Loader.File) => {
  fileText.setText(`Loading ${file.src}`);
});

this.load.on("complete", () => {
  fileText.setText("All done!");
});
```

Also: "start", "filecomplete", "loaderror"

10 / 20

Try it: Loading Screen

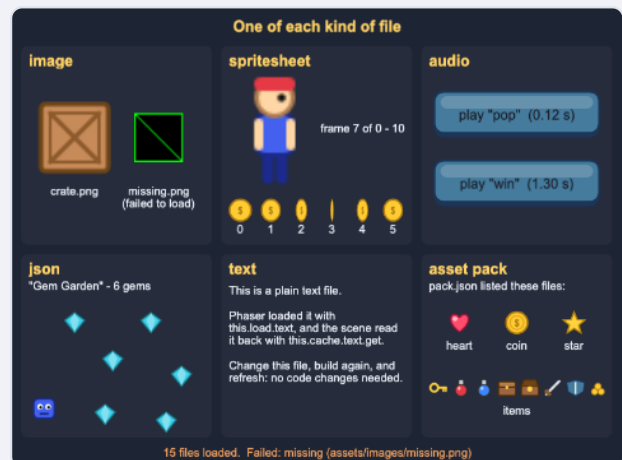
- build and run `ch03_loading_screen` - a flash
- F12 -> **Network** -> **Disable cache**, **Fast 4G** -> refresh
- set `COPIES` to 0 - what changes?
- press **R** in the menu: "Loaded **0** files" - why?



11 / 20

Every kind of file

```
this.load.image(CRATE_KEY, CRATE_FILE);
this.load.spritesheet(HERO_KEY, HERO_FILE, {
  frameWidth: 32, frameHeight: 48 });
this.load.audio(POP_KEY, POP_FILE);
this.load.json(LEVEL_KEY, LEVEL_FILE);
this.load.text(NOTES_KEY, NOTES_FILE);
this.load.pack(PACK_KEY, PACK_FILE,
  PACK_SECTION);
```



12 / 20

Sprite sheets

- one picture, cut into equal **frames**, numbered from 0
- `coin_spin.png` is 192 x 32 -> six 32 x 32 frames
- the frame is the fourth argument:

```
this.add.image(frameX, y + 215, COIN_SPIN_KEY, frame);
```

- wrong frame size = frames cut in the wrong place
- no frame size at all = `Invalid frameWidth given.`

13 / 20

JSON - and saying what is in it

```
export interface LevelData {  
  name: string;  
  player: Point;  
  gems: Point[];  
}  
  
const level: LevelData = this.cache.json.get(LEVEL_KEY);  
level.gems.forEach((gem) => {  
  this.add.image(originX + gem.x, originY + gem.y, GEM_KEY);  
});
```

`get` returns `any`. The interface checks the **code**, never the **file**.

14 / 20

Asset packs

```
{
  "pickups": {
    "path": "assets/",
    "files": [
      { "type": "image", "key": "heart", "url": "images/heart.png" },
      { "type": "image", "key": "coin", "url": "images/coin.png" },
      ...
    ]
  }
}
```

- the list of files moves **out of the code**
- sections; each can set its own `path`
- the pack is a file too - with a key, in the JSON cache

15 / 20

When a file is missing

```
this.load.on("loaderror", (file: Phaser.Loader.File) => {
  this.failed.push(`${file.key} (${file.src})`);
});
```

Missing	Result
picture	the missing texture - black, green outline
sound	Audio key "pop" not found in cache
JSON / text	undefined

First stop: the console's red **404** line.

16 / 20

Loading later

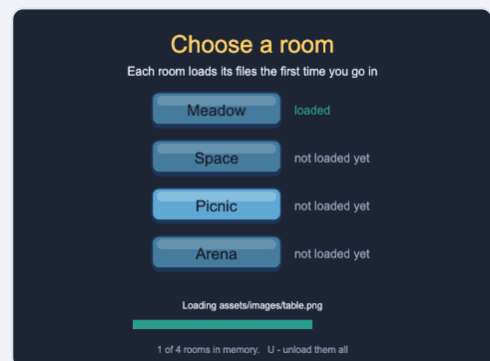
```
this.queueRoom(room);
this.load.once("complete", () => {
  this.enterRoom(index);
});
this.load.start();
```

- outside `preload()` , **you** start the loader
- check first: `this.textures.exists(key)` , `this.cache.audio.exists(key)`
- ignore clicks while `this.load.isLoading()`
- let go: `this.textures.remove(key)` , `this.cache.audio.remove(key)`

17 / 20

Try it: Load on Demand

- throttle the network, choose a room: watch it load
- ESC, choose it again: instant
- Meadow, then Arena: one file fewer - why?
- U, then go back in



18 / 20

Summary

- `load... queues`; `create()` runs when the queue is empty
- files are kept **by key**, in game-wide caches; loaded keys are skipped
- Boot -> Preload -> Menu; a `Graphics` bar moved by `"progress"`
- image, spritesheet, audio, json (+ interface), text, pack
- missing: `"loaderror"`, the missing texture, the console
- later: queue, `once("complete")`, `this.load.start()`

19 / 20

Challenges

1. **Restyle the bar** - rounded, outlined, red -> orange -> green
2. **Percentage** - whole numbers, never 100% too soon
3. **Hearts in the level** - JSON and interface together
4. **Keep it moving** - a spinner that turns while loading
5. **When files fail** - an error screen, try again or carry on
6. **Rooms from a pack** - every room's files in one asset pack

Next: **Chapter 4 - The life of a scene**

20 / 20



The life of a scene

Overview

Chapters 1-3 used scenes as boxes to put code in. This chapter opens the box: the exact order of a scene's methods and events, what happens in a frame, and the scene manager's other powers - `launch` , `pause` / `resume` , `sleep` / `wake` , `stop` , `remove` . The Phaser ideas are the **lifecycle**, **scene events**, **parallel scenes** (a HUD and a pause menu) and **cleaning up listeners**; the OO ideas are **overriding a method and calling** `super` (a `Sprite` 's `preUpdate`) and the **context** argument of a listener (JavaScript's `this` compared with Java's method references).

The chapter has one big practical lesson that students will need in every later chapter: **Phaser clears up a scene's game objects, timers, tweens and input listeners when it shuts down, but not listeners on `this.events` or on longer-lived emitters.** Leaks do not crash at once; they make things happen twice, three times... and then crash. The logger's leak mode makes this visible.

Prerequisites

- Chapter 1: `preUpdate` on an `Image` , `delta` , `override`
- Chapter 2: `scene.start(key, data)` , `init(data)` , scenes are reused, `on` and `once` , the registry
- Chapter 3: the loader and `preload()` (only lightly needed here)
- Java: overriding and `super.method()` ; listeners and method references (`this::onClick`)

Learning outcomes

Students can:

1. state the order in which Phaser calls a scene's constructor, `init` , `preload` , `create` and `update` , which of them run once and which on every start, and where game objects'

- `preUpdate` comes in a frame
- 2. listen for scene events (`CREATE` , `PAUSE` , `RESUME` , `SLEEP` , `WAKE` , `SHUTDOWN` , `DESTROY` , `UPDATE` , ...) with a method and a context, and remove the listener again
- 3. override `preUpdate` in a `Sprite` subclass correctly (`protected override` , calling `super`)
- 4. run scenes side by side with `launch` , and choose between `pause` , `sleep` and `stop`
- 5. explain which listeners Phaser removes when a scene shuts down, and clean up the rest in `SHUTDOWN`
- 6. measure time in a pausable game with timers or `delta` , not `time / this.time.now`
- 7. build a HUD scene and a pause menu scene around a game scene

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Recap Chapter 2's trap (fields keep old values). "Today: exactly what Phaser calls, and when"
0:05 - 0:20	Slides 3-5: the lifecycle diagram, then live : run the logger, read the log from the top. Restart with 5 and ask "what is missing?" (the constructor)
0:20 - 0:30	Slides 6-7: one frame in order; scene events; the context argument (Java <code>this::onClick</code> comparison)
0:30 - 0:35	Slide 8: <code>Sprite.preUpdate</code> - live : press S in the logger, then delete <code>super.preUpdate</code> in <code>Spinner.ts</code> and rebuild
0:35 - 0:45	Slides 9-10: the scene manager table; pause vs sleep (press 1, 2, 3, 4); requests are queued
0:45 - 0:55	Slides 11-12: cleaning up. Live : L then 5, 5 - watch every line of the log appear three times. Ask for the fix before showing <code>onShutdown()</code>
0:55 - 1:00	Slide 13: <code>time</code> vs <code>delta</code> - the question "what does <code>time</code> do while paused?"

Lab (2 hours)

Time	Activity
0:00 - 0:20	Everyone runs the logger and fills in a table: for pause, sleep, stop and remove - is it updated? drawn? do its game objects survive? what events? (Answers: the chapter's table)
0:20 - 0:35	Challenge 1
0:35 - 0:55	Play <code>ch04_pause_and_hud</code> ; slides 14-18. Read <code>GameScene</code> , <code>HudScene</code> , <code>PauseScene</code> together; find the three places where something is cleaned up in <code>SHUTDOWN</code> or by <code>stop</code>
0:55 - 1:40	Challenges 2-4
1:40 - 2:00	Challenges 5-6 for the fast; the rest finish 3-4. Collect the "which camera fades?" answers from challenge 5

Key points to stress

- **The constructor runs once.** Everything that must happen on every start goes in `init()` or `create()` . Everything that must happen once for the whole game (animations, registry defaults) must survive `create()` running again - check with `exists` , or do it in a scene that only runs once
- **Scene events are on `this.events` , and that emitter is reused.** This is the surprise: students assume shutting a scene down clears everything. It clears game objects, timers, tweens and input listeners - not listeners on `this.events` , `this.game.events` , `this.registry.events` or other scenes' emitters
- **Paused scenes are drawn; sleeping scenes are not; neither is updated nor hears input.** So the scene that reads the "unpause" key must be a different, running scene
- `start` **stops the caller;** `launch` **does not.** The most common HUD bug is `this.scene.start(HUD)` from the game scene
- `restart()` **has no key.** It restarts its own scene only
- **Scene manager calls are queued** (except `add` and `remove`). Code after `this.scene.start(...)` still runs, in the old scene
- `time` **is the game's clock and keeps going during a pause.** Timers and `delta` are the scene's, and stop. `this.time.now` is only brought up to date while the scene steps

Common problems and errors

What students see	Cause	Fix
This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	preUpdate in a Sprite subclass without override	add override (a Chapter 1 Image did not need it - a Sprite has its own preUpdate
Class 'Spinner' incorrectly extends base class 'Sprite'. / Property 'preUpdate' is private in type 'Spinner' but not in type 'Sprite'.	private override preUpdate	protected (or public) - a subclass cannot make a method less visible
A sprite moves but its animation is stuck on one frame; no error	super.preUpdate(time, delta) missing	call it, first thing in preUpdate
Console: AnimationManager key already exists: spin after a restart	this.anims.create(...) in create() , run again on restart	if (!this.anims.exists(key)) , or create animations in a scene that runs once
After each restart, events happen twice, three times... (sounds doubled, score counted twice)	listeners added in create() to this.events , this.game.events , this.registry.events or another scene's events , never removed	remove them in a SHUTDOWN handler: off(event, method, this)
off(...) does not remove the listener	on and off were each given a new arrow function	pass the same method and context to both, or keep the function in a variable

What students see	Cause	Fix
The pause menu opens but the game can never be unpaused	the unpause key is read by the paused scene, which hears no input	read it in the pause scene (which is running)
The game disappears when the HUD or pause menu appears	<code>this.scene.start(HUD_SCENE)</code> - <code>start</code> stops the calling scene	<code>this.scene.launch(...)</code>
Argument of type 'string' is not assignable to parameter of type 'object'.	<code>this.scene.restart(GAME_SCENE)</code> - <code>restart</code> takes data, not a key	<code>this.scene.start(GAME_SCENE)</code> from the other scene, or <code>restart()</code> inside the scene itse
Property 'score' does not exist on type 'Scene'.	<code>this.scene.get(GAME_SCENE).score</code> - <code>get</code> returns a plain <code>Scene</code> type	better: send the value as event data or registry value. If it really must be read, <code>this.scene.get(GAME_SCENE)</code> as <code>GameScene</code> and a public gette
TypeError: Cannot read properties of null (reading 'queue') in the console when removing a scene from a key press	<code>this.scene.remove(...)</code> runs immediately, in the middle of Phaser dispatching the key to every scene	defer it: <code>this.time.delayedCall(0, () => this.scene.remove(key))</code>
The HUD shows nothing until the first change	it listened for events the game emitted in its <code>create()</code> , before the HUD had started (launch is queued)	pass the starting values as <code>launch</code> data
The invulnerable time / power-up ends early if the game was paused	measured with <code>this.time.now</code> or <code>time</code>	use <code>this.time.delayedCall</code> , or add up <code>delta</code>

Discussion questions

- `DemoScene` 's constructor cannot use `this.log()` , and `EventLog` is a plain class that any file can import. What is available to a scene in its constructor, and why? (Only what `super(...)` sets up; Phaser gives it its systems when the game boots it.)
- The HUD listens to `GameScene` 's events rather than `GameScene` calling HUD methods. What would break, or become harder, if `GameScene` called `hud.showScore(...)` directly? (The HUD might not have started yet; `GameScene` would depend on the HUD existing; a second HUD is harder.)
- When should a scene be paused, slept, or stopped? Give a game example of each
- Why does the leak in leak mode make the log *triple* after two restarts, not double? Why do the leaked `UPDATE` listeners appear *before* `Spinner.preUpdate()` in the log? (Phaser's own update list re-registers its listener on each start, after the leftovers.)
- In challenge 5, why can the pause scene's camera fade the whole screen, but the game scene's cannot?

Extension ideas

- Add a `TRANSITION` between two scenes with `this.scene.transition({ target, duration, moveAbove })` and log its events (`transitionout` , `transitionstart` , `transitioncomplete`)
- Make the logger's frame limit adjustable while running: `this.game.loop.setFPSLimit(n)` (Phaser 4.2) on the number keys, and watch the coin's steps get bigger
- `this.scene.switch(key)` sleeps the current scene and starts (or wakes) another: build a two-room game where each room keeps its state when you leave it
- Run two copies of the same scene class side by side with `this.scene.add(key, new GameScene(key))` and camera viewports - a split screen

Assessment ideas

- Written: give a sequence of calls (`launch` , `pause` , `sleep` , `wake` , `stop` , `start`) and ask for the log a lifecycle logger would print, including which lifecycle methods run again

- Code reading: a scene with `this.game.events.on("blur", () => this.pauseGame())` in `create()` and no clean-up. Ask what happens after three restarts, how to prove it, and for the fix
 - Practical: "add a game-over scene launched over the paused game, showing the score, with R to play again" - checks `launch` , `pause` , data, and restarting a paused scene
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 4 - The life of a scene - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Count the frames drawn

Project: `solutions/ch04_challenge_1_count_the_frames` (from `ch04_lifecycle_logger`)

Goal: listen for a scene event; see for yourself that a paused scene is drawn and a sleeping one is not; and notice which scene can report on which.

`src/scenes/DemoScene.ts`

```
private onRender(): void {  
    // CHALLENGE 1: RENDER happens once each time the scene is drawn - even  
    while it is paused  
    this.renderers = this.renderers + 1;  
}
```

```
public getUpdates(): number {  
    return this.updates;  
}  
  
public getRenderers(): number {  
    return this.renderers;  
}
```

`src/scenes/LogScene.ts`

```
// CHALLENGE 1: ask DemoScene for its counts  
counts = `      updates: ${demo.getUpdates()}    renders:  
${demo.getRenderers()}`;
```

plus `this.renderers = 0;` in `init()`, so a restart starts the count again.

Why not in `DemoScene` 's own text? Its text is changed in `update()` - and `update()` is exactly what stops when the scene is paused. The count would freeze on screen even though it is still going up. `LogScene` is always running, so it can show it.

Tested with the harness: after 1 second paused, updates stayed at 32 while renders went from 32 to 76; after putting the scene to sleep, both stopped (35 and 89, unchanged a second later).

Look for: the counter reset in `init()` ; getters rather than making the fields public (both work - the getters say "read only"); a correct answer to "which numbers stop" - paused: updates; asleep: both. Students who put the count in `update()` (`renders++` there) have counted updates twice, not renders - a good discussion point.

2. Thirty frames a second

Project: `solutions/ch04_challenge_2_thirty_fps` (from `ch04_pause_and_hud`)

Goal: the `fps` config, and understanding what `actualFps` does and does not measure.

`src/main.ts`

```
fps: {  
  target: 60,  
  limit: 30,  
},
```

`src/scenes/HudScene.ts`

```

override update(_time: number, delta: number): void {
    this.framesCounted = this.framesCounted + 1;
    this.timeCounted = this.timeCounted + delta;

    if (this.timeCounted >= 1000) {
        const fps = this.framesCounted * 1000 / this.timeCounted;
        this.fpsText.setText(`fps: ${fps.toFixed(1)} (browser:
        ${this.game.loop.actualFps.toFixed(1)})`);
        this.framesCounted = 0;
        this.timeCounted = 0;
    }
}

```

With a limit, Phaser still receives every browser frame; it just skips calling the game step until enough time has built up. `actualFps` counts every browser frame, so it stays near 60. The HUD's `update()` runs once per real game step, so counting its calls gives the true rate. The HUD is a good home for it because it keeps running while the game is paused.

Tested with the harness (the headless browser offers about 48 frames a second): the HUD showed `fps: 26.5 (browser: 48.6)`. On a 60 Hz screen it shows 30. Holding the right arrow for a second moved the player about the same distance with and without the limit.

Look for: the counter based on `delta`, or on `time` differences - both fine; resetting the counters in `init()`. Many students will first display `actualFps`, see 60, and decide the limit "does not work" - worth letting them find that, then asking what `actualFps` is counting.

3. How long was I away?

Project: `solutions/ch04_challenge_3_time_away` (from `ch04_pause_and_hud`)

Goal: a scene that runs while another is paused; passing data with `resume`; a listener on `this.events` that must be removed.

`src/scenes/PauseScene.ts`

```

// CHALLENGE 3: this scene is running while the game is paused, so its
update() does the counting
override update(_time: number, delta: number): void {
    this.pausedFor = this.pausedFor + delta;
    this.awayText.setText(`Paused for ${this.pausedFor / 1000}.toFixed(1)}
s`);
}

// resume the game, and take this scene away
private carryOn(): void {
    // CHALLENGE 3: resume() can hand data to the scene it resumes - it
    arrives with the RESUME event
    const data: ResumeData = { pausedFor: this.pausedFor };
    this.scene.resume(GAME_SCENE, data);
    this.scene.stop();
}

```

src/scenes/GameScene.ts

```

// CHALLENGE 3: hear about each resume, and the data PauseScene sends with
it
this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);

```

```

// CHALLENGE 3: listeners on this.events outlive a shutdown - without this,
every restart
// would add another, and a pause would be counted twice, three times, ...
this.events.off(Phaser.Scenes.Events.RESUME, this.onResume, this);

```

```

// CHALLENGE 3: the RESUME event's listener is given the scene's Systems,
then the data
private onResume(_sys: Phaser.Scenes.Systems, data: ResumeData): void {
    this.timePaused = this.timePaused + data.pausedFor;
    this.events.emit(PAUSED_CHANGED, this.timePaused / 1000);
}

```

and the HUD listens for `PAUSED_CHANGED` like the other three events (and removes that listener in its `SHUTDOWN` handler too).

The "counted once" requirement is the point of the challenge: the `RESUME` listener is on `GameScene`'s own `this.events`, which Phaser does not clear on shutdown. Tested with

the harness: two restarts, then a pause - the HUD showed one pause's worth of time, and `listenerCount("resume")` was 1.

Look for: `pausedFor` reset in `init()` (the pause scene object is reused for every pause); the `off` in `SHUTDOWN`. Other good answers: the pause scene writes the total into the registry, or emits an event on `GameScene`'s emitter. A tempting wrong answer: read `this.time.now` in `GameScene` on `PAUSE` and on `RESUME`, and subtract. It always gives 0 (checked with the harness) - the scene's clock only updates its `now` when the scene steps, and `RESUME` arrives before the first step. (`this.game.loop.time` would work.) A good puzzle to set the class.

4. Settings

Project: `solutions/ch04_challenge_4_settings_overlay` (from `ch04_pause_and_hud`)

Goal: `sleep` and `wake` between two overlay scenes; `setVisible` on another scene; a setting in the registry that outlives every scene.

`src/scenes/PauseScene.ts`

```
keyboard.on("keydown-S", () => {
  this.scene.launch(SETTINGS_SCENE);
  this.scene.sleep();
});
```

`src/scenes/SettingsScene.ts`

```
keyboard.on("keydown-H", () => {
  const show = this.registry.get(SHOW_HUD) === false;    // it was hidden:
  show it
  this.registry.set(SHOW_HUD, show);
  this.scene.setVisible(show, HUD_SCENE);                // takes effect
  straight away
  this.showSettings();
});
```

```
// wake the pause menu, and take this scene away
private back(): void {
    this.scene.wake(PAUSE_SCENE);
    this.scene.stop();
}
```

src/scenes/HudScene.ts

```
this.scene.setVisible(this.registry.get(SHOW_HUD) !== false);
```

- the pause menu **sleeps**, so it is neither drawn nor listening to keys while the settings are open - and when it wakes, it is exactly as it was (its `once` listeners for P, R and Q are still waiting). Its S listener is `on`, so the settings can be opened again
- the HUD is hidden with `setVisible(false)`, not `slept`: it keeps running and keeps listening, so its numbers are right when it is shown again
- the setting lives in the registry; the HUD reads it in `create()`, so it survives the HUD being stopped and relaunched on a restart

Tested with the harness: S from the pause menu - `PauseScene` sleeping, `SettingsScene` running; H - HUD hidden; ESC - pause menu awake; R - new round, HUD still hidden; score changed while hidden, then H again - HUD visible showing the new score.

Look for: the pause menu slept, not stopped (stopping it would lose its data - the score, the title); the setting surviving a restart. `this.scene.sleep(HUD_SCENE)` for the HUD also works, but a student should be able to say what is different (a sleeping HUD would still receive the events - its listeners are still on `GameScene` - but would not be updated or drawn).

5. Fade in, fade out

Project: `solutions/ch04_challenge_5_fade_in_fade_out` (from `ch04_pause_and_hud`)

Goal: camera fades and their completion event; and the subtle part - a paused scene's camera does not run its effects.

src/scenes/MenuScene.ts


```

this.input.keyboard!.once("keydown-SPACE", () => {
  this.cameras.main.fadeOut(FADE_TIME);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE, ()
=> {
    this.scene.start(GAME_SCENE);
  });
});

```

src/scenes/PauseScene.ts

```

// CHALLENGE 5: fade to black, then do `next`.
//
// It is THIS scene's camera that fades. GameScene is paused, and a paused
// scene's cameras are
// not updated either - a fade started on its camera would never finish. A
// camera's fade covers
// everything under it, so fading the top scene fades the whole screen.
private fadeOutThen(next: () => void): void {
  if (this.leaving) {
    return;
  }
  this.leaving = true;
  this.cameras.main.fadeOut(FADE_TIME);
  this.cameras.main.once(Phaser.Cameras.Scene2D.Events.FADE_OUT_COMPLETE,
next);
}

```

plus `this.cameras.main.fadeIn(FADE_TIME)` at the start of `create()` in `MenuScene`, `GameScene` and `HudScene` (the HUD has its own camera, so it needs its own fade).

Tested with the harness: a fade started on the paused `GameScene`'s camera stayed at `fadeEffect.progress 0` for a full second; fading `PauseScene`'s camera darkened the whole screen, game and HUD included, and the restart happened when it finished. Pressing P and R during a Q fade did nothing.

Look for: the scene change inside the `FADE_OUT_COMPLETE` listener, not straight after `fadeOut()` (then there is no fade at all); protection against a second key during the fade (`once` on the menu; the `leaving` flag in the pause menu, which also stops P "carrying on" into a scene that is about to restart). Students who try to fade `GameScene`'s camera from the pause menu will find it never finishes - excellent, if they can explain why.

6. Auto-pause, without a leak

Project: `solutions/ch04_challenge_6_auto_pause` (from `ch04_pause_and_hud`)

Goal: listeners on the game's own emitter, and cleaning them up - proved, not assumed.

`src/scenes/GameScene.ts`

```
this.game.events.on(Phaser.Core.Events.BLUR, this.autoPause, this);
this.game.events.on(Phaser.Core.Events.HIDDEN, this.autoPause, this);
```

```
this.game.events.off(Phaser.Core.Events.BLUR, this.autoPause, this);
this.game.events.off(Phaser.Core.Events.HIDDEN, this.autoPause, this);
```

(in the `SHUTDOWN` handler, next to stopping the HUD)

```
private autoPause(): void {
    if (this.scene.isActive()) {
        this.pauseGame();
    }
}
```

`src/scenes/HudScene.ts`

```
override update(): void {
    const blur = this.game.events.listenerCount(Phaser.Core.Events.BLUR);
    const hidden = this.game.events.listenerCount(Phaser.Core.Events.HIDDEN);
    this.listenersText.setText(`game listeners - BLUR: ${blur}  HIDDEN:
    ${hidden}`);
}
```

`isActive()` is `false` while the scene is paused, so the pause menu is never opened twice, and nothing happens on the game-over screen.

The counts are not 1: Phaser itself listens on the game's emitter (and some of its listeners belong to whichever scenes are running), so the number to check is that it is **the same** in the same situation, round after round. Tested with the harness: during play HIDDEN stayed at 2 and BLUR at 4 through three restarts (BLUR is 5 in the very first round - one of Phaser's

own listeners goes away after the first start, which is why "does not grow" is the test, not "stays exactly the same"). In a copy with the `off` lines removed, three restarts took HIDDEN from 2 to 5 and BLUR from 5 to 7. `game.events.emit("blur")` (from the harness) opened the pause menu during play, did nothing when already paused, and did nothing on the menu after quitting.

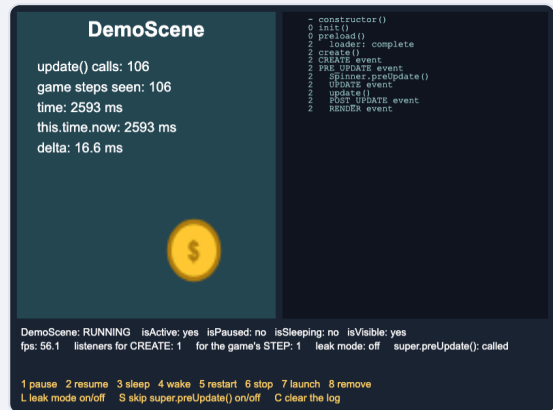
Look for: the `off` calls with the same method and context as the `on` calls (an arrow function passed to both `on` and `off` removes nothing); the "only while playing" check; a test plan. `once` is a plausible wrong answer: it would pause the first time only.

Chapter 4 - The life of a scene - Slides

Chapter 4

The life of a scene

What Phaser calls, and when - and the scene manager's other powers



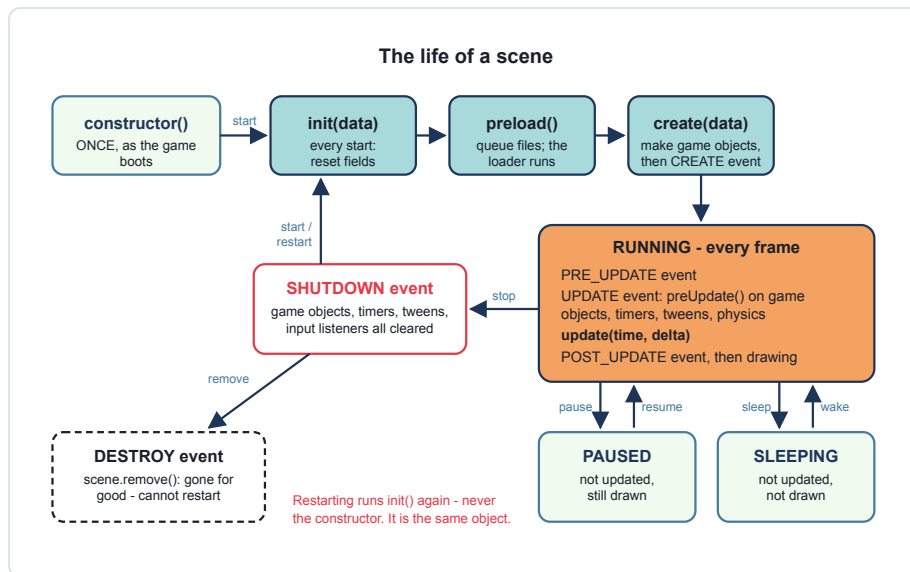
1 / 20

Today

- the order: `constructor` , `init` , `preload` , `create` , `update`
- one frame, step by step; scene **events**
- a `Sprite` 's own `preUpdate` - and `super`
- `launch` , `pause / resume` , `sleep / wake` , `stop` , `remove`
- cleaning up: the listeners Phaser does **not** remove
- `time` vs `delta` ; the frame rate
- a HUD and a pause menu, as scenes

2 / 20

The life of a scene



3 / 20

Once, or every start?

```

constructor() {
    super(DEMO_SCENE);
    eventLog.add("constructor()");
}

// Runs every time the scene starts: the place to reset fields
init(): void {
    this.log("init()");
    this.updates = 0;
    ...
}
    
```

- constructor: **once**, when the game boots
- `init()`, `preload()`, `create()` : **every** start - then the `CREATE` event
- restart: `init()` again, never the constructor. Same object

4 / 20

Try it: Lifecycle Logger

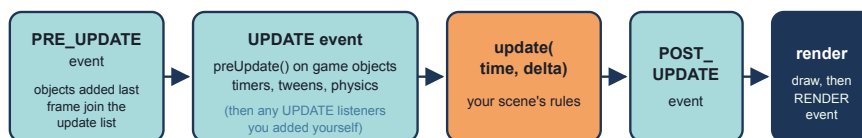
- build, serve, open `ch04_lifecycle_logger`
- read the log from the top: what happens in frame 0? frame 2?
- press **5** (restart): what is missing from the log this time?
- press **S**: what does `super.preUpdate()` do?



5 / 20

One frame, in order

One frame, for one running scene



With several scenes running

every scene is UPDATED first - the last scene in the list first, the first scene last - then every scene is DRAWN - the first scene in the list first, so later scenes are drawn on top.

A paused scene is skipped in the update pass but still drawn. A sleeping scene is skipped in both.

`update()` comes after every game object has moved itself.

6 / 20

Listening for scene events

```
this.events.on(Phaser.Scenes.Events.PAUSE, this.onPause, this);
this.events.on(Phaser.Scenes.Events.RESUME, this.onResume, this);
...
this.game.events.on(Phaser.Core.Events.STEP, this.onGameStep, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, this.onShutdown, this);
```

- `on(event, method, context)` - the **context** is the object to call it on
- Java: `this::onPause` remembers its object. JavaScript's `this.onPause` does not
- the same three arguments take it off again: `off(event, method, context)`

7 / 20

A Sprite's own `preUpdate`

Coin.ts

```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);
    ...
    this.age = this.age + delta;
    ...
}
```

- `Image` has no `preUpdate` - `Sprite` does: it moves the animation on
- `override` and `protected` - the compiler checks
- `super.preUpdate(...)` - **nobody** checks. Leave it out: moves, never animates

8 / 20

The scene manager

	Updated?	Drawn?	Game objects
<code>launch(key)</code> - alongside this one	yes	yes	new
<code>pause(key)</code> / <code>resume(key)</code>	no	yes	kept
<code>sleep(key)</code> / <code>wake(key)</code>	no	no	kept
<code>stop(key)</code>	no	no	destroyed
<code>remove(key)</code>	-	-	gone for good

`start` stops **the caller**. `restart()` has no key. Requests are **queued**: they happen as the scene manager's next update begins.

9 / 20

Paused or asleep?

- **paused**: frozen, still on screen
- **asleep**: not drawn either
- both keep everything
- neither hears the keyboard - so the keys live in **another** scene

```

- constructor()
0  init()
0  preload()
2  loader: complete
2  create()
2  CREATE event
2  PRE UPDATE event
2  Spinner.preUpdate()
2  UPDATE event
2  update()
2  POST UPDATE event
2  RENDER event
112 >> scene.pause(DEMO_SCENE)
112 PAUSE event
148 >> scene.resume(DEMO_SCENE)
148 RESUME event
148 PRE UPDATE event
148 Spinner.preUpdate()
148 UPDATE event
148 update()
148 POST UPDATE event
148 RENDER event
174 >> scene.sleep(DEMO_SCENE)
174 SLEEP event

```

DemoScene: SLEEPING isActive: no isPaused: no isSleeping: yes isVisible: no
 fps: 53.1 listeners for CREATE: 1 for the game's STEP: 1 leak mode: off super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
 L leak mode on/off S skip super.preUpdate() on/off C clear the log

10 / 20

Cleaning up

When a scene shuts down, Phaser clears:

- its game objects, timers, tweens, input listeners

It does **not** clear listeners on:

- the scene's own `this.events` (the object - and emitter - is reused)
- `this.game.events` , `this.registry.events` , another scene's `events`

Whatever you add to an emitter that outlives your scene, take off in `SHUTDOWN` .

11 / 20

A leak you can see

Leak mode (L), then restart twice (5, 5):

- every event logged **three** times
- "game steps seen" climbs 3x too fast
- 3 listeners for `CREATE` , 3 for `STEP`

In a game: doubled scores, tripled sounds... then a crash.

DemoScene

update() calls: 68
game steps seen: 201
time: 7776 ms
this.time.now: 7776 ms
delta: 20.0 ms



```
244 POST_UPDATE event
244 POST_UPDATE event
244 RENDER event
244 >> demo.scene.restart()
263 SHUTDOWN event
263 leak mode: listeners left behind!
263 init()
263 preload()
263 loader: complete
263 create()
263 CREATE event
263 CREATE event
263 CREATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 PRE_UPDATE event
263 UPDATE event
263 Spinner.preUpdate()
263 UPDATE event
263 update()
263 POST_UPDATE event
263 POST_UPDATE event
263 POST_UPDATE event
263 RENDER event
```

DemoScene: RUNNING isActive: yes isPaused: no isSleeping: no isVisible: yes
fps: 45.8 listeners for CREATE: 3 for the game's STEP: 3 leak mode: ON super.preUpdate(): called

1 pause 2 resume 3 sleep 4 wake 5 restart 6 stop 7 launch 8 remove
L leak mode on/off S skip super.preUpdate() on/off C clear the log

12 / 20

Time and delta

- `time` / `this.time.now` - the **game's** clock. Keeps going while a scene is paused
- `delta` - ms since the last frame. Only arrives while the scene **runs**
- `timers` (`this.time.delayedCall`) belong to the scene: they wait during a pause

```
fps: {  
  target: 60,  
  limit: 0,  
},
```

`limit: 10` - the coin moves just as fast, in bigger steps

13 / 20

Try it: Pause and HUD

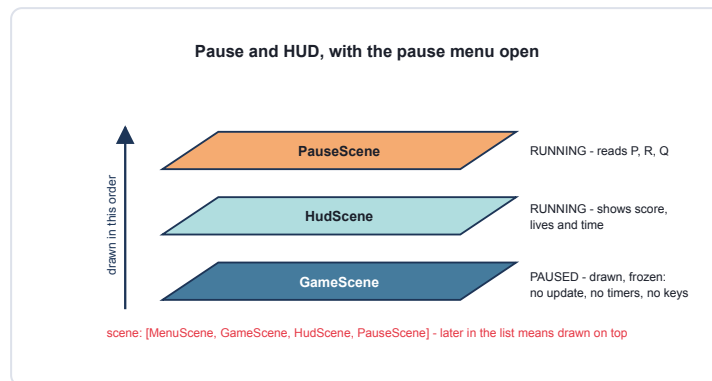
```
scene: [MenuScene, GameScene, HudScene,  
PauseScene],
```

- arrows, collect coins, avoid triangles
- P pauses; R, Q from the menu
- which scenes are running when paused?



14 / 20

Scenes side by side



```
const hudData: HudData = { score: this.score, lives: this.lives, seconds: this.secondsShown };
this.scene.launch(HUD_SCENE, hudData);
```

15 / 20

The HUD listens

```
this.events.emit(SCORE_CHANGED, this.score);
```

```
const gameScene = this.scene.get(GAME_SCENE);
gameScene.events.on(SCORE_CHANGED, this.showScore, this);
...
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  gameScene.events.off(SCORE_CHANGED, this.showScore, this);
  ...
});
```

GameScene emits; HudScene listens. GameScene 's emitter outlives the HUD - so the HUD cleans up.

16 / 20

Pausing

```
private pauseGame(): void {  
    const data: PauseData = { gameOver: false, title: "Paused",  
score: this.score };  
    this.scene.launch(PAUSE_SCENE, data);  
    this.scene.pause();  
}
```

```
private carryOn(): void {  
    this.scene.resume(GAME_SCENE);  
    this.scene.stop();  
}
```

GameScene pauses itself; PauseScene (running) resumes it



17 / 20

Time that survives a pause

```
this.player.setAlpha(0.4);  
this.time.delayedCall(SAFE_TIME, () => {  
    this.safe = false;  
    this.player.setAlpha(1);  
});
```

Not `this.safeUntil = this.time.now + SAFE_TIME - this.time.now` keeps going during a pause.
Measure time with **timers** and `delta` (as `Coin` ages itself).

18 / 20

Summary

- constructor **once**; `init` , `preload` , `create` every start; `update` every frame
- frame: `PRE_UPDATE` , `UPDATE` (objects' `preUpdate`), `update()` , `POST_UPDATE` , `draw`
- `Sprite` subclass: `protected override preUpdate + super.preUpdate(time, delta)`
- `launch` alongside; `pause` = frozen but drawn; `sleep` = hidden too; `stop` ; `remove`
- scenes are drawn in list order; `bringToTop`
- clean up listeners on long-lived emitters in `SHUTDOWN`
- pausable time: timers and `delta`

19 / 20

Challenges

1. **Count the frames drawn** - `RENDER` events vs `update()` calls
2. **Thirty frames a second** - `fps.limit` , and a real frame counter on the HUD
3. **How long was I away?** - time paused, counted once
4. **Settings** - a settings scene; the pause menu sleeps; H hides the HUD
5. **Fade in, fade out** - camera fades between scenes
6. **Auto-pause, without a leak** - `BLUR` / `HIDDEN` , proved leak-free

Next: **Chapter 5 - Audio**

20 / 20



Overview

Students learn to play sound effects and music in Phaser, and to control them. The Phaser ideas are **the sound manager** (`this.sound`), the difference between **fire-and-forget** `play()` and a kept sound object from `add()` , **sound configs** (volume, rate, detune, pan, loop, delay), a sound's **states and events**, **cross-fading with tweens**, and the browser's **autoplay rule**. The one big idea - the one that causes the bugs - is that **the sound manager belongs to the game, not to a scene**: sounds outlive the scene that started them, and so do listeners on the sound manager. It connects directly to Chapter 2 (scenes are reused) and Chapter 4 (clean up listeners at `SHUTDOWN`).

The chapter is fun to teach - students like making noise - but it is also the first chapter where the results cannot be seen, only heard. Headphones for everyone make the lab much calmer, and are needed to hear panning at all.

Prerequisites

- Chapter 2: scenes, `this.scene.start` , `init()` and the reused-scene trap, the registry, object literals, spread and union types
- Chapter 3: loading assets in `preload()`
- Chapter 4: scene events, especially `SHUTDOWN` , and removing listeners on emitters that outlive a scene
- Tweens are used lightly (fading a volume); Chapter 7 covers them properly. One sentence of explanation is enough here

Learning outcomes

Students can:

1. load sounds and play them with `this.sound.play(key, config)` , choosing volume, rate, detune, pan, loop and delay

2. choose between `this.sound.play()` and `this.sound.add()` , and destroy sounds they `add()`
3. control a playing sound - pause, resume, stop, seek, change its settings - and react to its `"complete"` event
4. explain why music carries on across scenes, and use `this.sound.get(key)` to avoid starting a second copy
5. fade and cross-fade music with tweens, and explain why the tweens must belong to the scene that is starting
6. mute or change the volume of the whole game, and explain the browser's autoplay rule and `this.sound.locked`

Suggested session plan (2 hours)

Time	Activity
0:00 - 0:10	Play <code>ch05_sound_board</code> on the projector (speakers on). Press a pad ten times: "what is wrong with that?" (it sounds like a machine). Then press R and do it again
0:10 - 0:30	Slides 3-7: loading, <code>play</code> with a config, rate and detune, the seconds trap, random pitch and pan. Students try every key on the sound board
0:30 - 0:45	Slides 8-10: <code>add</code> versus <code>play</code> , <code>"complete"</code> and <code>destroy</code> , the three states. Walk through <code>SoundPad.play()</code> and <code>playOrPauseMusic()</code> . Challenge 1
0:45 - 1:00	Live demo of the duplicate music bug (below). Slides 11-12: the game-wide sound manager, <code>this.sound.get</code>
1:00 - 1:15	Slides 13-15: <code>Music.play</code> , cross-fades, whose tweens, mute. Students play <code>ch05_music_across_scenes</code> and watch the monitor line while moving between scenes
1:15 - 1:25	Slides 16-18: autoplay, <code>locked</code> , the title scene - and the listener bug it had
1:25 - 2:00	Challenges 2-6

Key points to stress

- `this.sound` **is the same object in every scene**. Draw the game-wide diagram on the board. Sounds keep playing when a scene shuts down; that is a feature for music and a bug for everything else
- `play()` **for effects**, `add()` **for anything you need to control** - and `add()` means `destroy()` when you are done
- a scene's `create()` runs every time the scene starts, so "start the music in `create()` " means "start **another copy** of the music" unless it checks `this.sound.get(key)` first
- **tweens belong to a scene**; music belongs to the game. A fade made by a scene that is shutting down never finishes
- **listeners on the sound manager outlive the scene** - `TitleScene` shows the crash and the fix, exactly the Chapter 4 pattern
- `delay` **and** `seek` **are in seconds**, while nearly everything else in Phaser is milliseconds
- detune is in **cents** (100 = a semitone); rate is a multiplier; both change speed as well as pitch
- sound cannot start until the player interacts with the page: design for it with a "click to start" screen, rather than hoping

Common problems and errors

What students see	Cause	Fix
Music gets louder and louder, and sounds "phased", after going back and forth between menus	the music is added and played in <code>create()</code> every time the scene starts. (With that change to <code>Music.play</code> , <code>menu - credits - menu - credits - menu</code> leaves five <code>music_menu</code> sounds playing)	check <code>this.sound.get(key)</code> first, or keep the music in one place like <code>Music</code>
Game music still playing on the game-over screen	sounds are not stopped when their scene shuts down	stop or fade the music on purpose
Error: Audio key "music_menu" not found in cache (in the browser console), and the game stops	the sound was not loaded under that key: a misspelled key, a wrong path, or a file not copied into <code>public/assets/audio/</code> . (The loader's 404 appears as a red network error just above)	check the key, the path, and that the file is in <code>public/</code>
Property 'setVolume' does not exist on type 'BaseSound'.	the result of <code>this.sound.get(key)</code> is typed as <code>BaseSound</code> , which leaves out volume and the <code>set...</code> methods	keep the object from <code>add()</code> in a field, or tween <code>volume</code> (tweens take any object), or cast with a comment
Expected 1 arguments, but got 0. on <code>this.sound.getAll()</code>	the type definitions make the key compulsory on one of the three sound managers	pass a key, or use <code>this.sound.getAllPlaying()</code>
TypeError: Cannot read properties of null (reading 'drawImage') a frame after changing scene	a listener on the sound manager (e.g. "unlocked") changed a Text belonging to a scene that has shut down	remove the listener in a <code>SHUTDOWN</code> handler
A sound with <code>delay: 500</code> never plays	<code>delay</code> is in seconds - that is eight minutes	<code>delay: 0.5</code>
A fade that stops half-way; music stuck quiet	the tween was made in a scene that then shut down	make fades in the scene that is starting (as <code>Music.play</code> does)

What students see	Cause	Fix
No sound at all in the Celbridge webview or the browser, but no error	the sound is locked until the page is clicked or a key is pressed	click the game first; add a "click to start" screen
Every effect plays at once on the first click	effects were "played" while the sound was locked; Phaser holds them until it unlocks (tested: three effects played a second apart while locked all started on the click)	do not play sound before the first interaction
<code>pan</code> does nothing	the student is not wearing headphones - or the browser (Safari on iOS) has no stereo panner	headphones; never depend on <code>pan</code>
<code>this.sound.volume === 0.6</code> is false straight after setting it to 0.6	Web Audio stores volumes as 32-bit floats: it reads back as <code>0.6000000238418579</code>	never compare volumes with <code>===</code> ; keep your own setting (challenge 2)

Suggested demos

- **The duplicate music bug.** In `ch05_music_across_scenes/src/Music.ts`, change `let music: Phaser.Sound.BaseSound | null = scene.sound.get(key);` to `... = null;`. Build, and go menu - credits - menu - credits - menu. Ask the class what they hear (louder, slightly smeared music). The monitor line still shows one track, because `get` returns the *first* sound with that key - a nice lesson in itself. To count the copies, add `this.scene.sound.getAll(key).length` to the monitor's text: five. Put the line back
- **The listener leak.** In `TitleScene.ts`, remove the `this.events.once(Phaser.Scenes.Events.SHUTDOWN, ...)` block, build, open the page in a **fresh** browser window (so the sound is locked), open the console, and click. The error appears a frame after the menu starts
- **Seconds, not milliseconds.** In the sound board, change `ECHO_DELAY` to `250`. The echo "disappears". Ask why before revealing it
- **Rate vs detune.** On the sound board, press RIGHT ten times (rate 2), play a pad; press Z, then W twelve times (detune 1200), play the same pad. They sound the same: an octave

is a doubling

Discussion questions

- Why does Phaser make the sound manager game-wide, rather than giving each scene its own? (Music across scenes; one mute; one unlock.) What would be harder if it were per-scene?
- `Music` is a class with only static methods. In Java terms, what is it? Is that good OO? What would a `MusicPlayer` object stored in the registry look like instead?
- The mute setting lives in the registry *and* in `this.sound.mute`. Is that duplication? When could the two disagree? (Challenge 3 is exactly that question.)
- Why is it wrong to keep the rhythm of challenge 5 with `this.time.addEvent`?
- What makes a sound effect "feel" right? Try the random pitch with 50 cents and with 600 cents

Extension ideas

- **Audio sprites:** one file holding several sounds, with markers - `sound.addMarker({ name, start, duration })` and `sound.play(name)`. Make a marker for each half of `music_game.wav`
- **Spatial sound:** `this.sound.add(key, { source: { x, y, follow: sprite } })` and `this.sound.setListenerPosition(x, y)` - a sound that gets quieter as the player walks away
- a **settings scene** with sliders (drag a knob with the pointer) for music and effects volume, kept separate: two master volumes, one for each kind of sound
- an **analyser** that draws the music's waveform
(`this.sound.context.createAnalyser()`) - see the Phaser "audio-and-sound" notes; Web Audio only
- add sound to the Chapter 2 game: a click on start, a pop on each hit, music during play, a fanfare on the win screen, with the music stopped on the win screen

Assessment ideas

- Practical: "add a pause scene (Chapter 4) that pauses the game music while it is shown and resumes it when it closes - and plays its own quiet music meanwhile"
- Code reading: show a `create()` that does `this.sound.add("music", { loop: true }).play();` and ask what happens on the third visit to the scene, and how to fix it
- Short answer: the difference between `this.sound.play(key)` and `this.sound.add(key)`, and when to use each
- Short answer: why does `TitleScene` remove its `"unlocked"` listener at `SHUTDOWN`, when it does not remove its `"pointerdown"` listener?

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 5 - Audio - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. More pads

Project: `solutions/ch05_challenge_1_more_pads` (from `ch05_sound_board`)

Goal: see that a list of sounds kept as data means adding a sound is adding a line.

`src/assets.ts`

```
{ key: "win", file: "assets/audio/win.wav" },  
  // CHALLENGE 1: four more pads - a third row. Nothing else needs to know  
  // how many there are:  
  // preload() and makePads() loop over this array  
  { key: "door", file: "assets/audio/door.wav" },  
  { key: "whoosh", file: "assets/audio/whoosh.wav" },  
  { key: "correct", file: "assets/audio/correct.wav" },  
  { key: "wrong", file: "assets/audio/wrong.wav" },  
];
```

`src/scenes/SoundBoardScene.ts`

```
this.pads.forEach((pad, index) => {  
  // CHALLENGE 1: there are only nine number keys - pads 10, 11 and 12 are  
  // click-only  
  if (index < NUMBER_KEYS.length) {  
    keyboard.on(`keydown-${NUMBER_KEYS[index]}`, () => {  
      this.playPad(pad);  
    });  
  }  
});
```

plus the four WAV files copied into `public/assets/audio/`, and the settings text moved down to make room for the third row. Tested with the harness: key 9 plays `door`; clicking pad 12 plays `wrong`.

Look for: the files really copied into `public/` (a missing file shows no error until the pad is pressed: `Audio key "door" not found in cache`); `makePads()` left alone - the layout maths already wraps into a third row. Without the `if`, pads 10-12 would listen for `"keydown-undefined"`, which never fires - harmless, but worth asking students what `NUMBER_KEYS[9]` is.

2. Turn it down

Project: `solutions/ch05_challenge_2_turn_it_down` (from `ch05_music_across_scenes`)

Goal: use the sound manager's master volume, and make something work in every scene - the same pattern as the mute button.

A new game object, made in each scene next to the mute button:

`src/objects/VolumeKey.ts`

```
const LEVELS = [1, 0.6, 0.3];
...
// nothing in the registry yet (undefined) means the first level, 100%
private level(): number {
    return this.scene.registry.get(VOLUME_LEVEL) ?? 0;
}

private nextLevel(): void {
    const level = (this.level() + 1) % LEVELS.length;
    this.scene.registry.set(VOLUME_LEVEL, level);
    this.scene.sound.volume = LEVELS[level];
    this.showLevel();
}
```

`src/scenes/MenuScene.ts` (and `CreditsScene.ts`, `GameScene.ts`)

```
new MuteButton(this, 760, 40);
// CHALLENGE 2: the master volume, next to the mute button (V changes it)
new VolumeKey(this, 725, 40);
```

Why the registry, when `this.sound.volume` is game-wide? Because reading it back does not give exactly what was set: Web Audio stores volumes as 32-bit floats, so after `this.sound.volume = 0.6`, `this.sound.volume` is `0.6000000238418579` (checked with the harness), and `LEVELS.indexOf(...)` returns -1. Keeping the *level number* in the registry avoids comparing floats - a good discussion point.

Look for: `this.sound.volume` (the master volume), not each sound's volume; the level shown in every scene; `% LEVELS.length` to go round. Accept a solution that keeps the level in a `static` field, or that reads `this.sound.volume` and picks the nearest level. A solution that loops over `this.sound.getAllPlaying()` setting each volume is wrong: sounds started later are loud again.

3. Remember the mute

Project: `solutions/ch05_challenge_3_remember_the_mute` (from `ch05_music_across_scenes`)

Goal: save a setting in the browser, and restore it when the game starts.

`src/objects/MuteButton.ts`

```
this.scene.sound.mute = muted;
// CHALLENGE 3: save it in the browser too. localStorage only holds strings:
// "true" or "false"
localStorage.setItem(MUTED_STORAGE, String(muted));
```

`src/scenes/TitleScene.ts`

```
create(): void {
    // CHALLENGE 3: the game starts here, so this is where the saved setting
    // is read back.
    // getItem gives the string that was saved - or null if nothing was, which
    // counts as "not muted"
    const muted = localStorage.getItem(MUTED_STORAGE) === "true";
    this.registry.set(MUTED, muted);
    this.sound.mute = muted;
```

Tested with the harness: mute in the menu, `location.reload()` , and the game starts with `game.sound.mute` true and the menu's button showing `sound_off` .

Look for:

- the setting restored in **both** places - the registry (for the button's picture) and `this.sound.mute` (for the sound). Restoring only the registry gives a button that says "muted" over music that plays
 - comparing with the **string** "true" . `Boolean(localStorage.getItem(...))` is a classic bug: the string "false" is truthy, so the game is always muted once anything is saved
 - a specific storage name: every page from `http://127.0.0.1:8000/` shares one localStorage, so "muted" would be shared by every project students serve on that port
-
-

4. Duck the music

Project: `solutions/ch05_challenge_4_duck_the_music` (from `ch05_music_across_scenes`)

Goal: `add()` plus "complete" to know when an effect ends; tweening a sound's volume from outside the `Music` class without fighting its fades.

A new method in `Music` , so the class that owns the music's volume stays the only one that tweens it:

```
src/Music.ts
```



```

public static fadeTo(scene: Phaser.Scene, key: string, fraction: number,
duration: number): void {
    const music: Phaser.Sound.BaseSound | null = scene.sound.get(key);
    if (music === null || !music.isPlaying) {
        return;
    }

    // stop the fade-in (or an earlier duck) first, so only this tween moves
    the volume
    scene.tweens.killTweensOf(music);
    scene.tweens.add({
        targets: music,
        volume: MUSIC_VOLUME * fraction,
        duration: duration,
    });
}

```

src/scenes/GameScene.ts

```

private playPowerup(): void {
    const powerup = this.sound.add(POWERUP_KEY);
    powerup.once(Phaser.Sound.Events.COMPLETE, () => {
        powerup.destroy();
        // The sound belongs to the game, so it can finish after the player has
        pressed ESC and
        // this scene has shut down. Only bring the music back up if the game is
        still running -
        // otherwise the menu is already fading this music out, with its own
        tweens.
        if (this.scene.isActive()) {
            Music.fadeTo(this, MUSIC_GAME_KEY, 1, DUCK_TIME);
        }
    });

    Music.fadeTo(this, MUSIC_GAME_KEY, DUCK_LEVEL, DUCK_TIME);
    powerup.play();
}

```

Tested with the harness: after ten stars the music's volume is 0.12 (a fifth of 0.6) while `powerup` plays, and 0.6 again after it ends; pressing ESC during the power-up leaves the menu music at 0.6 and the game music paused.

Look for:

- `add()` and a "complete" listener - the hint rules out `this.sound.play()`
 - `destroy()` on the power-up sound
 - `killTweensOf` before the duck: ten quick stars in the first second collide with `Music.play`'s fade-in otherwise, and the fade-in wins
 - the "has the scene gone?" question. Many students will not think of it; ask what happens if ESC is pressed during the power-up. (Without the check, nothing visibly goes wrong in this project - tested with the harness - but the listener adds a tween to a scene that has shut down, which is working by luck, not by design.) Removing the listener in a `SHUTDOWN` handler, as `TitleScene` does, is an equally good answer
-
-

5. On the beat

Project: `solutions/ch05_challenge_5_on_the_beat` (from `ch05_music_across_scenes`)

Goal: drive the game from the music's own clock (`seek`), not from a separate timer.

`src/scenes/GameScene.ts`

```

const BEATS_PER_MINUTE = 128;
const BEAT_LENGTH = 60000 / BEATS_PER_MINUTE;
...
override update(): void {
    const music: Phaser.Sound.BaseSound | null =
this.sound.get(MUSIC_GAME_KEY);
    if (music === null || !music.isPlaying) {
        return;
    }

    // BaseSound's type has no seek, although every sound has one - as in
    MusicMonitor
    const seek = (music as Phaser.Sound.WebAudioSound).seek; // seconds
    const beat = Math.floor(seek * 1000 / BEAT_LENGTH);

    // a new beat (including going from the last beat back to beat 0 when the
    music loops)
    if (beat !== this.lastBeat) {
        this.lastBeat = beat;
        this.pulse();
    }
}

```

`pulse()` is a short `yoyo` tween on the circle's scale. `lastBeat` is reset to -1 in `init()` (the scene is reused - Chapter 2's trap again).

Tested with the harness: after 5.04 seconds of music the circle had pulsed 11 times (beats 0 to 10: $5.04 \text{ s} / 0.46875 \text{ s} = 10.75$). After leaving the game for two seconds and coming back, the beat number still matched `Math.floor(seek * 1000 / 468.75)` - because it is worked out from `seek`, it cannot drift.

Look for:

- the beat worked out from `seek`, not counted. A `this.time.addEvent({ delay: 468.75, loop: true })` pulses happily - until the music pauses, the tab loses focus, or the scene restarts, when it falls out of step. Ask students who did that to test with ESC and back
- seconds versus milliseconds: `seek` is in seconds, `BEAT_LENGTH` in milliseconds
- nothing happens while the music is not playing (for example during the first frame, before `Music.play` has made it)

- `music_game.wav` is exactly 15 seconds, which is exactly 32 beats, so the loop does not break the rhythm. A sharp student will ask what would happen if it were not

6. Jukebox

Project: `solutions/ch05_challenge_6_jukebox` (from `ch05_sound_board`)

Goal: several sound objects, `"complete"` to chain them, and knowing that `stop()` does not fire `"complete"`.

`src/scenes/SoundBoardScene.ts`

```
// CHALLENGE 6: one sound object per track, NOT looping - so each one fires
// "complete" when
// it reaches its end, and that starts the next. ("complete" does not fire
// when a sound is
// stopped with stop(), so skipping with K cannot set off a second skip.)
this.tracks = MUSIC_TRACKS.map((track) => {
  const sound = this.sound.add(track.key, { loop: false, volume:
MUSIC_VOLUME });
  sound.on(Phaser.Sound.Events.COMPLETE, () => {
    this.nextTrack();
  });
  return sound;
});
```

```
private get music(): GameSound {
  return this.tracks[this.trackIndex];
}
...
private nextTrack(): void {
  this.music.stop();
  this.trackIndex = (this.trackIndex + 1) % this.tracks.length;
  this.music.setVolume(this.volume *
MUSIC_VOLUME).setRate(this.rate).setDetune(this.detune);
  this.music.play();
}
```

The **getter** (`get music()`) is the neat trick: every existing line that says `this.music` - play, pause, stop, loop, skip, the progress bar - now means "the current track", with no other change. It is TypeScript's version of a Java `getMusic()` method, used without brackets.

Tested with the harness: K moves `music_menu` to `music_game` ; seeking to 0.3 seconds before the end moves on to `music_action` , then round to `music_menu` ; only one track is ever playing.

Look for:

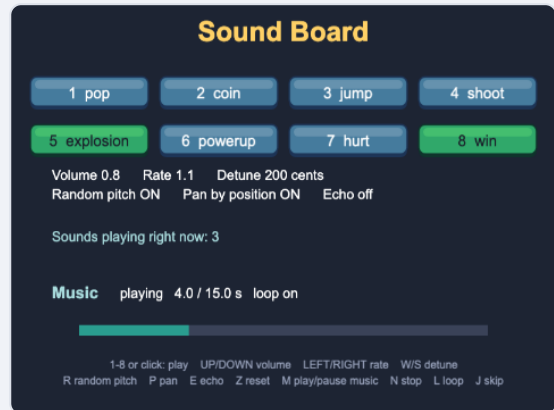
- `on` (not `once`) for `"complete"` - each track completes every time round the playlist
- the settings copied onto the next track (each track is a separate sound object that has not seen the volume, rate or detune keys)
- one track playing at a time: a common bug is starting the next track without stopping the current one on K, so two play at once
- a solution that uses a single sound object and `this.sound.add` again for each track (destroying the old one) is also fine - check it destroys

Chapter 5 - Audio - Slides

Chapter 5

Audio

Sound effects, music, and the rules of the browser



1 / 20

Today

- loading sounds; playing them with a **config**
- `play()` - fire and forget - or `add()` - keep hold
- a sound's states, and its "complete" event
- the sound manager belongs to the **game**
- cross-fading music with tweens; mute everywhere
- the browser's **autoplay** rule

2 / 20

Loading sounds

```
preload(): void {  
  for (const pad of PAD_SOUNDS) {  
    this.load.audio(pad.key, pad.file);  
  }  
  this.load.audio(MUSIC_KEY, MUSIC_FILE);  
}
```

- like pictures: a key and a file; loaded once, for every scene
- an **array** of files lets the browser pick a format it can play: `["theme.ogg", "theme.mp3"]`

3 / 20

Fire and forget

```
this.sound.play(COIN_KEY, {  
  detune: Phaser.Math.Between(-RANDOM_DETUNE, RANDOM_DETUNE),  
  pan: (star.x - centreX) / centreX * MAX_PAN,  
});
```

- makes a sound, plays it, **destroys** it when it ends
- config: `volume` `rate` `detune` `pan` `loop` `delay` `seek` `mute`
- leave a setting out and it keeps its default

4 / 20

Rate, detune - and seconds

- `rate: 2` - twice as fast, an octave higher
- `detune: 1200` - also an octave higher (cents: 100 = a semitone)
- both change speed **and** pitch; Phaser multiplies them

```
const ECHO_DELAY = 0.25; // seconds (SoundConfig delays are in seconds, not milliseconds)
```

`delay` and `seek` are in **seconds**. `delay: 250` waits four minutes.

5 / 20

Less mechanical

The same sound, again and again, sounds like a machine.

- a small **random detune** each time
- **pan** by position: -1 left, 0 middle, 1 right (headphones!)

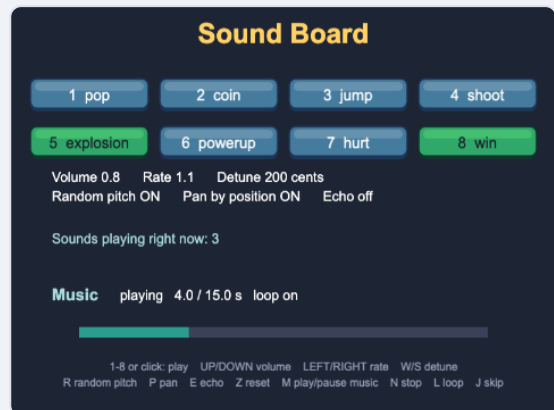
```
pan: this.panByPosition ? this.panFor(pad.x) : 0,
```

Pan is ignored where there is no stereo panner (iPhones, iPads): a finishing touch only.

6 / 20

Try it: Sound Board

- 1-8 play; arrows change volume and rate; W/S detune
- R random pitch, P pan, E echo
- M / N / L / J: the music - play and pause, stop, loop, skip
- find where each setting goes into a `SoundConfig`



7 / 20

Keep hold: `add`

```
const sound = this.scene.sound.add(this.soundKey);

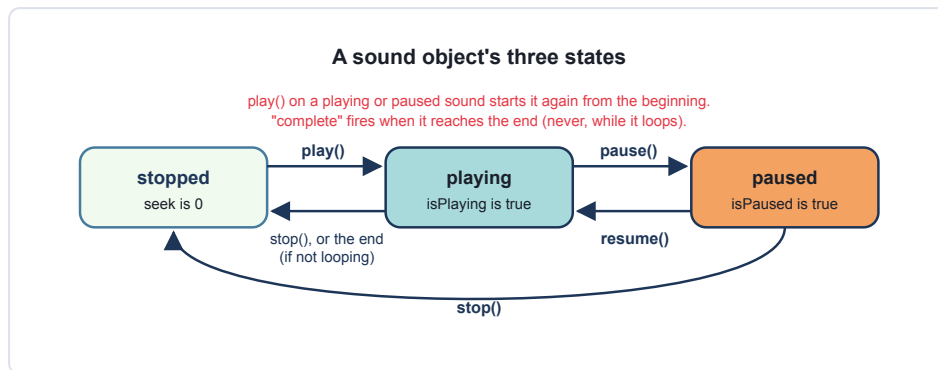
sound.once(Phaser.Sound.Events.COMPLETE, () => {
  this.playing = this.playing - 1;
  this.showState();
  sound.destroy();
});

sound.play(config);
```

- `add()` gives you the sound object - and does **not** start it
- it stays in the sound manager until **you** `destroy()` it

8 / 20

Three states



`play()` on a playing or paused sound starts it **again from the beginning**

9 / 20

Changing a playing sound

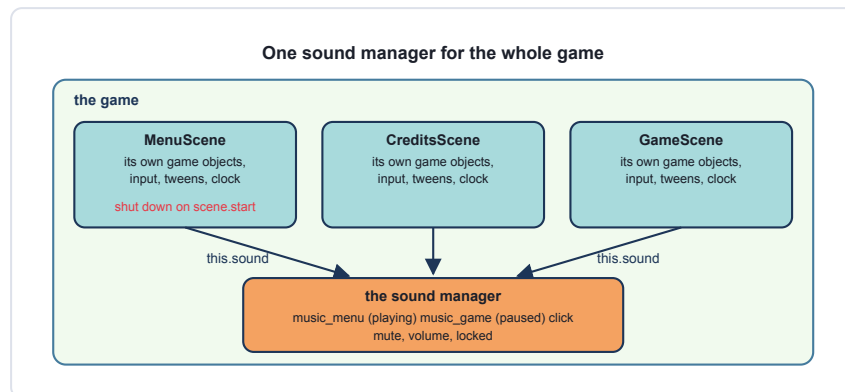
```
if (this.music.isPlaying) {  
  this.music.pause();  
} else if (this.music.isPaused) {  
  this.music.resume();  
} else {  
  this.music.play();  
}
```

```
this.music.setVolume(MUSIC_VOLUME).setRate(1).setDetune(0);
```

`setLoop` `setPan` `setSeek` - and `seek` , `duration` to read back

10 / 20

One sound manager



Sounds **keep playing** when their scene shuts down.

11 / 20

The duplicate music bug

```
// in create() - runs EVERY time the scene starts
this.sound.add(MUSIC_MENU_KEY, { loop: true }).play();
```

menu - credits - menu - credits - menu: **five** copies playing

Ask first:

```
let music: Phaser.Sound.BaseSound | null = scene.sound.get(key);
```

`null` if there is no sound with that key

12 / 20

Music.play(this, key)

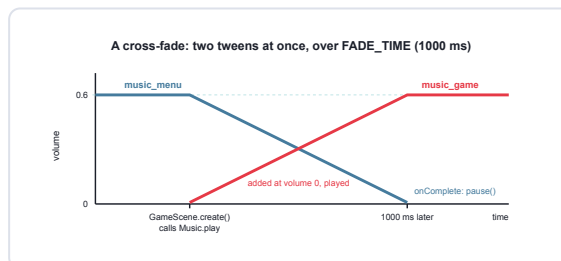
```
if (music === null) {  
    music = scene.sound.add(key, { loop: true, volume: 0 });  
    music.play();  
} else if (music.isPaused) {  
    music.resume();  
} else if (!music.isPlaying) {  
    music.play();  
}
```

- every scene names the music it wants, in `create()`
- already playing? leave it alone
- every **other** track fades out, and is paused

13 / 20

Cross-fading - whose tweens?

```
scene.tweens.add({  
    targets: music,  
    volume: MUSIC_VOLUME,  
    duration: FADE_TIME,  
});
```



Tweens belong to a **scene**, and die with it: make fades in the scene that is **starting**

14 / 20

Mute, everywhere

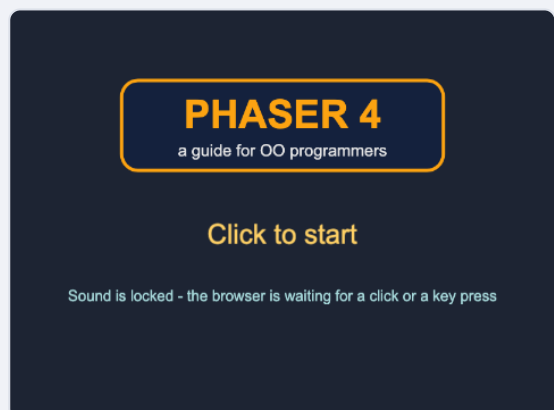
```
private toggle(): void {  
    const muted = !this.isMuted();  
    this.scene.registry.set(MUTED, muted);  
    this.scene.sound.mute = muted;  
    this.showState();  
}
```

- `this.sound.mute` , `this.sound.volume` - the whole game
- a `MuteButton` in every scene; one setting in the registry

15 / 20

The autoplay rule

- no sound until the player clicks, taps or presses a key
- `this.sound.locked` ; the `"unlocked"` event
- sounds played while locked wait - then all start at once
- so: a **"click to start"** screen



16 / 20

A listener that outlived its scene

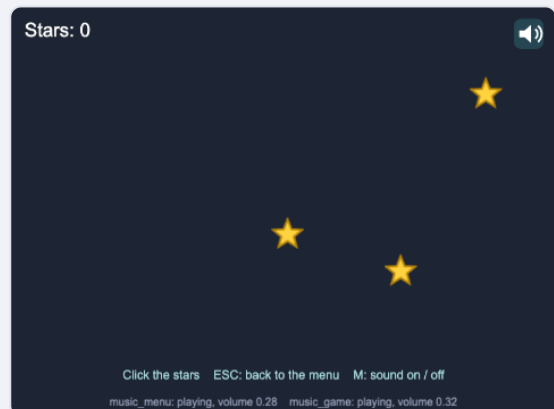
```
const onUnlocked = (): void => {
  status.setText("Sound is unlocked");
};
this.sound.once(Phaser.Sound.Events.UNLOCKED, onUnlocked);
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.sound.off(Phaser.Sound.Events.UNLOCKED, onUnlocked);
});
```

Without the `SHUTDOWN` part: `TypeError: Cannot read properties of null (reading 'drawImage')`

17 / 20

Try it: Music Across Scenes

- menu - credits - menu: the music carries on
- menu - game: watch the cross-fade on the bottom line
- M in any scene; then change scene
- leave the game quickly: does anything get stuck?



18 / 20

Summary

- `this.load.audio(key, file) ; this.sound.play(key, config)` - fire and forget
- `this.sound.add(key, config)` - keep it; `play` `pause` `resume` `stop` `set...` ; `destroy()`
- `"complete"` when a sound ends; `delay` and `seek` in seconds
- one sound manager for the game: check `this.sound.get(key)` before starting music
- `tween volume` to fade - in the scene that is starting
- `this.sound.mute` , `this.sound.volume` ; `locked` until the first click

19 / 20

Challenges

1. **More pads** - a third row of four
2. **Turn it down** - V: 100%, 60%, 30%, in every scene
3. **Remember the mute** - survives a refresh (`localStorage`)
4. **Duck the music** - quieter under the power-up
5. **On the beat** - a circle that pulses at 128 bpm, in time after pauses
6. **Jukebox** - three tracks in turn; K for the next

Next: **Chapter 6 - Scoring**

20 / 20



Scoring

Overview

Scoring looks like the easiest part of a game, and it is where students first meet a real design question: *who owns the score, and how does everyone else find out about it?* The chapter answers it in three steps. `ch06_hud_events` shows three Phaser emitters (a scene's, the game's, the registry's) feeding a HUD scene, and what each does when a scene restarts. `ch06_coin_collector` puts the rules and numbers in a plain `ScoreManager` class that **announces** changes as events, with a HUD scene that only listens. `ch06_high_score_table` saves a top-ten table in `localStorage`, validates what it reads back, and takes initials from the keyboard.

The Phaser ideas are **parallel scenes** (`launch` / `stop`, draw order), **event emitters** (`emit` / `on` / `once` / `off`, context, `changedata-`), **counter tweens** and generic `keydown` events. The TypeScript and JavaScript ideas are `extends Phaser.Events.EventEmitter`, `unknown` and **type guards**, `try / catch`, `JSON`, optional properties and optional chaining, `padStart`, and regular expressions. The OO idea - tell an object what happened, let it announce the results - is the one to leave them with.

Prerequisites

- Chapter 2: scenes, `init(data)` with an interface, the registry, the scene object being reused
- Chapter 4: `launch`, parallel scenes, `SHUTDOWN` and cleaning up listeners (the chapter recaps what it needs in a few lines, so it can be taught before Chapter 4 at a push)
- Java: interfaces, inheritance, the observer pattern / Swing listeners (for the analogy)

Learning outcomes

Students can:

1. run a HUD as a separate scene on top of the game, and explain why it is better than text in the game scene
2. send and listen for events on a scene, on the game and on the registry, and choose between them
3. explain why a late listener misses events, and read the current state as well as listening
4. remove listeners from longer-lived emitters on `SHUTDOWN` , passing the same function and context
5. design a plain class that owns game state and emits events, and keep scenes to *telling* and *listening*
6. save and load structured data with `localStorage` and JSON, and validate it with `unknown` , `try / catch` and a type guard
7. read typed characters from `keydown` events with `event.key`

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play <code>ch06_coin_collector</code> . Ask: "what is on screen that is not the game?"
0:05 - 0:15	Slides 3-4: why a HUD scene; <code>launch</code> , draw order, each scene's own camera (point out the shake does not move the HUD)
0:15 - 0:30	Slides 5-9 with <code>ch06_hud_events</code> running: the three routes. Live demo (below) of the missing clean-up
0:30 - 0:45	Slides 10-12: the <code>ScoreManager</code> , tell vs announce. Draw the "who talks to whom" diagram on the board before showing it
0:45 - 0:55	Slides 13-17: floating text and count-up; <code>localStorage</code> , JSON and validation
0:55 - 1:00	Slide 18: initials; the challenge list

Lab (2 hours)

Time	Activity
0:00 - 0:15	Build and play all three projects. In <code>ch06_high_score_table</code> , get onto the table, then find the key in the browser's developer tools (Application > Local Storage in Chrome)
0:15 - 0:30	Break it on purpose: edit the stored value to <code>this is not json</code> , reload - the game still starts. Then comment out the <code>try / catch</code> round <code>JSON.parse</code> and reload
0:30 - 1:15	Challenges 1-3
1:15 - 2:00	Challenges 4-6 (6 is a good take-home)

Suggested demos and live coding

- **The invisible leak.** In `ch06_hud_events/src/scenes/HudScene.ts` , delete the `this.events.once(Phaser.Scenes.Events.SHUTDOWN, ...)` block, rebuild, and press H a few times. There is **no error and the numbers are right** - but the "listeners" counts go 2, 3, 4. Ask why it still works (the scene object is reused, so old listeners call the same `HudScene` , whose fields now point at the new texts). Then ask: what if `showCoins` also played a sound?
- **Late listeners.** In `ch06_hud_events` , press 2 a few times, then H: `Gems: ?` . Ask why coins and stars survive a HUD restart and gems do not. Lead them to "events are news, not state".
- **Forgetting** `this.scene.stop(HUD_SCENE)` . Remove that line from `GameScene.endGame()` in `ch06_coin_collector` : the HUD bar stays on top of the game over screen (checked in a scratch copy: the active scenes were `HudScene` , `GameOverScene`). Pressing SPACE then relauches the HUD, which *restarts* it - so it looks fine again, hiding the bug.
- **Why validate?** Show `ch06_high_score_table` starting normally with a corrupt value in storage, then with the `try / catch` removed: a blank screen and a `SyntaxError` in the console.

Key points to stress

- **Tell, then listen.** Scenes call the `ScoreManager`'s methods to report what happened; they never set the score. The `ScoreManager` emits events and never knows who listens. Most students' first design has the game scene updating HUD text directly, or the HUD reading game-scene fields every frame - both work, and both are worth comparing to this
- **Events are news, not state.** Anything that starts later (a launched HUD starts a frame late) must read the current values as well as listen
- **Clean up what you add to someone else's emitter.** Phaser clears a scene's own objects, timers and input listeners on shutdown. It does *not* remove listeners the scene put on the game, the registry, another scene, or a `ScoreManager`
- `on(event, fn, this)` - the third argument matters, and `off` needs the same three things
- **Never trust stored data.** Storage is shared by the whole site (every project in this book is served from 127.0.0.1:8000), can be edited by the player, and outlives versions of your game
- a new `ScoreManager` per game is the cleanest reset there is - contrast with Chapter 2's `init()`

Common problems and errors

These messages were produced by making each mistake in a scratch copy of the chapter projects.

What students see	Cause	Fix
Nothing wrong, but <code>listenerCount</code> climbs; effects play twice after a restart	listeners on another emitter not removed on <code>SHUTDOWN</code>	<code>off(...)</code> them in a <code>SHUTDOWN</code> handler
<code>TypeError: Cannot read properties of undefined (reading 'addCounter')</code> when the score changes	<code>this.scoreManager.on(SCORE_CHANGED, this.countUpTo)</code> - no context, so <code>this</code> is the <code>ScoreManager</code>	pass <code>this</code> as the third argument to <code>off()</code>
Argument of type 'number null' is not assignable to parameter of type 'number'.	<code>this.showScore(tween.getValue())</code> in a counter tween	<code>tween.getValue() ?? 0</code>
Argument of type 'string null' is not assignable to parameter of type 'string'.	<code>JSON.parse(localStorage.getItem("scores"))</code>	check for <code>null</code> first
Type 'Scene' is missing the following properties from type 'GameScene': <code>coins</code> , <code>gems</code> , <code>preload</code> , <code>init</code> , and 6 more.	<code>this.gameScene = this.scene.get(GAME_SCENE);</code> into a <code>GameScene</code> field	<code>this.scene.get<GameScene>(GAME_SCENE)</code>

What students see	Cause	Fix
Blank screen; console: <code>SyntaxError: Unexpected token 'h', "this is not json" is not valid JSON</code>	<code>JSON.parse</code> with no <code>try / catch</code> , and bad data in storage	<code>try / catch</code> , and start table
The HUD never updates from the registry, or misses the first value	listening for <code>changedata-score</code> when the key is new: the first <code>set</code> sends <code>setdata</code>	set the key before anything <code>GameScene.init()</code> does the current value when it starts
The HUD restarts (score display resets) whenever the game scene restarts	<code>this.scene.launch(HUD_SCENE)</code> on a HUD that is already running restarts it	<code>if (!this.scene.isActive)</code>
HUD bar still showing over the game over screen	no <code>this.scene.stop(HUD_SCENE)</code> at game over	stop it before starting the
The old "new" row still flashes on the title screen after a later game	<code>this.scene.start(TITLE_SCENE)</code> with no data reuses the data it was last started with	pass <code>{}</code>
The initials screen types an extra letter, or the confirm prompt (challenge 4) cancels itself	a <code>keydown</code> listener added inside another key's listener receives the same key press	one <code>keydown</code> listener with
Scores "disappear"	playing in Celbridge's <code>game.webview</code> then in Chrome (or the other way round) - each browser has its own storage	expected; explain

What students see	Cause	Fix
Scores from another project appear, or vanish	two games using the same storage key on 127.0.0.1:8000	a key that names the gai

Discussion questions

- The ScoreManager extends `Phaser.Events.EventEmitter`. It could instead *have* one: `public readonly events = new Phaser.Events.EventEmitter();` - which is how `this.registry.events` works. What changes for the callers? Which do you prefer, and why? (Inheritance exposes `emit` to everyone, so any class can fake a `SCORE_CHANGED`; composition could keep `emit` private-ish.)
- `emit` is not type-checked. How could we make `SCORE_CHANGED`'s arguments checked by the compiler? (Typed wrapper methods such as `onScoreChanged(fn: (score: number) => void)`.)
- The game scene plays the level-up sound; the HUD shows the level-up banner. Both listen to `LEVEL_CHANGED`. Should the ScoreManager play the sound? Why not?
- Where should "best score this session" live - the registry, the ScoreManager, or the table?
- The high score table is loaded afresh by every scene that needs it, rather than kept in memory. What are the advantages? When would it be a bad idea?
- What could a player do to cheat the table? Can a browser game ever stop that? (No - anything in the browser can be edited. Only a server can keep honest scores.)

Extension ideas

- a pause scene (Chapter 4) that shows the ScoreManager's numbers - a third listener, for free
- unit-test the scoring rules: extract them into a class that does not extend a Phaser class (take an emitter in the constructor, or return results), and test it with `deno test`

- save the whole game's settings (volume from Chapter 5, difficulty from challenge 6) as one JSON object under one key, with a `version` number and a migration when the shape changes
 - show the table with a stagger tween, rows sliding in one after another (a preview of Chapter 7)
-

Assessment ideas

- Practical: "add a *time bonus*: points for every second survived, shown in the HUD, without the HUD reading anything from the game scene"
 - Code reading: show a HUD `create()` with `on` calls and no `SHUTDOWN` handler; ask what goes wrong, when, and how they would prove it (listener counts)
 - Short answer: what is the difference between `unknown` and `any`, and why is `JSON.parse`'s result better treated as `unknown`?
 - Design: draw the "tells / announces" diagram for a game of their choice (lives, ammo, keys)
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 6 - Scoring - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. A star is born

Project: `solutions/ch06_challenge_1_a_star_is_born` (from `ch06_coin_collector`)

Goal: find the places a pickup type is described, and add one - data, not new code.

`src/objects/Coin.ts`

```
export const GEM: CoinKind = { texture: GEM_KEY, points: 50, scale: 1.5,
lifetimeScale: 0.6 };
// CHALLENGE 1 - the rarest and quickest pickup
export const STAR: CoinKind = { texture: STAR_KEY, points: 100, scale: 1.6,
lifetimeScale: 0.45 };
```

`src/scenes/GameScene.ts`

```
const STAR_CHANCE = 0.05;           // CHALLENGE 1 - one pickup in twenty is
a star
...
// CHALLENGE 1 - one random number, split into three ranges: star, gem, coin
const roll = Math.random();
let kind = GOLD_COIN;
if (roll < STAR_CHANCE) {
    kind = STAR;
} else if (roll < STAR_CHANCE + GEM_CHANCE) {
    kind = GEM;
}
```

plus `STAR_KEY` / `STAR_FILE` in `assets.ts`, `this.load.image(STAR_KEY, STAR_FILE)` in `TitleScene.preload()`, a third picture on the title screen, and `star.png` copied into `public/assets/images/`.

Look for: `ScoreManager` , `HudScene` and `Coin` 's class untouched - the points come from the `CoinKind` , and the `ScoreManager` only ever sees a number. The most common slip is calling `Math.random()` twice (`if (Math.random() < 0.05) ... else if (Math.random() < 0.12)`), which quietly changes the odds: the gem chance becomes 12% of the remaining 95%. One roll, split into ranges, is the clean answer. Tested with the harness: 200 spawned pickups gave 167 coins, 24 gems and 9 stars, and a clicked star scored 100.

2. Bonus life

Project: `solutions/ch06_challenge_2_bonus_life` (from `ch06_coin_collector`)

Goal: see the payoff of events: change the rules in one place, and the display follows.

`src/ScoreManager.ts`

```
if (this.pickups % PICKUPS_PER_LEVEL === 0) {
  this.level = this.level + 1;
  this.emit(LEVEL_CHANGED, this.level);

  // CHALLENGE 2 - a new level gives back one lost life. The HUD already
  // listens for
  // LIVES_CHANGED, so it fills a heart in again without being changed at
  // all.
  if (this.lives < START_LIVES) {
    this.lives = this.lives + 1;
    this.emit(LIVES_CHANGED, this.lives);
  }
}
```

Why it works without touching the HUD (the question in the challenge): the HUD does not know *why* lives change. It listens for `LIVES_CHANGED` and redraws the hearts from the number it is given - so a life gained is shown exactly like a life lost.

Look for: the change in `ScoreManager` , not in `GameScene` (a student who adds a life in the game scene's `LEVEL_CHANGED` listener has to reach into the `ScoreManager` - there is no setter - and usually ends up adding one, which is worth discussing). The cap at three.

Tested with the harness: two lives lost, ten pickups - hearts go from 1, 0.2, 0.2 to 1, 1, 0.2 ; two more levels and all three are back, and no more.

3. Combo meter

Project: `solutions/ch06_challenge_3_combo_meter` (from `ch06_coin_collector`)

Goal: a new view of existing data, entirely in the HUD.

`src/ScoreManager.ts`

```
// CHALLENGE 3 - exported, so the HUD's meter can use the same numbers as  
the rules  
export const COMBO_STEP = 5;    // the multiplier goes up by 1 for every 5  
pickups in a row  
export const MAX_MULTIPLIER = 5;
```

`src/scenes/HudScene.ts`

```

this.add.rectangle(METER_X, METER_Y, METER_WIDTH, METER_HEIGHT, 0x000000,
0.6).setOrigin(0, 0.5);
this.meterFill = this.add.rectangle(METER_X, METER_Y, METER_WIDTH,
METER_HEIGHT, 0xf4a261).setOrigin(0, 0.5);
this.meterFill.scaleX = 0;
...
private showMeter(combo: number, multiplier: number): void {
    this.tweens.killTweensOf(this.meterFill);

    if (multiplier >= MAX_MULTIPLIER) {
        this.meterFill.scaleX = 1;    // the top multiplier: nothing more to
fill, so it stays full
        return;
    }

    const step = combo % COMBO_STEP;    // 0 to 4: pickups since the multiplier
last went up
    if (combo > 0 && step === 0) {
        // the multiplier has just gone up: show the bar full for a moment, then
empty it
        this.meterFill.scaleX = 1;
        this.tweens.add({ targets: this.meterFill, scaleX: 0, delay: 150,
duration: 300 });
    } else {
        // a fifth for each pickup - or empty when the combo is broken (combo 0,
step 0)
        this.tweens.add({ targets: this.meterFill, scaleX: step / COMBO_STEP,
duration: 120 });
    }
}

```

`showMeter` is called at the end of `showCombo`, which already runs on every `COMBO_CHANGED`.

- the fill is a `Rectangle` stretched with `scaleX`; origin `(0, 0.5)` makes it grow from the left. Changing `width` also works, but a `Rectangle`'s `width` does not redraw its shape - `setSize` does
- `%` (remainder) turns "combo 13" into "3 towards the next step"

Look for: the constants **exported** from `ScoreManager.ts`, not copied as a second `5` in the HUD (a copy goes wrong the day someone changes the rule). `killTweensOf` before starting a new tween, so an old one does not fight it. Accept a meter with no tween at all.

Tested with the harness: 3 pickups gives 0.6; the 5th fills it and it empties; a broken combo gives 0; the top multiplier keeps it full.

4. Wipe the slate

Project: `solutions/ch06_challenge_4_wipe_the_slate` (from `ch06_high_score_table`)

Goal: a small state machine for keyboard input, and removing something from storage.

`src/HighScoreTable.ts`

```
// CHALLENGE 4 - empty the table, and forget the saved copy too
public clear(): void {
  this.entries = [];
  try {
    localStorage.removeItem(STORAGE_KEY);
  } catch {
    // storage turned off: there was nothing saved to remove
  }
}
```

`src/scenes/TitleScene.ts`

```
// CHALLENGE 4 - R asks the question; then Y clears the table, and any other
key cancels
private handleKey(event: KeyboardEvent): void {
  const key = event.key.toLowerCase();
  if (!this.confirming) {
    if (key === "r") {
      this.confirming = true;
      this.prompt.setVisible(true);
    }
    return;
  }

  if (key === "y") {
    HighScoreTable.load().clear();
    const data: TitleData = {}; // no row to highlight any more
    this.scene.restart(data); // start this scene again: it reads the
    (now empty) table
  } else {
    this.confirming = false;
    this.prompt.setVisible(false);
  }
}
```

and the SPACE listener becomes `on` with a check, so it does nothing while the question is showing:

```
this.input.keyboard!.on("keydown-SPACE", () => {
  if (!this.confirming) {
    this.scene.start(GAME_SCENE);
  }
});
```

The trap most students fall into: listening for R, and then adding a `"keydown"` listener *inside* the R listener to hear the answer. Phaser sends `keydown-R` first and then `keydown` for the **same** key press - so the brand-new listener hears the R, and takes it as the answer. We checked this in a scratch copy: the answer listener logged `answer: r` straight after `asking`. One `keydown` listener with a `confirming` flag avoids it.

Look for: `confirming` reset in `init()` (the scene object is reused); `clear()` on `HighScoreTable` rather than `localStorage.removeItem` with the key copied into the scene; `restart({})` rather than `restart()` - without data, the new start could still carry

the old `highlight` . Tested with the harness: R shows the question, N or SPACE cancel and leave the table, R then Y empties it, the storage key is gone, and it is still empty after a reload.

5. Chasing the record

Project: `solutions/ch06_challenge_5_chasing_the_record` (from `ch06_high_score_table`)

Goal: a second listener on the same event, with its own state.

`src/scenes/HudScene.ts`

```

init(data: HudData): void {
    ...
    // CHALLENGE 5 - the HUD reads the table itself: it is the only one that
    needs the top score
    this.record = HighScoreTable.load().getTopScore();
    this.celebrated = false;
}
...
this.scoreManager.on(SCORE_CHANGED, this.checkRecord, this); // CHALLENGE
5 - a second listener
...
private showScore(score: number): void {
    this.shownScore = score;
    this.scoreText.setText(String(score).padStart(SCORE_DIGITS, "0"));
    // CHALLENGE 5 - HI shows the record, or the score on screen once that is
    higher, so the two
    // count up together
    this.hiText.setText(String(Math.max(this.record,
score)).padStart(SCORE_DIGITS, "0"));
}

private checkRecord(score: number): void {
    if (this.celebrated || this.record === 0 || score <= this.record) {
        return;
    }
    this.celebrated = true;
    this.hiText.setColor("#ffd166");
    ...
}

```

The rest of `checkRecord` makes a big "NEW HIGH SCORE!" text and blinks it with a tween before destroying it (the same pattern as the level banner).

Look for:

- `celebrated` reset in `init()` - otherwise the second game never celebrates (Chapter 2's trap)
- the matching `off` for the new listener in the `SHUTDOWN` handler
- `HI` updated in `showScore` (so it counts up with the score) rather than jumping to the new score in the listener - both are acceptable, but ask which looks better
- `record === 0` : with an empty table every first point would be a "new high score". A good discussion point - is it one?

Some students put the record in the `ScoreManager`. That is defensible (it is a scoring fact), but then the `ScoreManager` needs the table, or has to be given the record - ask them to argue for it. Tested with the harness: record 300; at 170 and 230 nothing; at 330 `HI` shows 000330, gold, and the message appears once.

6. Easy, normal, hard

Project: `solutions/ch06_challenge_6_easy_normal_hard` (from `ch06_high_score_table`)

Goal: data that travels through every scene; one storage key per table.

A new file describes the difficulties:

`src/Difficulty.ts`

```
export type Difficulty = "easy" | "normal" | "hard";

export interface DifficultySettings {
  name: string;
  lifetimeScale: number; // how long coins stay: 1.4 = 40% longer
  spawnScale: number;   // the time between coins: 0.75 = they come a
  // quarter faster
}

export const DIFFICULTIES: Record<Difficulty, DifficultySettings> = {
  easy: { name: "EASY", lifetimeScale: 1.4, spawnScale: 1.25 },
  normal: { name: "NORMAL", lifetimeScale: 1, spawnScale: 1 },
  hard: { name: "HARD", lifetimeScale: 0.7, spawnScale: 0.75 },
};
```

The answer to the hint's question - `load()` takes the difficulty, and uses it to choose the storage key; the table remembers its key for `save()` :

`src/HighScoreTable.ts`


```
// CHALLENGE 6 - each difficulty is saved under its own key. "normal" keeps
// the original key, so
// the scores saved before there were difficulties still count as normal
// ones.
function storageKeyFor(difficulty: Difficulty): string {
    return difficulty === "normal" ? STORAGE_KEY :
    `${STORAGE_KEY}.${difficulty}`;
}
...
public static load(difficulty: Difficulty = DEFAULT_DIFFICULTY):
HighScoreTable {
    const key = storageKeyFor(difficulty);
```

Then the difficulty is carried in scene data all the way round: `TitleData` and a new `GameData` gain `difficulty?`, `GameOverData` gains `difficulty`, and every `this.scene.start` passes it on. The title screen's 1, 2 and 3 keys restart it with the chosen difficulty:

`src/scenes/TitleScene.ts`

```
private choose(difficulty: Difficulty): void {
    const data: TitleData = { difficulty: difficulty };
    this.scene.restart(data);
}
```

and the game scene scales its timings:

`src/scenes/GameScene.ts`

```
const lifetime = levelLifetime *
DIFFICULTIES[this.difficulty].lifetimeScale; // CHALLENGE 6
```

Look for:

- a union type (or enum) for the difficulty rather than bare strings or numbers - then `Record<Difficulty, ...>` makes the compiler insist every difficulty has settings
- the difficulty in **every** hand-over. The usual bug: playing again from the game over screen (SPACE) silently goes back to normal, because `GameOverScene` starts `GameScene` with no data. The tidy fix is the one here; storing the current difficulty in the registry is also a fair answer

- keeping the old key for normal - a thoughtful touch, not required
- a default parameter (`difficulty: Difficulty = DEFAULT_DIFFICULTY`) so older calls still compile

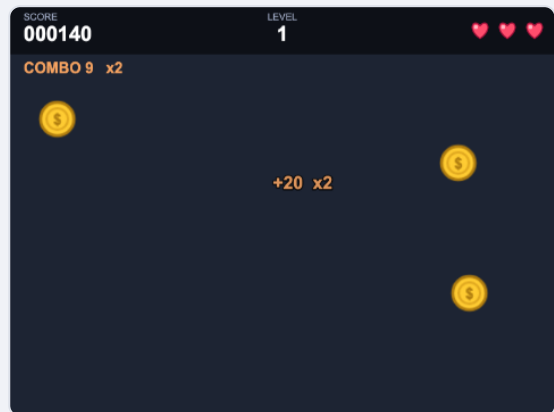
Tested with the harness: a seeded normal table shows under NORMAL; 3 shows an empty HARD table; a hard game's coins last 1680 ms instead of 2400; a hard high score goes into `phaser4-guide.coin-collector.high-scores.hard` and the normal table is untouched; after an easy game, SPACE on the game over screen plays easy again.

Chapter 6 - Scoring - Slides

Chapter 6

Scoring

A HUD, a ScoreManager, and high scores that last



1 / 20

Today

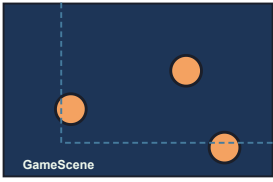
- a **HUD** in a scene of its own, on top of the game
- three ways to tell the HUD something changed
- a **ScoreManager**: tell it what happened; it announces the results
- floating "+10"s and a score that counts up
- high scores saved in the browser - and checked when they come back
- typing initials

2 / 20

A HUD is a scene

Two scenes running at once, one drawn on top of the other

HudScene (see-through, except its bar)



SCORE 000140 LEVEL 1 ♥♥♥

GameScene

main.ts

```
scene: [TitleScene, GameScene,
        HudScene, GameOverScene]

Later in the list = drawn later = on top.
```

GameScene

```
this.scene.launch(HUD_SCENE, data)
starts the HUD beside it, on the next frame

this.scene.stop(HUD_SCENE)
takes it away again when the game ends
```

```
this.scene.launch(HUD_SCENE, hudData); // beside this scene, from the next frame
...
this.scene.stop(HUD_SCENE);           // when the game ends
```

3 / 20

Why a separate scene?

- **its own camera** - `this.cameras.main.shake(...)` in the game does not shake the score
- draw order = order in the config's `scene` list: later is on top
- the game scene can restart without the HUD rebuilding itself
- one job per class: the game plays, the HUD shows

4 / 20

Emitters

An **event emitter**: `emit` events on it; listen with `on` , `once` , `off`

```
ball.on("pointerdown", ...) // you have used one already
```

Three that can reach a HUD:

- the game scene's own: `this.events`
- the game's: `this.game.events`
- the registry's: `this.registry.events`

5 / 20

Try it: HUD Events

- click the coin, gem and star (or 1, 2, 3)
- **R** restarts the game scene; **H** restarts the HUD
- watch the "listeners" counts
- after H, why does it say `Gems: ?`



6 / 20

Three routes

```
// 1 - the game scene's own emitter (the HUD needs the scene)
this.events.emit(COINS_CHANGED, this.coins);
this.gameScene.events.on(COINS_CHANGED, this.showCoins, this);

// 2 - the game's emitter (anyone can send, anyone can listen)
this.game.events.emit(GEMS_CHANGED, this.gems);
this.game.events.on(GEMS_CHANGED, this.showGems, this);

// 3 - the registry announces its own changes
this.registry.inc(STARS, 1);
this.registry.events.on(`changedata-${STARS}`, this.onStarsChanged, this);
```

First `set` of a new key sends `setdata`, not `changedata`

7 / 20

News and state

- an event is **news**: only listeners there *at the time* hear it
- a launched HUD starts a frame late - so it also reads the current values

```
this.showCoins(this.gameScene.getCoins()); // route 1: ask the scene
this.gemsText.setText("Gems: ?"); // route 2: nobody to ask
this.showStars(this.registry.get(STARS)); // route 3: registry = state + news
```

8 / 20

Clean up

```
this.events.once(Phaser.Scenes.Events.SHUTDOWN, () => {
  this.gameScene.events.off(COINS_CHANGED, this.showCoins, this);
  this.game.events.off(GEMS_CHANGED, this.showGems, this);
  this.registry.events.off(`changedata-${STARS}`, this.onStarsChanged, this);
});
```

- those emitters **outlive the HUD** - Phaser will not remove these listeners
- forget it: **no error**, the listeners just pile up
- `off` needs the same event, function **and context** as `on`

9 / 20

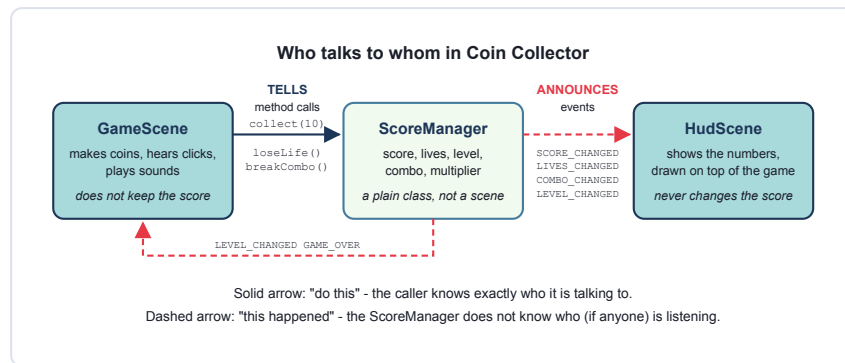
A ScoreManager

```
export class ScoreManager extends Phaser.Events.EventEmitter {
  private score = 0;
  private lives = START_LIVES;
  ...
  public collect(basePoints: number): number {
    this.combo = this.combo + 1;
    const points = basePoints * this.getMultiplier();
    this.score = this.score + points;
    this.emit(SCORE_CHANGED, this.score, points);
    ...
  }
}
```

A plain class: no text, no sounds, no scenes

10 / 20

Tell - announce



The observer pattern - Swing's `addActionListener`

11 / 20

The HUD listens

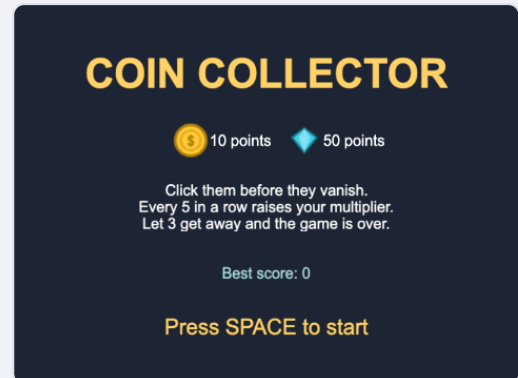
```
init(data: HudData): void {  
    this.scoreManager = data.scoreManager;  
    ...  
}  
  
create(): void {  
    ...  
    this.showScore(this.scoreManager.getScore()); // now  
    this.scoreManager.on(SCORE_CHANGED, this.countUpTo, this); // and later  
    ...  
}
```

New game? `this.scoreManager = new ScoreManager();` - nothing to reset

12 / 20

Try it: Coin Collector

- 5 in a row raises the multiplier
- 10 pickups = a faster level
- 3 escape = game over
- find every `emit` and every `on`



13 / 20

Feel: float and count

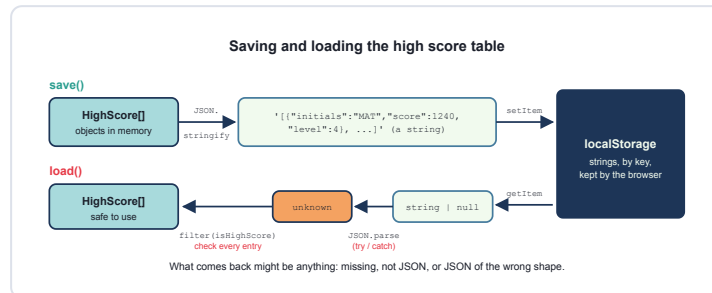
```
// FloatingText: rise, fade, then destroy itself
scene tweens.add({ targets: this, y: y - RISE, alpha: { value: 0, ease: "Cubic.easeIn" },
  duration: DURATION, ease: "Cubic.easeOut", onComplete: () => this.destroy() });

// HudScene.countUpTo(score)
this.countUp?.stop();
this.countUp = this.tweens.addCounter({
  from: this.shownScore, to: score, duration: COUNT_UP_TIME, ease: "Cubic.easeOut",
  onUpdate: (tween) => {
    this.showScore(Math.round(tween.getValue() ?? 0));
  },
});
```

14 / 20

localStorage and JSON

- `setItem(key, text)`, `getItem(key)` (or `null`), `removeItem(key)`
- **strings only** - `JSON.stringify` / `JSON.parse`
- kept **per web site**: every project on 127.0.0.1:8000 shares one - name your key
- each browser has its own (Celbridge's webview is not Chrome)



15 / 20

Never trust what comes back

```

let data: unknown;
try {
  data = JSON.parse(text);
} catch {
  return new HighScoreTable([]); // not JSON at all
}
if (!Array.isArray(data)) {
  return new HighScoreTable([]); // JSON, but not a list
}
const entries = data.filter(HighScoreTable.isHighScore);
  
```

`unknown` : check before use. `isHighScore(value: unknown): value is HighScore` - a **type guard**

16 / 20

Try it: High Score Table

- get onto the table; find the key in the developer tools
- change it to `this is not json` and reload
- what happens to a score that *ties*?

COIN COLLECTOR			
HIGH SCORES			
1.	ACE	002450	level 7
2.	BOB	001980	level 6
3.	CAT	001620	level 5
4.	DAN	001400	level 5
5.	MAT	001240	level 4
6.	EVE	001130	level 4
7.	FOX	000960	level 4
8.	GUS	000720	level 3
9.	HAL	000540	level 3
10.	IVY	000350	level 2

Press SPACE to start

Click coins before they vanish. 5 in a row raises the multiplier.

17 / 20

Typing initials

```
this.input.keyboard!.on("keydown", (event: KeyboardEvent) => {  
  this.handleKey(event);  
});  
...  
if (LETTER.test(event.key) && this.initials.length < MAX_INITIALS) {  
  this.initials = this.initials + event.key.toUpperCase();  
} else if (event.key === "Backspace") {  
  this.initials = this.initials.slice(0, -1);  
}
```

`event.key` - what the key types. `event.code` - where the key is

18 / 20

Summary

- a HUD is a scene, launched on top; it has its own camera
- emitters: `this.events` , `this.game.events` , `this.registry.events` , your own
- events are news: read the current state too, and clean up on `SHUTDOWN`
- scenes **tell** the ScoreManager; it **announces**
- tweens and counter tweens make scores feel good
- `localStorage` + JSON for scores that last - validated on the way back in

19 / 20

Challenges

1. **A star is born** - a rare 100-point star
2. **Bonus life** - a new level gives a life back; the HUD needs no change
3. **Combo meter** - a bar that fills towards the next multiplier
4. **Wipe the slate** - R, then Y, clears the table
5. **Chasing the record** - `HI` in the HUD, and a celebration
6. **Easy, normal, hard** - a table for each difficulty

Next: **Chapter 7 - Animations**

20 / 20



Animations

Overview

The chapter covers three separate tools that all make a game feel alive: **frame animations** from sprite sheets (`this.animations` , `play` , animation events), **tweens** (`this.tweens` : easing, yoyo, repeat, chains, stagger, counters) and **particles** (`this.add.particles` , `explode`). The design idea is the **state machine** for a character's animations - the first time in the book that an object's behaviour is organised around explicit states, and a pattern students will meet again in Chapters 12, 13, 15 and 17. The TypeScript ideas are **string-literal union types** as enums, `Record<K, V>` , `switch` on strings, `Partial<T>` , `??` and a justified `as` .

It is a full chapter. If time is short, the lab and the hero are the core; the tween playground can be explored by students on their own with the README beside them.

Prerequisites

- Chapter 1: `preUpdate` and `delta` , speeds in pixels per second, origins
- Chapter 2: `on / once` , `setInteractive` , scenes reused and reset in `init()`
- Chapter 3: loading sprite sheets (a quick recap is in the chapter)
- Chapter 4: `Sprite.preUpdate` and why `super.preUpdate` matters (the chapter points back to it)
- Chapter 6: floating text with a tween is referred to in passing

Learning outcomes

Students can:

1. make named animations from a sprite sheet with `this.animations.create` and `generateFrameNumbers` , choosing `frameRate` , `repeat` and `yoyo` , and explain why they are made once, in a preload scene

2. play and control an animation on a sprite (`play` , `play(key, true)` , `chain` , `playAfterRepeat` , `stop` , `pause` , `timeScale` , `setFlipX`) and read its state (`currentAnim` , `currentFrame`)
3. react to animation events, including `ANIMATION_COMPLETE_KEY + key`
4. structure a character's animations as a state machine with a single `changeState` method
5. write tweens with `duration` , `ease` , `yoyo` , `hold` , `repeat` , `delay` and `onComplete` ; use chains, stagger and counters; and prevent tweens fighting with `killTweensOf`
6. choose a suitable ease, and describe the difference between `easeIn`, `easeOut` and `easeInOut`
7. make a burst of particles with an emitter created once with `emitting: false`

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Show a still of the hero next to the running game. "What is different?" - frames, flipping, and movement
0:05 - 0:20	Slides 3-6: frame animation vs tween; the sprite sheet diagram; <code>animations.create</code> ; global animations; Sprite vs Image
0:20 - 0:30	Live demo in <code>ch07_animation_lab</code> : press 2 repeatedly, then hold it - the hero freezes on frame 2. Then 3. Ask for an explanation before showing slide 7
0:30 - 0:40	Slides 8-9: chain, playAfterRepeat, events. In the lab: C, then 2 followed by A, watching the event log
0:40 - 0:55	Slides 10-13: the state machine. Draw the states on the board with the class first, then show <code>changeState</code> and <code>updateState</code>
0:55 - 1:00	Slides 14 and 18 as a preview of tweens and particles, for the lab

Lab (2 hours)

Time	Activity
0:00 - 0:15	Build and play all three projects. In the hero project, read <code>Hero.ts</code> from top to bottom in pairs
0:15 - 0:30	Challenge 1 (frame rates) - everyone
0:30 - 0:50	Tweens: slides 14-18 on screen; students press every button in the playground, then read the matching method. The easing gallery: 1, 2, 3
0:50 - 1:00	Live demo: comment out <code>resetCrate()</code> in <code>pulse()</code> , press Pulse four times quickly - the crate ends up stuck at a bigger size (2.4 instead of 1.5). Ask why
1:00 - 1:45	Challenges 2-4, then 5 and 6 for those who are ahead
1:45 - 2:00	Show-and-tell of title sequences (challenge 3) and stomping (challenge 6)

Key points to stress

- **frames vs tweens.** A frame animation changes the picture; a tween changes a number. Students often try to "tween the frames" or "animate the position" - naming the difference early avoids it
- **animations belong to the game.** `this.anims` is the game's animation manager in every scene; making animations in a scene that restarts gives a console warning, not an error, so it can go unnoticed for a long time
- **play restarts.** The single most common animation bug is `play(key)` in `update()`, which restarts the animation every frame so it never gets past its first frame. Two cures: `play(key, true)`, or (better) only call `play` when the state changes
- **one place changes state.** The state machine's value comes from `changeState` being the only place the state and the animation change. If students set `currentState` in several places, the benefit is lost
- `super.preUpdate`: a `Sprite` subclass that overrides `preUpdate` without calling `super` shows a frozen frame and gives no error at all
- **ask, don't reach in** (from Chapter 2) returns in the hero: the scene calls `hero.hurt()`; it does not set the hero's velocity or state

- **effects clean up after themselves:** `onComplete: () => popup.destroy()` , and `once(ANIMATION_COMPLETE, () => boom.destroy())` . Otherwise the scene fills with invisible objects
- **make an emitter once**, and use `explode` or `emitting` - never a new emitter per burst

Common problems and errors

What students see	Cause	Fix
The character slides about frozen on one frame; no error	<code>preUpdate</code> overridden without <code>super.preUpdate(time, delta)</code>	call <code>super.preUpdate</code> first
The animation is stuck on its first frame while a key is held	<code>play(key)</code> called every frame (or on every key repeat) restarts it	<code>play(key, true)</code> , or play only when the state changes
Property 'play' does not exist on type 'Image'.	the object was made with <code>this.add.image</code>	<code>this.add.sprite</code> (or extend <code>Sprite</code>)
The sprite keeps its old animation; console: Missing animation: hero-run	misspelled animation key	keep animation keys as constants
Console: AnimationManager key already exists: hero-idle	animations made in a scene that starts more than once	make them in the preload scene
This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	<code>preUpdate</code> in a <code>Sprite</code> subclass without <code>override</code>	add <code>override</code> (and keep it <code>protected</code>)
Class 'Hero' incorrectly extends base class 'Sprite'. Property 'state' is private in type 'Hero' but not in type 'Sprite'.	a field named <code>state</code> - every game object already has one	another name, e.g. <code>currentState</code> (likewise a method called <code>setState</code>)
Property 'hurt' is missing in type '{ idle: string; run: string; jump: string; fall: string; }' but required in type 'Record<HeroState, string>'.	a state added to the union but not to the <code>Record</code>	this is the type doing its job - add the animation
An <code>ANIMATION_COMPLETE</code> listener never runs	the animation has <code>repeat: -1</code> and never completes	give it a finite <code>repeat</code> , or listen for <code>ANIMATION_REPEAT</code> / <code>ANIMATION_STOP</code>

What students see	Cause	Fix
After several hurts, the "hurt is over" code runs several times at once	<code>on</code> used where <code>once</code> was meant, adding a listener per hurt	<code>once</code>
A yoyo tween "comes back" to the wrong place, or the object creeps	a second tween started while the first was running, from a mid-way value	<code>killTweensOf</code> and reset first, or ignore the input while <code>isTweening</code>
The ease looks linear	misspelled ease string (<code>"Bounce.easout"</code>) - Phaser silently falls back to linear	check the spelling; no error is given
Argument of type 'number null' is not assignable to parameter of type 'number'. on <code>Math.round(tween.getValue())</code>	<code>getValue()</code> returns <code>number null</code>	<code>tween.getValue()</code> <code>?? 0</code>
Particles appear at (0, 0)	<code>explode(count)</code> with no position, on an emitter made at (0, 0)	<code>explode(count, x, y)</code>
The game slows down after a minute of the trail/explosions	a new emitter made for every burst or every pointer move	make one emitter, in <code>create()</code>

The error messages above were produced in scratch copies of the chapter projects.

Suggested demos and live-coding moments

- **The stuck run.** In the lab, hold 2 then hold 3. Nothing explains `ignoreIfPlaying` as well
- **Delete** `super.preUpdate` in `Hero.ts`, build, run right. The hero glides, frozen. Put it back
- **Change one number:** the hurt animation's `frameRate` from 8 to 3. The hero stays hurt for longer, with no other change - the animation event ties game rules to animations
- **Pulse without** `resetCrate` (see the lab plan), then with it

- **Easing on the board.** Before showing the gallery, ask students to sketch how a door, a ball dropped on the floor, and a menu sliding in should move. Then find their curves in the gallery
 - **Particle knobs:** in `addExplosions`, change `gravityY` to -300, `speed` to a single number, remove `blendMode: "ADD"`, make `quantity` 200. Discuss what each setting does to the "feel"
-

Discussion questions

- The hero has five states. Which pairs of states can never follow each other directly? Where does the code make sure of that?
 - Why is it `this.once(...)` in `hurt()` and not `this.on(...)` in the constructor? Could it be `on` in the constructor instead? (Yes - with the listener registered once. Both are defensible; discuss which is easier to follow.)
 - What is the difference between `animations.timeScale` on a sprite and changing `frameRate` in `animations.create`?
 - When would you use a tween to move a character, and when physics or `preUpdate`? (Tweens: a known start, end and time - cut-scenes, menus, platforms on rails. Physics: anything that responds to the player or to collisions.)
 - `killTweensOf` then reset, or ignore the click while `isTweening`? Which feels better for a button? For a character's attack?
-

Extension ideas

- give the hero a **land** state with a short squash (a `scaleY` tween, yoyo) between fall and idle
- add `this.animations.addMix(HERO_RUN, HERO_IDLE, 100)` and compare stopping with and without it
- a **particle trail** from the hero's feet while running (a flowing emitter with `startFollow`, switched by the state machine)
- use `animations.staggerPlay` to start a row of coins spinning out of step
- tween along a path: `Phaser.Curves.Path` and `this.add.follower` for an enemy that flies loops

- explore `Stepped` easing (`ease: "Stepped", easeParams: [5]`) for a deliberately jerky, retro look
-

Assessment ideas

- **Practical:** "add a `duck` state to the hero: DOWN held on the ground shows frame 10 and stops running; releasing DOWN returns to idle". Marks for: the union, the `Record` , the transitions in `updateState` , no `play` outside `changeState`
 - **Code reading:** show a `Sprite` subclass with `play(RUN)` in `preUpdate` and no `super` call; ask for both bugs and their symptoms
 - **Short answer:** explain `yoyo` , `hold` , `repeat: 2` and `repeatDelay` by sketching the value against time (the timeline diagram in the chapter is the model answer)
 - **Design:** draw the state machine for a door (closed, opening, open, closing, locked), naming the animation and the event that ends each state
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 7 - Animations - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Different speeds

Project: `solutions/ch07_challenge_1_different_speeds` (from `ch07_animation_lab`)

Goal: find where an animation's speed is decided, and see that a new animation needs no new art.

`src/scenes/PreloadScene.ts`

```
this.animations.create({
  key: BAT_FLAP,
  frames: this.animations.generateFrameNumbers(BAT_SHEET, { start: 0, end: 3 }),
  frameRate: 24, // CHALLENGE 1: twice as fast (was 12)
  repeat: -1,
});
...
// CHALLENGE 1: the same frames as hero-run, faster - a new animation needs
// no new pictures
this.animations.create({
  key: HERO_SPRINT,
  frames: this.animations.generateFrameNumbers(HERO_SHEET, { start: 2, end: 7 }),
  frameRate: 20,
  repeat: -1,
});
```

`src/scenes/LabScene.ts`

```
keyboard.on("keydown-SEVEN", () => this.hero.play(HERO_SPRINT)); //
CHALLENGE 1
```

plus the ghost's `frameRate: 2`, the new `HERO_SPRINT` key in `assets.ts`, the two labels, the help line and `index.html`.

Look for: the new key added to `assets.ts` as a constant, not typed as a string in two places. Changing the frame rate *where the animation is made* rather than setting `timeScale` on the sprite - both work, but only one changes the animation for every sprite that plays it. Tested with the harness: key 7 plays `hero-sprint` at 20 fps; `game.anims.get("bat-flap").frameRate` is 24.

2. Coin run

Project: `solutions/ch07_challenge_2_coin_run` (from `ch07_hero_animations`)

Goal: load a second sprite sheet and animation; a simple distance test; a tween that cleans up after itself.

`coin_spin.png` is copied into `public/assets/spritesheets/`, loaded in `PreloadScene`, and given a looping `coin-spin` animation there. The scene makes the coins:

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: a row of spinning coins, alternately low and high
COIN_XS.forEach((x, i) => {
  const y = i % 2 === 0 ? COIN_LOW_Y : COIN_HIGH_Y;
  const coin = this.add.sprite(x, y, COIN_SHEET).setScale(COIN_SCALE);
  coin.play(COIN_SPIN);
  this.coins.push(coin);
});
```

and checks them every frame, measuring from the middle of the hero's body (its `y` is its feet):

```

private checkCoins(): void {
    const heroMiddleY = this.hero.y - this.hero.displayHeight / 2;
    for (const coin of [...this.coins]) {          // a copy: collect()
        removes coins from the list
        const distance = Phaser.Math.Distance.Between(this.hero.x, heroMiddleY,
        coin.x, coin.y);
        if (distance < PICKUP_DISTANCE) {
            this.collect(coin);
        }
    }
}

private collect(coin: Phaser.GameObjects.Sprite): void {
    this.coins = this.coins.filter((c) => c !== coin);
    this.collected = this.collected + 1;
    this.coinText.setText(`Coins: ${this.collected} / ${COIN_XS.length}`);

    this.tweens.add({
        targets: coin,
        y: coin.y - 80,
        alpha: 0,
        scale: COIN_SCALE * 1.5,
        duration: 500,
        ease: "Quad.easeOut",
        onComplete: () => {
            coin.destroy();
        },
    });
}

```

Look for:

- the coin taken **out of the list at once**, before its tween. The most common bug is to destroy it only in `onComplete` while it is still in the list: the next frames count it again, and one coin scores ten or twenty times
- `destroy()` in `onComplete` (or the faded coins stay in the scene)
- `init()` resetting the list and the count, in case the scene is restarted (Chapter 2's trap)
- measuring from the hero's `y` (its feet) without noticing - it half works, which makes it hard to spot. A good moment to revisit origins from Chapter 1

Tested with the harness: running along the ground collects the low coins; jumping collects the high ones; the count matches.

3. Title sequence

Project: `solutions/ch07_challenge_3_title_sequence` (from `ch07_tweens_and_particles`)

Goal: a `tweens.chain` of several steps, and making it safe to restart.

`src/scenes/PlaygroundScene.ts`

```
private playTitle(): void {
  this.tweens.killTweensOf(this.title);
  this.title.setPosition(TITLE_X,
    TITLE_START_Y).setScale(1).setAngle(0).setAlpha(1);

  this.tweens.chain({
    targets: this.title,
    tweens: [
      { y: TITLE_Y, duration: 1000, ease: "Bounce.easeOut" },
      { scale: 1.3, duration: 200, ease: "Quad.easeOut", yoyo: true },
      { angle: { from: -6, to: 6 }, duration: 100, yoyo: true, repeat: 3 },
      { angle: 0, duration: 60 },
      { alpha: 0, delay: 800, duration: 500 },          // delay: hold still,
    ],
    then fade
  });
  this.show("T: a chain of five tweens on the title");
}
```

The title is made once in `create()`, above the top of the screen, and T calls `playTitle()`.

- `{ from, to }` sets the start value as well as the end: the wobble starts at -6 degrees wherever the angle was
- `delay` on the last tween is the "hold for a moment"; a `hold` on the tween before would also work
- `killTweensOf` stops the chain as well as single tweens

Look for: the reset before the chain - without it, a second T part way through starts from wherever the title had got to (half faded, or tilted). Accept `onComplete` callbacks nested

inside each other, but show the chain as the tidier answer. Tested with the harness: T, then T again after half a second, leaves exactly one tween running and the title lands at `y = 110`.

4. Sparkle trail

Project: `solutions/ch07_challenge_4_sparkle_trail` (from `ch07_tweens_and_particles`)

Goal: a flowing emitter, following something, turned on and off.

`src/scenes/PlaygroundScene.ts`

```
private addTrail(): void {
  this.trail = this.add.particles(0, 0, PARTICLE_KEY, {
    speed: { min: 5, max: 40 },
    lifespan: 600,
    scale: { start: 1.5, end: 0 },
    alpha: { start: 0.8, end: 0 },
    tint: [0xa8dadc, 0xffffffff, 0xfffd166],
    blendMode: "ADD",
    frequency: 15,           // a burst every 15 milliseconds...
    quantity: 2,             // ...of two particles
    emitting: false,         // not until the pointer is in the playground
  });
  this.trail.startFollow(this.input.activePointer);

  // only in the playground: stop over the buttons (and when the pointer
  // leaves the game)
  this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
    this.trail.emitting = pointer.x > PLAY_LEFT;
  });
  this.input.on("gameout", () => {
    this.trail.emitting = false;
  });
}
```

- the explosion emitter uses `emitting: false` + `explode()`; this one **flows** (`frequency`, `quantity`) and is switched with `emitting`

- `startFollow` takes anything with an `x` and a `y` - the pointer, a sprite, a plain object
- particles already alive carry on when `emitting` goes false, so the trail fades out rather than vanishing

Look for: one emitter, made once. A common wrong answer calls

`this.add.particles(...)` inside `pointermove`, making a new emitter dozens of times a second - it works for a minute, then the game slows to a crawl. Another is

`emitter.stop() / start()` every move; it works, but `emitting` says what is meant.

Tested with the harness: moving across the playground gives 60+ live particles; moving onto the buttons sets `emitting` to false and they are all gone within a second.

5. Speed you can see

Project: `solutions/ch07_challenge_5_speed_you_can_see` (from `ch07_hero_animations`)

Goal: acceleration with `delta`, and an animation's playback speed driven by the game.

`src/objects/Hero.ts`

```

private readKeys(seconds: number): void {
    let direction = 0;
    if (this.cursors.left.isDown) {
        direction = -1;
    } else if (this.cursors.right.isDown) {
        direction = 1;
    }

    if (direction !== 0) {
        // pushing against the way the hero is going: brake AND accelerate, for
        a quick turn
        const turning = this.velocityX * direction < 0;
        const rate = turning ? ACCELERATION + DECELERATION : ACCELERATION;
        this.velocityX = Phaser.Math.Clamp(this.velocityX + direction * rate *
seconds, -RUN_SPEED, RUN_SPEED);
    } else {
        // slow down towards 0 - and stop exactly AT 0, not just past it
        const slowBy = DECELERATION * seconds;
        if (Math.abs(this.velocityX) <= slowBy) {
            this.velocityX = 0;
        } else {
            this.velocityX = this.velocityX - Math.sign(this.velocityX) * slowBy;
        }
    }
    ...
}

```

```

private matchStrideToSpeed(): void {
    if (this.currentState === "run") {
        this.anims.timeScale = Math.max(MIN_TIME_SCALE, Math.abs(this.velocityX)
/ RUN_SPEED);
    } else {
        this.anims.timeScale = 1;
    }
}

```

`matchStrideToSpeed()` is called at the end of `preUpdate()`. The state machine did not change: the hero is "run" while `velocityX` is not 0, and "idle" when it is.

Look for:

- slowing down that **snaps to exactly 0**. Without it the speed wobbles either side of zero for ever (-0.3, 0.2, -0.1...), the hero never becomes idle, and it twitches left and right as

the flip follows the sign

- `timeScale` put back to 1 in other states - otherwise the next animation keeps the last run speed (a jump after a slow start plays slowly)
- a minimum `timeScale` : at a speed of 2 pixels a second, a `timeScale` of 0.008 looks frozen
- `delta` used for both acceleration and deceleration

Tested with the harness: holding RIGHT, the speed goes 50, 140, 260 with `timeScale` 0.30, 0.54, 1.00; after letting go the hero is still `run` at 170, then `idle` at 0.

6. Stomp

Project: `solutions/ch07_challenge_6_stomp` (from `ch07_hero_animations`)

Goal: bring the chapter together - a second animated class, animation events, particles - and keep the state machine in charge of the hero.

A new `Slime` class (`src/objects/Slime.ts`) is a `Sprite` that patrols between two x positions in its own `preUpdate()` - calling `super.preUpdate()`. `PreloadScene` loads `slime.png`, `explosion.png` and `particle.png` and makes `slime-walk` and `explode`. `Hero` gains three things the scene can ask for:

`src/objects/Hero.ts`

```
// CHALLENGE 6: true while coming down from a jump - the only time a slime
// can be stomped
public isFalling(): boolean {
    return this.currentState === "fall";
}

// CHALLENGE 6: called by the scene after a stomp - spring back up off the
// slime
public bounceOff(): void {
    this.velocityY = -STOMP_BOUNCE;
    this.changeState("jump");
}
```

and `hurt(fromX?: number)`, which knocks the hero away from where the hurt came from. The scene decides what a touch means:

`src/scenes/GameScene.ts`

```
private checkSlimes(): void {
    const heroBounds = this.hero.getBounds();
    for (const slime of [...this.slimes]) {          // a copy: squash()
        removes slimes from the list
        if (!Phaser.Geom.Intersects.RectangleToRectangle(heroBounds,
            slime.getBounds())) {
            continue;
        }
        if (this.hero.isFalling()) {
            this.squash(slime);
            this.hero.bounceOff();
        } else {
            this.hero.hurt(slime.x);                // does nothing if
            already hurt
        }
    }
}

private squash(slime: Slime): void {
    this.slimes = this.slimes.filter((s) => s !== slime);
    const centreY = slime.y - slime.displayHeight / 2;

    const boom = this.add.sprite(slime.x, centreY, EXPLOSION_SHEET);
    boom.play(EXPLODE);
    boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
        boom.destroy();
    });
    this.sparks.explode(SPARKS, slime.x, centreY);

    slime.destroy();
    this.showSlimesLeft();
}
```

- "cannot be hurt again until the hurt state is over" comes free: `hurt()` already returns at once if the hero is hurt
- `isFalling()` is "fall", not "`velocityY > 0`": a hero knocked into the air by a hurt is falling too, but should not stomp

- `bounceOff()` goes through `changeState`, so the jump animation plays and the state machine stays the only thing that changes the hero's state

Look for:

- the scene **asking** the hero (`isFalling()`) and **telling** it (`bounceOff()` , `hurt()`), not setting `hero.velocityY` - the fields are private, so a student who tries gets `Property 'velocityY' is private and only accessible within class 'Hero'`. Some will make the fields public to get round it; this is the moment to talk about why not
- a separate `Slime` class, rather than slime movement written into the scene's `update()`
- the explosion sprite destroying itself, and one emitter reused for every squash
- the slime list copied before looping (`[...this.slimes]`), because squashing removes from it

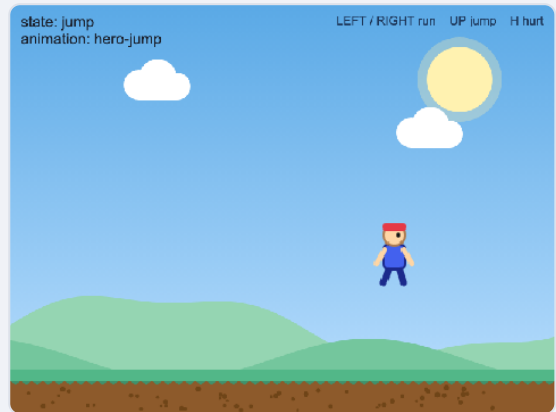
Tested with the harness: dropping the hero onto a slime removes it, bounces the hero into `jump`, and shows "Slimes left: 2"; walking into a slime gives `hurt` with the knockback away from it. With the keys, running left and jumping after 0.1 seconds lands on the first slime.

Chapter 7 - Animations - Slides

Chapter 7

Animations

Frames, tweens and particles



1 / 20

Today

- sprite sheets become **animations**: `this.ans.create`
- playing, chaining and listening to animations
- `setFlipX`, and a **state machine** for the hero
- **tweens**: smooth changes to any number
- easing, chains, stagger, counters
- **particles**: bursts of sparks

2 / 20

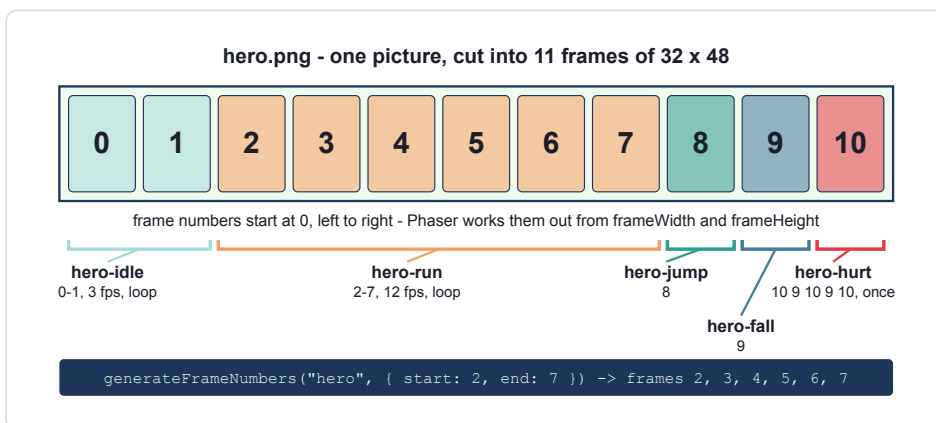
Two kinds of animation

Frame animation	Tween
changes which picture a sprite shows	changes a number smoothly
frames from a sprite sheet	<code>x</code> , <code>y</code> , <code>scale</code> , <code>alpha</code> , <code>angle</code> , ...
a running hero, a spinning coin	a sliding menu, a fading popup
<code>this.ans</code> , <code>sprite.play(key)</code>	<code>this.tweens.add({...})</code>

A character usually needs both.

3 / 20

Sprite sheets



```
this.load.spritesheet(HERO_SHEET, HERO_FILE, { frameWidth: HERO_WIDTH, frameHeight: HERO_HEIGHT });
```

4 / 20

Making an animation

```
this.anims.create({
  key: HERO_RUN, // the animation's own name
  frames: this.anims.generateFrameNumbers(HERO_SHEET, { start: 2, end: 7 }),
  frameRate: 12, // frames per second
  repeat: -1, // -1: for ever
});

this.anims.create({
  key: HERO_HURT,
  frames: this.anims.generateFrameNumbers(HERO_SHEET, { frames: [10, 9, 10, 9, 10] }),
  frameRate: 8, // no repeat: plays once, then COMPLETES
});
```

Also: `yoyo: true`, `repeatDelay`, `delay`, `duration`

5 / 20

Once, for the whole game - on a Sprite

- `this.anims` is the **game's** animation manager, in every scene
- make animations **once**, in the preload scene
- twice? `AnimationManager` key already exists: `hero-idle`

```
const sprite = this.add.sprite(x, 72, sheet); // a Sprite can play
sprite.play(anim);

this.add.image(x, 505, HERO_SHEET, frame) // an Image shows ONE frame
```

`image.play(...)` - Property 'play' does not exist on type 'Image'

6 / 20

play restarts

```
// play() starts the animation again from its first frame - even if it is already playing
keyboard.on("keydown-TWO", () => this.hero.play(HERO_RUN));
// play(key, true): "ignore this if that animation is already playing"
keyboard.on("keydown-THREE", () => this.hero.play(HERO_RUN, true));
```

- `play(key)` in `update()` = stuck on the first frame for ever
- lab: hold **2**, then hold **3**

7 / 20

Controlling, and listening

```
this.hero.play(HERO_HURT);
this.hero.chain(HERO_IDLE); // then idle, automatically

keyboard.on("keydown-A", () => this.hero.anims.playAfterRepeat(HERO_IDLE));
this.hero.anims.timeScale = Math.min(this.hero.anims.timeScale * 2, MAX_TIME_SCALE);

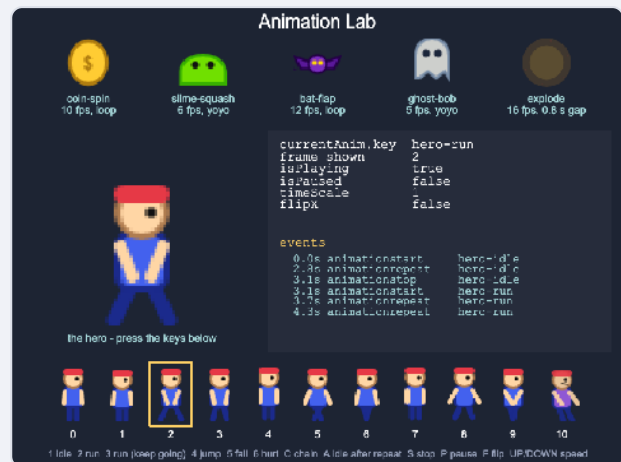
this.hero.on(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, () => {
    this.addToLog(" (hurt is over)");
});
```

repeat: -1 never completes - no `ANIMATION_COMPLETE`

8 / 20

Try it: the Animation Lab

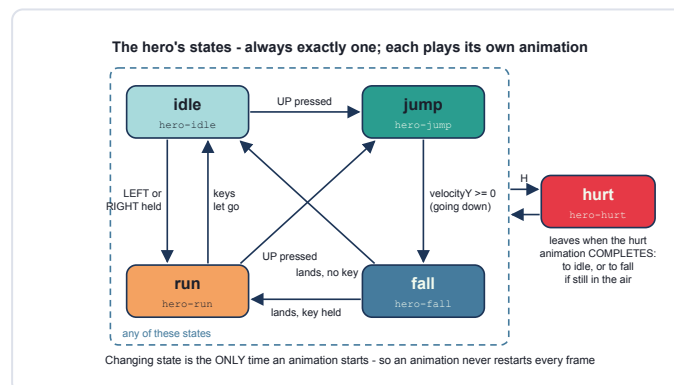
- keys **1-6**: play each animation
- **C** chain, **A** after repeat, **S** stop, **P** pause
- **F** flip, **UP/DOWN** speed
- watch the **event log**



9 / 20

Facing both ways - and a state machine

```
if (this.velocityX !== 0) {
  this.setFlipX(this.velocityX < 0); // frames face right; mirrored = left
}
```



10 / 20

States in code

```
export type HeroState = "idle" | "run" | "jump" | "fall" | "hurt";

const ANIMATIONS: Record<HeroState, string> = {
  idle: HERO_IDLE,
  run: HERO_RUN,
  ...
};

private changeState(next: HeroState): void {
  if (next === this.currentState) {
    return; // already in that state: do NOT restart its animation
  }
  this.currentState = next;
  this.play(ANIMATIONS[next]);
}
```

The **only** place the state - and the animation - changes

11 / 20

Hurt: ended by the animation

```
public hurt(): void {
  ...
  this.changeState("hurt");
  this.setTint(HURT_TINT);

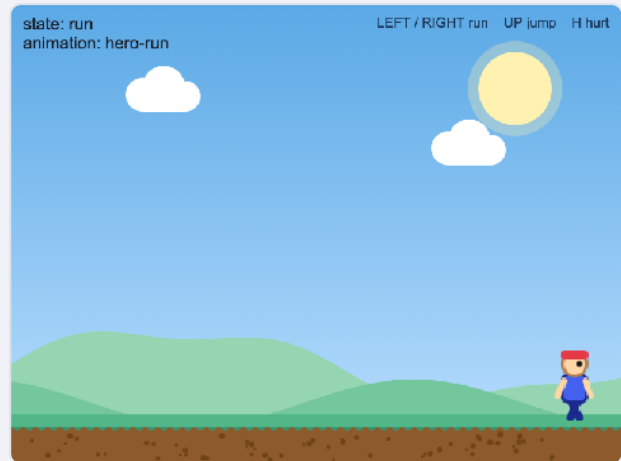
  this.once(Phaser.Animations.Events.ANIMATION_COMPLETE_KEY + HERO_HURT, () => {
    this.clearTint();
    this.changeState(this.isOnGround() ? "idle" : "fall");
  });
}

protected override preUpdate(time: number, delta: number): void {
  super.preUpdate(time, delta); // leave out: frozen on one frame, NO error
}
```

12 / 20

Try it: Hero Animations

- LEFT / RIGHT, UP, and H
- watch the state at the top
- change the hurt `frameRate` - what else changes?
- delete `super.preUpdate` - then put it back



13 / 20

Tweens

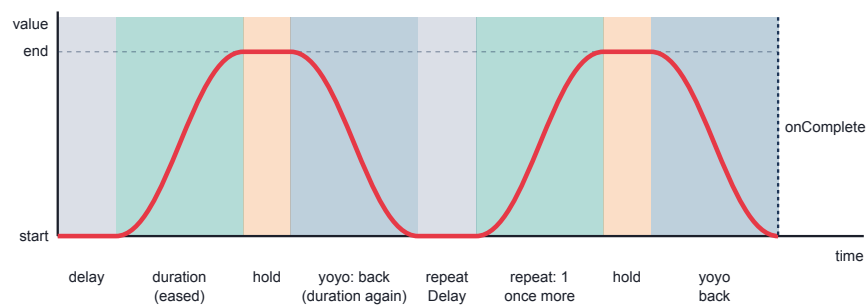
```
this.tweens.add({
  targets: this.crate,           // one object, or an array
  x: this.restX,                 // the values, and where they end up
  y: this.restY,
  duration: 800,                 // milliseconds
  ease: "Cubic.easeInOut",
});
```

- returns at once; Phaser does the changing, every frame
- relative values: `angle: "+=360"`
- `onComplete: () => popup.destroy()` - effects clean up after themselves

14 / 20

Yoyo, hold, repeat, delay

One tween, over time: { delay, duration, hold, yoyo: true, repeat: 1, repeatDelay }



repeat: 2 = three times in all. repeat: -1 = for ever.

15 / 20

Easing

- easeIn - slow start (leaving)
- easeOut - slow finish (arriving)
- easeInOut - both (moving)
- Back, Elastic, Bounce - character
- misspelled ease = **linear**, silently



16 / 20

Chains, stagger, counters

```
this.tweens.chain({
  targets: this.crate,
  tweens: [
    { y: this.restY - 120, duration: 400, ease: "Quad.easeOut" },
    { angle: 360, duration: 500, ease: "Cubic.easeInOut" },
    { y: this.restY, duration: 700, ease: "Bounce.easeOut" },
    ...
  ],
});
```

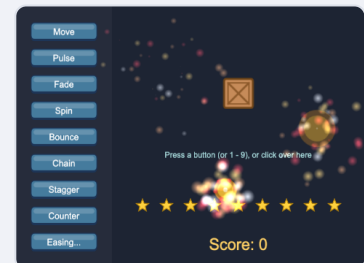
- **stagger**: one tween, many targets - `delay: this.tweens.stagger(60),`
- **counter**: a plain number - `this.tweens.addCounter({ from, to, onUpdate })`
- **no fighting**: `this.tweens.killTweensOf(this.crate);` before starting another

17 / 20

Particles

```
this.sparks = this.add.particles(0, 0, PARTICLE_KEY, {
  speed: { min: 80, max: 360 }, // random in a range
  ...
  scale: { start: 2.5, end: 0 }, // changes over its life
  ...
  tint: [0xffd166, 0xf4a261, 0xe63946, 0xffffffff], // one of these
  ...
  emitting: false, // made ONCE, silent...
});
this.sparks.explode(SPARKS, x, y); // ...until a burst

boom.once(Phaser.Animations.Events.ANIMATION_COMPLETE, () => {
  boom.destroy(); // the explosion sprite
  removes itself
});
```



18 / 20

Summary

- `this.anims.create + generateFrameNumbers` - once, for the game
- only a `Sprite` plays; `play(key, true); chain, playAfterRepeat, timeScale`
- animation **events**; `ANIMATION_COMPLETE` needs a finite `repeat`
- `super.preUpdate` in a `Sprite` subclass
- a **state machine**: one state, one place that changes it
- `this.tweens.add / chain / stagger / addCounter / killTweensOf`
- `this.add.particles(...), emitting: false, explode(n, x, y)`

19 / 20

Challenges

1. **Different speeds** - new frame rates; a `hero-sprint` on key 7
2. **Coin run** - spinning coins to collect, with a floating tween
3. **Title sequence** - one `tweens.chain`, restartable
4. **Sparkle trail** - particles that follow the pointer
5. **Speed you can see** - acceleration; stride matched with `timeScale`
6. **Stomp** - patrolling slimes: squash from above, hurt from the side

Next: **Chapter 8 - Collisions**

20 / 20



Collisions

Overview

Students learn to detect and respond to things touching, in two stages. First **by hand**: shapes from `Phaser.Geom`, the tests in `Phaser.Geom.Intersects`, bounding boxes from `getBounds()`, and a circular hit area for clicks - enough to understand what any collision system does underneath, and why rectangles are unfair to round things. Then **Arcade Physics**: turning it on, giving game objects bodies, fitting bodies to pictures, and the central choice of the chapter - `collider` (separate and bounce) or `overlap` (just report) - with groups, callbacks, process callbacks, and switching collisions and bodies off. The chapter ends with a complete breakout game.

This is the first chapter that uses a physics engine. Keep to *collisions*: velocity is used, and bounce and immovable bodies are mentioned, but Chapter 9 covers movement physics (acceleration, drag, gravity, mass) properly. When students ask "how do I make it slide / fall / speed up?", point them forward.

Prerequisites

- Chapter 1: game objects, `update()`, speeds in pixels per second
- Chapter 2: `setInteractive`, `pointerdown`, the reused-scene trap and `init()`
- Chapter 7: tweens (used for the player's flashing), and `delayedCall` from earlier chapters
- Java: casts, `instanceof`, `HashSet` - all have close TypeScript equivalents here

Learning outcomes

Students can:

1. test whether two rectangles, two circles, or a circle and a rectangle overlap with `Phaser.Geom.Intersects`, and explain the maths behind the first two

2. explain why bounding rectangles are unfair to round objects, and give a circle a circular hit area with `setInteractive({ hitArea, hitAreaCallback })`
3. turn on Arcade Physics, give game objects bodies (`this.physics.add.image/sprite/existing`), and use `debug` to see them
4. fit a body to its picture with `setSize`, `setOffset` and `setCircle`
5. choose correctly between `collider` and `overlap`, for single objects and for groups (`group`, `staticGroup`), and use the objects passed to the callback (with `as`)
6. use a process callback to decide whether a collision counts; switch collisions off with `collider.active`, and bodies with `disableBody` / `enableBody`

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play <code>ch08_breakout</code> for a minute. Ask: "how many different collisions can you name in this game? Which ones bounce, and which ones just make something happen?"
0:05 - 0:20	Slides 3-7: geometry shapes, the two tests (draw the four rectangle comparisons on the board), <code>getBounds</code> , testing every pair, unfair corners, a circular hit area. Demo <code>ch08_hand_made_collisions</code>
0:20 - 0:25	Live demo: in the shapes project, arrange two circles corner to corner, then press SPACE. "Would you accept that as a hit in a game?"
0:25 - 0:35	Slides 8-10: Arcade Physics, bodies, <code>debug</code> , <code>setSize</code> / <code>setOffset</code> / <code>setCircle</code> . Turn <code>debug: true</code> on in <code>ch08_collect_and_avoid</code> and rebuild
0:35 - 0:50	Slides 11-15: collider vs overlap, groups, callbacks and <code>as</code> , switching off. Live demo: replace the stars' overlap with <code>this.physics.add.collider(this.player, this.stars)</code> - no callback - and play: the player shoves the stars, which slide away across the screen. Then put the callback back on the collider: the star is still collected, after a one-frame bump. Ask why the overlap is still the right choice
0:50 - 1:00	Slides 16-18: breakout - the process callback and the paddle angle. Slides 19-20: summary and challenges

Lab (2 hours)

Time	Activity
0:00 - 0:20	Build and play all three projects. Answer the "What to look at" questions in each README
0:20 - 0:40	Challenge 1 (debug key) and 2 (hover highlight)
0:40 - 1:20	Challenges 3 and 4 (tough bricks, wide paddle)
1:20 - 2:00	Challenges 5 and 6 (glass bricks, fair triangles) for those who are ready; the extension ideas below for the rest

Key points to stress

- a collision test is **just geometry**. Arcade Physics does the same tests as Part 1 - it only does them for you, efficiently, and responds to them. Students who understand Part 1 debug Part 2 much faster
- **the body is not the picture**. Every "it hit me but it didn't touch me" bug is a body that does not match what the player sees. The first move is always `debug: true`
- **collider or overlap** is a design decision: *should these two push each other apart?* Walls, crates, bricks, paddles: yes. Pickups, hazards, trigger zones: no
- colliders are set up **once**, in `create()`, not every frame in `update()`
- an overlap callback fires **every frame** the two things touch - students must design for that (invulnerability, disabling the pickup's body, a flag)
- move bodies with **velocity**, not by setting `x / y` - otherwise the physics engine cannot stop them at walls. (The paddle following the mouse is the exception, and the chapter says why it is safe)
- **static bodies do not follow their game objects**: `refreshBody()` after moving or scaling one

Common problems and errors

What students see	Cause	Fix
<code>TypeError: Cannot read properties of undefined (reading 'add') at create , on this.physics.add...</code>	no <code>physics</code> section in the game config	add <code>physics: { d "arcade", arcade: }</code> to <code>main.ts</code>
<code>TypeError: Cannot read properties of null (reading 'setSize') in a class that extends Phaser.Physics.Arcade.Image</code>	extending the Arcade class does not make a body; <code>scene.physics.add.existing(this)</code> is missing (or comes after the <code>setSize</code>)	call <code>scene.physics.ad</code> in the constructor, k method
<code>TS2339 [ERROR]: Property 'disableBody' does not exist on type 'Body StaticBody Tile GameObjectWithBody'.</code>	using a callback parameter directly	cast it: <code>star as Phaser.Physics.Ar</code>
<code>TS2531 [ERROR]: Object is possibly 'null'. on this.player.body.velocity</code>	a game object's <code>body</code> is typed <code>... null</code>	<code>this.player.body</code> use the object's ow (<code>setVelocity</code>)
The player bumps or pushes a pickup instead of collecting it	<code>collider</code> used where <code>overlap</code> was meant	use <code>overlap</code> for pi
All lives lost at once when touching an enemy	the overlap callback runs every frame they touch	invulnerability: <code>col false</code> for a while, c callback
The player stops short of a crate, or walks into its edge	a static body scaled or moved without <code>refreshBody()</code> , or a body the wrong size	<code>refreshBody()</code> ; t
The body is offset from the sprite in the wrong direction	<code>setOffset</code> / <code>setCircle</code> offsets measured from the centre by mistake	they are measured top-left corner
A click just outside a circle still picks it up	default rectangular hit area	<code>hitArea: new Phaser.Geom.Circle</code> <code>hitAreaCallback: Phaser.Geom.Circle</code> the centre is <code>(r, r</code>

What students see	Cause	Fix
A process callback seems to do nothing - everything still collides	the callback forgot <code>return</code> . Phaser only skips a collision when it gets exactly <code>false</code> , and <code>undefined</code> is not <code>false</code> . TypeScript does not warn: Phaser types the callback as returning <code>void</code>	<code>return</code> the boolean
With debug on, outlines of bricks already knocked out are still drawn (breakout)	the bricks were removed with <code>disableBody</code> : a disabled <i>static</i> body no longer collides, but the debug drawing still shows it	harmless, but confusing bricks that will never be the chapter project
After a restart, debug drawing is off again (challenge 1)	each scene start gets a new physics world	expected; store the registry if it should persist

Discussion questions

- The shapes project tests 15 pairs a frame. How many would a bullet-hell game with 500 bullets and one player need, if bullets never hit each other? (500 - collisions *between* groups are far cheaper than every object against every other.) Why does `collider(enemies, enemies)` cost more than `collider(player, enemies)`?
- For each of these, collider or overlap: a door you walk through to change room; a spring that launches you; a moving platform; a checkpoint flag; a bullet hitting a wall; a bullet hitting an enemy
- The enemy body is smaller than the triangle. Is it fair? Who benefits? Should an enemy bullet's body be smaller or bigger than its picture? What about a coin's?
- Stars are disabled and re-enabled; bricks are destroyed. When is each better? (Reuse vs. gone for good; Chapter 14 comes back to this as object pools.)
- The paddle sets its `x` directly when following the mouse, but the player never does. Why is it safe for the paddle but not for the player?

Extension ideas

- in the shapes project, add a `Phaser.Geom.Triangle` shape and the three new tests it needs (`RectangleToTriangle` , `TriangleToCircle` , `TriangleToTriangle`)
- breakout: a second ball ("multi-ball") - one more collider line per ball, or put the balls in a group so the existing colliders just work
- collect and avoid: a bomb pickup that clears every enemy on screen (`this.enemies.clear(true, true)`)
- collision **categories** (`this.physics.nextCategory()` , `setCollisionCategory` , `setCollidesWith`) as an alternative to choosing which pairs get colliders
- breakout: levels read from an array of strings (`"XX..XX"`), a preview of tilemaps (Chapter 10)

Assessment ideas

- Practical: "add spikes to `ch08_collect_and_avoid` - a static group of hazards that cost a life when touched, but that enemies bounce off"
- Code reading: show a scene with `this.physics.add.collider(...)` inside `update()` and ask what is wrong and what happens as the game runs (a new collider every frame - it gets slower and slower, and callbacks fire many times)
- Short answer: explain the difference between a collide callback and a process callback, with an example of a game rule that needs a process callback
- Paper exercise: given two circles `(100, 100, r 30)` and `(150, 140, r 25)` , do they overlap? Do their bounding boxes? (Distance $64.0 > 55$, so no; the boxes overlap: yes)

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 8 - Collisions - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. Debug key

Project: `solutions/ch08_challenge_1_debug_key` (from `ch08_collect_and_avoid`)

Goal: find the physics world, and learn that `debug` is a setting you can change while the game runs.

`src/scenes/GameScene.ts`

```
this.input.keyboard!.on("keydown-D", () => {
  this.toggleDebug();
});
...
private toggleDebug(): void {
  const world = this.physics.world;
  if (!world.debugGraphic) {
    world.createDebugGraphic();
  } else {
    world.drawDebug = !world.drawDebug;
    world.debugGraphic.clear();
  }
  this.debugText.setVisible(world.drawDebug);
}
```

With `debug: false` in the config, the world has no `Graphics` to draw on, so the first press calls `createDebugGraphic()`, which makes one and turns `drawDebug` on. After that, `drawDebug` is flipped. The `clear()` matters: the world clears and redraws the debug graphic every frame *while* `drawDebug` is on - switch it off without clearing, and the last frame's outlines stay frozen on screen.

Look for: the frozen-outlines bug (very common - ask the student to toggle it off while things are moving); the "DEBUG" label kept in step with `world.drawDebug`, not with a separate flag that can drift. A student who sets `debug: true` in the config and then only

flips `drawDebug` has a valid answer, but the game no longer starts with it off. After game over and SPACE the debug drawing is off again: each scene start makes a new physics world. Tested with the harness: D, D, D, then a game over and restart.

2. Hover highlight

Project: `solutions/ch08_challenge_2_hover_highlight` (from `ch08_hand_made_collisions`)

Goal: point-in-shape tests by hand, with the right test for each shape.

`src/geometry.ts`

```
export function contains(shape: DragShape, x: number, y: number): boolean {
  if (shape instanceof Phaser.GameObjects.Arc) {
    return Phaser.Geom.Circle.Contains(circleOf(shape), x, y);
  }
  return Phaser.Geom.Rectangle.Contains(shape.getBounds(), x, y);
}
```

`src/scenes/ShapesScene.ts` (called from `update()`)


```

private showHover(): void {
  const pointer = this.input.activePointer;
  let hovered: DragShape | undefined = undefined;

  for (const shape of this.shapes) {
    if (contains(shape, pointer.x, pointer.y)) {
      if (hovered === undefined || this.children.getIndex(shape) >
this.children.getIndex(hovered)) {
        hovered = shape;
      }
    }
  }

  for (const shape of this.shapes) {
    if (shape === hovered) {
      shape.setStrokeStyle(HOVER_OUTLINE_WIDTH, HOVER_OUTLINE_COLOUR);
    } else {
      shape.setStrokeStyle(); // no line width: no outline
    }
  }
}

```

`this.input.activePointer` is the mouse (or the last finger), in game coordinates. When shapes overlap, only the one drawn on top is outlined - the one a click would pick up - found with its index in the scene's display list.

Look for: a circle test for circles (the challenge says the corner must not count - test it by hovering just outside a circle, diagonally); outlines removed from the other shapes. Highlighting *every* shape under the pointer is an acceptable simpler answer. Doing it in `"pointermove"` instead of `update()` works while the mouse moves, but misses a shape being dragged *under* a still pointer. Tested with the harness: centre of the big circle (outlined), its box corner (nothing), a box, and a circle dragged on top of a box (the circle wins).

3. Tough bricks

Project: `solutions/ch08_challenge_3_tough_bricks` (from `ch08_breakout`)

Goal: keep state on each game object, and let a collision callback decide what a hit means.

`src/scenes/BreakoutScene.ts` - when the bricks are made

```
const tough = row < TOUGH_ROWS;
brick.setData(HITS_LEFT, tough ? TOUGH_HITS : 1);
brick.setData(POINTS, tough ? TOUGH_POINTS : POINTS_PER_BRICK);
```

and in `hitBrick()`

```
const hitsLeft = (brick.getData(HITS_LEFT) as number) - 1;
if (hitsLeft > 0) {
  // still standing: remember the hit, and look cracked
  brick.setData(HITS_LEFT, hitsLeft);
  brick.setTint(CRACKED_COLOUR);
  return;
}

// CHALLENGE 3: read the points BEFORE destroy() - destroying a game object
// clears its data too
this.score += brick.getData(POINTS) as number; // was POINTS_PER_BRICK

// gone for good: destroy() removes the brick from the scene, its body from
// the physics world,
// and the brick from the group
brick.destroy();
```

Every game object has a **data manager**: `setData(name, value)` and `getData(name)` store values of your own on the object itself - no subclass needed. The ball still bounces off a tough brick on the first hit, because the collider does the bounce before the callback runs.

Look for: the order of `getData` and `destroy()` - reading the points after destroying the brick gives `undefined`, and the score becomes `NaN` - this happened while writing the solution, and only playing it showed the problem. A `Brick` subclass with a `hitsLeft` field is an equally good, more OO answer - it needs `classType: Brick` on the group, or bricks made with `new Brick(...)` and `this.bricks.add(brick)`. Tested with the harness: the ball dropped onto a top-row brick twice - first hit leaves it (cracked colour, `hitsLeft 1`), second removes it and scores 30.

4. Wide paddle

Project: `solutions/ch08_challenge_4_wide_paddle` (from `ch08_breakout`)

Goal: a new dynamic group created during play, an overlap, a timer, and a body that changes size.

`src/scenes/BreakoutScene.ts`

```
this.powerUps = this.physics.add.group();
this.physics.add.overlap(this.paddle, this.powerUps, (_paddle, powerUp) => {
  // the paddle was passed first, so the power-up is the second object
  (powerUp as Phaser.Physics.Arcade.Sprite).destroy();
  this.sound.play(PADDLE_SOUND);
  this.paddle.widen(WIDE_FACTOR, WIDE_TIME);
});
...
private dropPowerUp(x: number, y: number): void {
  const powerUp = this.powerUps.create(x, y, POWER_UP_KEY) as
  Phaser.Physics.Arcade.Sprite;
  powerUp.setVelocityY(POWER_UP_SPEED);
}
```

`src/objects/Paddle.ts`

```
public widen(factor: number, duration: number): void {
  this.setScale(factor, 1);
  this.narrowTimer?.remove(); // `?.` - only call remove() if there is a
  timer
  this.narrowTimer = this.scene.time.delayedCall(duration, () => {
    this.setScale(1, 1);
  });
}
```

`hitBrick()` calls `dropPowerUp(brick.x, brick.y)` one time in five, and `update()` destroys any power-up that has fallen below the screen (looping over a copy of the group's list, because `destroy()` removes items from it). The picture is `gem.png`, copied from the asset library.

Look for:

- an **overlap**, not a collider - with a collider the paddle (immovable) would bounce the gem away
- the body grows with the picture: a *dynamic* body follows its game object's scale by itself. The harness confirms the paddle's body goes from 104 to 156 pixels wide and back. (A static body would need `refreshBody()`)
- `moveTo` and `bounceOff` already use `displayWidth` , so the wider paddle clamps and aims correctly with no other changes - worth pointing out as a reward for not hard-coding the width
- a second power-up caught while wide should not end the effect early: removing the old timer handles that. A common bug is stacking timers, so the first one to fire shrinks the paddle while the second power-up's time is still running
- missed power-ups destroyed, not left falling for ever

Tested with the harness: a power-up caught (scale 1.5, body 156), one missed (destroyed), a second catch 6 seconds later keeps the paddle wide past the first timer's end, and it returns to normal 10 seconds after the second catch.

5. Glass bricks

Project: `solutions/ch08_challenge_5_glass_bricks` (from `ch08_breakout`)

Goal: a process callback that does something, then cancels the collision.

`src/scenes/BreakoutScene.ts`

```

this.physics.add.collider(
  this.ball,
  this.bricks,
  (_ball, brick) => {
    // Phaser's types allow for a body or a tile too; we passed the bricks
    group, so it is one
    // of its members - and a group makes Arcade Sprites unless told
    otherwise
    this.hitBrick(brick as Phaser.Physics.Arcade.Sprite);
  },
  // CHALLENGE 5: a process callback. Glass is smashed HERE, and false tells
  Phaser not to
  // collide - so no bounce, and the collide callback above is not called
  for glass. Every
  // other brick answers true, and collides as before.
  (_ball, brick) => {
    const hit = brick as Phaser.Physics.Arcade.Sprite;
    if (hit.getData(GLASS) === true) {
      this.hitBrick(hit);
      return false;
    }
    return true;
  },
);

```

and, when the bricks are made, the glass row is marked:

```

if (row === GLASS_ROW) {
  brick.setTint(GLASS_COLOUR);
  brick.setAlpha(GLASS_ALPHA);
  brick.setData(GLASS, true);
}

```

The process callback runs *before* separation. Returning `false` means no separation, no bounce and no collide callback - so the glass must be smashed inside the process callback itself.

Look for:

- `return true` for ordinary bricks. Without it the callback returns `undefined` for them, which Phaser treats as "collide" (it only skips on exactly `false`) - so this bug is invisible here, but it is sloppy and TypeScript does not catch it

- smashing in the collide callback instead: when the process callback returns `false` the collide callback never runs, so glass is never smashed - the ball passes through it and it stays there for ever. (And if the process callback returns `true` for glass, the ball bounces)
- an alternative some students find: an `overlap` for glass and a `collider` for the rest, with the bricks split into two groups. Perfectly good - and worth comparing with the process callback version

Tested with the harness: with the bricks above and below a glass brick removed, a ball fired up through it smashes it and keeps going up (velocity still -420); an ordinary brick still bounces the ball back.

6. Fair triangles

Project: `solutions/ch08_challenge_6_fair_triangles` (from `ch08_collect_and_avoid`)

Goal: combine both halves of the chapter: physics finds the candidates, geometry decides.

`src/scenes/GameScene.ts`

```

this.enemyOverlap = this.physics.add.overlap(
  this.player,
  this.enemies,
  () => {
    this.hurtPlayer();
  },
  (_player, enemy) => {
    return this.touchesTriangle(enemy as Phaser.Physics.Arcade.Sprite);
  },
);
...
// CHALLENGE 6: does the player's body really touch this enemy's red
// triangle?
private touchesTriangle(enemy: Phaser.Physics.Arcade.Sprite): boolean {
  // the enemy picture's top-left corner, in the game (its origin is its
  // centre)
  const left = enemy.x - enemy.width / 2;
  const top = enemy.y - enemy.height / 2;
  const [ax, ay, bx, by, cx, cy] = TRIANGLE_POINTS;
  const triangle = new Phaser.Geom.Triangle(left + ax, top + ay, left + bx,
  top + by, left + cx, top + cy);

  // the player's body, as a rectangle. body.x and body.y are its top-left
  // corner
  const body = this.player.body!;
  const box = new Phaser.Geom.Rectangle(body.x, body.y, body.width,
  body.height);

  return Phaser.Geom.Intersects.RectangleToTriangle(box, triangle);
}

```

and each enemy's body now covers the whole triangle:

```

enemy.setSize(40, 38);
enemy.setOffset(4, 4);

```

The body is a cheap first test ("close enough to be worth checking"); the process callback does the exact test, and only a real touch reaches `hurtPlayer()`. This two-stage idea - a quick **broad phase**, then an exact **narrow phase** - is how every physics engine works.

Look for:

- the body made **bigger** to cover the whole triangle. Keeping the small body means real touches near the triangle's bottom corners are never even checked - the most common half-right answer
- the test in the process callback, not in the overlap callback. In the overlap callback it can still work (`if (!this.touchesTriangle(...)) return;`), which is acceptable; the process callback is the idiomatic place
- the triangle built in game coordinates from the picture's top-left corner (`x - width / 2`), not from its centre
- the player's shape: its body rectangle is the fair choice (it matches the blob). Using `this.player.getBounds()` - the whole 48 x 48 picture - reintroduces unfair corners

Tested with the harness, against the chapter project: an enemy placed so only an empty corner of its picture overlaps the player costs no life in either; one placed so the player touches the triangle's bottom-left corner costs a life in the solution but not in the chapter project (the small body misses it); a full overlap costs a life in both.

Chapter 8 - Collisions - Slides

Chapter 8

Collisions

When this touches that



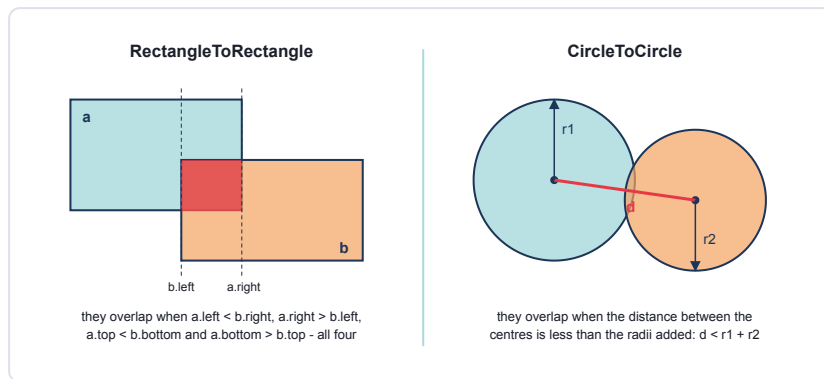
1 / 20

Today

- collisions **by hand**: `Phaser.Geom` shapes and `Intersects` tests
- why rectangles are **unfair** to round things
- **Arcade Physics**: bodies, and `debug` to see them
- fitting a body to its picture
- `collider` or `overlap` ? Groups against groups
- process callbacks, and switching collisions off
- a complete **breakout**

2 / 20

Shapes made of numbers



Phaser.Geom.Rectangle , Circle , Triangle ... - never drawn.

Phaser.Geom.Intersects.RectangleToRectangle(a, b) , CircleToCircle , CircleToRectangle

3 / 20

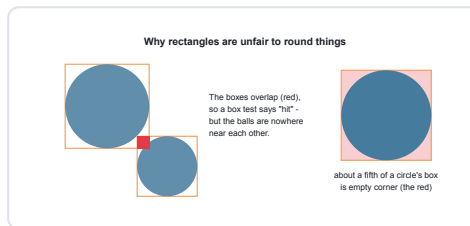
Which test?

```
if (a instanceof Phaser.GameObjects.Arc && b instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToCircle(circleOf(a), circleOf(b));
}
if (a instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToRectangle(circleOf(a), b.getBounds());
}
if (b instanceof Phaser.GameObjects.Arc) {
    return intersects.CircleToRectangle(circleOf(b), a.getBounds());
}
return intersects.RectangleToRectangle(a.getBounds(), b.getBounds());
```

- `getBounds()` - any game object's **bounding box**
- `instanceof` **narrows** the type: `a.radius` is allowed inside the `if`
- every pair, once: `for (j = i + 1 ...)` - 6 shapes, 15 pairs; 100 shapes, 4,950

4 / 20

Unfair corners



Round things: test **circles**. Boxy things: rectangles are exact.

5 / 20

Clicking a circle

```
circle.setInteractive({
  hitArea: new Phaser.Geom.Circle(radius, radius, radius),
  hitAreaCallback: Phaser.Geom.Circle.Contains,
  draggable: true,
  useHandCursor: true,
});
```

- default hit area: the **rectangle** - a click in the corner picks it up
- hit area coordinates: `(0, 0)` is the **top-left**, so the centre is `(radius, radius)`
- `Contains(shape, x, y)` - on every `Phaser.Geom` class

6 / 20

Try it: Hand-made Collisions

- build, serve, drag the shapes together
- press SPACE: bounding boxes only. Find a "hit" that is not a hit
- click just outside a circle, diagonally. Does it move?



7 / 20

Arcade Physics

```
// main.ts
physics: {
  default: "arcade",
  arcade: {
    debug: false,    // true: draw every body
  },
},
```

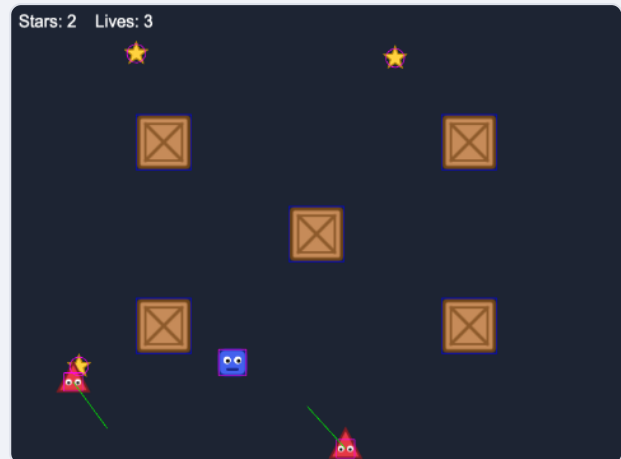
```
this.physics.add.image(x, y, key);    // a new Image, with a body
this.physics.add.sprite(x, y, key);    // a new Sprite, with a body
this.physics.add.existing(gameObject); // a body for an object you have
```

8 / 20

Bodies - and seeing them

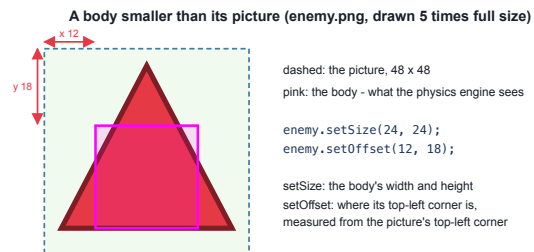
```
export class Player extends
Phaser.Physics.Arcade.Image {
  constructor(scene: Phaser.Scene, x:
number, y: number) {
    super(scene, x, y, PLAYER_KEY);
    scene.add.existing(this);
    scene.physics.add.existing(this);
```

- move with **velocity**:
 `this.setVelocity(speedX, speedY)`
- pink: dynamic bodies; blue: static



9 / 20

Fitting the body



```
star.setCircle(12, 4, 5); // radius, then offset from the top-left
```

10 / 20

Collider or overlap?

this.physics.add.collider(a, b)

1. a moves into b

2. Phaser pushes them apart



separates them, changes their velocities (bounce), THEN calls your callback

walls, crates, bricks, the paddle

this.physics.add.overlap(a, b)

a moves into b - and nothing is pushed



only calls your callback, every frame that they still touch

pickups, hazards, trigger areas

Should these two push each other apart?

11 / 20

All the collisions - set up once, in `create()`

```
this.physics.add.collider(this.player, this.crates);
this.physics.add.collider(this.enemies, this.crates);
this.physics.add.collider(this.enemies, this.enemies); // a group against itself

this.physics.add.overlap(this.player, this.stars, (_player, star) => {
  this.collectStar(star as Phaser.Physics.Arcade.Sprite);
});
this.enemyOverlap = this.physics.add.overlap(this.player, this.enemies, () => {
  this.hurtPlayer();
});
```

No collision code in `update()` at all

12 / 20

Groups

```
this.crates = this.physics.add.staticGroup();    // never move
...
this.enemies = this.physics.add.group({          // dynamic
  bounceX: 1,
  bounceY: 1,
  collideWorldBounds: true,
});
...
const crate = this.crates.create(x, y, CRATE_KEY) as Phaser.Physics.Arcade.Sprite;
crate.setScale(CRATE_SCALE);
crate.refreshBody();    // a static body does not follow its game object
```

13 / 20

Callbacks, casts, and switching off

TS2339: Property 'disableBody' does not exist on type 'Body | StaticBody | Tile | GameobjectWithBody' -> star as Phaser.Physics.Arcade.Sprite

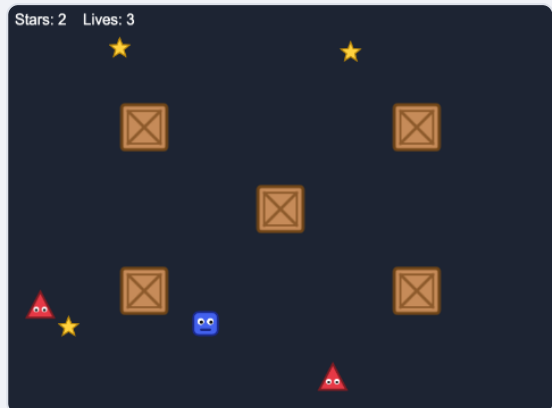
```
star.disableBody(true, true);    // body off, hidden
...
star.enableBody(true, spot.x, spot.y, true, true);    // back, somewhere else
...
this.enemyOverlap.active = false;    // stop checking for a while
...
this.enemyOverlap.active = true;    // ...and start again
```

An overlap fires **every frame** they touch

14 / 20

Try it: Collect and Avoid

- arrow keys; stars in, triangles out
- set `debug: true` in `main.ts`, rebuild
- replace the stars' overlap with `this.physics.add.collider(this.player, this.stars)`. What happens?

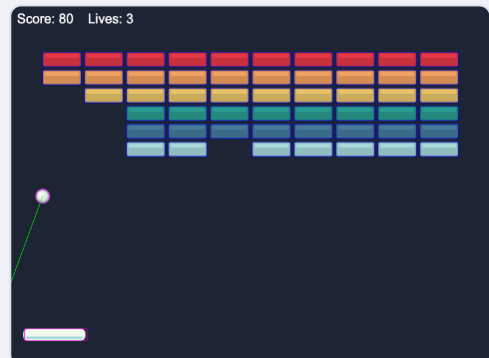


15 / 20

Try it: Breakout

- paddle: dynamic, **immovable**
- ball: `setCircle`, `setBounce(1)`, world bounds
- bricks: a **static group**, destroyed when hit

```
// no bottom edge: the ball falls out  
this.physics.world.setBoundsCollision(true, true, true,  
false);
```



16 / 20

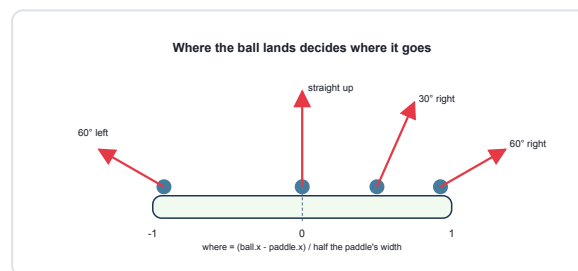
A process callback

```
this.physics.add.collider(  
  this.ball,  
  this.paddle,  
  () => {  
    this.ball.bounceOff(this.paddle);  
    this.sound.play(PADDLE_SOUND);  
  },  
  () => {  
    return this.ball.isFalling(); // false: no collision at all  
  },  
);
```

Runs **first**. Only exactly `false` cancels - forget the `return` and everything collides

17 / 20

Aiming the ball



```
const where = Phaser.Math.Clamp((this.x - paddle.x) / halfWidth, -1, 1);  
this.setDirection(where * MAX_BOUNCE_ANGLE);
```

18 / 20

Summary

- `Phaser.Geom + Intersects ; getBounds() ;` circles for round things
- physics config; `this.physics.add.image / sprite / existing ; debug`
- `setSize + setOffset , setCircle` - from the **top-left**
- `collider` separates; `overlap` reports. Once, in `create()`
- `group / staticGroup ; refreshBody() ;` cast callback objects with `as`
- process callbacks; `collider.active ; disableBody / enableBody ; destroy()`

19 / 20

Challenges

1. **Debug key** - D toggles debug drawing
2. **Hover highlight** - outline the shape under the pointer, fairly
3. **Tough bricks** - the top row takes two hits
4. **Wide paddle** - a falling power-up widens the paddle for 10 s
5. **Glass bricks** - smashed, but no bounce
6. **Fair triangles** - only a real touch of the triangle counts

Next: **Chapter 9 - 2D physics**

20 / 20



2D physics

Overview

Chapter 8 used Arcade Physics to find out when things touch. This chapter is about the physics itself: velocity and acceleration, gravity, bounce, drag and damping, mass, friction, the kinds of body, rotation, speed limits and world wrapping - and, briefly, Matter.js. The big idea to land is that **a physics body is a set of numbers the engine integrates for you**: students set a velocity or an acceleration and stop writing `x += speed * delta`. The playground makes every setting something to press and watch; the space ship shows the same settings adding up to a game; the Matter project shows what Arcade cannot do.

The chapter's one real trap is new, and silent: **adding a body to a physics group resets it** to the group's defaults. Expect "my ball doesn't move" in every lab until students have met it.

Prerequisites

- Chapter 8: bodies, `physics.add.existing`, `setCircle`, `collider` vs `overlap`, groups, callbacks cast with `as`, `disableBody` / `enableBody`, process callbacks
- Chapter 7: particles and the explosion animation (used, not taught, in the space ship)
- Chapter 2: scenes are reused, so fields reset in `init()`
- school physics: speed, acceleration, momentum. Worth five minutes of revision for some groups

Learning outcomes

Students can:

1. explain the difference between velocity and acceleration, and set both on an Arcade body
2. use world and body gravity, bounce, world bounds, linear drag and damping, and explain why drag has no effect while a body accelerates

3. choose between dynamic, immovable, non-pushable and static bodies, and say what mass and Arcade friction do
4. read `body.blocked` / `body.touching` , and listen for `"worldbounds"`
5. make a ship turn with angular acceleration and fly the way it points with `velocityFromRotation` , with a true top speed (`setMaxSpeed`) and wrapping (`world.wrap`)
6. explain the fixed time step, and pause or slow the physics world
7. make a simple Matter.js scene with shaped bodies, and say when Matter is the better choice

Suggested session plan (2 x 1 hour lab, or 1 hour lecture + 2 hour lab)

Time	Activity
0:00 - 0:10	Run <code>ch09_physics_playground</code> . Let students press every key for five minutes before any explanation. Ask: "what did D do? why do crates not bounce?"
0:10 - 0:25	Slides 3-6: the physics step, velocity vs acceleration, gravity and bounce, drag vs damping (draw the two curves on the board before showing the diagram)
0:25 - 0:35	Live demo of the group trap (below). Slide 7
0:35 - 0:50	Slides 8-11: kinds of body, friction on the moving platform, <code>blocked</code> / <code>touching</code> , pause and <code>timeScale</code> , the fixed step
0:50 - 1:00	Challenges 1 and 2
1:00 - 1:10	Fly <code>ch09_space_ship</code> . Ask: "why does the ship keep going? why does it turn with a little lag?"
1:10 - 1:30	Slides 12-15: angles and facing, angular acceleration, thrust with <code>velocityFromRotation</code> , damping and <code>setMaxSpeed</code> , wrap
1:30 - 1:40	Slides 16-18: Matter.js - drag the tower over, fire cannonballs; Arcade vs Matter
1:40 - 2:00	Challenges 3 and 4 (5 and 6 for the fast ones, or as homework)

Key points to stress

- **physics does the delta maths.** Once a game object has a body, move it with the body (velocity, acceleration), never by setting `x` and `y` - that teleports it past the collision checks. Chapter 8 said this; this chapter shows why the engine can do better
- **acceleration changes velocity; velocity changes position.** Students who confuse the two write `setAcceleration` for a walking player and wonder why it never stops
- **drag only works with zero acceleration** on that axis. It is the answer to half the "my ship never slows down" questions
- **damping inverts the drag number:** 0.9 is gentle, 0.1 is strong
- **Arcade bodies never rotate.** Rotation only turns the picture. Circles hide this; boxes do not. That, and stacking, is the case for Matter
- **the group trap:** add to the group first, then configure the body
- **angles start pointing right.** A picture drawn pointing up needs `rotation - Math.PI / 2`
- **Matter has its own units:** velocities per step, gravity 1, forces around 0.01

Live demo: the group trap

In `ch09_physics_playground`, `PlaygroundScene.addBall()`, move `this.things.add(ball);` from the top to the bottom of the method, build, and drop some balls. They fall straight down with no bounce and no sideways speed, and any that miss the crates and platforms drop straight through the floor (world bounds collision was reset too). There is no error. Ask students to find out why before showing slide 7. Put the line back and rebuild.

A second, shorter demo: in `Ship.thrust()`, delete `- Math.PI / 2`. The ship flies sideways.

Common problems and errors

What students see	Cause	Fix
A body added to a group does not move, has no bounce, or ignores world bounds; no error	the group reset it to its defaults when it was added	add it to the group first, then set
TS2551 [ERROR]: Property 'setAllowGravity' does not exist on type 'Body StaticBody'. Did you mean 'allowGravity'?	<code>this.body!.setAllowGravity(false)</code> - the body's type includes static bodies, which have no gravity	<code>(this.body as Phaser.Physics.Arcade.Body</code>
TS2551 [ERROR]: Property 'setAllowGravity' does not exist on type 'MovingPlatform'. Did you mean 'setGravity'?	calling a body-only method on the game object	call it on the body, as above
The ship never slows down	drag set, but the acceleration is never set back to 0	<code>setAcceleration(0)</code> when th
Damping stops everything instantly	<code>setDrag(0.05)</code> with damping on means "keep 5%"	use values near 1 (0.6 - 0.95) f
The ship flies sideways	the picture points up; angle 0 is right	subtract <code>Math.PI / 2</code> , or dra
The ship is faster diagonally	<code>setMaxVelocity</code> limits x and y separately	<code>body.setMaxSpeed(...)</code>
Objects vanish at the edge and pop back	<code>world.wrap</code> with no padding	pass a padding of about half th

What students see	Cause	Fix
Stacked Arcade crates jitter and sink	Arcade cannot really stack	expected - use Matter for stack
TS2345 [ERROR]: Argument of type 'Scene' is not assignable to parameter of type 'World'.	<code>super(scene, ...)</code> in a class extending <code>Phaser.Physics.Matter.Image</code>	Matter game objects take <code>scene</code>
<code>TypeError: Cannot read properties of undefined (reading 'add')</code> in <code>create</code> , in a Matter game	<code>this.physics.add...</code> when the config's default is "matter" - there is no Arcade plugin	use <code>this.matter.add...</code> (it c <code>this.physics</code> exists on every
Matter bodies fly off the screen	Arcade-sized numbers: <code>setVelocity(300, 0)</code>	Matter velocities are per step: 1
Debug drawing cannot be switched on later in Matter	bodies made while debug was off have no outline style	<code>debug: true</code> in the config, an the start of <code>create()</code> (as the j

Discussion questions

- Chapter 1's ball moved itself in `preUpdate` with `delta`. What would you have to add to it to get gravity? Bounce with a bounce factor? Drag? (A good way to show how much the engine does.)
- Why does drag only apply when there is no acceleration? What would a car game feel like if it applied all the time?
- Linear drag or damping - which suits a car? A spaceship? A puck on ice? A ball rolling on grass?
- In Arcade, friction belongs to the platform, not to the crate. Why might the designers have chosen that? What can Arcade friction not do?
- Why is the physics step fixed, when frames are not? What goes wrong in a game that moves by `delta` per frame when the computer is slow?
- When is Matter worth its cost? Name a game for each engine.

Extension ideas

- give the playground a wind: a key that sets `world.gravity.x` to a small value
- show the velocity of the ship as an arrow drawn with `Graphics` every frame
- use `this.physics.accelerateTo0bject` to make a homing missile that follows the ship
- set `fps: 30` in the playground config, turn gravity up to jupiter and watch collisions get rougher; then try `fps: 120` . Ask what it costs
- in Matter, build a chain or a pendulum with `this.matter.add.constraint` , or a Newton's cradle with `this.matter.add.newtonsCradle`

Assessment ideas

- Practical: "make a lunar lander: gravity 50, UP thrusts the way the lander points, land gently (`blocked.down` with a small `velocity.y`) or crash"
- Code reading: show `addBall()` with the group line moved to the end, and ask what the player will see and why
- Short answer: the difference between velocity and acceleration, and between linear drag and damping, with a sketch of speed against time for each
- Short answer: two reasons to choose Arcade over Matter, and one game where Matter is the right choice

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/` .

Chapter 9 - 2D physics - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment. All six were played with the test harness (key presses, and `eval` to read bodies' velocities, bounces, masses and angles) as well as type checked and linted.

1. Gravity switch

Project: `solutions/ch09_challenge_1_gravity_switch` (from `ch09_physics_playground`)

Goal: see that gravity is a vector - a strength *and* a direction - and that the world's gravity can be changed at any time.

`src/scenes/PlaygroundScene.ts`

```
interface Direction {
  name: string;
  x: number;
  y: number;
}

const DOWN: Direction = { name: "down", x: 0, y: 1 };
const UP: Direction = { name: "up", x: 0, y: -1 };
...
keyboard.on("keydown-LEFT", () => {
  this.setGravityDirection(LEFT);
});
...
private applyGravity(): void {
  const strength = GRAVITY_CHOICES[this.gravityIndex].value;
  this.physics.world.gravity.set(this.gravityDirection.x * strength,
  this.gravityDirection.y * strength);
}
```

The G handler now calls `applyGravity()` too, so a new strength keeps the chosen direction. Tested: LEFT gives gravity (-600, 0) and everything piles against the left wall; G

then gives (-1500, 0); UP gives (0, -1500), and things pile against the top edge and the underside of the ledge; R restarts with (0, 600) and "down".

Look for:

- **both** components set. The most common bug is `gravity.x = -600` for LEFT without zeroing `gravity.y`, so things fall diagonally
- the direction reset in `init()` - but note the world itself is rebuilt on restart, so the world gravity goes back to the config's value on its own; only the scene's field needs resetting
- the moving platform is unaffected (`allowGravity` is false) - worth asking students why

2. Bouncy and dead

Project: `solutions/ch09_challenge_2_bouncy_and_dead` (from `ch09_physics_playground`)

Goal: per-body settings, the group trap again, and a little inheritance.

A new class, `SpecialBall` extends `Ball`, remembers its own bounce and mass and applies them in `setUp()`; `Ball`'s constructor gains a default parameter for the picture.

`src/objects/SpecialBall.ts`

```
export class SpecialBall extends Ball {
  private readonly ownBounce: number;
  private readonly ownMass: number;

  constructor(scene: Phaser.Scene, x: number, y: number, key: string,
    bounce: number, mass: number) {
    super(scene, x, y, key);
    this.ownBounce = bounce;
    this.ownMass = mass;
  }

  public setUp(): void {
    this.setBounce(this.ownBounce);
    this.setMass(this.ownMass);
  }
}
```

src/scenes/PlaygroundScene.ts

```
// CHALLENGE 2: a SpecialBall is a Ball too, so ask about it FIRST - it
// keeps its own bounce
if (thing instanceof SpecialBall) {
    thing.setUp();
} else if (thing instanceof Ball) {
    thing.setBounce(BOUNCE_CHOICES[this.bounceIndex].value);
}
```

```
private addSpecialBall(x: number, y: number, key: string, bounce: number,
mass: number, tint: number): void {
    const ball = new SpecialBall(this, x, y, key, bounce, mass);
    ball.setTint(tint);
    this.things.add(ball);    // first - the group resets the body...
    this.applySettings(ball); // ...then drag, world bounds, and its own
    bounce and mass
}
```

Tested: after pressing 1, 2 and then B three times, the red balls have bounce 0 (dead), the blue ball still has bounce 1 and mass 0.5, and the grey ball bounce 0 and mass 5.

ball_blue.png is copied into public/assets/images/ and loaded in preload().

Look for:

- the order of the instanceof tests. With Ball first, the special balls match it and get B's bounce - a good discussion of "is-a" and the most specific type first
- setMass / setBounce called **after** this.things.add(...). Setting them in the constructor looks right and silently does nothing - the group wipes them
- simpler answers are fine: a keepsOwnBounce flag on Ball, or two plain images added to the group and set up in the scene. The key test is that B does not change them
- heavy on light: a grey ball dropped on a pile of blue ones scatters them; blue ones bounce off a grey one that hardly moves. Mass only matters between dynamic bodies

3. Brakes and reverse

Project: solutions/ch09_challenge_3_brakes_and_reverse (from ch09_space_ship)

Goal: understand that drag only applies with no acceleration, and that brakes are "just more damping".

src/objects/Ship.ts

```
// CHALLENGE 3: power 1 is full thrust forwards, -0.5 half thrust backwards, 0 none
public thrust(power: number): void {
  if (power !== 0) {
    ...
    this.scene.physics.velocityFromRotation(facing, THRUST * power,
    this.arcadeBody.acceleration);
  }
}
```

```
public brake(on: boolean): void {
  this.setDrag(on ? BRAKE_DAMPING : DAMPING);
}
```

with `BRAKE_DAMPING = 0.0001` : from 350 px/s, about 3.5 px/s is left after half a second ($350 \times 0.0001^{0.5}$).

src/scenes/SpaceScene.ts

```
const braking = this.brakeKey.isDown;
let power = 0;
if (this.cursors.up.isDown) {
  power = 1;
} else if (this.cursors.down.isDown) {
  power = -REVERSE_POWER;
}
this.ship.brake(braking);
this.ship.thrust(braking ? 0 : power);
```

Tested: full speed up (350), SHIFT held for 500 ms leaves 4 px/s; DOWN held for a second from rest reaches 147 px/s backwards (half thrust, less damping).

Look for:

- thrust switched **off** while braking. Students who only change the drag find that braking while holding UP does nothing - exactly the rule from the chapter. A great "why?" moment

- brake strength chosen by reasoning about damping (kept fraction per second), not by trial alone
- `addKey(KeyCode.SHIFT)` or `this.cursors.shift` - both work

4. Rocks that split

Project: `solutions/ch09_challenge_4_rocks_that_split` (from `ch09_space_ship`)

Goal: a class with state (`size`), creating bodies inside a collision callback, directions from a velocity vector, and mass that finally matters.

`src/objects/Rock.ts`

```
export const BIG = 3;
export const MEDIUM = 2;
export const SMALL = 1;
const SCALES = [0, 0.7, 1.1, 1.5];           // indexed by size (index 0 is not
used)
const SPEED_UP = [0, 1.9, 1.4, 1];          // how much faster than a big rock
const POINTS = [0, 100, 50, 20];
...
public launch(angle: number = Phaser.Math.Between(0, 359)): void {
```

`launch` now takes a direction (a default parameter keeps the old call working), multiplies the speed by `SPEED_UP[this.size]`, and sets `setMass(this.size * this.size)`.

`src/scenes/SpaceScene.ts`

```

if (rock.size > SMALL) {
    const bulletBody = bullet.body as Phaser.Physics.Arcade.Body; // bullets
    have dynamic bodies
    const heading = Phaser.Math.RadToDeg(bulletBody.velocity.angle());
    for (const turn of [-SPLIT_ANGLE, SPLIT_ANGLE]) {
        // start each piece a little way along its own path, so the two do not
        overlap
        const nudge = this.physics.velocityFromAngle(heading + turn, SPLIT_GAP);
        const piece = new Rock(this, rock.x + nudge.x, rock.y + nudge.y,
rock.size - 1);
        this.rocks.add(piece);           // first: adding to a physics group
        resets the body...
        piece.launch(heading + turn);    // ...then set it moving
    }
}

```

The existing `countActive() === 0` check still starts the next wave, and now only fires when every piece is gone (the new pieces are added before the check). Tested: shooting every rock of wave 1 takes $4 + 8 + 16 = 28$ shots, scores $4 \times 20 + 8 \times 50 + 16 \times 100 = 2080$, and starts wave 2.

Look for:

- the pieces added to the group **before** `launch()`
- `velocity.angle()` (radians) converted for `velocityFromAngle` (degrees) - mixing the two units is the usual bug, and gives pieces flying off in odd directions
- pieces made at exactly the same point overlap and are pushed apart by the rocks' collider - it works, but looks odd; the nudge is a nicety
- the body follows the scale: `setScale` before `setCircle` in the constructor, radius in the picture's own pixels. (The body's size catches up with the scale by the next step)

Note for teachers - in the headless browser used to test these projects, rotated images sometimes draw with a wedge missing (a software-rendering quirk). In a normal browser the rocks draw whole; if students report "broken rocks", check they are not using a software renderer.

5. Demolition

Project: `solutions/ch09_challenge_5_demolition` (from `ch09_matter_stack`)

Goal: read the state of Matter bodies (angle, position) every frame to make a game rule.

`src/objects/Crate.ts`

```
// CHALLENGE 5: tipped over, or fallen? Matter bodies rotate, so angle (in
degrees, -180 to 180)
// tells us how far over it has gone
public get isDown(): boolean {
    return Math.abs(this.angle) > TIP_ANGLE || this.y > this.startY + DROP;
}
```

`src/scenes/StackScene.ts`

```
private fire(): void {
    // CHALLENGE 5: only SHOTS cannonballs; after the last, wait for things to
    settle, then finish
    if (this.shotsLeft === 0) {
        return;
    }
    this.shotsLeft = this.shotsLeft - 1;
    if (this.shotsLeft === 0) {
        this.time.delayedCall(SETTLE_TIME, () => {
            this.finished = true;
        });
    }
}
```

```
const down = this.targets.filter((crate) => crate.isDown).length;
```

`buildTower()` and `buildPyramid()` push each crate into `this.targets`, so crates dropped with 1 do not count; `init()` resets the array, the shots and `finished`, because R restarts the same scene object. Tested: five shots, then after four seconds "5 of 17 crates down"; R resets to "0 / 17, Shots left: 5".

Look for:

- `Math.abs(angle)` - a crate tipped the other way has a negative angle

- a crate lying on its side has angle 90 (or -90); one that has rolled right over may be back near 0, which is why the height test is there too
- counting every frame rather than once at the end (the score climbs as things fall - more fun)
- not counting before the stack has settled: Matter's crates wobble a little at the start

6. Pinball flipper

Project: `solutions/ch09_challenge_6_pinball_flipper` (from `ch09_matter_stack`)

Goal: a constraint, and driving a Matter body by its angular velocity.

`src/objects/Flipper.ts`

```
super(scene.matter.world, pinX + PIN_TO_CENTRE, pinY, FLIPPER_KEY,
undefined, {
  chamfer: { radius: THICKNESS / 2 }, // rounded ends, like the picture
  density: 0.05, // heavy: a ball landing on it
  // hardly moves it
  ignoreGravity: true, // it is held up by its pin and our
  // code, not gravity
  ignorePointer: true, // the mouse spring cannot drag it
  // off its pin
});
scene.add.existing(this);

// the pin: pointA is a place in the world, pointB a point on the body,
// measured from its
// centre (and turning with it). Length 0 and stiffness 1: the two are held
// exactly together
scene.matter.add.worldConstraint(this.body as MatterJS.BodyType, 0, 1, {
  pointA: { x: pinX, y: pinY },
  pointB: { x: -PIN_TO_CENTRE, y: 0 },
});
```



```

public swing(up: boolean): void {
    const target = up ? UP_ANGLE : DOWN_ANGLE;
    const speed = up ? FLIP_SPEED : RETURN_SPEED;
    const error = Phaser.Math.DegToRad(target - this.angle);
    this.setAngularVelocity(Phaser.Math.Clamp(error, -speed, speed));
}

```

The scene calls `this.flipper.swing(this.flipKey.isDown)` every frame. Because the angular velocity is never more than the angle still to go, the flipper stops exactly at its limits - the "check of its angle every frame" from the hint. A static peg just past the tip stops a ball rolling off the end. Tested: the flipper rests at 20 degrees and flips to -30 while Z is held; a ball dropped just above it rolls against the peg, and one flick sends it up and right into the tower, tipping four crates.

Look for:

- which way is "up". The tip is on the right, so lifting it is **anticlockwise** - a smaller angle. Students who pin the right end must reverse the signs; the ball then flies up and *left*
- Matter's `setAngularVelocity` is radians per step: 0.1 is brisk; 1 flings the ball off the top of the world (we tried 0.35 - the ball left the screen)
- `ignoreGravity` on the flipper, or it sags and fights the code; `ignorePointer`, or the mouse spring pulls it off its pin
- static stoppers instead of the angle check are a fine answer too - the flipper bounces against them a little, which looks quite like a real flipper
- the red ball is bouncy (restitution 0.7): dropped from high up, it bounces off the flipper and over the peg. Dropping it just above the flipper, or lowering its restitution, fixes that

Chapter 9 - 2D physics - Slides

Chapter 9

2D physics

Velocity, gravity, drag, bounce - and a spaceship



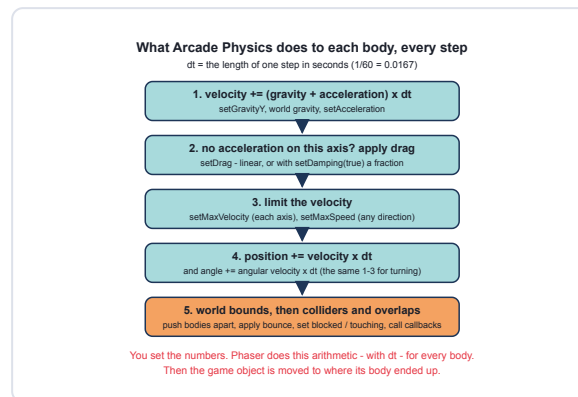
1 / 20

Today

- what the physics engine does to every body, every step
- gravity, bounce, drag and damping, mass, friction
- kinds of body: dynamic, immovable, static
- a trap: groups reset their bodies
- turning, thrust, top speed, wrapping round the world
- Matter.js: shapes that rotate and stack

2 / 20

Every body, every step



You set the numbers. Phaser does the `delta` maths.

3 / 20

Velocity and acceleration

- **velocity** - pixels per second. Changes the **position**
- **acceleration** - pixels per second, per second. Changes the **velocity**
- **gravity** - an acceleration every body shares

`setVelocityX(200)` - right at 200 px/s, from now on

`setAccelerationX(200)` - 200 px/s faster every second: 600 px/s after three seconds

4 / 20

Gravity and bounce

```
// G: the world's gravity - shared by every body (a body can add its own with setGravityY)
keyboard.on("keydown-G", () => {
  this.gravityIndex = (this.gravityIndex + 1) % GRAVITY_CHOICES.length;
  this.physics.world.gravity.y = GRAVITY_CHOICES[this.gravityIndex].value;
});
```

```
// the physics world starts as big as the game; stop it at the top of the ground instead
this.physics.world.setBounds(0, 0, 800, FLOOR_Y);
```

- `setBounce(b)` : 0 dead, 1 perfect, more than 1 gains speed
- `setCollideWorldBounds(true)` : the world's edges are walls

5 / 20

Drag, two ways



- `setDrag(150)` - lose 150 px/s every second
- `setDamping(true)` + `setDrag(0.3)` - keep 30% every second
- **drag only works when acceleration is 0**

6 / 20

The trap: groups reset their bodies

```
private addBall(x: number, y: number): void {
  const ball = new Ball(this, x, y);
  // add to the group FIRST: a physics group resets the bodies it is given to its own defaults
  // (no bounce, no velocity...), so anything set before this line would be lost
  this.things.add(ball);
  bodyOf(ball).onWorldBounds = true;
  ball.setVelocityX(Phaser.Math.Between(-LAUNCH_SPEED, LAUNCH_SPEED));
  this.applySettings(ball);
}
```

No error. No warning. The ball just stops.

7 / 20

Kinds of body

Body	Moves itself?	Pushed?
dynamic	yes	yes, shared by mass
<code>setPushable(false)</code>	yes	no - pushes back
<code>setImmovable(true)</code>	if given a velocity	never
static	never	never (cheapest)

Arcade **friction** belongs to a moving immovable body: how much of its movement it passes to a rider.

`setFriction(1, 0)` carries crates; `0` slides out from under them.

8 / 20

What is it touching?

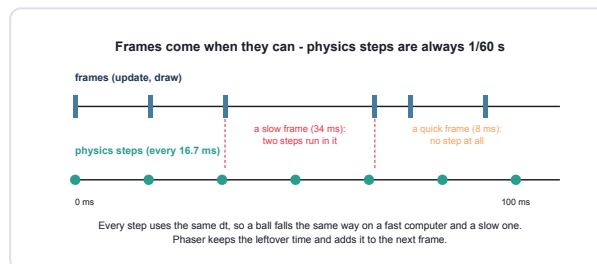
- `body.blocked.down` - resting on something that cannot give way
- `body.touching.down` - touching any body below
- worked out afresh every step: check in `update()`

```
this.physics.world.on(
  Phaser.Physics.Arcade.Events.WORLD_BOUNDS,
  (body: Phaser.Physics.Arcade.Body, _up: boolean, down: boolean) => {
    // ...
  },
);
```

Only for bodies with `onWorldBounds = true`

9 / 20

The physics clock

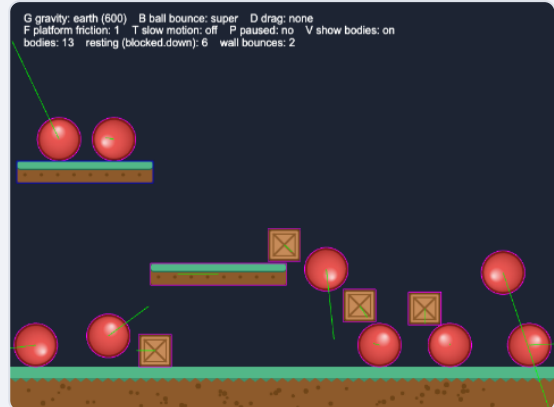


- `this.physics.pause()` / `resume()` - the scene keeps running
- `world.timeScale = 3` - slow motion (**bigger is slower**)

10 / 20

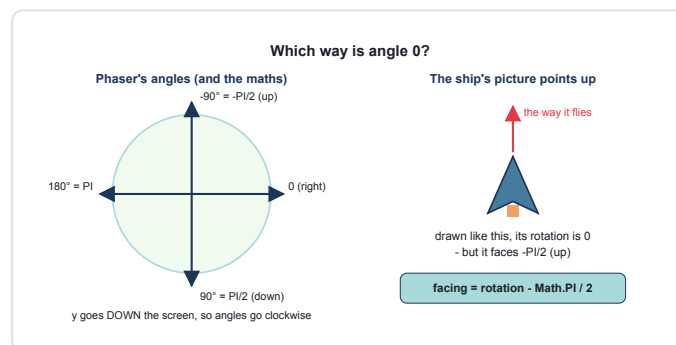
Try it: the playground

- G B D F T P - change the laws of physics
- V - see the real bodies
- watch a ball roll: the picture turns, the **body never does**
- drop a crate on the platform; press F



11 / 20

Which way is angle 0?



12 / 20

Turning and thrust

```
public turn(direction: number): void {
    this.setAngularAcceleration(direction * TURN_ACCELERATION);
}

public thrust(on: boolean): void {
    if (on) {
        ...
        const facing = this.rotation - Math.PI / 2;
        ...
        this.scene.physics.velocityFromRotation(facing, THRUST, this.arcadeBody.acceleration);
    } else {
        ...
        this.setAcceleration(0);
    }
}
```

13 / 20

Drift, top speed, wrap

```
this.setDamping(true);
this.setDrag(DAMPING);
...
this.arcadeBody.setMaxSpeed(MAX_SPEED);
```

DAMPING = 0.6 - keep 60% a second. setMaxSpeed - in any direction

- setMaxVelocity(x, y) limits each axis: diagonals go 1.4 times faster

```
this.physics.world.wrap(this.ship, SHIP_WRAP);
this.physics.world.wrap(this.rock, ROCK_WRAP);
```

14 / 20

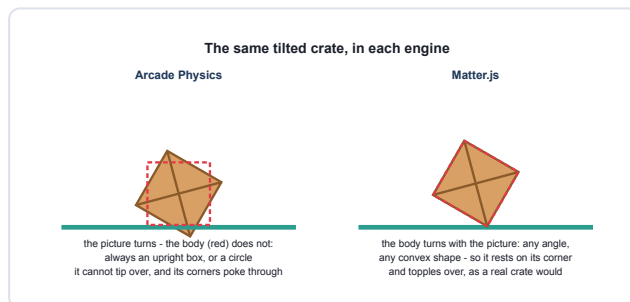
Try it: the space ship

- LEFT / RIGHT turn, UP thrusts, SPACE fires
- turn round and thrust to stop
- bullets get the ship's velocity added
- `this.physics.pause()` at game over



15 / 20

Matter.js: why a second engine?



```
physics: {  
  default: "matter",  
  matter: {  
    ...  
    gravity: { x: 0, y: 1 },  
  },  
}
```

16 / 20

Matter game objects

```
export class Crate extends Phaser.Physics.Matter.Image {  
  constructor(scene: Phaser.Scene, x: number, y: number) {  
    super(scene.matter.world, x, y, CRATE_KEY, undefined, {  
      friction: 0.6,          // wood on wood: grippy, so a stack stays up (0 = ice, 1 = rubber)  
      restitution: 0.05,      // bounce: crates hardly bounce at all  
    });  
    scene.add.existing(this);  
  }  
}
```

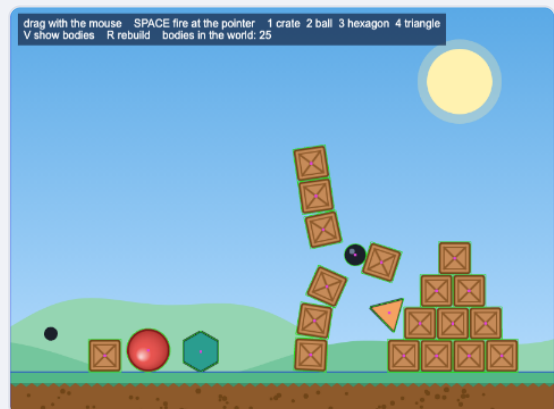
- `shape: { type: "circle", radius }, { type: "polygon", sides, radius }`
- `density` gives mass; velocities are **pixels per step**

17 / 20

Try it: knock it down

```
this.matter.add.mouseSpring({ stiffness: 0.2 });
```

- drag anything; SPACE fires at the pointer
- 1 - 4 drop shapes; V shows them
- Arcade for most games; Matter when the game is the physics



18 / 20

Summary

- set velocity and acceleration - the engine integrates, in fixed steps
- gravity, bounce, drag (linear or damping), max speed, mass, friction
- dynamic, immovable, static; `blocked` , `touching` , `"worldbounds"`
- groups reset bodies: **add first, then set up**
- angular velocity and acceleration; `velocityFromRotation` ; `world.wrap`
- Matter: real shapes, rotation, stacking - in its own units

19 / 20

Challenges

1. **Gravity switch** - arrow keys choose which way gravity pulls
2. **Bouncy and dead** - a light super-ball and a heavy dead one
3. **Brakes and reverse** - DOWN for retro-rockets, SHIFT to brake
4. **Rocks that split** - big to medium to small
5. **Demolition** - five cannonballs, count the crates down
6. **Pinball flipper** - a pinned Matter body, flicked with Z

Next: **Chapter 10 - Tilemaps**

20 / 20

Tilemaps

Overview

Students build tile worlds entirely in code. The Phaser ideas are the **three objects of a tilemap** (`Tilemap` = data, `Tileset` = picture, `TilemapLayer` = game object), **tile collision** with Arcade Physics, **finding and changing tiles** while the game runs, **converting between pixels and tiles**, and the **camera** for a world bigger than the screen (bounds, follow, scroll factor, a second camera as a minimap). The TypeScript ideas are 2D arrays (`number[][]`), `unknown` and a **type guard, function types** for callbacks, and `Record<K, V>` (in the solutions). There is also a small algorithm - cellular automata plus a flood fill - and a model/view split (`Cave` is data; the scene draws it).

Chapter 11 does the same with maps made in Tiled, so this chapter deliberately builds every map in code: students should come out understanding that a map is *just a grid of numbers*, which makes Tiled's JSON unsurprising next week.

Prerequisites

- Chapter 2: scene reuse and `init()` (used in `ch10_scrolling_world`)
- Chapter 6: `localStorage` and `JSON` (used in `ch10_tilePainter`)
- Chapter 7: sprite sheet animations and tweens (the hero's walk; challenge 6)
- Chapter 8: Arcade Physics bodies, `setSize`, colliders and their callbacks
- Java 2D arrays (`int[][]`), for the analogy with `number[][]`

Learning outcomes

Students can:

1. explain what a tileset, a tile index and an empty tile (`-1`) are, and why games use tiles
2. build a map from a 2D array with `make.tilemap`, `addTilesetImage` and `createLayer`, and describe the job of each of the three objects

3. make tiles solid with `setCollision` / `setCollisionByExclusion` and a physics collider, and debug it with `renderDebug`
4. find tiles with `getTileAt` / `getTileAtWorldXY`, and convert between pixels and tiles with `worldToTileXY` / `tileToWorldXY`
5. change a map while the game runs with `putTileAt`, `removeTileAt`, `putTilesAt`, `fill`
6. set up a camera for a world bigger than the screen, keep a HUD fixed with `setScrollFactor(0)`, and add a second camera
7. contrast free movement with grid movement, and choose between them for a given game

Suggested session plan (2 x 2 hours)

Session 1 - maps, collision and editing

Time	Activity
0:00 - 0:10	Show a screenshot of a well-known tile game (any top-down RPG or platformer) and ask students to find the repeated squares. How many different tiles? How big would the level be as one picture?
0:10 - 0:25	Slides 3-4: why tiles, indexes, the level as a 2D array. Students read <code>level.ts</code> and find the house and the bridge in the numbers
0:25 - 0:45	Slides 5-6: the three objects, the five steps in <code>create()</code> . Live-code the five steps into an empty scene, running after each one (see Demos)
0:45 - 1:00	Slide 7: collision; the smaller body; press D. Challenge 1
1:00 - 1:10	Slide 8: <code>getTileAtWorldXY</code> , a <code>Tile</code> 's <code>x / y</code> vs <code>pixelX / pixelY</code>
1:10 - 1:30	Slides 9-12: the tile painter - blank layer, <code>worldToTileXY</code> / <code>tileToWorldXY</code> , put/remove, <code>-1</code> . Students paint a level, press P, paste it into <code>ch10_array_map</code>
1:30 - 2:00	Challenges 2 and 3

Session 2 - big worlds and movement

Time	Activity
0:00 - 0:15	Recap quiz: what does <code>worldToTileXY(100, 70)</code> return? What does <code>tileToWorldXY(3, 2)</code> return? What index is an empty cell?
0:15 - 0:35	Slide 13: the cave generator. Do the cellular automaton on paper first (see Activities), then show <code>Cave.ts</code>
0:35 - 0:55	Slides 14-17: two layers, <code>setCollisionByExclusion</code> , camera bounds and follow, scroll factor, the minimap
0:55 - 1:05	Slide 18: free vs grid movement - discussion
1:05 - 2:00	Challenges 4-6 (6 is a good stretch for the strongest students; 5 suits everyone)

Key points to stress

- **a map is a grid of numbers.** Everything else - drawing, collision, the minimap - is Phaser reading those numbers. Changing the map at run time is changing a number
- **rows first:** `LEVEL[row][column]`, i.e. `[y][x]`. This is the single most common tile bug
- **the Tilemap is not drawn.** Layers are. A map can have several layers, and only the layers you collide with need collision set
- `setCollision` **is by index**, and remembered: tiles put down later with those indexes collide too. Collision has nothing to do with pictures - a map whose tileset fails to load still has walls
- `tileToWorldXY` **gives the top left corner** of a tile - add half a tile for the centre
- **a Tile's x and y are in tiles**, not pixels (`pixelX`, `pixelY` are pixels). Students who have been using `sprite.x` for pixels all term will trip on this
- `pointer.worldX`, **not** `pointer.x`, whenever the camera can move. The painter's camera never moves, so both work - a good moment to say why the book uses `worldX` anyway
- the player body is **smaller than a tile** on purpose: a 32 x 32 body through a 32-pixel door has to be pixel-perfect
- generated levels: keep the **model** (`Cave`) separate from the **view** (the tilemap). The model can then be tested, reused, or drawn differently

Common problems and errors

What students see	Cause	Fix
The map is invisible, but the player bumps into invisible walls; the console says Texture key "tilez" not found	the key passed to <code>addTilesetImage</code> does not match the key the image was loaded with	use the constant; remember the argument is used as the key whe there is no second
Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.	storing <code>createLayer(...)</code> in a field typed <code>TilemapLayer</code> without the cast	as <code>Phaser.Tilemaps.TilemapLayer</code> (and explain why it is safe here)
Argument of type 'Tileset null' is not assignable to parameter of type 'string string[] Tileset Tileset[]'.	the result of <code>addTilesetImage</code> passed straight to <code>createLayer</code>	<code>!</code> after <code>addTilesetImage(...)</code> check for <code>null</code>
<code>TypeError: Cannot read properties of undefined (reading '7')</code>	<code>LEVEL[x][y]</code> instead of <code>LEVEL[y][x]</code> : a column number used as a row, past the last row	rows first

What students see	Cause	Fix
<pre>TypeError: Cannot read properties of undefined (reading 'index') from GetTilesWithin , only when the player reaches one part of the map</pre>	a row of the level array is shorter than the first row	every row the same length (the strings of challenge 5 can check
The player walks through walls and water	<pre>setCollision</pre> not called, wrong indexes given, or no <pre>this.physics.add.collider(player, layer)</pre>	both are needed; press D to see collides
<pre>TypeError: Cannot read properties of undefined (reading 'add') in new Player</pre>	the project has no <code>physics</code> section in the config, so <code>scene.physics</code> does not exist	add <code>physics: { default: "arcade" }</code> to the config
The player gets stuck in doorways	body as big as a tile	<pre>setSize</pre> smaller than a tile
The player starts in the corner of a tile, or half inside a wall	<pre>tileToWorldXY</pre> is the top left corner	add <code>TILE_SIZE / 2</code>
<pre>Property 'load' in type 'PainterScene' is not assignable to the same property in base type 'Scene'. (plus four more errors, including Property 'image' does not exist on type '() => void')</pre>	a method called <code>load()</code> in a scene - it replaces the scene's loader	rename it (<code>loadMap()</code>); add , <code>make</code> , <code>physics</code> , <code>cameras</code> , <code>tile</code> <code>input</code> , <code>sound</code> are all taken

What students see	Cause	Fix
Painting happens on the palette strip, or the eraser paints	not checking the result of <code>worldToTileXY</code> is on the map; or comparing <code>current === 0</code> for the eraser	check bounds; the eraser is <code>-1</code> , <code>0</code> (0 is grass)
The camera shows black beyond the map edge	no <code>cameras.main.setBounds(...)</code>	set the camera's bounds to the r size
The player stops at an invisible line at x = 800 or y = 600	physics world bounds left at the game size while the player has <code>setCollideWorldBounds(true)</code>	<code>this.physics.world.setBounds(0, map.widthInPixels, map.heightInPixels)</code>
The HUD scrolls away	no <code>setScrollFactor(0)</code>	fix it to the screen
The HUD appears, tiny, inside the minimap	the minimap camera draws everything	<code>minimap.ignore([...])</code>
"My painted map is lost when I reload"	expected: S saves, L loads - the map is not loaded automatically	a nice mini-extension: load on st; there is a save

Suggested demos and live-coding moments

- **Build the map in five runs.** Start from an empty scene with only `preload()`. Add `make.tilemap` - run: nothing drawn (it is data). Add `addTilesetImage` - still nothing. Add `createLayer` - the map appears. Add the player - it walks through everything. Add `setCollision`
 - still walks through! Add the collider - now it stops. Each run makes one object's job visible
- **Change one number** in `level.ts` (a wall in the doorway) and rebuild: the level is data
- **The invisible map:** misspell the tileset key. Nothing is drawn, but the player still stops at the walls - a memorable way to show that collision is about indexes
- **Press D** and slide the player along a wall: show the white "interesting faces"
- **Painter to game:** paint a small level, press P, open the console, copy, paste into `ch10_array_map`'s `level.ts`, rebuild, walk round it

- **Two cameras:** in `ch10_scrolling_world`, comment out `minimap.ignore(...)` and then `this.cameras.main.ignore(this.marker)` in turn, and ask what will happen before running
-

Activities

- **Cellular automaton on paper.** Give pairs a 10 x 8 grid with random shaded squares (about half). They apply one smoothing pass to a *second* grid (rock if 5 or more of the 9 squares are rock). Then ask: what goes wrong if you write the result into the same grid as you go? (Cells already changed affect their neighbours - the result depends on the order.) This is exactly why `smooth()` builds a new grid
 - **Flood fill by hand.** On the smoothed grid, colour in everything reachable from one square. Which caves are cut off? Link to `fillUnreachable()`, and to the paint bucket in any drawing program
 - **Design a tileset.** In groups, list the tiles a Sokoban, a Pac-Man maze or a farm game would need, and which of them collide
-

Discussion questions

- A 60 x 40 level as one picture is about 10 MB of graphics memory; as tiles it is a 128 x 32 image and 2,400 numbers. What else do tiles make easy that a picture does not?
- Why does `setCollision` take indexes rather than a list of positions? When would you want to make a *single* tile solid? (`tile.setCollision(...)`; a secret door)
- `Cave` knows nothing about Phaser. What does that make possible? (Unit testing it; a different tileset; drawing it as a minimap; making levels on a server)
- Free movement or grid movement for: a Zelda-like, Sokoban, Pac-Man, a racing game, a roguelike? Why? (Pac-Man is interesting: it moves smoothly but is locked to the grid's lines)
- The minimap is a camera. What else could a second camera be used for? (Split-screen two-player; a rear-view mirror; a zoomed "picture in picture")

Extension ideas

- load the painter's saved map automatically at start-up, if there is one
- add a second layer to the painter (a "decorations" layer drawn over the ground, with its own palette) and a key to switch layers
- a **fog of war** minimap: only tiles the player has been near are shown (a third layer of black tiles removed as the player walks)
- **seeded caves**: replace `Math.random()` with `Phaser.Math.RandomDataGenerator` and show the seed
- smooth camera: `startFollow(player, true, 0.1, 0.1)` and discuss lerp
- try `createLayer(0, tileset, 0, 0, true)` (a `TilemapGPULayer`) in the cave, and see what breaks when a tile changes (`generateLayerDataTexture()`)

Assessment ideas

- Practical: "add lava (use `dungeon_tiles.png`, or tint a tile) that sends the player back to the start when stepped on" - tests `getTileAtWorldXY`, tile indexes, and a physics reset
- Practical: "add a door tile that disappears (`removeTileAt`) when the player has collected a key"
- Code reading: give `LEVEL[x][y]` code and the `Cannot read properties of undefined (reading '7')` error; ask for the bug and the fix
- Short answer: what are the jobs of the `Tilemap`, the `Tileset` and the `TilemapLayer`? Which one is a game object?
- Short answer: the player is at pixel (200, 150) in a map of 32-pixel tiles. Which column and row? What are the pixel coordinates of the centre of that tile?

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 10 - Tilemaps - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment.

1. A new island

Project: `solutions/ch10_challenge_1_new_island` (from `ch10_array_map`)

Goal: read and write a map as a 2D array; understand that the level is data.

Only `src/level.ts` changes. The house has moved to the top left, inside a moat, with a sand path from its door across the water; the river has a second bridge; the bottom half is a maze:

`src/level.ts`

```

export const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2],
  [2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 2, 2, 2, 2, 1, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 2, 2, 0, 2, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
  [2, 0, 0, 1, 1, 1, 3, 1, 1, 1, 0, 0, 0, 0, 0, 0, 3, 3, 3, 0, 0, 0, 0, 0, 0],
  ...
  [2, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1, 1, 3, 3, 3, 1, 1, 1, 1, 1, 1, 1],
  ...
];

// CHALLENGE 1: a new start - inside the house, which has moved
export const START_COLUMN = 6;
export const START_ROW = 4;

```

Look for: every row still 25 numbers long. A short row is not caught when the map is made - the map takes its width from the first row - but the game crashes with `TypeError: Cannot read properties of undefined (reading 'index')` (inside `GetTilesWithin`) the moment the player walks near the missing tiles, which can be long after the level starts. Also: the start on a tile the player can stand on; everything reachable. Tested with the harness: from the start, walking down leaves the house by the door and crosses the moat on the sand path; the new bridge crosses the river. Students who paint the map in `ch10_tilePainter` and paste it in have done exactly what the chapter suggests - accept it.

2. Sand slows you down

Project: `solutions/ch10_challenge_2_sand_slows` (from `ch10_array_map`)

Goal: read the tile under the player every frame, and keep the `Player` class free of map knowledge.

`src/objects/Player.ts`

```
// CHALLENGE 2: how fast to walk, as a fraction of SPEED. The player does
// not know about tiles -
// the scene looks at the map and tells it
private speedFactor = 1;
...
// CHALLENGE 2: 1 is full speed, 0.5 is half speed
public setSpeedFactor(factor: number): void {
  this.speedFactor = factor;
}
...
// CHALLENGE 2: times the speed factor
const velocity = new Phaser.Math.Vector2(dx, dy).normalize().scale(SPEED *
this.speedFactor);
```

`src/scenes/GameScene.ts`

```
const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);

// CHALLENGE 2: slow on sand, full speed anywhere else
const onSand = tile !== null && tile.index === SAND;
this.player.setSpeedFactor(onSand ? SAND_SPEED : 1);
```

The scene already found the tile for the info strip, so the check is two lines. Tested with the harness: walking down onto the sand below the house, the player's velocity drops from 160 to 80 pixels per second at the edge of the sand.

Look for:

- the player told its speed, not made to look at the map. A student who passes the layer into `Player` and checks tiles there has a working answer - ask what happens when a second map, or a different game, wants to reuse `Player`
- `tile !== null` checked first (off the map, or an empty cell, gives `null`)

- speed reset to full off the sand - a common bug is a player who slows down and never speeds up
 - where the check happens: the tile under the player's *centre*. Some students use the feet (a few pixels lower) - both are fine; discuss which looks right
-
-

3. Eyedropper

Project: `solutions/ch10_challenge_3_eyedropper` (from `ch10_tilePainter`)

Goal: tell the mouse buttons apart; read a tile back from the map.

`src/scenes/PainterScene.ts`

```
// CHALLENGE 3: stop the browser's own menu appearing when the right button
is clicked
this.input.mouse!.disableContextMenu();

// paint when the button goes down, and while it is held and the mouse moves
// CHALLENGE 3: ...but the RIGHT button picks up the tile instead
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) => {
    if (pointer.rightButtonDown()) {
        this.pick(pointer);
    } else {
        this.paint(pointer);
    }
});
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
    this.hover(pointer);
    // CHALLENGE 3: only the left button paints while dragging (isDown is true
    for any button)
    if (pointer.leftButtonDown()) {
        this.paint(pointer);
    }
});
```

```
private pick(pointer: Phaser.Input.Pointer): void {
  const cell = this.tileUnder(pointer);
  if (cell === null) {
    return;
  }
  const tile = this.layer.getTileAt(cell.x, cell.y);
  this.choose(tile === null ? EMPTY : tile.index);
}
```

Calling the existing `choose()` means the palette's yellow frame and the status text update with no extra code. Tested with the harness (right-button mouse events sent to the canvas): right-clicking a wall chose wall, an erased cell chose the eraser, grass chose grass; nothing was painted; the `contextmenu` event was cancelled.

Look for:

- `disableContextMenu()` - without it the browser's menu pops up over the game
- the drag handler changed too. `pointer.isDown` is true for *any* button, so the unchanged version paints while you right-drag - a subtle bug worth asking about
- an empty cell handled (`getTileAt` gives `null` , which must become `-1`)

4. Treasure tiles

Project: `solutions/ch10_challenge_4_treasure_tiles` (from `ch10_scrolling_world`)

Goal: put special tiles into a generated map, and react when the player touches one.

The chests are written into the wall layer's data before it goes into the layer, on open cells away from the start (the same choice of places the gems used):

```
src/scenes/WorldScene.ts
```



```

const wallData = this.wallTiles(cave);
this.placeChests(cave, wallData);
walls.putTilesAt(wallData, 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not
empty collides
...
private placeChests(cave: Cave, wallData: number[][]): void {
  ...
  for (const cell of places.slice(0, CHEST_COUNT)) {
    wallData[cell.row][cell.column] = CHEST;
  }
}

```

Because the chests are on the wall layer, they are solid - the player bumps into them. A collider can take a callback, which for a sprite and a layer is given the sprite and the **tile**:

```

this.physics.add.collider(this.player, walls, (_player, tile) => {
  // Phaser's callback type allows bodies and game objects too; with a layer
  it is always a Tile
  const bumped = tile as Phaser.Tilemaps.Tile;
  if (bumped.index === CHEST) {
    walls.removeTileAt(bumped.x, bumped.y);
    this.sound.play(COIN_KEY);
    this.chestsFound++;
    this.updateHud();
  }
});

```

Tested with the harness: placing the player two tiles above a chest (with open floor between) and walking down collects it - the tile disappears and the HUD counts it; ten chests are placed in every new cave.

Other good answers:

- `walls.setTileIndexCallback(CHEST, (sprite, tile) => { ... }, this)` - the callback fires when the player touches any chest. Note that if the callback returns `true`, Phaser skips the collision for that tile; returning nothing (as most students will) means the player is still bumped back
- a third layer holding only the chests (`-1` everywhere else) and `this.physics.add.overlap(player, chestLayer, ...)` - the player walks onto a

chest instead of bumping into it. Check the tile's index in the callback: an overlap with a layer reports every tile under the body, empty ones included

Look for: chests placed only on open floor (not in rock, where they could never be reached); the chest removed from the map, not just counted (otherwise the count runs up while the player leans on it); the count reset in `init()` for a new cave.

5. A map from strings

Project: `solutions/ch10_challenge_5_map_from_strings` (from `ch10_array_map`)

Goal: turn text into data; lookup tables; helpful errors.

`src/level.ts`

```
const MAP: string[] = [
  "#####",
  "#.....#",
  "#...SSS.....~~~.....#",
  "#..SS~SS.....~~~~~.....#",
  ...
  "#....#.@.#.....#",
  ...
];

const TILE_FOR: Record<string, number> = {
  ".": GRASS,
  "~": WATER,
  "#": WALL,
  "s": SAND,
  "@": GRASS,
};
```

```

export function parseLevel(rows: string[]): Level {
  const width = rows[0].length;
  const tiles: number[][] = [];
  let startColumn = -1;
  let startRow = -1;

  rows.forEach((text, row) => {
    if (text.length !== width) {
      throw new Error(`Map row ${row} is ${text.length} characters long, not
${width}`);
    }

    const line: number[] = [];
    text.split("").forEach((character, column) => {
      if (!(character in TILE_FOR)) {
        throw new Error(`Unknown map character "${character}" at column
${column}, row ${row}`);
      }
      if (character === START) {
        startColumn = column;
        startRow = row;
      }
      line.push(TILE_FOR[character]);
    });
    tiles.push(line);
  });

  if (startColumn < 0) {
    throw new Error(`The map has no ${START} to show where the player
starts`);
  }
  return { tiles, startColumn, startRow };
}

// CHALLENGE 5: parsed once, when the game loads
export const LEVEL: Level = parseLevel(MAP);

```

The scene uses `LEVEL.tiles`, `LEVEL.startColumn` and `LEVEL.startRow`. Tested with the harness: the tile data in the running game is identical, tile for tile, to `ch10_array_map`'s, and the player starts in the same place. With an `x` put into the map (in a scratch copy), the game stops with `Error: Unknown map character "x" at column 16, row 8` in the console.

- `Record<string, number>` is an object used as a lookup table - the TypeScript way to write a small `Map<Character, Integer>` as a literal
- `character in TILE_F0R` asks whether the object has that key. `TILE_F0R[character]` alone would give `undefined` for an unknown character, and the tilemap would quietly get a hole
- `forEach((value, index) => ...)` gives the index as well as the value - just what a map needs

Look for: the character table in **one** place; a sensible start (the `@` also needs a tile under it - grass here); errors that say *where* the problem is. Accept a plain nested `for` loop with `text[column]` or `text.charAt(column)` instead of `split("")`. Strong students will notice the strings do not show the house's door very clearly in a proportional font - an argument for a monospace editor, and for Tiled next week.

6. Grid movement

Project: `solutions/ch10_challenge_6_grid_movement` (from `ch10_array_map`)

Goal: a different model of movement - positions in tiles, checks before moves, tweens between.

`Player` becomes a plain `Sprite` (no physics - the config's `physics` section is removed too) that knows the layer and its own column and row:

```
src/objects/Player.ts
```

```

protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);    // without this, the animations would not
    play

    // CHALLENGE 6: no new move until the last one has finished - so no
    turning halfway
    if (this.moving) {
        return;
    }
    ...
    // CHALLENGE 6: look at the next tile first. Off the map (null), or a tile
    that collides (wall or
    // water - the scene's setCollision still says which), and the player
    turns but stays put
    const next = this.layer.getTileAt(this.column + dx, this.row + dy);
    if (next === null || next.collides) {
        this.stand();
        return;
    }

    // CHALLENGE 6: glide to the centre of the next tile; ignore the keys
    until it is there
    this.column = this.column + dx;
    this.row = this.row + dy;
    this.moving = true;
    this.anims.play(`walk-${this.facing}`, true);

    const corner = this.layer.tileToWorldXY(this.column, this.row);
    this.scene.tweens.add({
        targets: this,
        x: corner.x + TILE_SIZE / 2,
        y: corner.y + TILE_SIZE / 2,
        duration: STEP_TIME,
        onComplete: () => {
            this.moving = false;    // the next preUpdate moves again if a key is
            still held
        },
    });
}

```

src/scenes/GameScene.ts

```

this.player = new Player(this, this.layer, START_COLUMN, START_ROW);

```

Holding a key keeps moving because, once a tween completes, the next `preUpdate` sees the key still down and starts another. Only one direction is read at a time (`else if` all the way down), so there are no diagonals. `tile.collides` - set by the scene's unchanged `setCollision([WATER, WALL])` - is the "can I go there?" test, so the list of solid tiles is still in one place.

Tested with the harness: a short tap of DOWN moves exactly one tile (y 240 to 272); holding DOWN from the start walks out through the door and stops at row 10, above the river, at the exact centre of the tile (y 336); walking into the house wall does not move the player; holding RIGHT and DOWN together moves right only.

Look for:

- the position held in tiles (`column` , `row`), with pixels worked out from them - not the other way round. Students who convert `x / y` back to tiles each move get rounding errors
- the next tile checked **before** the move starts
- input ignored while moving, so the player cannot stop or turn between tiles
- `null` handled (off the edge of the map) - not needed with this map's wall border, but the right habit
- physics removed. Students who keep the Arcade body and tween a physics sprite get a body that lags behind or fights the tween; it is worth seeing once why grid movement and physics do not mix

Chapter 10 - Tilemaps - Slides

Chapter 10

Tilemaps

Worlds built from tiles - in code



1 / 20

Today

- why games use **tiles**; tilesets and tile **indexes**
- a map from a **2D array**: map, tileset, layer
- **collision** by tile index
- finding tiles; **pixels and tiles**
- changing the map while the game runs
- a world **bigger than the screen**: camera follow, bounds, a minimap
- free movement or **grid** movement?

2 / 20

Why tiles?

- **memory:** one small picture, reused. A 60 x 40 level as one picture: ~10 MB
- **level design:** a level is a grid of numbers - change a number, change the level
- **rules:** "is this a wall?" is a question about a number, not about pixels

Tilesets in the library (all 32 x 32): `simple_tiles.png` (4), `dungeon_tiles.png` (16), `platform_tiles.png` (16)

3 / 20

Tiles, indexes, and the level

A tileset is one picture; a map is a grid of numbers that pick from it

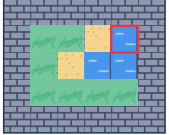
`simple_tiles.png` - 128 x 32 pixels, four 32 x 32 tiles



each tile's INDEX is its place in the picture, counting from 0 - left to right, then row by row in a taller tileset
-1 means "no tile": nothing is drawn

```
const LEVEL: number[][] = [
  [2, 2, 2, 2, 2, 2],
  [2, 0, 0, 3, 1, 2],
  [2, 0, 3, 1, 1, 2],
  [2, 0, 0, 0, 0, 2],
  [2, 2, 2, 2, 2, 2],
];
```

make.tilemap



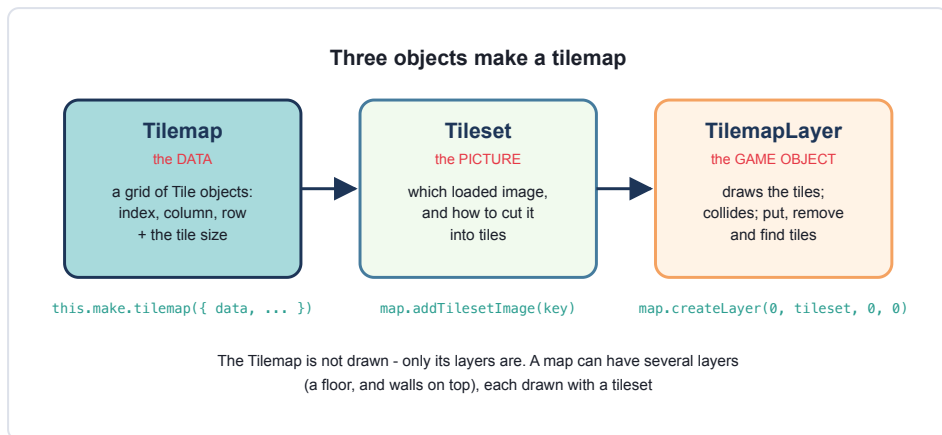
`LEVEL[1][4]` is 1:
row 1, column 4
is water

rows first: `LEVEL[row][column]` - the array looks like the map

`LEVEL[row][column]` - rows first. -1 = no tile

4 / 20

Three objects make a tilemap



5 / 20

Five steps

```
const map = this.make.tilemap({ data: LEVEL, tileWidth: TILE_SIZE, tileHeight: TILE_SIZE });

const tileset = map.addTilesetImage(TILES_KEY!);

this.layer = map.createLayer(0, tileset, 0, 0) as Phaser.Tilemaps.TilemapLayer;

this.layer.setCollision([WATER, WALL]);

this.physics.add.collider(this.player, this.layer);
```

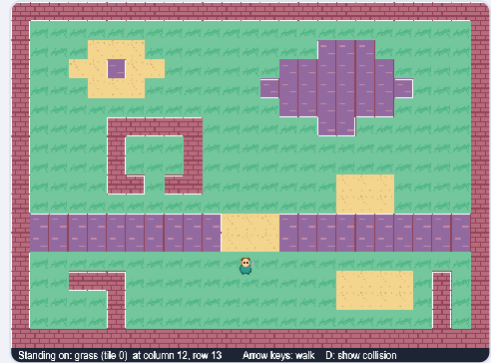
- `!` - "not `null` this time"
- `as` - `createLayer` could also make a GPU layer; we know it did not

6 / 20

Collision

```
// a body smaller than the picture, centred on it  
this.setSize(BODY_SIZE, BODY_SIZE);
```

- a 32 x 32 body through a 32-pixel door: pixel-perfect or stuck
- `setCollision` is **by index**, and remembered - tiles put later collide too
- press **D**: `layer.renderDebug(graphics, {...})`



7 / 20

Try it: Array Map - which tile am I on?

```
const tile = this.layer.getTileAtWorldXY(this.player.x, this.player.y);  
if (tile) {  
  const name = TILE_NAMES[tile.index];  
  ...  
}
```

- `tile.x` , `tile.y` - column and row, **in tiles**; `pixelX` , `pixelY` - pixels
- walk round `ch10_array_map` ; watch the strip; press D
- put a wall in the doorway (one number in `level.ts`)

8 / 20

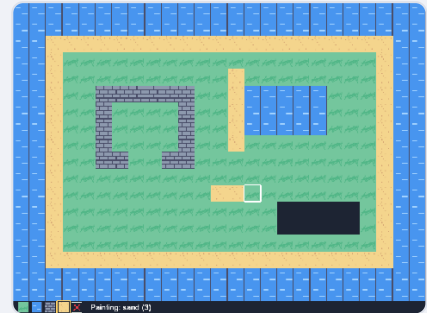
The tile painter: put and remove

```

this.layer = map.createBlankLayer("ground", tileset!);
this.layer.fill(GRASS);
...
if (this.current === EMPTY) {
    this.layer.removeTileAt(cell.x, cell.y);    // leaves
    a hole: nothing is drawn there
} else {
    this.layer.putTileAt(this.current, cell.x, cell.y);
}

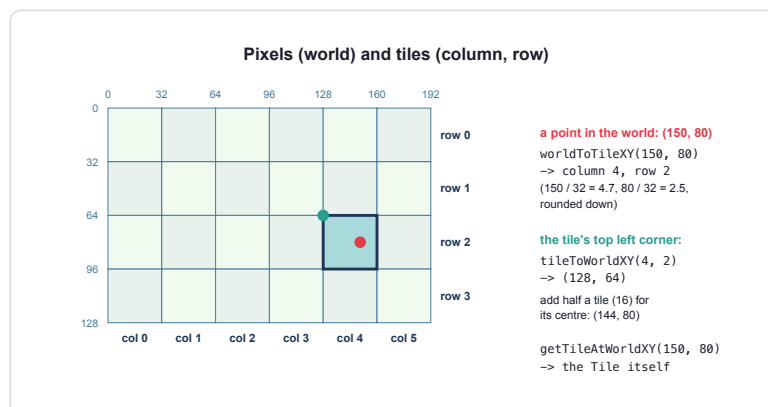
```

- `EMPTY` is `-1` : no tile. `0` is grass!



9 / 20

Pixels and tiles



`pointer.worldX`, not `pointer.x` - it allows for the camera

10 / 20

Save, load, print

```
localStorage.setItem(SAVE_KEY, JSON.stringify(this.toArray()));
...
let data: unknown = null;
...
if (!this.isMap(data)) {
  ...
}
this.layer.putTilesAt(data, 0, 0);
...
private isMap(value: unknown): value is number[][] {
```

- `unknown` - could be anything: check it
- `value is number[][]` - a **type guard**
- P prints the map as TypeScript - paste it into `ch10_array_map`

11 / 20

Try it: Tile Painter

- paint an island: 1-4 choose, 0 erases, drag to paint
- S, C, L - save, clear, load
- P, then look in the browser's console (F12)
- paste the result into `ch10_array_map/src/level.ts` and walk round it

12 / 20

A cave made in code

Making a cave: noise, smoothed, then cut-off caves filled in



- `Cave` is plain data - no Phaser. **Model**, not view
- the scene decides what it looks like

13 / 20

Two layers

```
const floor = map.createBlankLayer("floor", tileset!);
floor.weightedRandomize([
  { index: FLOOR, weight: 12 },
  { index: FLOOR_CRACKED, weight: 1 },
  { index: FLOOR_MOSSY, weight: 1 },
]);

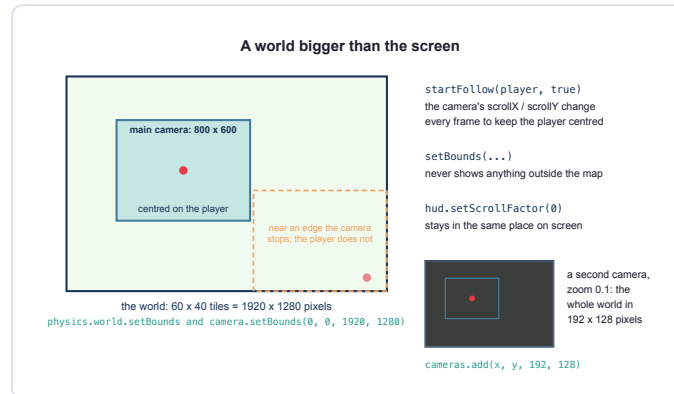
const walls = map.createBlankLayer("walls", tileset!);
walls.putTilesAt(this.wallTiles(cave), 0, 0);
walls.setCollisionByExclusion([EMPTY]); // every tile that is not empty collides
```

Walls: `-1` where the cave is open, so the floor shows through

14 / 20

The camera

```
this.physics.world.setBounds(0, 0, width, height);
this.cameras.main.setBounds(0, 0, width, height);
this.cameras.main.startFollow(this.player, true);    // true: round to whole pixels - no shimmer
this.hudText.setScrollFactor(0);                  // stays put on the screen while the camera moves
```



15 / 20

A minimap is just another camera

```
const minimap = this.cameras.add(MINI_X, MINI_Y, MINI_WIDTH, MINI_HEIGHT);
minimap.setZoom(MINI_ZOOM);
minimap.centerOn(width / 2, height / 2);

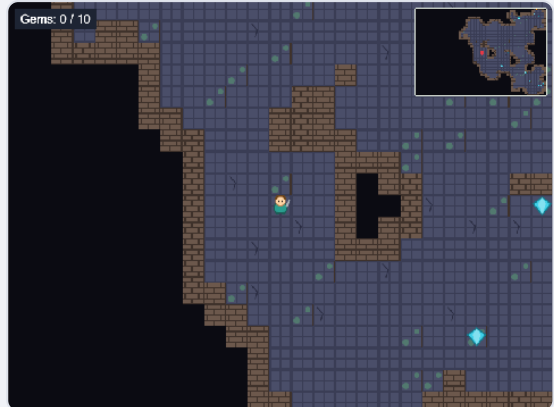
this.marker = this.add.circle(this.player.x, this.player.y, MARKER_RADIUS, 0xe63946);
this.cameras.main.ignore(this.marker);
minimap.ignore([frame, this.hudText]);
```

Every camera draws everything - unless told to `ignore` it

16 / 20

Try it: Scrolling World

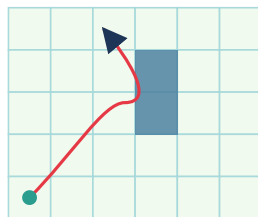
- find the ten gems; R for a new cave
- the same `Player` class as `ch10_array_map` - unchanged
- what happens without `minimap.ignore(...)` ? Without `setScrollFactor(0)` ?



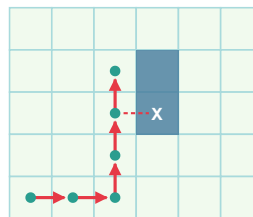
17 / 20

Free or grid movement?

Free movement and grid movement



FREE: velocity from the keys;
any pixel; the collider stops you
(this chapter's projects)



GRID: one whole tile per move, centre
to centre; look at the next tile first
(a wall or water there: do not move)

Zelda? Sokoban? Pac-Man? A roguelike?

18 / 20

Summary

- `tileset` = one picture; index from 0; `-1` = empty; `data[row][column]`
- `make.tilemap` (`data`) -> `addTilesetImage` (`picture`) -> `createLayer` (game object)
- `setCollision` + `physics.add.collider`
- `getTileAt(WorldXY)`, `worldToTileXY`, `tileToWorldXY` (top left corner)
- `putTileAt`, `removeTileAt`, `putTilesAt`, `fill`, `weightedRandomize`
- `camera.setBounds`, `startFollow`, `setScrollFactor(0)`, `cameras.add`, `ignore`

19 / 20

Challenges

1. **A new island** - moat, second bridge, maze, new start
2. **Sand slows you down** - half speed on sand
3. **Eyedropper** - right-click picks up a tile
4. **Treasure tiles** - chests in the map, collected by walking into them
5. **A map from strings** - `#`, `~`, `.`, `s`, `@`
6. **Grid movement** - one tile at a time, with tweens

Next: **Chapter 11 - Tilemaps with Tiled**

20 / 20

Tilemaps with Tiled

Overview

Chapter 10 built tilemaps from arrays in code. This chapter hands the level over to a tool: students draw maps in **Tiled**, export them as JSON, and load them into Phaser. The Phaser side is small - `tilemapTiledJSON`, `make.tilemap({ key })`, `addTilesetImage`, `createLayer`, `setCollisionByProperty`, and three ways into an object layer - so most of the learning is about the **workflow** (source file, export, build, refresh) and about the idea that a **level is data**: the code says what things do, the map says where they are and how they are tuned.

The one conceptual trap is **names as strings shared between two programs**. Tiled and the code must agree on the tileset name, every layer name, every object name and every property name, and a mismatch is not a build error - it is a console warning and a `null`. Teach students to keep the names as constants and to look in the browser console first.

Tiled is not on every lab machine. It is a free download (mapeditor.org) for Windows, macOS and Linux; check it is installed, and that it opens the chapter's maps, before the session. Without it, students can still do the Phaser half of the chapter with the maps provided, and challenges 2 and 4 can be done by editing the `.tmj` by hand (instructive in itself).

Prerequisites

- Chapter 10: tilemaps from arrays, tilesets, layers, `setCollision`, colliders with a layer, the camera following the player
- Chapters 7-9 for the hero animation and Arcade Physics used by the level (the chapter does not re-teach them)
- Chapter 2's `scene.start(key, data)` / `init(data)`, used again as `scene.restart(data)`

Learning outcomes

Students can:

1. make an orthogonal, fixed-size map in Tiled with an embedded tileset, tile layers, an object layer and custom properties on tiles, objects and the map
2. save the source (`.tmx`) and export the JSON (`.tmj`) to the right place, and explain what a GID and a firstgid are
3. load a Tiled map in Phaser and create its layers, matching the names used in Tiled
4. make tiles solid with `setCollisionByProperty` , and read any tile's custom properties
5. read an object layer with `findObject` , `filterObjects` , `createFromObjects` and `getObjectLayer(...).objects` , converting Tiled's rectangle positions to centres where needed
6. read custom properties on objects and maps, and explain why the code should cope with missing or wrongly typed values
7. join maps together with door objects whose properties name the target map and entrance

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play <code>ch11_tiled_level</code> . Ask: "where is this level in the code?" (It is not.) Open <code>level11.tmx</code> in Tiled on the projector
0:05 - 0:25	Live demo in Tiled (slides 3-8): New Map, New Tileset (embedded!), two tile layers, paint a few tiles, add <code>collides</code> to the ground tiles, an object layer with a <code>player</code> point and a couple of <code>coin</code> points, a map property. Save As in <code>tiled/</code> , Export As into <code>public/assets/maps/</code>
0:25 - 0:35	Slides 9-10: open the <code>.tmj</code> in Celbridge. GIDs and firstgid - ask "why does the first tile get GID 1, not 0?"
0:35 - 0:50	Slides 11-15: the four steps in <code>createMap()</code> , names that must match, <code>setCollisionByProperty</code> , press D. The three ways into an object layer; <code>getTiledProperty</code>
0:50 - 1:00	Slides 16-18: doors between maps, <code>scene.restart(data)</code> , loading a map on demand

Lab (2 hours)

Time	Activity
0:00 - 0:15	Install/start Tiled; open both projects' maps; build and run both projects
0:15 - 0:45	Guided: change <code>level11</code> in Tiled (move the goal, add a platform and two coins, change a slime's speed), export, build, refresh. Everyone must get round the edit-export-build loop at least twice
0:45 - 1:15	Challenges 1-3
1:15 - 2:00	Challenges 4-6

Key points to stress

- **Two files.** The game loads the `.tmj` , never the `.tmx` . "I changed the map and nothing happened" is nearly always a missing export (or a missing build)
- **Embed the tileset, fixed size, CSV.** Phaser cannot read external tilesets, infinite maps are awkward, and compressed tile data is not supported

- **Names are the contract** between Tiled and the code - constants in one file, spelled exactly
 - **GIDs are not tile indexes.** In a Tiled map, `tile.index` is the GID (grass top is 1, not 0). Choosing tiles by property avoids the whole question
 - **Tile properties vs object properties.** Phaser turns tile properties into an object (`tile.properties.hazard`) but leaves object and map properties as Tiled's list of `{ name, type, value }` - hence `getTiledProperty`
 - **Rectangles are measured from the top-left**, game objects from their centre
 - **Data can be wrong.** The designer is a person: default values and checks (`?? 0` , the fallback in `getTiledProperty` , `null` checks) are part of reading a map
 - In Phaser 4, **Tiled's tile animations play by themselves** (Phaser 3 needed a plugin or hand-written code - students will find old tutorials that say otherwise)
-

Common problems and errors

These messages were produced by breaking copies of the chapter projects.

What students see	Cause	Fix
console: Tilemap has no tileset "platform". Its tilesets are ['platform_tiles'], then the project's own error No tileset "platform" in the map, or no picture "platform_tiles" loaded	first argument of addTilesetImage does not match the tileset's name in Tiled	copy the name from Tiled's Tilesets panel (the warning lists the real names)
console: Texture key "platform_tiles" not found, then the same No tileset... error	the tileset picture was never loaded with this.load.image, or with a different key	load it; the second argument of addTilesetImage is the key
console: Invalid Tilemap Layer ID: ground and Valid tilelayer names: ['Background', 'Ground'], then TypeError: Cannot read properties of null (reading 'setCollisionByProperty')	layer name spelled differently (here, a lower-case g)	match Tiled exactly - capitals matter
console: No map data found for key level1, every layer invalid; Network tab shows a 404	wrong path in tilemapTiledJSON, or the map was never exported into public/assets/maps/	export to the right folder, then build
console: Failed to process file: tilemapJSON "level1" and SyntaxError: Unexpected token '<', "<?xml vers"... is not valid JSON	loading the .tmx instead of the .tmj	export JSON; load the .tmj
console: External tilesets unsupported. Use Embed Tileset and re-export, then TypeError: Cannot read properties of undefined (reading '2')	"Embed in map" was not ticked when the tileset was made	Embed Tileset button in the Tilesets panel, or the "Embed tilesets" export option, then export again

What students see	Cause	Fix
the player falls through everything; no error	no <code>collides</code> property, a differently spelled one, or a string property <code>"true"</code> rather than a bool	press D; check the property's name and type in Tiled
changes made in Tiled do not appear	not exported, or not built, or the browser shows the old page	File > Export (Ctrl+E), build, refresh
a coin or door is half a tile out	a rectangle's <code>x, y</code> used as its centre	add half the width and height
build error: <code>Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.</code>	storing <code>createLayer(...)</code> in a <code>TilemapLayer</code> field without the cast	<code>as Phaser.Tilemaps.TilemapLayer</code> (we did not ask for a GPU layer)
build error: <code>'spawn' is possibly 'null'.</code>	using <code>findObject</code> 's result without checking it	check for <code>null</code> (the map might not have the object)
build error: <code>Type 'number undefined' is not assignable to type 'number'.</code>	<code>spawn.x</code> used directly - Phaser's type makes <code>x</code> optional	<code>spawn.x ?? 0</code>
the door sends the player straight back	the entrance point is inside the other map's door rectangle	move the entrance a tile or two into the room
every coin tile collected at once (challenge 6)	the overlap with a tile layer reports empty places too	check <code>tile.index !== -1</code>

Suggested demos and live-coding moments

- **The contract, broken.** Rename the Ground layer in Tiled to "ground", export, refresh: the console warning lists the valid names. Rename it back. Then rename the tileset: a different warning

- **Data, not code.** Change a slime's `speed` property to 300 in Tiled, export, refresh. Then type `"fast"` into it (change its type to string): the fallback in `getTiledProperty` keeps the game running. Ask what would happen without the `typeof` check
- **GIDs.** Add `console.log(this.ground.getTileAt(0, 17).index)` at the end of `createMap()` : it prints 1, not 0, for a grass tile. Ask why
- **Draw order.** In Tiled, paint a bush on the Background layer over one of the crates, and drag Background above Ground in the Layers panel. Export: the bush is still hidden behind the crate, because Phaser draws layers in the order the code calls `createLayer` . Swap the two `createLayer` lines and it appears in front. Which wins - Tiled's order, or the code's?

Discussion questions

- What belongs in the map, and what in the code? (Positions, sizes, speeds, messages, which tiles are solid - map. What a coin does when touched, how a slime patrols - code.) Where would you put a slime's *damage*? The level's music?
- Why is `collides` better than `setCollision([1, 2, 3, 4, 5, 6, 11])` ? What happens to each when a designer adds a new solid tile to the tileset?
- `RoomScene` has no list of rooms. What are the advantages? What can go wrong that a list would have caught at start-up? (A mistyped `target` is only found when someone walks through that door.)
- The maps in this chapter were written by a script, not drawn in Tiled. When might a game generate its maps rather than have them drawn? (Roguelikes do exactly that.)

Extension ideas

- a level select: several `levelN.tmx` files and a menu that starts `GameScene` with the map key
- a tile property `friction` (float) on ice tiles, read under the player's feet to change their drag
- use a Tiled **object template** (`.tx`) for coins - and find out what Phaser does with it (templates must be detached on export: Preferences > Export Options > Detach templates)

- tile objects (Insert Tile, T) instead of points for coins, so the designer sees coins in the editor; `createFromObjects` with `gid` handles them
 - a minimap of the Tiled level with a second camera (Chapter 10)
-

Assessment ideas

- Practical: "Add a second level to `ch11_tiled_level`, reached by touching the flag, with at least one new kind of object configured by a custom property." Mark the map (in Tiled) as well as the code
 - Code reading: show `createMap()` and a screenshot of Tiled's Layers and Tilesets panels with one name different; ask what the console will say and which line fails
 - Short answer: explain GID and firstgid, and why a map with two tilesets needs them
 - Short answer: why does `getTiledProperty` take a fallback, and why does it check `typeof`?
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`. The changed maps are written by `tools/make_solution_maps.ts`.

Chapter 11 - Tilemaps with Tiled - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every code change is marked with a `// CHALLENGE n` comment.

Every challenge in this chapter also changes a **map**, which students do in Tiled. JSON cannot hold comments, so the solutions' maps were written by a script, `tools/make_solution_maps.ts`, which builds the chapter's maps and changes them - each change marked `CHALLENGE n` there instead. The maps open in Tiled like any other, so you can show the class what was changed. Each solution's README says which maps changed.

When marking, ask for both files: the `.tmx` in `tiled/` **and** the exported `.tmj` in `public/assets/maps/`. A student whose game does not show their change has almost always forgotten to export.

1. Above the player

Project: `solutions/ch11_challenge_1_above_the_player` (from `ch11_tiled_doors`)

Goal: see that layers are drawn in order, that depth can override it, and that a layer with no collision is walked through.

In Tiled: Layer > New > Tile Layer, named **Above**, dragged above Walls in the Layers panel. The solution paints a stone beam across the hall between the pillars, and the tops of the four pillars. Nothing on this layer has `collides` set, and the scene does not ask for it anyway.

`src/scenes/RoomScene.ts`

```
const ABOVE_DEPTH = 10; // CHALLENGE 1 - higher than the player (0), so
drawn over them
```

```

this.createPlayer(map);
this.physics.add.collider(this.player, walls);

// CHALLENGE 1 - the "Above" layer, drawn over the player. No collision is
// set on it, so the
// player walks underneath. Not every room has one: getLayer gives null for
// a layer that is not
// in the map (createLayer would warn in the console, and give null too).
if (map.getLayer(ABOVE_LAYER) !== null) {
    map.createLayer(ABOVE_LAYER, tiles).setDepth(ABOVE_DEPTH);
}

```

Making the layer after the player would be enough on its own - game objects are drawn in the order they are added - but `setDepth` says what is meant and survives someone moving the line. No cast is needed here: `setDepth` exists on both kinds of layer `createLayer` can return.

Tested with the harness: the player walks up through row 8 (the beam) without stopping, and is hidden while under it; in the cellar, which has no Above layer, there is no console warning.

Look for:

- the layer made **after** the player, or given a depth - students who make it with the other layers find it drawn *under* the player and are puzzled
- the cellar still working: a student who calls `createLayer("Above", ...)` unconditionally gets *Invalid Tilemap Layer ID: Above* in the console for the cellar (harmless here, but a good moment to talk about checking). Adding an empty Above layer to every map is an equally good answer
- not setting collision on the Above layer

2. Gems

Project: `solutions/ch11_challenge_2_gems` (from `ch11_tiled_level`)

Goal: a custom property on an object, read through `createFromObjects` ' data.

In Tiled: select the coins to change (several can be selected at once with Select Objects), and add an **int** property **value** = 5. The solution changes the four coins above the brick wall.

src/scenes/GameScene.ts

```
// CHALLENGE 2 - createFromObjects has put each coin's custom properties
// into its data. A coin
// with a "value" is shown as a gem; the others spin. (The whole group can
// no longer be told to
// play the spin animation - it would turn the gems back into coins.)
for (const sprite of coinSprites) {
    const coin = sprite as Phaser.GameObjects.Sprite; // createFromObjects
    // made Sprites (its default)
    const value: number = coin.getData(VALUE) ?? 1;
    coin.setData(VALUE, value);
    if (value > 1) {
        coin.setTexture(GEM_KEY);
    } else {
        coin.play(COIN_SPIN);
    }
}
```

```
this.score = this.score + coin.getData(VALUE); // CHALLENGE 2
```

A coin with no property gets `undefined` from `getData`, so `?? 1` gives it the default - and `setData` stores it, so collecting is one line. `gem.png` is copied into `public/assets/images/` and loaded in `preload()`.

Tested with the harness: 12 coins have value 1 and 4 are gems with value 5; collecting a gem makes the score 5, then a coin makes it 6.

Look for:

- the property read from the sprite's **data** (`getData`), not from `obj.properties` - both work, but `createFromObjects` has already done the work. Students who loop over `getObjectLayer` and use `getTiledProperty` have a perfectly good answer too
- `this.coins.playAnimation(COIN_SPIN)` removed or replaced - left in, it turns every gem back into a spinning coin, a nice bug to discuss
- the score reset in `init()` (Chapter 2's trap: R restarts the scene)

- a common wrong turn: a **string** property `"5"` . `getData` then returns the string, and `score + "5"` makes `"05"` . The type in Tiled matters

3. A third room

Project: `solutions/ch11_challenge_3_a_third_room` (from `ch11_tiled_doors`)

Goal: understand that the level is data. The right answer changes **no code**.

In Tiled:

1. make a new map, `tiled/treasure.tmx` (the solution's is 17 x 9 tiles). The quickest way is to open the cellar, **File > Save As** `treasure.tmx` , and change the size (Map > Resize Map) and the tiles: the embedded tileset, its `collides` properties and the layers all come with it. Starting from File > New works too, but then the tileset must be made again - embedded, named **dungeon_tiles**, with `collides` ticked on the same tiles
2. layers **Floor**, **Walls** and **Objects**, spelled as in the other rooms; a map property **name** = `The Treasure Room`
3. in the new room: a point `from_hall` , and a `door` rectangle with `target` = `hall` , `entrance` = `from_treasure`
4. in the hall: repaint the closed door (tile 5) as an open door (tile 6), add a `door` rectangle over it with `target` = `treasure` and `entrance` = `from_hall` , and a point `from_treasure` two tiles below it
5. export both maps to `public/assets/maps/` - the new one's file name must be the `target : treasure.tmj`

`RoomScene` loads `assets/maps/<target>.tmj` on demand, finds the entrance by name and centres the camera on whatever size of room it gets - so nothing in `src/` changes.

Tested with the harness: walking up the carpet goes from the hall to the treasure room, arriving at `from_hall` ; walking down through its door returns to the hall at `from_treasure` .

Look for:

- no code changes. If a student changed code, ask what for - often they added the map to a list in `preload()` , which works but misses the point

- entrance points placed a tile or two **away** from the doors (otherwise the player arrives in the doorway and is sent straight back - students meet this and have to reason about it)
 - the three names that must agree across two files: the `target` and the new map's file name; the `entrance` and the point's name; the layer and tileset names
 - common failures: the file exported as `Treasure.tmj` with `target = treasure` (a 404 - file names on the web are case-sensitive), a non-embedded tileset (*External tilesets unsupported. Use Embed Tileset and re-export*), or forgetting to export the hall after changing it
-
-

4. Signposts

Project: `solutions/ch11_challenge_4_signposts` (from `ch11_tiled_level`)

Goal: trigger areas from rectangle objects, a string property, and asking "am I in it?" every frame.

In Tiled: rectangles named **sign**, each with a **string** property **message**. The solution has three (by the sign at the start, before the lava, and before the flag) and paints two more sign tiles in the Background layer. The first message has two lines; in the JSON a line break is written as `\n`, and Phaser's `Text` shows it as a new line.

`src/scenes/GameScene.ts`

```
// CHALLENGE 4 - a signpost: a rectangle from Tiled, and the message to show
while the player is in it
interface Sign {
  zone: Phaser.GameObjects.Zone;
  message: string;
}
```

```

for (const obj of signObjects) {
    const width = obj.width ?? 32;
    const height = obj.height ?? 32;
    // Tiled measures a rectangle from its top-left corner; a Zone is placed
    // by its centre
    const zone = this.add.zone((obj.x ?? 0) + width / 2, (obj.y ?? 0) + height
    / 2, width, height);
    this.physics.add.existing(zone, true);
    this.signs.push({ zone: zone, message: getTiledProperty(obj.properties,
    "message", "") });
}

```

```

// CHALLENGE 4 - show the message of the sign the player is standing by, or
// hide the box.
// physics.overlap with no callback just answers the question: are these two
// touching right now?
private updateSignMessage(): void {
    const sign = this.signs.find((s) => this.physics.overlap(this.player,
    s.zone));
    if (sign === undefined) {
        this.messageText.setVisible(false);
    } else {
        this.messageText.setText(sign.message).setVisible(true);
    }
}

```

The message box is one `Text` with a `backgroundColor` and `padding`, fixed to the camera with `setScrollFactor(0)`, and hidden until needed.

Tested with the harness: at the start the box shows the controls; walking right out of the first sign's rectangle hides it; standing by the lava sign shows "Lava ahead - jump it!".

Look for:

- the top-left to centre conversion (the classic bug: the message appears half a sign to the left)
- hiding the box again. The usual first attempt uses `this.physics.add.overlap(player, zone, callback)`, which says when the player is **in** a sign but never when they have **left** - so the message never goes away. The hint points at `physics.overlap` in `update()`; a "was I overlapping last frame?" flag is another good answer

- the message stored with its zone (an interface, or `zone.setData("message", ...)`) rather than a second array kept in step

5. Moving platforms

Project: `solutions/ch11_challenge_5_moving_platforms` (from `ch11_tiled_level`)

Goal: a polyline's points, a chain of tweens, an immovable body - and discovering that Arcade Physics does not carry a rider sideways on a tweened platform.

In Tiled: erase the floating platform over the wide pool, then draw a polyline with Insert Polygon (click the points, then press Enter - or right-click - to finish without closing it), and name it **platform**. The solution's goes across the pool at ground level and then up, so the player can ride it to the higher platform.

`src/scenes/GameScene.ts`

```
// a polyline's points are measured from the object's x and y: turn them  
into world positions  
const points = obj.polyline.map((p) => new Phaser.Math.Vector2((obj.x ?? 0)  
+ p.x, (obj.y ?? 0) + p.y));  
  
const platform = this.add.image(points[0].x, points[0].y,  
PLATFORM_KEY).setScale(PLATFORM_SCALE);  
this.physics.add.existing(platform);  
const body = platform.body as Phaser.Physics.Arcade.Body; // dynamic: made  
by physics.add.existing  
body.setAllowGravity(false); // it floats  
body.setImmovable(true); // the player cannot push it
```

```

// there and back: each point after the first, then back through each point
before the last
const route = [...points.slice(1), ...points.slice(0, -1).reverse()];
let from = points[0];
const steps: Phaser.Types.Tweens.TweenBuilderConfig[] = [];
for (const to of route) {
    const distance = Phaser.Math.Distance.BetweenPoints(from, to);
    steps.push({
        targets: platform,
        x: to.x,
        y: to.y,
        duration: (distance / PLATFORM_SPEED) * 1000,
        completeDelay: PLATFORM_WAIT,
    });
    from = to;
}
this.tweens.chain({ tweens: steps, loop: -1 });

```

Each step's duration comes from its length, so the platform moves at a steady speed. `completeDelay` makes it wait at each point. (`hold` looks tempting, but it only applies to tweens that `yoyo` .)

Arcade's collider lifts the player with the platform, but a tween changes the platform's `x` directly, so its body has no velocity and the player is not carried sideways - the platform slides out from under them. The solution carries the player itself, every frame:

```

private carryPlayer(): void {
    const feet = this.player.body as Phaser.Physics.Arcade.Body;
    for (const platform of this.platforms) {
        const top = platform.image.body as Phaser.Physics.Arcade.Body;
        const standingOn = Math.abs(feet.bottom - top.top) < 2 && feet.right >
top.left && feet.left < top.right;
        if (standingOn) {
            this.player.x = this.player.x + (platform.image.x - platform.lastX);
        }
        platform.lastX = platform.image.x;
    }
}

```

Tested with the harness: over two complete trips the player's position relative to the platform stayed exactly the same (7 pixels left of its centre, 32 above), across, up, down

and back.

Look for:

- the points turned into world positions (forgetting to add `obj.x / obj.y` puts the platform near the top-left corner of the map)
 - a body that ignores gravity and cannot be pushed - otherwise it falls into the pool, or the player pushes it along
 - some way of carrying the player. A good, alternative answer is `body.setDirectControl(true)`, which tells Arcade the body is moved by setting its position and makes it work out a velocity, so the collider carries the rider. In testing it carried the player less reliably than the hand-written version (the rider drifted across the platform over a few trips), which is worth discussing. Moving the platform by velocity instead of tweens (`this.physics.moveTo`) also works, and Arcade then carries the player by itself; it needs more code to know when each point is reached. Chapter 15 comes back to moving platforms
-
-

6. Spinning coins, as tiles

Project: `solutions/ch11_challenge_6_spinning_coins_as_tiles` (from `ch11_tiled_level`)

Goal: a second tileset and firstgid, Tiled's tile animations, and collecting tiles with an overlap and `removeTileAt` .

In Tiled:

1. File > New > New Tileset: `coin_spin.png` (in `public/assets/spritesheets/`), 32 x 32, **embedded**, named **coins**. Tiled gives it firstgid 17, because `platform_tiles` uses 1-16
2. Edit Tileset, select tile 0, and open the **Tile Animation Editor** (a button on the tileset editor's tool bar). Drag the six tiles into the frame list, each 100 ms
3. a new tile layer, **Pickups**, above Ground; paint coin tile 0 where the coins should be. The editor shows them spinning

`src/scenes/GameScene.ts`

```
// CHALLENGE 6 - the second tileset, "coins", joined to the coin_spin
// picture. Its tiles have GIDs
// from 17 (its firstgid), so they cannot be mixed up with platform_tiles'
// 1-16. Phaser 4 plays
// the tile animation made in Tiled's Tile Animation Editor by itself.
const coinTiles = this.map.addTilesetImage(COINS_TILESET_NAME, COIN_KEY);
if (coinTiles === null) {
    throw new Error(`No tileset "${COINS_TILESET_NAME}" in the map, or no
    picture "${COIN_KEY}" loaded`);
}
this.pickups = this.map.createLayer(PICKUPS_LAYER, coinTiles) as
Phaser.Tilemaps.TilemapLayer;
```

```
// CHALLENGE 6 - an overlap with a tile layer reports EVERY tile under the
// player's body,
// including the empty places (index -1), so only a real coin tile counts
this.physics.add.overlap(this.player, this.pickups, (_player, object) => {
    const tile = object as Phaser.Tilemaps.Tile; // from a tile layer, the
    second object is a Tile
    if (tile.index !== -1) {
        this.collectCoinTile(tile);
    }
});
```

```
// CHALLENGE 6 - take the coin tile out of the layer, and count it
private collectCoinTile(tile: Phaser.Tilemaps.Tile): void {
    this.pickups.removeTileAt(tile.x, tile.y);
}
```

The `coin_spin` sprite sheet, already loaded for the object coins, is used as the tileset's picture as well - one texture can be both. Coin tiles are counted with

```
this.pickups.filterTiles((tile: Phaser.Tilemaps.Tile) => tile.index !== -1) .
```

Tested with the harness: the map has tilesets `platform_tiles` (firstgid 1, 16 tiles) and `coins` (firstgid 17, 6 tiles); `getAnimatedTileId(17, ms)` steps 17, 18, 19, 20 as time passes, and two screenshots 170 ms apart show the coins at different frames; running right along row 16 collects three coin tiles, and the total counts 16 object coins + 10 tiles.

Look for:

- `addTilesetImage` called twice, once per tileset, and the Pickups layer given the right one (or an array of both, if a student paints both tilesets on one layer)
- `tile.index !== -1` in the overlap - without it the first empty place counts as a coin, and the count races up as soon as the game starts
- students who follow Phaser 3 tutorials and write their own tile animation code: Phaser 3 needed a plugin or hand-written code for Tiled's tile animations; **Phaser 4 plays them itself** (both kinds of tilemap layer). A student who animates by hand with a timer and `putTileAt` has done more work than needed, but understood the idea - credit it
- the firstgid question from the hint answered: 17

Chapter 11 - Tilemaps with Tiled - Slides

Chapter 11

Tilemaps with Tiled

Draw the level. Load the level.



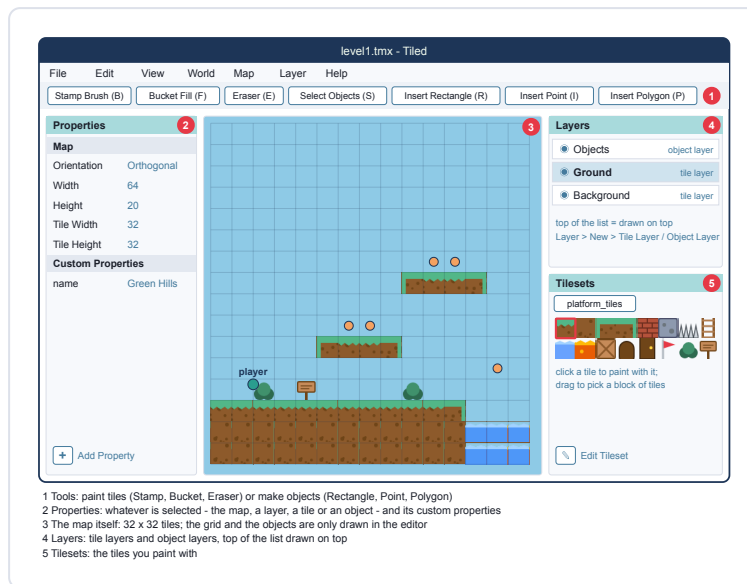
1 / 20

Today

- **Tiled:** tilesets, tile layers, object layers, custom properties
- `.tmx` (source) and `.tmj` (what the game loads)
- GIDs
- `tilemapTiledJSON`, `make.tilemap`, `addTilesetImage`, `createLayer`
- solid tiles **by property**
- reading objects: spawn points, coins, enemies, doors
- the level is **data**, not code

2 / 20

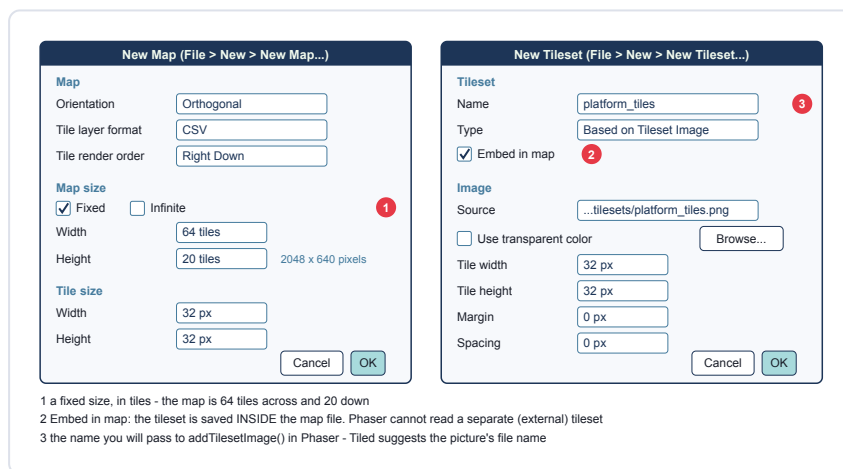
Tiled



Free, mapeditor.org - Windows, macOS, Linux

3 / 20

A new map, a new tileset



Orthogonal · fixed size · CSV · tileset embedded in map

4 / 20

Layers, and painting

- the map starts with one tile layer: rename it **Ground**
- Layer > New > Tile Layer: **Background**, below Ground
- top of the Layers list = drawn on top
- Stamp Brush (B), Bucket Fill Tool (F), Eraser (E)
- **check which layer is selected before you paint**

5 / 20

Custom properties on tiles

The tileset, with a custom property on some of its tiles

0 grass top, 1 dirt, 2 grass left, 3 grass right, 4 brick, 5 stone, 6 spikes, 7 ladder, 8 water, 9 lava, 10 crate, 11 door top, 12 door, 13 flag, 14 bush, 15 sign

☒ collides = true (solid: the player stands on these)
☐ hazard = true (touch one and you start again)

Tiles with no box have no custom properties at all.
The numbers are tile INDEXES within the tileset, from 0 (see the GIDs diagram).

Properties (7 tiles selected)

Tile	
ID	0
Class	
Probability	1

Custom Properties

collides ☒ true 1

1 tick it - the value is true
2 select several tiles first to add it to them all

+ Add Property: name collides, type bool 2

Edit Tileset → select tiles → + → **collides** , **bool**, ticked

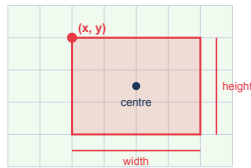
6 / 20

Objects

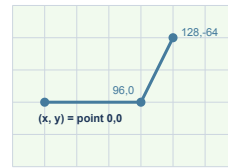
Where Tiled measures each kind of object from



Point: x and y are the point.
A sprite placed there is centred on it.



Rectangle: x, y is the TOP-LEFT corner.
A Zone is placed by its centre:
(x + width / 2, y + height / 2)



Polyline: a list of points,
each measured FROM (x, y).

Layer > New > Object Layer · Insert Point (I) · Insert Rectangle (R) · **Name** them: `player` , `coin` , `enemy` , `goal`

7 / 20

Save, then export

`tiled/level1.tmx`

`public/assets/maps/level1.tmj`

File > Save

- you edit this

File > Export As...

- the game loads this

- after the first time: **File > Export** (Ctrl+E / Cmd+E)
- then **build**, then **refresh**
- "I changed the map and nothing happened" = not exported

8 / 20

Inside a .tmj

level1.tmj - the map

```
"type": "map",
"orientation": "orthogonal",
"width": 64, "height": 20,
"tilewidth": 32,
"tileheight": 32,
"infinite": false,
"properties": [ ... ],
"tilesets": [ ... ],
"layers": [ ... ],
"nextlayerid": 4,
"nextobjectid": 21,
"version": "1.10"
```

Keys are written in alphabetical order;
"..." marks what has been left out here.

Phaser reads all of it with
`this.load.tilemapTiledJSON(key, file)`
and builds the map with
`this.make.tilemap({ key })`

tilesets[0] - embedded in the map

```
"name": "platform_tiles",      <- addTilesetImage("platform_tiles", ...)
"firstgid": 1,
"image": "../tilesets/platform_tiles.png",
"columns": 8, "tilecount": 16, "tilewidth": 32, ...
"tiles": [ { "id": 0, "properties": [
  { "name": "collides", "type": "bool", "value": true } ] }, ... ]
```

layers[0] and [1] - tile layers

```
"name": "Ground", "type": "tilelayer",      <- createLayer("Ground", ...)
"width": 64, "height": 20,
"data": [ 0, 0, 0, ... ]                  one GID per tile, row by row:
      1, 1, 1, 1, 4, 0, 0, 0, 3, ... ] 64 x 20 = 1280 numbers
```

layers[2] - the object layer

```
"name": "Objects", "type": "objectgroup", <- getObjectLayer("Objects")
"objects": [
  { "id": 1, "name": "player", "point": true, "x": 64, "y": 520, ... },
  { "id": 2, "name": "coin", "point": true, "x": 200, "y": 432, ... },
  { "id": 18, "name": "enemy", "point": true, "x": 1232, "y": 520,
    "properties": [ { "name": "speed", "type": "float", "value": 60 } ] },
  { "id": 20, "name": "goal", "x": 1952, "y": 480,
    "width": 32, "height": 64, ... }
]
```

9 / 20

GIDs

Tile index (in the tileset) and GID (in the map)

tileset												
index	0	1	2	3	4	5	6	7	8	9	10	11
GID	1	2	3	4	5	6	7	8	9	10	11	12

GID = firstgid + index. This tileset's firstgid is 1, so GID 0 is left free to mean "no tile".
A second tileset in the same map would start where this one ends - firstgid 17.

row 17 of the Ground layer, columns 7 to 16, as saved in the .tmj:

drawn												
tile.index	1	1	1	1	4	-1	-1	-1	3	1		

In Phaser a Tiled tile's index IS its GID - grass top is 1, not 0 - and an empty place is -1.
So `setCollision(1, 2, 3)` means GIDs. Choosing tiles by property avoids counting at all.

10 / 20

Loading the map

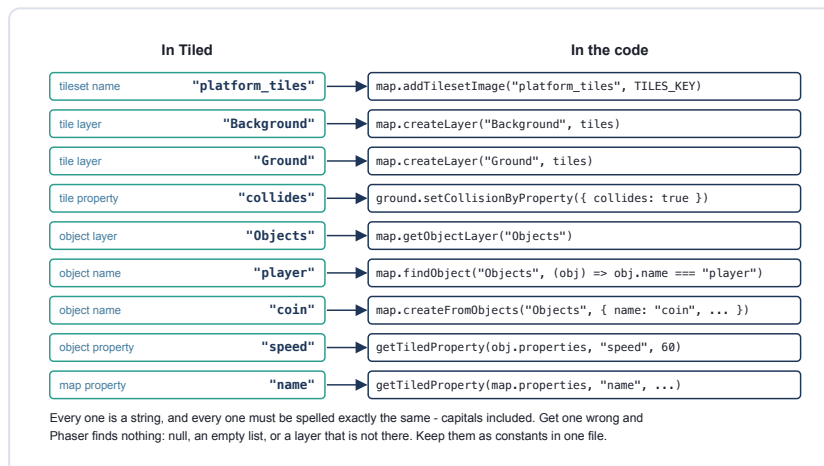
```
// preload()
this.load.tilemapTiledJSON(MAP_KEY, MAP_FILE); // the .tmj
this.load.image(TILES_KEY, TILES_FILE);       // its picture - separately

// create()
this.map = this.make.tilemap({ key: MAP_KEY });
const tiles = this.map.addTilesetImage(TILESET_NAME, TILES_KEY);
this.map.createLayer(BACKGROUND_LAYER, tiles);
this.ground = this.map.createLayer(GROUND_LAYER, tiles) as Phaser.Tilemaps.TilemapLayer;
this.ground.setCollisionByProperty({ collides: true });
```

`addTilesetImage(name in Tiled, key in Phaser)`

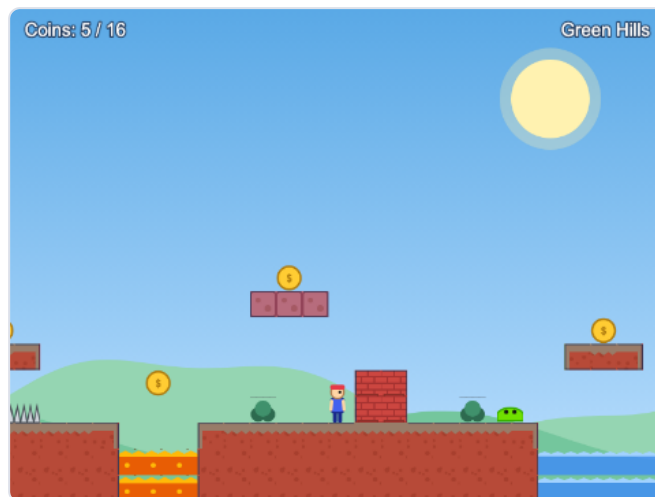
11 / 20

The names must match



12 / 20

Press D



```
tiles.some((tile) => tile.properties.hazard === true); // any tile property
```

13 / 20

Three ways into an object layer

```
// one object
const spawn = this.map.findObject(OBJECTS_LAYER, (obj) => obj.name === "player");

// a Sprite for every match
const coinSprites = this.map.createFromObjects(OBJECTS_LAYER, { name: "coin", key: COIN_KEY });

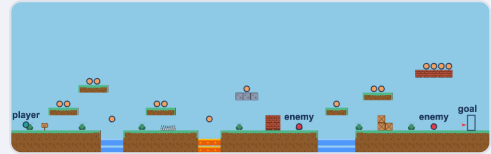
// all of them, to do as you like with
for (const obj of objectLayer.objects) {
  if (obj.name === "enemy") {
    const speed = getTiledProperty(obj.properties, "speed", DEFAULT_SLIME_SPEED);
  }
}
```

Object and map properties: a list of { name, type, value }

14 / 20

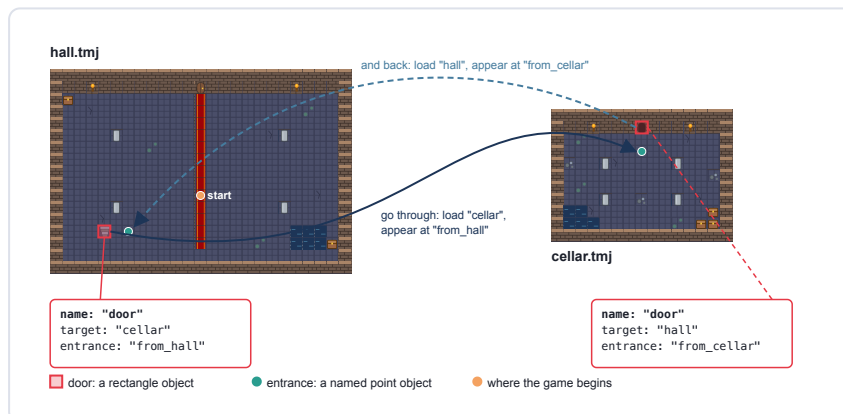
Try it: Tiled Level

- build, serve, play `ch11_tiled_level`
- open `tiled/level1.tmx` in Tiled
- change a slime's `speed` ; add a platform and two coins
- export, build, refresh



15 / 20

Doors between maps



16 / 20

One scene, restarted

```
init(data: Partial<RoomData>): void {
  this.room = {
    map: data.map ?? FIRST_ROOM.map,
    entrance: data.entrance ?? FIRST_ROOM.entrance,
  };
  this.leaving = false;
}

// preload(): only this room's map - skipped if already in the cache
this.load.tilemapTiledJSON(this.room.map, MAPS_FOLDER + this.room.map + MAP_EXTENSION);

// goThrough(door), after the fade:
const next: RoomData = { map: door.target, entrance: door.entrance };
this.scene.restart(next);
```

17 / 20

Try it: Tiled Doors

- down the stairs; back through the cellar door
- where is the list of rooms? (there isn't one)
- why are the entrance points away from the doors?



18 / 20

Summary

- Tiled: tilesets, tile layers, object layers, custom properties
- edit `.tmx` , **export** `.tmj` ; embedded tileset, fixed size, CSV
- GID = firstgid + index; in Phaser, `tile.index` is the GID, empty = -1
- `tilemapTiledJSON` → `make.tilemap({ key })` → `addTilesetImage` → `createLayer`
- `setCollisionByProperty({ collides: true })` ; `tile.properties`
- `findObject` , `filterObjects` , `createFromObjects` , `getObjectLayer(...).objects`
- names are the contract: constants, spelled exactly

19 / 20

Challenges

1. **Above the player** - a layer drawn over the player
2. **Gems** - coins with a `value` property
3. **A third room** - behind the closed door (how much code?)
4. **Signposts** - rectangles with messages
5. **Moving platforms** - along a polyline
6. **Spinning coins, as tiles** - a second tileset, animated in Tiled

Next: **Chapter 12 - Higher or lower**

20 / 20

Higher or lower

Overview

The first genre chapter. Students meet one small game three times - simple, intermediate, advanced - and the real subject is **how a game's code grows**: from one scene with a few fields, to a model (`Card` , `Deck` , `rules.ts`) kept apart from a view (`CardSprite` , `Button` , `HistoryRow`), with a scene in between. Along the way: a union-of-strings state machine that ignores input at the wrong moment, a Fisher-Yates shuffle (and why the popular `sort` trick is biased), a flip made of two tweens, a reusable button, and a risk-and-reward scoring system. There is little new Phaser here; there is a lot of design.

It is also the first chapter where students should *play critically*: why is the advanced version more fun than the simple one, when the underlying luck is the same?

Prerequisites

- Chapters 1-2: scenes, `init(data)` , interfaces, pointer input, the "scenes are reused" trap
- Chapter 3: a preload scene; Chapter 5: playing sounds; Chapter 6: local storage
- Chapter 7: tweens, tween chains, particles
- Java: `final` fields, getters, enums, `Predicate<T>` (for challenges 3 and 6)

Learning outcomes

Students can:

1. separate a game's model (plain TypeScript data and rules) from its view (Phaser game objects), and explain what each change touches
2. use unions of string literals for small sets of values (suits, guesses, outcomes, states)
3. implement a Fisher-Yates shuffle and explain why `sort(() => Math.random() - 0.5)` is biased
4. keep a game state, and ignore input while the game is busy

5. animate a card flip with a tween chain and change a sprite's frame (or texture) mid-animation
6. build a reusable `Container`-based button with hover, pressed and disabled states
7. design scoring that rewards risk (a pot, a multiplier, a cash-out choice) and persist records

Suggested session plan (2 x 2 hours)

Session 1 - simple and intermediate

Time	Activity
0:00 - 0:10	Play all three versions for two minutes each, in order. Ask: "which is most fun, and <i>why</i> ? The luck is identical." Collect answers on the board (risk, streak, reveal)
0:10 - 0:30	Slides 3-7: genre notes, the simple version - <code>Card</code> , frames, <code>judge()</code> , <code>GameState</code> . Walk through <code>GameScene.guess()</code>
0:30 - 0:45	Live demo: delete the state check at the top of <code>guess()</code> , rebuild, and mash H. Ask what went wrong before explaining
0:45 - 1:00	Challenge 1 (aces high). Then ask who changed <code>rank</code> instead of <code>value</code> - see Common problems
1:00 - 1:25	Slides 8-11: model and view, the deck, the shuffle bias (run the experiment - see Demos), <code>draw()</code> and <code>Card undefined</code>
1:25 - 1:40	Slides 12-14: the flip, buttons, the state machine. Live-code: change <code>Sine.easeIn / easeOut</code> to <code>Linear</code> , then change <code>scaleX</code> to <code>scale</code>
1:40 - 2:00	Challenges 2 and 3

Session 2 - advanced

Time	Activity
0:00 - 0:20	Slides 15-17: the pot, multipliers, cash out. Discussion: "with only a score, what is the best strategy?" (always guess the likely way - no decisions). "With a pot?"
0:20 - 0:35	Slide 18: dealing, sprites that swap jobs, the history row, records, polish
0:35 - 0:45	Polish exercise: turn off one piece of polish at a time (comment out the shake, the float text, the sparkles). Which one did the class miss most?
0:45 - 1:15	Challenge 4 (jokers wild)
1:15 - 2:00	Challenges 5 and 6. Finish with the computer playing (challenge 6) on the projector

Key points to stress

- **The model never imports Phaser.** Point out that `Card.ts`, `Deck.ts` and `rules.ts` could run in a plain Deno script (they can - see Demos). The next chapter (memory match) needs a deck too
- **The scene asks, then tells.** `judge()` returns an outcome; the scene decides what sound and text go with it. Rules never play sounds
- **One gate for all input.** H, L and both buttons all call `guess()`; the state check lives there once. If students add a new input (challenge 5's D, challenge 6's computer), it must go through the same gate
- **Order of the reveal.** `reveal()` runs *after* the flip completes. Play the result sound before the card is visible and the tension goes - worth demonstrating
- **The shuffle is a correctness issue, not style.** The chart in the chapter is real data from V8, the engine in Chrome and Deno
- `value` **versus** `rank`. Two names for what are equal today and different tomorrow (aces high)

Common problems and errors

What students see	Cause	Fix
Every card shows as the next rank up; the king of spades shows as a card back	frame formula missing <code>- 1</code> : <code>suit * 13 + rank</code>	<code>suit * 13 + (rank - 1)</code>
Aces high "works" but the ace of spades is a card back	challenge 1 done by changing <code>rank</code> to 14, so the frame is <code>suit * 13 + 13</code>	change <code>value</code> , not <code>rank</code>
Mashing H deals two cards and judges both against the same card to beat	no <code>state</code> check at the top of <code>guess()</code>	<code>if (this.state !== "waiting") return;</code>
Card shrinks to a dot instead of flipping	tweening <code>scale</code> instead of <code>scaleX</code>	<code>scaleX</code> only
<code>TypeError: this.showFace is not a function</code> at the first flip (in the browser console); the build was fine	<code>onComplete: function () { this.showFace(card); } - a function has its own <code>this</code></code>	arrow function: <code>onComplete: () => { ... }</code>
<code>Type 'Card undefined' is not assignable to type 'Card'.</code>	<code>draw()</code> returning <code>this.cards.pop()</code> directly	check for <code>undefined</code> first (and throw)
<code>Error: The deck is empty</code> (console, in <code>Deck.draw</code>)	drawing from an empty deck - e.g. a challenge that plays through the whole deck	check <code>isEmpty()</code> before drawing
<code>Property 'add' in type 'HistoryRow' is not assignable to the same property in base type 'Container'. (and This member must have an 'override' modifier because it overrides a member in the base class 'Container'.)</code>	naming a method <code>add</code> in a <code>Container</code> subclass	pick another name, e.g. <code>addCard</code>
<code>Argument of type '"down"' is not assignable to parameter of type 'Guess'.</code>	<code>this.guess("down")</code>	the union type doing its job - use <code>"lower"</code>

What students see	Cause	Fix
This comparison appears to be unintentional because the types 'Guess' and '"hihger"' have no overlap.	a typo in a string being compared	the union catches typos in comparisons too
Cannot assign to 'rank' because it is a read-only property.	trying to change a card (often while attempting aces high)	cards do not change; change <code>value</code> 's getter
Texture "cards" has no frame "joker" (console warning) and the ace of clubs appears	<code>setFrame(JOKER_KEY)</code> for a texture that is not in <code>cards.png</code>	<code>setTexture(JOKER_KEY)</code>
No sound until the first click	the browser's autoplay rule (Chapter 5)	expected; the title screen's PLAY click unlocks audio

Suggested demos and live-coding moments

- **The shuffle experiment.** In a scratch file, run in Deno:

```
const counts: Record<string, number> = {};
for (let i = 0; i < 600000; i++) {
  const order = ["A", "B", "C"].sort(() => Math.random() - 0.5).join("");
  counts[order] = (counts[order] ?? 0) + 1;
}
console.log(counts);
```

ABC and BAC come out about 150,000 times each; the other four about 75,000. Then swap in Fisher-Yates and run again (100,000 each). Students believe a number they watched appear

- **The model runs without Phaser.** `deno eval 'import { Deck } from "./src/Deck.ts"; const d = new Deck(); console.log(String(d.draw()), String(d.draw()));'` from the intermediate project folder. No browser, no Phaser - that is what model/view buys
- **Remove the state check** (session 1) - and then, in the advanced version, remove the `multiplier === 0` part of the check in `guess()` and try H on a king (nothing breaks; the guess is just always lost - why was it greyed out?)

- **Easing:** `Sine.easeIn` / `easeOut` versus `Linear` on the flip; `Back.easeOut` on the title's axes
 - **Turn the polish off** one piece at a time (session 2)
-

Discussion questions

- The simple, intermediate and advanced versions have the same odds. Why is the advanced one more fun? Which single feature adds the most?
 - A tie "does not count". What would change if a tie counted as a loss? As a win? Where in the code would you change it, and what else would need to change (text, the title screen)?
 - `Card` has no `x`, `y` or `faceUp`. Where do those live, and why? What would go wrong if `Card` extended `Phaser.GameObjects.Image`?
 - `winningRanks()` pretends all 13 ranks are equally likely. When is that badly wrong? (Late in the deck.) Should the multipliers use the real deck? Would that make the game better or just harder to understand?
 - Is the pot-and-cash-out design fair? What does a cautious player's score look like against a reckless one's? (Challenge 6 lets you test it.)
 - `GameOverScene` in the intermediate version handles both winning and losing. When should an ending be its own scene?
-

Extension ideas

- a "bet" before each guess: the player chooses how much of their banked score to risk
- sound pitch rising with the streak (`this.sound.play(key, { detune: inARow * 100 })`)
- a two-player version, taking turns, with the history row shared
- move `Card`, `Deck` and `Button` into a shared folder and use them in Chapter 13's memory game
- unit tests for `judge()`, `riskMultiplier()` and `Deck` with `Deno.test` - they need no browser

Assessment ideas

- Practical: "add a 'same' guess that pays x5 and wins only on a tie" - touches `Guess`, `judge()`, `riskMultiplier()`, a new button, and nothing in `Card` or `CardSprite`. Marking focuses on where the changes went
- Code reading: show a shuffle written with `sort(() => Math.random() - 0.5)` and ask what is wrong and how to show it is wrong
- Short answer: explain, with the flip as the example, why `reveal()` is called from the tween's `onComplete` rather than straight after `flipTo()`
- Design: "list three changes that would only touch the view, and three that would only touch the model"

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 12 - Higher or lower - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (`<!-- CHALLENGE n -->` in HTML).

1. Aces high

Project: `solutions/ch12_challenge_1_aces_high` (from `ch12_higher_or_lower_simple`)

Goal: see the point of `value` being separate from `rank`.

`src/Card.ts`

```
// what "higher" and "lower" compare.  
// CHALLENGE 1: aces are high - an ace is worth 14, one more than a king.  
// The rank stays 1, so the  
// ace's picture (and toString) are unchanged: only the rules see the  
// difference.  
public get value(): number {  
    return this.rank === 1 ? 14 : this.rank;  
}
```

plus the rule text at the bottom of `GameScene` changed to "Aces are high".

One line of rules. `judge()` compares `value`, so it needs no change; `frameFor()` uses `rank`, so the ace still shows the ace picture. Tested with the harness: king face up, H, an ace drawn - "Ace of clubs - right!".

Look for: the change made in `value`, not `rank`. The most common wrong answer is to make an ace's **rank** 14 (in `random()` or the constructor). The rules then work, but the picture breaks: `frameFor()` gives `suit * 13 + 13`, the *next* suit's ace - and the ace of spades becomes frame 52, a card back. This is the model/view point of the chapter in miniature; worth showing the class.

2. Cards left

Project: `solutions/ch12_challenge_2_cards_left` (from `ch12_higher_or_lower_intermediate`)

Goal: add to a class's public interface without exposing its insides.

`src/Deck.ts`

```
// CHALLENGE 2: how many cards are still in the deck. A getter, so it reads like a field -  
// deck.size - but cannot be changed from outside.  
public get size(): number {  
    return this.cards.length;  
}
```

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: called after every draw()  
private showCardsLeft(): void {  
    this.cardsLeftText.setText(`Cards left: ${this.deck.size}`);  
}
```

`showCardsLeft()` is called straight after both `draw()` calls (the first card in `create()`, and in `guess()`). Tested: 51 at the start, 50 after one guess.

Look for: a getter or method on `Deck`, not `cards` made `public` - that would let the scene push and pop cards itself. A student who updates the text in `update()` every frame has a working answer; ask whether it needs to change 60 times a second.

3. Probability hint

Project: `solutions/ch12_challenge_3_probability_hint` (from `ch12_higher_or_lower_intermediate`)

Goal: ask the model a question by passing it a function.

`src/Deck.ts`

```
// CHALLENGE 3: how many of the cards left pass a test. The test is a
function that takes a card
// and says true or false - e.g. deck.countWhere((card) => card.value > 7)
public countWhere(test: (card: Card) => boolean): number {
    return this.cards.filter(test).length;
}
```

src/scenes/GameScene.ts

```
private showChances(): void {
    const current = this.currentCard.value;
    const higher = this.deck.countWhere((card) => card.value > current);
    const lower = this.deck.countWhere((card) => card.value < current);
    this.higherHint.setText(`${this.percent(higher)} chance`);
    this.lowerHint.setText(`${this.percent(lower)} chance`);
}

// CHALLENGE 3: e.g. 31 of 50 cards -> "62%"
private percent(count: number): string {
    return `${Math.round((count / this.deck.size) * 100)}%`;
}
```

`showChances()` is called at the end of `waitForGuess()`, and `guess()` clears both hints. Tested: king of diamonds face up, 51 cards left - "0% chance" and "94% chance" (48 of 51).

Look for:

- counting the *deck*, not the 13 ranks - the challenge says so, and it is what makes the hint interesting late in a game
- the two chances not adding up to 100%: the cards of the same rank are ties. A good discussion point
- `(card: Card) => boolean` as a parameter type - Java's `Predicate<Card>`. A student who adds `countHigher(value)` and `countLower(value)` to `Deck` has a fine answer; `countWhere` is more general
- hints hidden during the reveal (otherwise they show stale numbers for the old card)

4. Jokers wild

Project: `solutions/ch12_challenge_4_jokers_wild` (from `ch12_higher_or_lower_intermediate`)

Goal: extend the model and the view in step, and make a texture in code.

The model - `src/Card.ts`, `src/Deck.ts`, `src/rules.ts`:

```
// CHALLENGE 4: a third parameter with a default value, so every existing  
new Card(suit, rank)  
// still works and makes an ordinary card  
constructor(suit: Suit, rank: number, isJoker: boolean = false) {  
  ...  
  // CHALLENGE 4: two jokers, shuffled in with the rest  
  this.cards.push(Card.joker(), Card.joker());  
  ...  
  // CHALLENGE 4: jokers are wild - whatever you guessed, a joker makes it  
right  
  if (next.isJoker) {  
    return "right";  
  }  
}
```

The picture - `src/scenes/PreloadScene.ts` draws a cream card with a gold star using `Graphics`, and saves it:

```
graphics.generateTexture(JOKER_KEY, CARD_WIDTH, CARD_HEIGHT);  
graphics.destroy();
```

The view - `src/objects/CardSprite.ts`:

```
// CHALLENGE 4: a joker is a different texture, so setTexture (picture AND  
frame), not just setFrame  
public showFace(card: Card): void {  
  if (card.isJoker) {  
    this.setTexture(JOKER_KEY);  
  } else {  
    this.setTexture(CARDS_KEY, CardSprite.frameFor(card));  
  }  
}
```


and `showBack()` becomes `setTexture(CARDS_KEY, BACK_FRAME)`, or a sprite that once showed a joker could never show the back again.

The scene - `src/scenes/GameScene.ts`:

```
// CHALLENGE 4: a card that can be guessed against - jokers drawn here are put aside
private drawPlayable(): Card {
  let card = this.deck.draw();
  while (card.isJoker) {
    card = this.deck.draw();
  }
  return card;
}
```

used for the first card, and in `moveOn()` when the revealed card is a joker. Tested with a rigged deck: 2 face up, L, a joker - "Joker - right!", streak 1; the next card to beat is the 9 under it, and the next card turns back to the blue back (texture `cards`, frame 52).

Look for:

- the default parameter, so no existing `new Card(...)` call changed
- `setTexture` rather than `setFrame` - a student who writes `setFrame(JOKER_KEY)` gets the console warning `Texture "cards" has no frame "joker"`, and the sprite shows frame 0 (the ace of clubs) instead
- the joker handled in `judge()` (the model), not by special-casing the scene's messages
- the texture made after loading, in `create()` (or at the start of `GameScene`), with the `Graphics` destroyed afterwards
- a joker never becoming the card to beat - otherwise `judge()` compares against rank 0, and "lower" can never win

5. Double or nothing

Project: `solutions/ch12_challenge_5_double_or_nothing` (from `ch12_higher_or_lower_advanced`)

Goal: add a new state to the state machine, and a new action that uses the existing pieces.

src/Card.ts

```
// CHALLENGE 5: diamonds and hearts are the red suits
public get isRed(): boolean {
    return this.suit === "diamonds" || this.suit === "hearts";
}
```

src/scenes/GameScene.ts

```
// CHALLENGE 5: "gambling" - a double-or-nothing card is being dealt; no
// guesses or cash outs
type GameState = "dealing" | "waiting" | "revealing" | "gambling" | "over";
...
private doubleOrNothing(): void {
    if (this.state !== "waiting" || this.pot === 0) {
        return;
    }
    this.state = "gambling";
    this.setButtonsEnabled(false);
    this.messageText.setText("Double or nothing: red doubles the pot, black
loses it...");

    const card = this.deck.draw();
    this.cardsPlayed = this.cardsPlayed + 1;
    this.deckPile.setVisible(!this.deck.isEmpty());
    this.dealTo(this.nextSprite, NEXT_X, card, () => {
        this.settleGamble(card);
    });
}
```

`settleGamble(card)` doubles and banks the pot on red (with the cash-out coin sound and sparkles), or empties it on black (with the wrong sound and a shake), sets `inARow` to 0, and after a pause adds the card to the history row, hides the next-card sprite, and either ends the game (deck empty) or calls `waitForGuess()`. The card to beat does not change. D is added to the keys line, the controls in `index.html` and the README.

Tested with a rigged deck: D with an empty pot does nothing; H (pot 10), D, queen of hearts - score 20; H pressed during the gamble is ignored; H (pot 20), D, 3 of clubs - pot gone, score still 20, lives still 3.

Look for:

- a new state (not reusing `"revealing"`) and the existing `state !== "waiting"` checks doing the rest - H, L and C are all blocked during the gamble with no extra code
 - `isRed` in `Card` (the model) rather than checking suits in the scene
 - the deck-empty case handled - a gamble can draw the last card
 - reuse of `dealTo()`, `floatText()`, `pulse()` and the sparkles rather than copies
-

6. The computer plays the odds

Project: `solutions/ch12_challenge_6_computer_player` (from `ch12_higher_or_lower_advanced`)

Goal: drive the game from code through the same methods the player uses, and write a simple strategy.

`Deck` gains `countWhere()` (as in challenge 3) and a `size` getter. In `src/scenes/GameScene.ts`, the end of `waitForGuess()`:

```
// CHALLENGE 6: every time the game is ready for a move, the computer (if playing) makes one
if (this.autoPlay) {
    this.scheduleComputerMove();
}
```

and the strategy:

```

private computerMove(): void {
  if (!this.autoPlay || this.state !== "waiting") {
    return; // switched off while thinking, or something else is happening
  }
  const current = this.currentCard.value;
  const higher = this.deck.countWhere((card) => card.value > current);
  const lower = this.deck.countWhere((card) => card.value < current);
  const chance = Math.max(higher, lower) / this.deck.size;

  if (this.pot > 0 && chance < CASH_OUT_BELOW) {
    this.cashOut(); // cashOut() calls waitForGuess(), which schedules the
    next move
    return;
  }
  // (only equal counts - e.g. both 0 when every card left is a tie - need
  the multiplier check,
  // so the computer never picks a guess the game will refuse, such as
  "higher" on a king)
  const guess: Guess = higher > lower || riskMultiplier("lower",
  this.currentCard) === 0 ? "higher" : "lower";
  this.guess(guess);
}

```

`scheduleComputerMove()` is a `this.time.delayedCall(THINK_TIME, ...)`. The A key toggles `autoPlay`, shows "COMPUTER PLAYING", and schedules a move straight away if the game is waiting. `autoPlay` is reset in `init()`, so each game starts with the player in charge.

Tested by switching the computer on and letting it play a whole game with the scene's clocks sped up (`time.timeScale` and `tweens.timeScale` set to 4 from the harness): it played 36 cards on its own, cashed out several times, and finished "Out of lives!" with 990 points and a longest run of 10.

Look for:

- the computer calling `guess()` and `cashOut()` - the same methods as the keys - so it cannot cheat or skip the state checks
- the check at the top of `computerMove()` : without it, pressing A twice quickly schedules two moves, or the computer moves after being switched off
- no `update()` polling ("is it waiting now? now?") - the event-driven version is simpler

- a guess the game would refuse (`riskMultiplier` 0) would leave the computer stuck waiting for a move that never happens - ask students how they would spot that bug
- extension: run many games and print the average score for different `CASH_OUT_BELOW` values - a real experiment in strategy

Chapter 12 - Higher or lower - Slides

Chapter 12

Higher or lower

One game, three versions



1 / 20

Today

- what makes a guessing game fun
- a `Card`, a `Deck`, the rules - **no Phaser**
- a `CardSprite` and a `Button` - **the view**
- a proper shuffle (and a broken one)
- game state: ignoring the player at the right moments
- a flip made of two tweens
- risk, reward, and cashing out

2 / 20

What makes it fun?

The luck is the same in all three versions. So why is one more fun?

- **risk** - "higher" on a 2 is safe; on a jack it is a gamble. Show it; pay for it
- **streaks** - five in a row feels like skill. Let the player lose it
- **the reveal** - the half-second while the card turns *is* the game

Rules: aces low (1); a tie does not count

3 / 20

Try it: simple

- `ch12_higher_or_lower_simple`
- H = higher, L = lower
- five right in a row wins
- wrong: back to zero
- one scene, one small class



4 / 20

A card is data

```
export type Suit = "clubs" | "diamonds" | "hearts" | "spades";

export class Card {
  public readonly suit: Suit;
  public readonly rank: number; // 1 = ace, 2 - 10, 11 = jack, 12 = queen, 13 = king
  ...
  public get value(): number {
    return this.rank;
  }
}
```

- union of string literals: only those four
- `readonly` = Java's `final` ; `get` = read like a field
- no picture, no position, no Phaser

5 / 20

Which picture?

Frames of cards.png: $\text{frame} = \text{suit} * 13 + (\text{rank} - 1)$

	A (1)	2 (2)	3 (3)	4 (4)	5 (5)	6 (6)	7 (7)	8 (8)	9 (9)	10 (10)	J (11)	Q (12)	K (13)
0 clubs	0	1	2	3	4	5	6	7	8	9	10	11	12
1 diamonds	13	14	15	16	17	18	19	20	21	22	23	24	25
2 hearts	26	27	28	29	30	31	32	33	34	35	36	37	38
3 spades	39	40	41	42	43	44	45	46	47	48	49	50	51
	52	53	52 and 53: the two backs										

Queen of hearts: suit 2, rank 12, so $\text{frame } 2 * 13 + (12 - 1) = 37$

```
return SUITS.indexOf(card.suit) * 13 + (card.rank - 1);
```

6 / 20

The rules - and when to listen

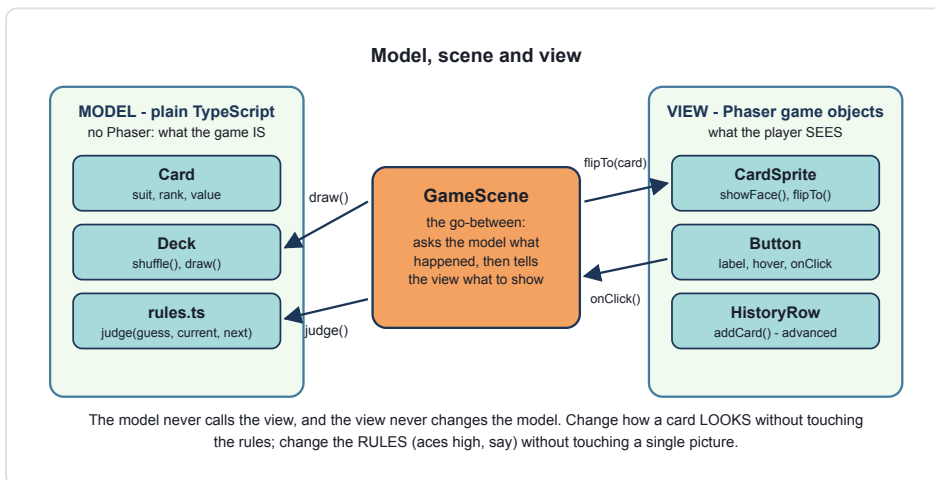
```
type Outcome = "right" | "wrong" | "tie";
type GameState = "waiting" | "revealing" | "won";

private guess(guess: Guess): void {
  if (this.state !== "waiting") {
    return; // a card is still being revealed, or the game is over
  }
  this.state = "revealing";
  ...
  const outcome = this.judge(guess, this.currentCard, next);
}
```

`judge()` only looks at cards. The scene decides what each outcome *looks* like.

7 / 20

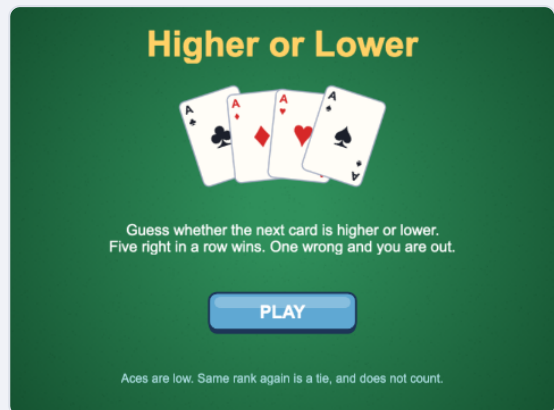
Model and view



8 / 20

Try it: intermediate

- `ch12_higher_or_lower_intermediate`
- a shuffled deck: no repeats
- cards flip; buttons; five lights
- one wrong and you are out
- find: `Card.ts`, `Deck.ts`, `rules.ts` - which import Phaser?



9 / 20

Fisher-Yates

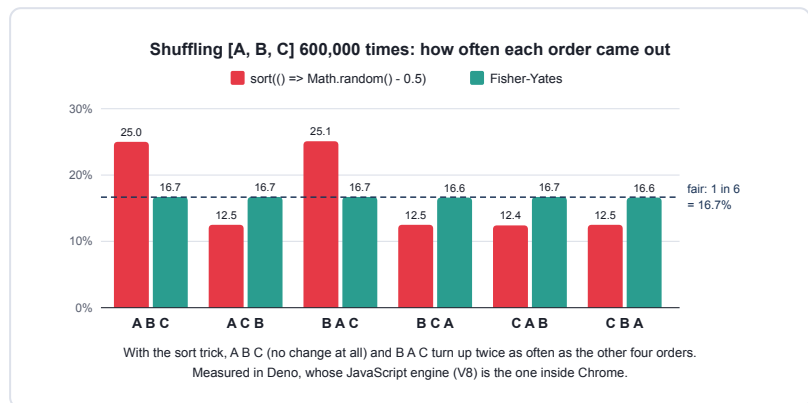
```
public shuffle(): void {
  for (let i = this.cards.length - 1; i > 0; i--) {
    const j = Math.floor(Math.random() * (i + 1)); // 0 to i, inclusive
    const swap = this.cards[i];
    this.cards[i] = this.cards[j];
    this.cards[j] = swap;
  }
}
```

- fill each place, from the back, with a fair pick of what is left
- one pass; every order equally likely
- `Phaser.Utls.Array.Shuffle` does the same

10 / 20

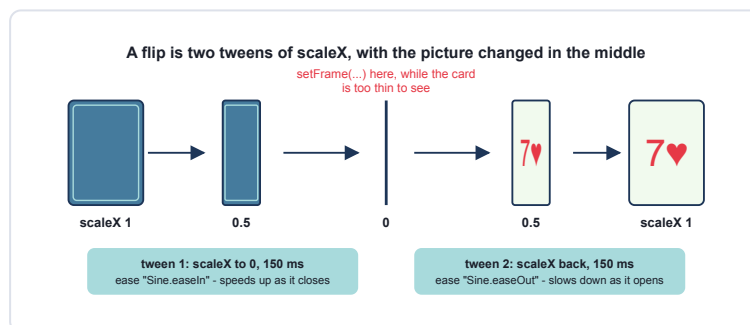
The shuffle you will find online

```
cards.sort(() => Math.random() - 0.5); // DON'T
```



11 / 20

A flip is two tweens



```
this.scene.tweens.chain({
  targets: this,
  tweens: [
    {
      scaleX: 0,
      duration: FLIP_TIME / 2,
      ease: "Sine.easeIn",
      onComplete: () => {
        ...
      },
    },
    { ... },
  ],
  onComplete: onComplete,
});
```

The first tween's `onComplete` calls `showFace(card)` (or `showBack()`) while the card is edge-on

12 / 20

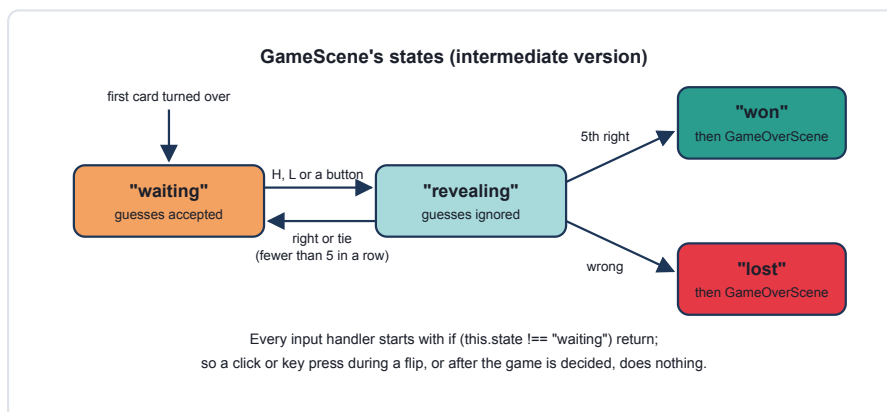
A button

```
export class Button extends Phaser.GameObjects.Container {  
  ...  
  this.add([this.background, this.label]);  
  this.background.setInteractive({ useHandCursor: true });  
  
  this.background.on("pointerover", () => {  
    this.background.setTexture(BUTTON_OVER_KEY);  
  });  
  ...  
  this.background.on("pointerup", () => {  
    ...  
    onClick();  
  });  
}
```

- a **Container**: one object holding a picture and a label
- normal / over / down pictures; act on **pointerup**
- the caller passes in what to do

13 / 20

The state machine



`reveal()` runs from the flip's `onComplete` : see the card **first**, then hear the result

14 / 20

Try it: advanced

- `ch12_higher_or_lower_advanced`
- one deck, three lives
- a **pot**: more for risk, more for a long run
- **cash out** (C) to bank it
- wrong: lose the pot **and** a life



15 / 20

Paying for risk

```
export function winningRanks(guess: Guess, current: Card): number {
  if (guess === "higher") {
    return 13 - current.value;
  }
  return current.value - 1;
}

export function pointsFor(multiplier: number, inARow: number): number {
  return BASE_POINTS * multiplier * inARow;
}
```

- `riskMultiplier()` : 7+ winning ranks x1, 4-6 x2, 1-3 x3, 0 = impossible
- the buttons show it: `HIGHER x1` , `LOWER x3`

16 / 20

The pot and cash out

- `score` - banked, safe
- `pot` - at risk
- only a score: always guess the likely way. No decisions
- with a pot: **when do you stop?**

```
this.score = this.score + this.pot;  
this.pot = 0;  
this.inARow = 0;
```



17 / 20

Dealing, history, records, polish

```
const oldCurrent = this.currentSprite;  
this.currentSprite = this.nextSprite;  
this.nextSprite = oldCurrent;
```

- two sprites swap jobs - they are references, as in Java
- `HistoryRow` - a `Container` : tell every card where it belongs
- `Records` - best score and longest run in local storage
- floating "+30", pulses, sparkles, a shake, a heart that pops

18 / 20

Summary

- risk, streaks, the reveal
- **model** (`Card` , `Deck` , `rules.ts`) apart from **view** (`CardSprite` , `Button`)
- unions of strings for suits, guesses, outcomes, states
- Fisher-Yates - not `sort(random)`
- a `state` checked at the top of every input
- flip = two tweens of `scaleX` , frame changed in between
- a pot you can lose makes the score interesting

19 / 20

Challenges

1. **Aces high** - an ace beats a king
2. **Cards left** - count down the deck
3. **Probability hint** - the real chance, from the deck
4. **Jokers wild** - two jokers, drawn in code
5. **Double or nothing** - red doubles, black loses
6. **The computer plays the odds** - press A and watch

Next: **Chapter 13 - Memory match**

20 / 20

Memory match

Overview

The first "grid" game of the book, built three times. The Phaser content is light - sprite sheet frames on an `Image`, `setFrame`, tweens and tween chains, `delayedCall`, a `Container`, particles - and almost all of it was met in earlier chapters. The real subject is **program structure**: laying out a grid with arithmetic rather than typed-in positions, writing game rules as a **state machine**, and waiting for animations with **callbacks**. The advanced version adds an OO design discussion that suits Java programmers well: a `Theme` interface with two implementations, and a `TileFace` that separates what a tile *shows* from what it *matches*.

The one idea to get across above all others: **the board must be locked while tiles are showing**, and a single `state` field is the cleanest way to do it. Students who "fix" the three-tiles-up bug with a growing collection of booleans are the ones to talk to.

Prerequisites

- Chapter 2: scenes, `init(data)` and resetting fields, `setInteractive` and `"pointerdown"`, the registry
- Chapter 5 (sounds), Chapter 6 (local storage, validating what is read back), Chapter 7 (tweens, chains, stagger, particles)
- Chapter 12: the `Card / Deck` split, Fisher-Yates, a flip made of two `scaleX` tweens, game state as a union of string literals, and the `cards.png` frame sum
- Java interfaces and lambdas (`Runnable`) for the analogies

Learning outcomes

Students can:

1. lay out an $R \times C$ grid of game objects with nested loops, centred in the game, and convert between (row, col) and a flat index with `row * COLS + col`

2. make shuffled pairs (or groups) of values with `Phaser.Utils.Array.Shuffle`
3. show and change the frame of a sprite sheet on an `Image`
4. describe a game's rules as a state machine (states, events, transitions), implement it with a union type, and use it to ignore input in "locked" states
5. animate a flip with two tweens, and run code when it finishes by passing a callback
6. turn a performance (moves, time) into a rating, and show it
7. (advanced) design an interface that lets the same game use different sets of content, and describe levels as data

Suggested session plan (2 x 2 hours)

Session 1 - the simple and intermediate versions

Time	Activity
0:00 - 0:10	Play a physical round of pairs with a dozen cards, if you have some. Ask: "what are the rules, exactly? What happens if I turn over a third card while two are showing?"
0:10 - 0:30	Slides 3-6: what makes the genre work, tiles as sprite sheet frames, making pairs, the grid maths. Students draw the 4 x 3 grid on paper and work out the centre of tile 7 before seeing <code>layoutGrid</code>
0:30 - 0:50	Slides 7-9: the state machine, and trying the simple version. Live demo: delete <code>this.state = "twoUp"</code> in the simple version and click three tiles quickly - see "Common problems" below for the crash. Discuss why
0:50 - 1:20	Challenges 1 and 5 (both on the simple version)
1:20 - 1:40	Slides 10-13: the flip, callbacks, the fourth state, stars. Live-code the flip from scratch in a blank tile (one tween first, then the second in <code>onComplete</code>)
1:40 - 2:00	Start challenge 2 or 3

Session 2 - the advanced version

Time	Activity
0:00 - 0:15	Play the advanced version, both themes. Ask: "what had to change so that two <i>different</i> cards can match?"
0:15 - 0:40	Slides 14-15: levels as data, the board that fits, the <code>Theme</code> interface and <code>matchKey</code> . Design discussion (below)
0:40 - 0:55	Slides 16-18: peek and shuffle as two more locked states; buttons and best results
0:55 - 2:00	Challenges 2, 3, 4 and 6

Key points to stress

- **Positions are calculated, not typed.** A grid that centres itself from `ROWS` , `COLS` , `GAP` and the tile size can change size without anyone touching the layout code - which is exactly what challenge 1 and the advanced levels rely on
- **Gaps are one fewer than tiles.** `COLS * TILE_SIZE + (COLS - 1) * GAP` . The off-by-one here only shows as a grid slightly off-centre, so students often do not notice it
- **One `state` field, not several booleans.** A state machine can only be in one state at once, so impossible combinations cannot happen. Each new feature (a flip, a peek, a shuffle, a hint, a time limit) is a new state and a few new transitions
- **Test what is allowed, not what is forbidden.** The intermediate version locks with `state === "twoUp" || state === "checking"` ; the advanced one with `state !== "idle" && state !== "oneUp"` . Only the second stays correct when a new locked state is added. Challenge 3 walks straight into this (see solutions.md)
- **Animations take time, so logic must wait for them.** The pair is checked in the second flip's callback, not straight after starting it; the board unlocks in the callback of the flip back
- **The tile knows what it is; the scene knows the rules.** `Tile` has `flipUp` , `celebrate` , `shake` , `matches` - nothing about pairs, moves or turns. That is why challenges 4 and 5 barely touch `Tile`
- **Shape and meaning are different things.** `frame` vs `matchKey` in the advanced version - the same idea as Chapter 12's model/view split

Common problems and errors

What students see	Cause	Fix
Three (or four) tiles face up at once, then <code>TypeError: Cannot read properties of null (reading 'matches')</code> in the console	the lock is missing: after a wrong pair the state stayed <code>"oneUp"</code> with <code>firstTile</code> already <code>null</code> (produced by deleting <code>this.state = "twoUp"</code> from the simple version)	put the game in a locked state when wrong pair shows
Argument of type <code>'Tile null'</code> is not assignable to parameter of type <code>'Tile'</code> . Type <code>'null'</code> is not assignable to type <code>'Tile'</code> .	passing <code>this.firstTile</code> (a <code>Tile null</code>) where a <code>Tile</code> is needed	the state guarantees it is set: <code>this.firstTile!</code> - or check for
Type <code>"oneup"</code> is not assignable to type <code>'State'</code> . Did you mean <code>"oneUp"</code> ?	a typo in a state name	this is why the states are a union not plain strings
The game restarts the moment the last pair is found	<code>this.input.once("pointerdown", ...)</code> added inside the tile's click handler: the same click reaches the scene's listener straight after (tested: the scene restarts on the winning click)	use a key, a <code>delayedCall</code> , or a separate win scene
Second game cannot be won; or tiles from the first game are still "there"	<code>tiles</code> (or <code>pairsFound</code> , <code>state</code>) not reset in <code>init()</code> ; the array still holds the old, destroyed tiles	reset every field in <code>init()</code> - Chapter 2's trap
Grid is a little off-centre	gaps counted as <code>COLS * GAP</code> instead of <code>(COLS - 1) * GAP</code> , or the half-tile for the centre origin forgotten	check the sums against the diagram

What students see	Cause	Fix
The result sound plays before the second tile can be seen	the pair is checked straight after <code>flipUp()</code> starts, not in its callback	pass <code>() => this.checkPair()</code> to <code>flipUp</code>
After a flip, a scaled-down tile is suddenly full size	the flip tweens <code>scaleX</code> back to <code>1</code>	tween back to the tile's own base (advanced <code>Tile</code>)
A dark tile tinted yellow looks darker, not brighter	<code>setTint</code> multiplies colours	<code>setTintMode(Phaser.TintModes</code> (challenge 3's solution), or tween <code>alpha</code> / <code>scale</code> instead
Class <code>'CardTheme'</code> incorrectly implements interface <code>'Theme'</code> . Property <code>'backFrame'</code> is missing in type <code>'CardTheme'</code> but required in type <code>'Theme'</code> .	a class that implements <code>Theme</code> has left a property out	the interface doing its job - add it
Property <code>'cards'</code> is missing in type <code>'{ pictures: PictureTheme; }'</code> but required in type <code>'Record<ThemeName, Theme>'</code> .	a name added to <code>ThemeName</code> with no theme in <code>THEMES</code> (or the reverse)	add the theme; <code>Record</code> insists every key has a value
No sound on the first flip	the browser blocks audio until the page has been clicked	the title scene's "Click to start" so this: mention Chapter 5

Demos and live-coding

- **Remove the lock** (simple version): delete `this.state = "twoUp";` in `checkPair`, rebuild, and click three tiles quickly. Three tiles stay up and the console shows the `TypeError` above. Ask the class to trace, state by state, what happened

- **The flip from nothing:** in a blank scene, one tile. Tween `scaleX` to 0 - "now it has vanished, what next?" - add `onComplete` with `setFrame` , then the second tween. Then change the eases to `"Linear"` and compare
 - **The easing matters:** in the intermediate project, set `HALF_FLIP` to 600 and watch the flip in slow motion; then try `"Back.easeOut"` on the second half
 - **The trap of `once("pointerdown")` :** in the simple version, change `win()` to use `this.input.once("pointerdown", ...)` . Find the last pair - the game restarts instantly. Explain the order Phaser delivers the event (game object first, then the scene)
-

Discussion questions

- Draw the state machine for the simple version on the board. Which transitions does each click cause? Which state has no transition on a click, and why is that the important one?
 - Why is `face` `readonly` on a `Tile` ? What could go wrong if the scene could change it?
 - The intermediate version checks the pair in a callback. What would players notice if it checked straight away? (The sound and shake come before the second face is visible.)
 - In the card theme, the seven of hearts matches the seven of diamonds. Why could the simple version's `Tile` (with `face: number`) not express that? What is the smallest change that could?
 - `LEVELS` is an array of plain objects. What would change if each level were a subclass of a `Level` class instead? When would that be worth it? (When levels have different *behaviour*, not just different numbers.)
 - Is the peek worth a star? How would you tune the star limits - by feel, or by recording real players' moves?
-

Extension ideas

- a "flip everything face up for two seconds at the start" memorisation phase, as a new state
- a third theme - letters, numbers, or shapes drawn with `Graphics` and `generateTexture`
- matching by a *rule* rather than a picture: a number and a sum that makes it (7 and "3 + 4")

- a replay: record every click (time and tile) in an array, then play the game back
 - a proper undo of the last pair? (Why does that not make sense for this genre?)
-

Assessment ideas

- Practical: "add a fourth theme with its own `makeFaces` rule" (tests the interface), or "make the wrong-pair delay get shorter as more pairs are found" (tests the state machine and timers)
 - Code reading: give the intermediate `tileClicked` and `checkPair` without comments, and a list of click times; ask which clicks are ignored
 - Short answer: explain why the pair is checked in `flipUp`'s callback, and what a callback is, using a Java comparison
 - Design: draw the state diagram for challenge 4 (two players) or challenge 5 (triples)
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 13 - Memory match - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment. Every solution was played with the test harness; what was checked is noted under each.

1. Choose the size

Project: `solutions/ch13_challenge_1_choose_the_size` (from `ch13_memory_simple`)

Goal: turn constants into state that survives a restart - `scene.restart(data)` and `init(data)`.

`ROWS` and `COLS` become fields, and the three sizes a table:

`src/scenes/GameScene.ts`

```
const SIZES: BoardSize[] = [
  { rows: 3, cols: 4 },
  { rows: 4, cols: 4 },
  { rows: 4, cols: 6 },
];
...
export interface BoardSize {
  rows: number;
  cols: number;
}
...
private rows = SIZES[0].rows; // CHALLENGE 1: the size is a field now, not
a constant
private cols = SIZES[0].cols;
```

The keys restart the scene with a size; `init` takes it, or keeps the size it had:

```
init(data: Partial<BoardSize>): void {
  this.rows = data.rows ?? this.rows;
  this.cols = data.cols ?? this.cols;
  ...
  this.input.keyboard!.on("keydown-ONE", () => {
    this.scene.restart(SIZES[0]);
  });
}
```

- the first start (from the game config) and `restart()` with no data both hand `init` an **empty object**, so the fields may be missing. `Partial<BoardSize>` is "a `BoardSize` with every field optional", and `??` (the nullish coalescing operator) falls back to the current size when a field is missing. So SPACE after a win - `this.scene.restart()`, unchanged - plays again at the same size, for free
- `layOutGrid` swaps `ROWS / COLS` for `this.rows / this.cols`; nothing else in it changes, because it was already centring by arithmetic. That is the point of the challenge

Tested: 1, 2, 3 give 12, 16 and 24 tiles; the 6 x 4 grid spans x 60-740 and y 76-524 (centred); SPACE after winning the 4 x 4 board gives another 4 x 4 board.

Look for: the size in the restart data (not a field set before `restart`, which works but by accident - the scene object is reused); the grid still centred at every size; the message saying the size. Students who keep the size in the **registry** instead have a good answer too. Some will ask about a 5 x 5 board: 25 tiles cannot make pairs - a nice moment to add a check that `rows * cols` is even.

2. Against the clock

Project: `solutions/ch13_challenge_2_against_the_clock` (from `ch13_memory_intermediate`)

Goal: a losing condition - and the discovery that it can happen in the *middle* of something.

The countdown is the same clock, shown the other way round:

```
src/scenes/GameScene.ts
```



```

if (this.seconds >= TIME_LIMIT) {
  this.seconds = TIME_LIMIT;
  this.outOfTime();
}
...
const left = TIME_LIMIT - this.seconds;
this.timeText.setText(`Time left: ${left.toFixed(1)}`);
this.timeText.setColor(left <= HURRY_TIME ? "#e63946" : "#ffffff");

```

Running out of time is a new, permanently locked state, "timeUp" :

```

private outOfTime(): void {
  this.clockRunning = false;
  this.state = "timeUp";

  // cancel a wrong pair that is waiting to turn back (the tiles simply stay
  // face up)
  this.time.removeAllEvents();

  // anything face down - or on its way down - turns face up
  for (const tile of this.tiles) {
    if (!tile.isFaceUp()) {
      tile.reveal();
    }
  }
}

```

The hard part is that time can run out at any moment: while the second tile is flipping up, while a wrong pair is waiting to turn back, or while it is half-way through turning back. Three small pieces cover all of them:

- `this.time.removeAllEvents()` cancels the pending "turn the wrong pair back" call
- `checkPair` starts with `if (this.state === "timeUp") { return; }`, so a flip that finishes late does nothing
- `Tile.reveal()` stops any flip in progress - and with it the callback that would have unlocked the board - and flips the tile up:

`src/objects/Tile.ts`

```
public reveal(): void {
    this.scene.tweens.killTweensOf(this);
    this.scaleX = 1;
    this.flipUp();
}
```

`WinData` gains `pairsFound` and `outOfTime`; the win scene shows "Out of time!", no earned stars (all three grey) and "Pairs found: 1 of 8", and plays the wrong sound instead of the win sound.

Tested: time forced to run out (by moving `startTime` back) while a wrong pair was half-way through turning back: every tile ends face up on its own face with `scaleX` 1, the state stays `"timeUp"`, and the win scene reports `outOfTime: true`. The time turns red below ten seconds.

Look for: the timer counting down from the first click (not from the scene starting - either is defensible, but be consistent with the chapter); a locked state for the end; and whether the student thought about the "in the middle of a flip" cases. Many solutions work only when time runs out while the board is idle - play one with a wrong pair showing at 0.5 seconds left and see. A new scene for losing is also a fine answer.

3. A hint

Project: `solutions/ch13_challenge_3_a_hint` (from `ch13_memory_intermediate`)

Goal: another locked state - and a lesson in how to write a lock.

```
src/scenes/GameScene.ts
```

```

private hint(): void {
  if (this.state !== "idle") {
    return;
  }
  const hidden = this.tiles.filter((tile) => !tile.isFaceUp());
  if (hidden.length === 0) {
    return;
  }
  const face = Phaser.Utls.Array.GetRandom(hidden).face;
  const pair = hidden.filter((tile) => tile.face === face);

  this.state = "hinting";
  this.moves = this.moves + HINT_COST;
  this.updateHud();
  for (const tile of pair) {
    tile.flash(() => {
      this.state = "idle";
    });
  }
}

```

In "idle" no unmatched tile is face up, so "face down" means "not yet found" - pick one at random, and flash it and its partner. `Tile.flash` pulses the tile three times and glows it yellow:

src/objects/Tile.ts

```

public flash(onDone: () => void): void {
  this.setTint(HINT_TINT).setTintMode(Phaser.TintModes.ADD);
  this.scene tweens.add({
    targets: this,
    scale: 1.12,
    duration: 150,
    yoyo: true,
    repeat: 2,
    onComplete: () => {
      this.clearTint().setTintMode(Phaser.TintModes.MULTIPLY);
      onDone();
    },
  });
}

```

The ordinary tint *multiplies* colours, which can only make the dark tile darker; `ADD` brightens it. (Both tiles call `onDone`, so the state is set to `"idle"` twice - harmless.)

The bug the solution walked into. The first version of this solution added `"hinting"` to the `State` type and left `tileClicked` as it was:

```
if (this.state === "twoUp" || this.state === "checking") {
```

Clicking a tile during the hint then fell through to the "second tile" branch, with no first tile, and the harness showed `TypeError: Cannot read properties of null (reading 'matches')`. The fix is to list the states that **allow** a click, as the advanced version does:

```
if (this.state !== "idle" && this.state !== "oneUp") {  
  return;  
}
```

Now any state added later is locked unless someone decides otherwise. This is the most useful thing to discuss from this challenge.

Tested: H in `"idle"` flashes two tiles with the same face and adds 2 moves; clicks during the flash are ignored; H with one tile up does nothing.

Look for: the hint only when idle; the moves added; a new state or an equivalent lock while the flash plays (without one, a click starts a flip - a `scaleX` tween - on a tile that is also being scaled by the flash, and the two fight). Flipping the pair face up briefly is also an acceptable "flash".

4. Two players

Project: `solutions/ch13_challenge_4_two_players` (from `ch13_memory_intermediate`)

Goal: find the one place in the state machine where a turn ends.

```
src/scenes/GameScene.ts
```

```

private player = 0;
private scores = [0, 0];
...
this.scores[this.player] = this.scores[this.player] + 1;
this.updateHud();
...
second.flipDown(() => {
    // CHALLENGE 4: THE place where a turn ends - a wrong pair, now face down
    again
    this.player = 1 - this.player;
    this.updateHud();
    this.state = "idle";    // unlock only once both are face down again
});

```

A pair scores for the current player, who simply goes again - nothing to do. A wrong pair passes the turn, and the right moment is when the tiles are face down and the board unlocks. Switching when the wrong pair is *found* would show "Player 2's turn" while player 1's tiles are still up. `1 - this.player` flips between 0 and 1.

The HUD shows both scores and whose turn it is, in that player's colour. `WinData` carries `scores`, and the win scene announces the winner or a draw:

`src/scenes/WinScene.ts`

```

const [p1, p2] = this.result.scores;
let heading = "A draw!";
if (p1 > p2) {
    heading = "Player 1 wins!";
} else if (p2 > p1) {
    heading = "Player 2 wins!";
}

```

`const [p1, p2] = ...` is **destructuring**: the first two items of the array, as two constants.

Stars are dropped: moves are shared by both players, so they no longer say how well either played.

Tested: player 1 finds a pair and keeps the turn (score 1-0); a wrong pair passes to player 2, who finds three (1-3); a wrong pair passes back; the game ends 5-3 and the win scene says "Player 1 wins!". Playing again starts at 0-0, player 1.

Look for: the turn switched in exactly one place; scores reset in `init()` - and reset with a **new** array (`this.scores = [0, 0]`), because the old array was handed to the win scene inside `WinData` , and changing it in place would change the result being shown. Worth asking students why that matters even though the win scene has already drawn it.

5. Triple match

Project: `solutions/ch13_challenge_5_triple_match` (from `ch13_memory_simple`)

Goal: generalise from "a first tile" to "the tiles showing", and redesign the states.

`src/scenes/GameScene.ts`

```
type State = "idle" | "picking" | "wrong";    // CHALLENGE 5
...
private showing: Tile[] = [];
```

```
tile.showFace();
this.showing.push(tile);

if (!tile.matches(this.showing[0])) {
  // a different face - this group is wrong, however many are showing
  this.state = "wrong";
  const wrongGroup = this.showing;
  this.showing = [];
  this.time.delayedCall(MISMATCH_DELAY, () => {
    for (const wrongTile of wrongGroup) {
      wrongTile.hideFace();
    }
    this.state = "idle";
  });
} else if (this.showing.length === GROUP_SIZE) {
```

- `makeFaces` pushes each face `GROUP_SIZE` times; the board holds `ROWS * COLS / GROUP_SIZE` groups
- comparing each new tile with the **first** one is enough: if every tile matches the first, they all match each other

- `wrongGroup` keeps hold of the array for the delayed call, while `this.showing` starts afresh

With `GROUP_SIZE = 2` this is the ordinary game again - a good test that the generalisation is right. How big can the board be? Twelve faces make 36 tiles - 6 x 6, which does not fit at 100 pixels a tile (the grid would be 680 pixels tall), so it would need smaller tiles or scaling as in the advanced version.

Tested: three the same stay up; two different are wrong straight away (after two clicks); two the same and a third different is wrong with three showing; clicks during "wrong" are ignored; all four groups found wins.

Look for: the array rather than `firstTile / secondTile / thirdTile`; the lock still there; a wrong guess ending the group as soon as it is wrong (a student who waits for three tiles before comparing has made a different - but playable - game; ask which is better).

6. Keyboard control

Project: `solutions/ch13_challenge_6_keyboard_control` (from `ch13_memory_advanced`)

Goal: separate *where* from *what*: a cursor is a place on the board, and a tile is whatever is there now.

`layOutGrid` keeps the numbers it already worked out, so a column and row can be turned into x and y on any level, in either theme, at any scale:

`src/scenes/GameScene.ts`

```
this.gridLeft = left;
this.gridTop = top;
this.stepX = (width + GAP) * scale;
this.stepY = (height + GAP) * scale;
```

The cursor is a `Rectangle` with a stroke and no fill, drawn above the tiles, and the keys move it (clamped to the board) or pick the tile under it:

```

private moveCursor(dCol: number, dRow: number): void {
  this.cursorCol = Phaser.Math.Clamp(this.cursorCol + dCol, 0,
this.level.cols - 1);
  this.cursorRow = Phaser.Math.Clamp(this.cursorRow + dRow, 0,
this.level.rows - 1);
  this.cursor.setPosition(this.placeX(this.cursorCol),
this.placeY(this.cursorRow));
}

private pickAtCursor(): void {
  const x = this.placeX(this.cursorCol);
  const y = this.placeY(this.cursorRow);
  const tile = this.tiles.find((t) => Phaser.Math.Distance.Between(t.x, t.y,
x, y) < 1);
  if (tile !== undefined) {
    this.tileClicked(tile);
  }
}

```

`pickAtCursor` hands the tile to the same `tileClicked` the mouse uses, so every rule - the locks, face-up tiles, matched tiles - applies to the keyboard without a line of new logic. And because it looks for the tile *at the place* every time, it keeps working after a shuffle has moved the tiles round (`find` returns `undefined` if nothing is there - it never is, but the type says it could be).

Tested on level 4 with cards (6 x 4): the cursor starts on the top-left card, clamps at the edges, SPACE and ENTER turn over the card under it; after `shuffleUnmatched()` the card picked is the one now at the cursor's place.

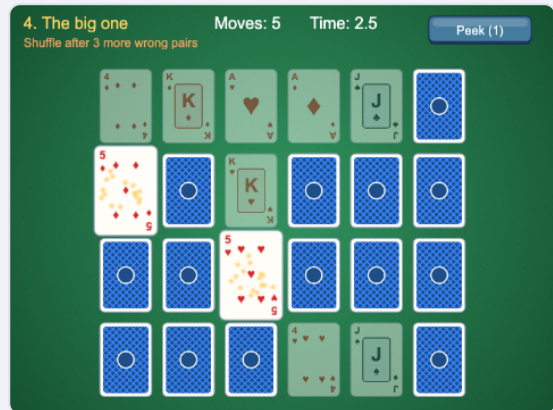
Look for: the cursor as (col, row) rather than a reference to a tile - a student who stores "the tile under the cursor" finds it wrong after the first shuffle, which is exactly the point of the hint; the tile found by position (or a 2D array of tiles kept up to date when the shuffle happens - also good, but more to keep in step); the mouse still working; the cursor sized for both themes.

Chapter 13 - Memory match - Slides

Chapter 13

Memory match

Turn over two. Keep them if they match.



1 / 20

Today

- a grid laid out **by arithmetic**
- shuffled pairs, dealt onto the grid
- the rules as a **state machine** - and locking the board
- a flip made of two tweens, and **callbacks**
- moves, time and stars
- levels as data; a **Theme** interface; peek and shuffle

2 / 20

What makes a memory game work

- it is a game about **information**: every tile turned tells you something
- the player must **see** a wrong pair - about a second
- the board must be **fair**: no third tile while two are showing
- score **efficiency**: moves and time
- **feedback**: flip, sound, pop, shake

3 / 20

A tile is one frame of a sprite sheet

[image: w:900]

```
this.load.spritesheet(TILES_KEY, TILES_FILE, { frameWidth: TILE_SIZE, frameHeight: TILE_SIZE });
...
super(scene, x, y, TILES_KEY, BACK_FRAME);
...
public showFace(): void {
    this.faceUp = true;
    this.setFrame(this.face);
}
```

A **Tile** is an **Image** that starts on frame 0 (the back). An **Image** can show any frame; **Sprite** is for animation.

4 / 20

Pairs, then a grid

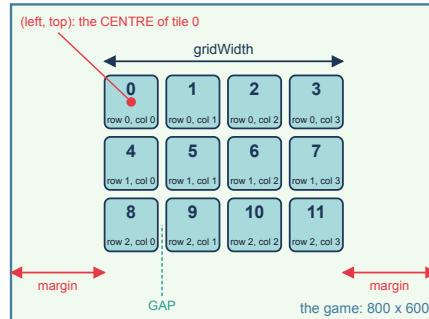
```
for (let face = 1; face <= pairs; face++) {  
    faces.push(face, face);  
}  
return Phaser.Utls.Array.Shuffle(faces);
```

```
for (let row = 0; row < ROWS; row++) {  
    for (let col = 0; col < COLS; col++) {  
        const x = left + col * (TILE_SIZE + GAP);  
        const y = top + row * (TILE_SIZE + GAP);  
        const face = faces[row * COLS + col];
```

5 / 20

Centring the grid

Laying out a 4 x 3 grid, centred in the game



The sums in layOutGrid()

```
gridWidth = COLS * TILE_SIZE + (COLS - 1) * GAP  
four tiles across, but only three gaps between them
```

```
left = (800 - gridWidth) / 2 + TILE_SIZE / 2  
two equal margins, so the grid is centred - and  
+ TILE_SIZE / 2, because a tile's origin is its centre
```

```
x = left + col * (TILE_SIZE + GAP)
```

```
y = top + row * (TILE_SIZE + GAP)  
each step across or down is one tile and one gap
```

```
face = faces[row * COLS + col]
```

the shuffled faces are dealt out row by row; the
number on each tile is its index in the array

6 / 20

The rules are a state machine

```
type State = "idle" | "oneUp" | "twoUp";

switch (this.state) {
  case "idle":
    tile.showFace();
    this.firstTile = tile;
    this.state = "oneUp";
    break;
  case "oneUp":
    tile.showFace();
    this.checkPair(this.firstTile!, tile);
    break;
  case "twoUp":
    break;
}
```

"twoUp" : a wrong pair is showing - the click is ignored. That is the **lock**

7 / 20

A wrong pair locks the board

```
} else {
  // leave the wrong pair showing long enough to remember, then turn both back
  this.state = "twoUp";
  this.time.delayedCall(MISMATCH_DELAY, () => {
    first.hideFace();
    second.hideFace();
    this.state = "idle";
  });
}
```

- one `state` field - never two states at once
- remove the lock: three tiles up, then a `TypeError`

8 / 20

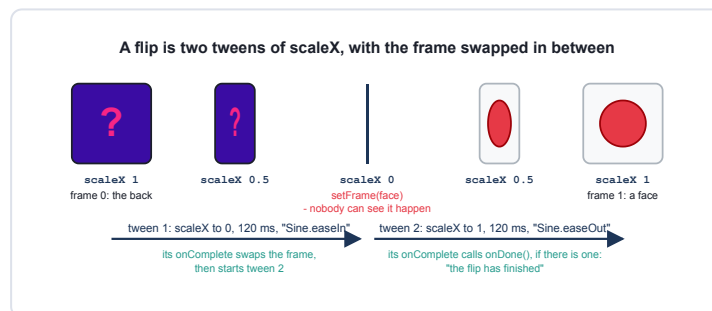
Try it: the simple version

- build, serve, play `ch13_memory_simple`
- change `ROWS`, `COLS` and `GAP` - the grid lays itself out
- what goes wrong with a 5 x 5 board?



9 / 20

The flip: two tweens of `scaleX`



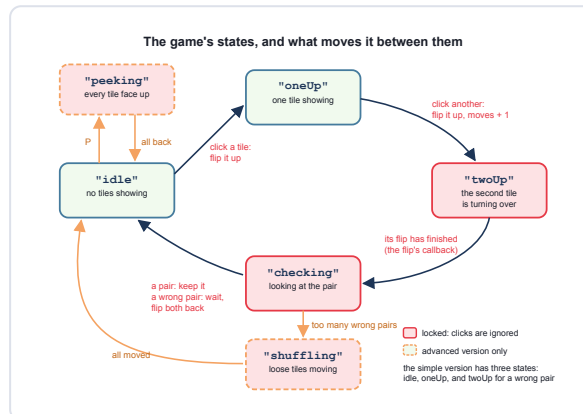
```
this.scene.tweens.add({
  targets: this,
  scaleX: 0,
  duration: HALF_FLIP,
  ease: "Sine.easeIn",
  onComplete: () => {
    this.setFrame(frame);
    this.scene.tweens.add({
      ...
```

10 / 20

Wait for it: callbacks, and a fourth state

```
public flipUp(onDone?: () => void): void {
  ...
  this.state = "twoUp";
  ...
  tile.flipUp(() => {
    this.checkPair();
  });
}
```

`onDone` is a function (like a Java `Runnable`); `?` makes it optional

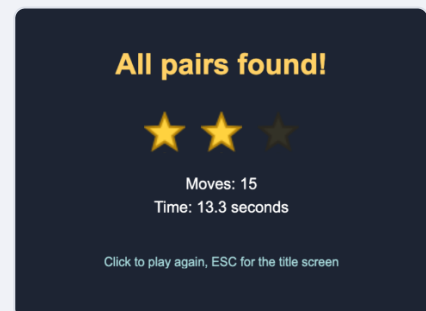


11 / 20

Moves, time and stars

```
export function starsFor(moves: number, pairs: number): number
{
  if (moves <= Math.ceil(pairs * THREE_STARS)) {
    return 3;
  }
  if (moves <= Math.ceil(pairs * TWO_STARS)) {
    return 2;
  }
  return 1;
}
```

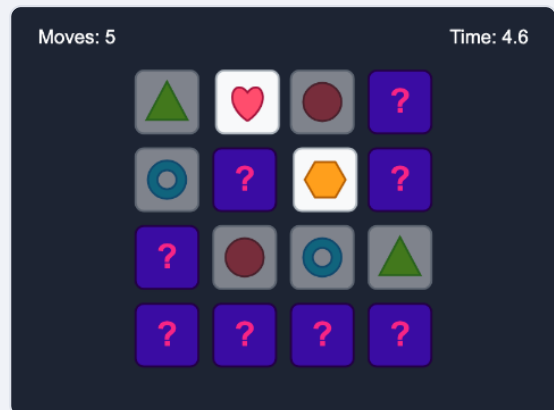
- a **move** = two tiles; the clock starts at the first click
- show the stars you *missed*, too



12 / 20

Try it: the intermediate version

- play `ch13_memory_intermediate` - how many moves for three stars?
- set `HALF_FLIP` to 600: watch the flip slowly
- try "Linear" instead of the `Sine` eases



13 / 20

Levels are data; the board fits

```
export const LEVELS: LevelData[] = [  
  { name: "Warm up", cols: 4, rows: 3, shuffleAfter: 0 },  
  { name: "Four square", cols: 4, rows: 4, shuffleAfter: 0 },  
  { name: "Shifting sands", cols: 5, rows: 4, shuffleAfter: 4 },  
  { name: "The big one", cols: 6, rows: 4, shuffleAfter: 3 },  
];
```

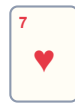
```
const scale = Math.min(1, AREA_WIDTH / gridWidth, AREA_HEIGHT / gridHeight);
```

Flips tween back to the tile's `baseScale`, not to 1.

14 / 20

Themes: what a tile shows vs what it means

Same picture, or same meaning? A `TileFace` has both

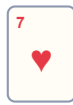


frame: 32
"7 red"



frame: 19
"7 red"

match



frame: 32
"7 red"



frame: 45
"7 black"

no match

frame
what the tile shows -
a different card for each

matchKey
what the tile means -
tiles match when these
are the same

```
export interface TileFace {  
  frame: number;      // the sprite sheet frame shown when the tile is face up  
  matchKey: string;   // two tiles match when their matchKeys are the same  
}
```

`PictureTheme` and `CardTheme` both implements `Theme`

15 / 20

Two more locked states

```
private peek(): void {  
  if (this.state !== "idle" || this.peeksLeft === 0) {  
    return;  
  }  
  this.state = "peeking";  
  ...  
}
```

```
const loose = this.tiles.filter((tile) => !tile.isMatched());  
const places = loose.map((tile) => ({ x: tile.x, y: tile.y }));  
Phaser.Utls.Array.Shuffle(places);
```

Shuffle the **places**, not the tiles

16 / 20

Buttons and best results

- `Button` extends `Phaser.GameObjects.Container` (Chapter 12) - an image and a text that move as one; `onClick` is a callback
- `BestResults` - local storage (Chapter 6), checked when read back

```
if (a.stars !== b.stars) {  
    return a.stars > b.stars;  
}  
if (a.moves !== b.moves) {  
    return a.moves < b.moves;  
}  
return a.seconds < b.seconds;
```

17 / 20

Try it: the advanced version

- play both themes: which is harder, and why?
- level 3 or 4: make wrong guesses until the board shuffles
- add a fifth level - one line



18 / 20

Summary

- grids: two loops, `(COLS - 1) * GAP`, equal margins, `row * COLS + col`
- `Shuffle` for pairs; `setFrame` to turn a tile over
- rules as a **state machine**; locked states ignore the player
- flips are two tweens; **callbacks** say when they have finished
- moves and time -> stars, in one small function
- an `interface` for themes; `matchKey` vs `frame`; levels as data

19 / 20

Challenges

1. **Choose the size** - keys 1, 2, 3 for three board sizes
2. **Against the clock** - 60 seconds, counting down
3. **A hint** - H flashes a pair, for two moves
4. **Two players** - take turns; a pair means another go
5. **Triple match** - groups of three
6. **Keyboard control** - a cursor, arrows and SPACE

Next: **Chapter 14 - Catch, avoid and shoot**

20 / 20

Catch, avoid and shoot

Overview

The first action-game chapter. Students meet one genre three times - a fruit catcher, a space shooter, and a space shooter with levels, bosses and power-ups - and see how each version grows from the last. The Phaser ideas are **spawn timers** (`this.time.addEvent`), **object pools** (a physics group with `classType` and `maxSize` , `get()` , `enableBody` / `disableBody`), **waves** built from tweens, timers and a curved path, a **fire rate**, **invulnerability**, particles, a `TileSprite` starfield, and (in the advanced version) levels as data, a boss, power-ups and a HUD scene fed by events. The design ideas - readable threats, rising pressure, fairness - matter as much as the code.

The one idea to make sure every student leaves with is **pooling**, including its price: an object that is reused must be *fully* reset every time. Most of the chapter's bugs, and most of the students' bugs, come from that.

Prerequisites

- Chapter 2: scene keys, `init()` and the restart trap, `scene.start(key, data)`
- Chapters 4 and 6: launching a scene in parallel; events between scenes; `localStorage`
- Chapter 5: the game-wide sound manager (for the advanced version's music)
- Chapter 7: tweens, animations, particles
- Chapter 8: Arcade bodies, groups, `overlap` , process callbacks, `as` casts in callbacks
- Chapter 12 or 13: a small state machine held in a union type

Learning outcomes

Students can:

1. keep a player on one line with velocity and `setCollideWorldBounds` , driven by held keys

2. spawn objects on a timer with `this.time.addEvent({ delay, loop | repeat, callback })` , and stop it
3. explain why a shooter pools its bullets, and build a pool with a physics group, `get()` , `enableBody(true, x, y, true, true)` and `disableBody(true, true)`
4. explain why `killAndHide` is not enough for a physics object, and why a pooled object must be fully reset
5. build waves from staggered tweens, repeating timers and a tween along a path, and count a wave down to its end
6. add a fire rate, flashing invulnerability with a process callback, explosions with particles, and a difficulty ramp
7. describe how a game grows: data-driven levels, a phase state machine, one job per class, a HUD that only listens

Suggested session plan (1 hour lecture + 2 hour lab)

Lecture (1 hour)

Time	Activity
0:00 - 0:05	Play all three versions on the projector, back to back. Ask: "what stayed the same?"
0:05 - 0:15	Slides 3-6: the genre, the three versions, the catcher - player, timer, falling groups
0:15 - 0:35	Slides 7-11: pooling. Live demo of <code>killAndHide</code> (below). The three switches table
0:35 - 0:45	Slides 12-14: waves and counting a wave down, invulnerability, the starfield
0:45 - 0:55	Slides 15-18: the advanced version - data, phases, the swoop path, the boss, power-ups, HUD
0:55 - 1:00	Slides 19-20: summary, the challenges, and who should start where

Lab (2 hours)

Time	Activity
0:00 - 0:15	Everyone builds and plays the catcher; change <code>SPAWN_DELAY</code> , <code>BOMB_CHANCE</code> and <code>FALL_GRAVITY</code> and feel the difference
0:15 - 0:50	Challenges 1 and 2 (catcher)
0:50 - 1:00	Group check-in: open the intermediate version and find all three places a pooled object is switched on or off
1:00 - 2:00	Challenge 3 or 6 (intermediate), then 4 or 5 (advanced) for those who get there

Key points to stress

- **Pool what you make many of, often.** Bullets and enemies are pooled; explosions and power-ups are not. Ask students to justify each - it is a judgement, not a rule
- **Three switches:** `active` (updates, and "free" to the pool), `visible` (drawn), `body.enable` (moves and collides). `disableBody(true, true)` turns off all three; `killAndHide` only two
- **Reset everything on reuse.** Tweens (`killTweensOf`), velocities (`enableBody` with `reset` clears them), angles, tints, fields such as a bullet's owner
- **Count the wave; don't ask the pool.** `countActive() === 0` is true in the gaps of a stream wave
- `get()` **can return** `null` - type it as `Bullet | null` so the compiler insists on a check
- **The player's class does not know its keys.** The scene reads input and calls `move(-1 | 0 | 1)`. Challenges 1 and 6 both depend on this, and neither changes the ship or basket class
- **Fairness is design, not luck:** small hit boxes (`setBodySize`), invulnerability after a hit, enemies that stop firing when close

Common problems and errors

What students see	Cause
<code>TypeError: Cannot read properties of null (reading 'fire')</code> in the console, and the game freezes, when firing fast	<code>this.bullets.get()</code> as <code>Bullet</code> with no null check: the pool is full
Invisible bullets still destroy enemies	switched off with <code>killAndHide</code> or <code>setVisible(false)</code> - the body still on
A reused enemy flies off to an old position, or starts moving at the wrong speed	a tween from its last life is still running
The next wave never comes, or two come at once	<code>enemiesLeft</code> not decremented on every exit (shot, crash, off screen, pool empty), or decremented twice
The basket falls off the bottom of the screen	gravity set in the game config rather than on the falling groups
<code>TS2345 [ERROR]: Argument of type '(bullet: Bullet, enemy: Enemy) => void' is not assignable to parameter of type 'ArcadePhysicsCallback'. followed by Type 'Body StaticBody GameObjectWithBody Tile' is not assignable to type 'Bullet'.</code>	typing the overlap callback's parameters as your classes
<code>TS4114 [ERROR]: This member must have an 'override' modifier because it overrides a member in the base class 'TileSprite'.</code>	<code>preUpdate</code> in a <code>TileSprite</code> subclass without <code>override</code>

What students see	Cause
TS2741 [ERROR]: Property 'shield' is missing in type '{ spread: string; rapid: string; life: string; }' but required in type 'Record<PowerUpKind, string>'. (challenge 4)	a new kind added to the union but not to the <code>Record</code>
Things can still be caught after GAME OVER	spawn timer not removed, physics not paused
The second game starts with the old score, or ends at once	fields reset only where declared
The HUD works, but after a few games the game slows, or a HUD value updates several times	HUD listeners on the game scene's emitter never removed. There is error message - in the console, <code>game.scene.getScene("GameScene").events.listenerCount("s changed")</code> shows 2 after the second game, 3 after the third...
Holding SPACE fires one shot, a pause, then a fast stream (the keyboard's auto-repeat)	<code>keyboard.on("keydown-SPACE")</code> used for firing

The first error and the three `TS` errors above were produced by making those mistakes in copies of the projects; the listener count was read the same way, by removing the `SHUTDOWN` handler.

Suggested demos and live-coding moments

- `killAndHide` **is not enough**. In the intermediate version, change the body of `Bullet.kill()` from `this.disableBody(true, true);` to `this.setActive(false).setVisible(false);` - which is what `killAndHide` does. Play: bullets still vanish when they hit, and it looks fine - until a stream wave lines ships up in a column, and one shot destroys the whole column. Ask why before explaining. We checked this: one bullet fired up a column of three ships destroyed all three (score 300), with no bullet visible after the first.

- **The leftover tween.** Remove `this.scene tweens.killTweensOf(this);` from `Enemy.kill()` , shoot ships while they are still swooping into the row, and watch the next stream wave: some ships fly up into a row that is not there. We checked this: an enemy switched off mid-swoop and handed out again at (700, 500) was dragged straight back up to the row.
- **Pool size.** Set `BULLET_POOL_SIZE` to 2 in the intermediate version: firing becomes a staccato pair. Then remove the `null` check - the game stops with the `TypeError` .
- **Tuning live.** Open `levels.ts` in the advanced version, make level 1's boss 5 health and the first wave 20 ships. "Level design without touching game code."

Discussion questions

- Explosions are not pooled. Should they be? What would you need to reset? (Animation, position, and the `ANIMATION_COMPLETE` listener - a pooled sprite would need `once` re-added each time, or a single `on` added when made.)
- Why does the intermediate version count `enemiesLeft` instead of asking `enemies.countActive() === 0` ? When would `countActive` be fine?
- Power-ups are "active until" times on the clock, not timers. What are the advantages? What would go wrong with a `delayedCall` that switched spread shot off, if the player picked up a second one?
- `WavePattern` is `"row" | "stream" | "swoop"` . In Java you would use an `enum` . What does each give you that the other does not?
- The HUD only listens to events. What would be lost if it read `GameScene` 's fields every frame instead?
- Where is the line between "fair" and "easy"? Which of the fairness rules in the chapter would you remove for an "expert" mode?

Extension ideas

- enemy variety: a class hierarchy (`Enemy` -> `Diver` , `Tank`) with different health and scores, still in one pool per class
- a boss with phases: a new attack pattern below half health
- replace the row wave's tween with `Phaser.GameObjects.PathFollower` and compare

- a pause scene (Chapter 4) that also pauses the music
 - screen shake strength that scales with what exploded; a hit-stop (pause physics for 50 ms)
-

Assessment ideas

- Practical: "add a fourth wave pattern of your own design to the advanced version's `WaveSpawner` , and use it in `levels.ts` " - look for the new pattern in the union type, a reset-safe enemy, and correct wave counting
 - Code reading: show `Enemy.kill()` without `killTweensOf` and a description of the bug; ask for the cause and the fix
 - Short answer: "What does each of `setActive(false)` , `setVisible(false)` and `disableBody()` switch off? Which does a pooled bullet need, and why?"
 - Short answer: "Why is a fire rate a stored time rather than a `delayedCall` ?"
-

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 14 - Catch, avoid and shoot - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change is marked with a `// CHALLENGE n` comment (and `<!-- CHALLENGE n -->` in `index.html`). All six were played with the test harness, as described under each one.

1. Pointer control

Project: `solutions/ch14_challenge_1_pointer_control` (from `ch14_catcher_simple`)

Goal: scene-wide pointer events, and moving an object *towards* a target rather than onto it.

The pointer sets a target; the keys, when pressed, take over again:

`src/scenes/GameScene.ts`

```
this.input.on("pointermove", (pointer: Phaser.Input.Pointer) => {
    this.pointerTargetX = pointer.x;
});
this.input.on("pointerdown", (pointer: Phaser.Input.Pointer) => {
    this.pointerTargetX = pointer.x;
});
```

```
// CHALLENGE 1: a key press takes over from the pointer, until the pointer
moves again
if (this.cursors.left.isDown) {
    this.pointerTargetX = null;
    this.basket.move(-1);
} else if (this.cursors.right.isDown) {
    this.pointerTargetX = null;
    this.basket.move(1);
} else if (this.pointerTargetX !== null) {
    this.moveTowardsPointer(this.pointerTargetX);
} else {
    this.basket.move(0);
}
```

```
private moveTowardsPointer(targetX: number): void {
  const distance = targetX - this.basket.x;
  if (Math.abs(distance) < CLOSE_ENOUGH) {
    this.basket.move(0);
  } else {
    this.basket.move(Math.sign(distance));
  }
}
```

`Basket` is unchanged - the scene still only calls `move(-1 | 0 | 1)`. That is the pay-off of the chapter's design: a new way to control the basket, and not one line of the basket's class changes.

Look for:

- `pointerdown` as well as `pointermove`: a touch screen has no "move without pressing"
- a dead zone (`CLOSE_ENOUGH`). Without it the basket overshoots by a few pixels every frame and jitters from side to side - a good thing to let students see, then explain
- `pointerTargetX` reset in `init()` - otherwise a restarted game remembers where the pointer was
- a common simpler answer sets `this.basket.x = pointer.x`. It works, but the basket teleports and ignores its speed; the challenge asks it to slide. Some students use `this.physics.moveTo(basket, pointer.x, basket.y, SPEED)` - fine, if they stop it on arrival

Tested with the harness: mouse moved to `x = 100`, the basket stopped at 96; clicked at `x = 650`, it stopped at 656; LEFT held, it moved left and the pointer target was cleared.

2. Combo scoring

Project: `solutions/ch14_challenge_2_combo_scoring` (from `ch14_catcher_simple`)

Goal: a piece of game state with a clear reset rule, and feedback with a tween.

`src/scenes/GameScene.ts`

```
// CHALLENGE 2: each fruit in a row is worth 10 more than the last, up to
MAX_COMBO times 10
this.combo = this.combo + 1;
const points = FRUIT_POINTS * Math.min(this.combo, MAX_COMBO);
this.showPoints(fruit.x, fruit.y, points);
```

The combo breaks in one place - `loseLife()`, which both a dropped fruit and a caught bomb already go through:

```
this.sound.play(HURT_SOUND);
this.combo = 0; // CHALLENGE 2: a dropped fruit or a caught bomb
breaks the combo
this.lives = this.lives - 1;
```

The floating score is a text that tweens up and fades, then destroys itself:

```
this.tweens.add({
  targets: text,
  y: y - 80,
  alpha: 0,
  duration: 700,
  onComplete: () => {
    text.destroy();
  },
});
```

Look for:

- the reset in **one** place, reached by every way of missing - students who reset in both `catchBomb` and the drop code often forget one
- `this.combo = 0` in `init()` as well (the restart trap again)
- `text.destroy()` in `onComplete` : without it, every caught fruit leaves an invisible text behind
- reading `fruit.x` / `fruit.y` **before** `fruit.destroy()` . (A destroyed Phaser object still has its last `x` and `y`, so reading them after works too - but it is a bad habit.)

Tested with the harness: three fruit caught in a row scored $10 + 20 + 30 = 60$, a caught bomb set the combo to 0, and the next fruit scored 10.

3. Zig-zag enemies

Project: `solutions/ch14_challenge_3_zig_zag` (from `ch14_shooter_intermediate`)

Goal: a new wave pattern, combining a velocity with a repeating tween; a difficulty ramp.

The zig-zag is a yoyo tween on `x` that repeats for ever, while the velocity carries the ship down:

`src/objects/Enemy.ts`

```
public zigZag(width: number, time: number): void {
  this.scene tweens.add({
    targets: this,
    x: { from: this.x - width / 2, to: this.x + width / 2 },
    duration: time,
    ease: "Sine.easeInOut",
    yoyo: true,
    repeat: -1,
  });
}
```

`src/scenes/GameScene.ts`

```
// CHALLENGE 3: three patterns in turn - row, stream, zig-zag, row, ...
const pattern = this.wave % 3;
if (pattern === 1) {
  this.rowWave(size, ROW_SPEED + extraSpeed);
} else if (pattern === 2) {
  this.streamWave(size, STREAM_SPEED + extraSpeed);
} else {
  this.zigZagWave(size, ZIGZAG_SPEED + extraSpeed);
}
```

```
const width = Math.min(ZIGZAG_WIDTH + this.wave * ZIGZAG_WIDTH_PER_WAVE,
  ZIGZAG_MAX_WIDTH);
const time = Math.max(ZIGZAG_TIME - this.wave * ZIGZAG_TIME_PER_WAVE,
  ZIGZAG_MIN_TIME);
```

"Wilder" is wider (up to a limit) and quicker (down to a limit). `Math.min` and `Math.max` are the limits - without them, wave 30 would be unplayable.

The important point to draw out: `Enemy.kill()` already calls `killTweensOf(this)`, so the endless zig-zag tween is stopped when the ship is shot down or leaves - and the pool hands out a clean ship next time. Tested with the harness: a zig-zag wave (wave 3), then the row wave that followed reused the same enemy objects, and every one of them had 0 tweens and flew in a straight row.

Look for:

- ships spawned far enough from the edges for the whole sway to stay on screen
- an answer that sways with `Math.sin(time)` in a `preUpdate()` is also good - check that it stores the ship's starting x and resets it in `spawn()`, because a pooled enemy keeps its fields
- a `repeat: -1` tween **without** `killTweensOf` in `kill()` - the classic pooled-object bug. Ask what the next row wave looks like

4. Shield power-up

Project: `solutions/ch14_challenge_4_shield` (from `ch14_shooter_advanced`)

Goal: extend a union type and follow the compiler to every place that must change; a picture made in code; a new rule for being hit.

Adding `"shield"` to `PowerUpKind` and nothing else gives exactly the two errors the hint promises:

```
TS2741 [ERROR]: Property 'shield' is missing in type '{ spread: string; rapid: string; life: string; }' but required in type 'Record<PowerUpKind, string>'.
```

```
const TEXTURES: Record<PowerUpKind, string> = {  
    ~~~~~
```

```
TS2345 [ERROR]: Argument of type '"spread" | "rapid" | "shield"' is not assignable to parameter of type 'WeaponPowerUp'.
```

```
    Type '"shield"' is not assignable to type 'WeaponPowerUp'.
```

```
    this.ship.powerUp(powerUp.kind, this.time.now);  
    ~~~~~
```

The first needs a picture, which `PreloadScene` draws with `Graphics` (a blue orb for the pickup, and a see-through bubble for the ship). The second needs a new branch in `collect()` :

`src/scenes/GameScene.ts`

```
} else if (powerUp.kind === "shield") {  
    this.ship.giveShield();          // CHALLENGE 4  
} else {
```

The ship owns its bubble, and keeps it in place in a new `preUpdate()` :

`src/objects/PlayerShip.ts`

```
preUpdate(_time: number, _delta: number): void {  
    this.bubble.setPosition(this.x, this.y);  
}
```

and `shipHit()` checks the shield first:

`src/scenes/GameScene.ts`

```
if (this.ship.hasShield()) {  
    this.ship.popShield();  
    this.sound.play(HIT_SOUND);  
    this.ship.makeInvulnerable();  
    return;  
}
```

The shield never wears off, so it is a `boolean` , not an "until" time like spread and rapid. The HUD needed no new code: `describeWeapon()` adds `"SHIELD"` , and the existing `WEAPON_CHANGED` event carries it. The menu legend gained a fourth entry.

Look for:

- the `KINDS` array updated too (the compiler does **not** catch this one - without it, shields never drop). A good question for the class: why can the compiler check the `Record` but not the array?
- the moment of safety after the shield pops - without it, two bullets arriving together take the shield *and* a life
- the bubble drawn above the ship (made after it), and hidden when the shield pops

Tested with the harness: a shield power-up collected (HUD showed SHIELD, bubble visible), an enemy bullet popped it with lives still 3, a second bullet two seconds later cost a life.

5. Smart bomb

Project: `solutions/ch14_challenge_5_smart_bomb` (from `ch14_shooter_advanced`)

Goal: act on every live member of a pool; keep the wave count correct; add a HUD value through the event system.

`src/scenes/GameScene.ts`


```

private useBomb(): void {
    if (this.bombs === 0 || this.phase === "over") {
        return;
    }
    this.bombs = this.bombs - 1;
    this.events.emit(BOMBS_CHANGED, this.bombs);
    this.cameras.main.flash(BOMB_FLASH_TIME);

    const { width, height } = this.scale;
    for (const enemy of this.enemies.getMatching("active", true) as Enemy[]) {
        if (enemy.x >= 0 && enemy.x <= width && enemy.y >= 0 && enemy.y <=
height) {
            this.explode(enemy.x, enemy.y);
            enemy.kill();
            this.addScore(ENEMY_POINTS);
            this.enemyGone();
        }
    }
    for (const bullet of this.enemyBullets.getMatching("active", true) as
EnemyBullet[]) {
        bullet.kill();
    }
    if (this.phase === "boss" && this.boss.active) {
        this.boss.weaken(BOMB_BOSS_DAMAGE);
        this.events.emit(BOSS_HEALTH_CHANGED, this.boss.getHealth(),
this.boss.getMaxHealth());
    }
}

```

The two traps the hint points at:

- **"on screen"** - `getMatching("active", true)` also returns ships that are active but waiting off the edge: a row wave's ships above the top, a swoop's ships queued at the start of the path. They should not explode
- `enemiesLeft` - every destroyed ship goes through `enemyGone()`, exactly as a shot-down ship does, so the count stays right and the next wave starts exactly once. The tempting shortcut, `this.enemiesLeft = 0`, is wrong: in a stream wave the ships still to come would then count it below zero, and the wave would never end

The boss "loses a chunk but survives" through a new `Boss.weaken()`, which never takes the last point of health. The HUD shows the bombs through a new `BOMBS_CHANGED` event, and `HudData` gained a `bombs` field for the starting value.

Look for:

- the new listener removed in the HUD's `SHUTDOWN` handler, like the others
- `bombs` reset in `init()`
- `keyboard.on("keydown-B")` , not `once` - it is used up to three times

Tested with the harness: in a stream wave with 2 of 6 ships on screen, B destroyed those 2 (`enemiesLeft` 6 to 4) and the wave still ended when the other 4 had gone; pressed while a swoop's ships were still off screen, it destroyed none; against a 30-health boss it took off 10 and the boss lived; a fourth press did nothing.

6. Second player

Project: `solutions/ch14_challenge_6_second_player` (from `ch14_shooter_intermediate`)

Goal: turn "the player" into data the scene loops over; share a pool between two owners.

Everything that belongs to one player goes in one object:

`src/scenes/GameScene.ts`

```
interface Player {
  name: string;
  ship: PlayerShip;
  left: Phaser.Input.Keyboard.Key;
  right: Phaser.Input.Keyboard.Key;
  fire: Phaser.Input.Keyboard.Key;
  score: number;
  lives: number;
  hudText: Phaser.GameObjects.Text;
}
```

and the old one-player code runs once per player still in the game:

```
for (const player of this.livingPlayers()) {
  if (player.left.isDown) {
    player.ship.move(-1);
  } else if (player.right.isDown) {
    player.ship.move(1);
  } else {
    player.ship.move(0);
  }
}
```

`PlayerShip` does not change at all - it never knew its keys, which is the point of the hint.

Both ships share the bullet pool (made twice as big). A bullet remembers who fired it:

`src/objects/Bullet.ts`

```
public fire(x: number, y: number, owner: number): void {
  this.owner = owner;
  this.enableBody(true, x, y, true, true);
}
```

`src/scenes/GameScene.ts`

```
const shooter = this.players[bullet.owner]; // CHALLENGE 6: read before
kill(), to be clear
```

The stream wave aims at a random living player

(`Phaser.Utls.Array.GetRandom(living)`). Each ship has its own overlap with the enemies, so the callback knows which player was hit. A player with no lives has their ship switched off with `disableBody(true, true)`; when `livingPlayers()` is empty, the game ends, and `GameOverScene` gets `scores: number[]` and says who won (or that it was a draw).

Look for:

- no duplicated "player 2" copies of methods - a loop, or a method taking a `Player`
- the owner set on **every** fire: a pooled bullet keeps its last owner otherwise, and the wrong player scores
- the stream wave not aiming at a player who is out
- a second `PlayerShip` class for player 2, or `if (player === 2)` inside `PlayerShip`, both miss the point of the hint - worth discussing

Tested with the harness: both ships moved on their own keys; bullets fired as player 2 scored for player 2; player 2 knocked out (ship gone, HUD "OUT") while player 1 played on; the next stream wave dropped all five ships above player 1; when player 1 was out too, the game over screen said "Player 2 wins!".

Chapter 14 - Catch, avoid and shoot - Slides

Chapter 14

Catch, avoid and shoot

One genre, three games



1 / 20

Today

- a player on one line; spawning on a timer
- **object pools** - and the price of reuse
- waves: tweens, timers, a curved path
- fire rate, invulnerability, explosions, a starfield
- growing a game: levels as data, a boss, power-ups, a HUD scene

2 / 20

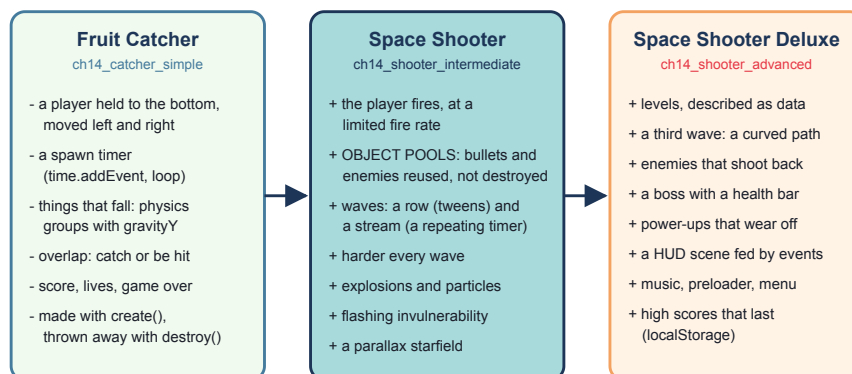
The genre

- the player can do almost nothing - the game is in **what comes down the screen**
- **readable** threats: a bomb looks like a bomb
- **rising pressure**: easy, then harder and harder
- **fairness**: small hit boxes, a moment of safety, no point-blank shots

3 / 20

Three versions

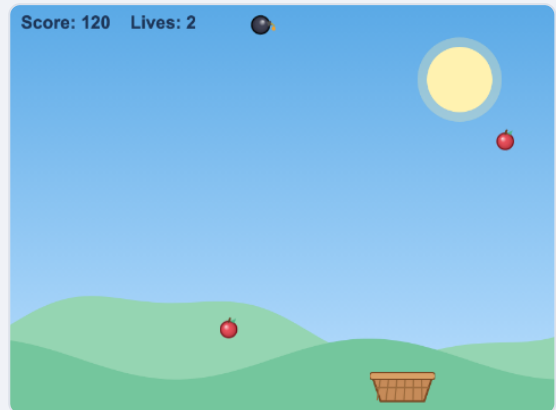
One game, three times - each version keeps the last one's ideas and adds more



4 / 20

Try it: Fruit Catcher

- build and play `ch14_catcher_simple`
- change `SPAWN_DELAY`, `BOMB_CHANCE`, `FALL_GRAVITY` - how does it feel?
- why does a fruit brushing the side not count?



5 / 20

A timer, and things that fall

```
this.fruit = this.physics.add.group({ gravityY: FALL_GRAVITY });

this.spawnTimer = this.time.addEvent({
  delay: SPAWN_DELAY,
  loop: true,
  callback: () => {
    this.dropSomething();
  },
});

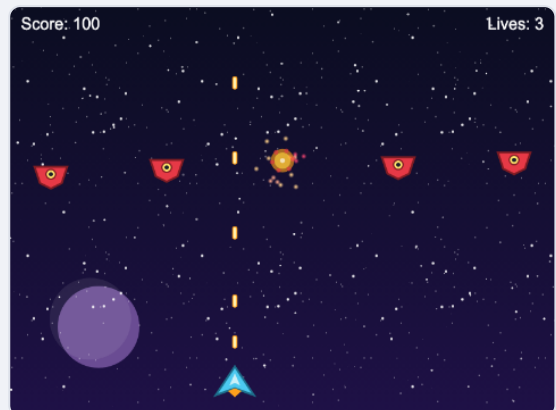
this.physics.add.overlap(this.basket, this.fruit, (_basket, fruit) => {
  this.catchFruit(fruit as Phaser.Physics.Arcade.Sprite);
});
```

`loop: true` - for ever; `repeat: n` - $n + 1$ times; `remove()` - stop

6 / 20

Try it: Space Shooter

- hold SPACE: about 5 shots a second
- how many bullet **objects** does a minute of firing need?
- (we measured: **9**)

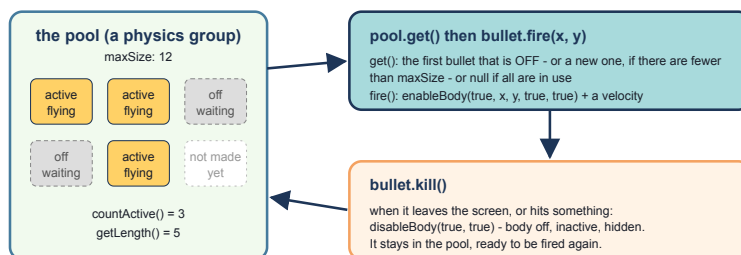


7 / 20

Why pool?

- make + destroy hundreds of objects a minute -> the **garbage collector** -> stutter
- instead: make a few, **switch them off**, switch them on again

An object pool: the same few bullets, switched on and off again and again



No new objects while the game runs, and nothing for the garbage collector to clear up - just switches.

8 / 20

A pool in Phaser

```
this.bullets = this.physics.add.group({ classType: Bullet, maxSize: BULLET_POOL_SIZE });

// firing
const bullet = this.bullets.get() as Bullet | null;
if (bullet === null) {
    return;
}
bullet.fire(this.ship.x, this.ship.y - MUZZLE_OFFSET);

// in Bullet
this.enableBody(true, x, y, true, true); // on
this.disableBody(true, true);           // off
```

9 / 20

Three switches

Switch	Off means
active	no <code>preUpdate()</code> ; free to the pool
visible	not drawn
body.enable	not moved, overlaps nothing

`killAndHide` = the first two only. **Demo:** an invisible bullet that still shoots down ships.

10 / 20

The price of reuse

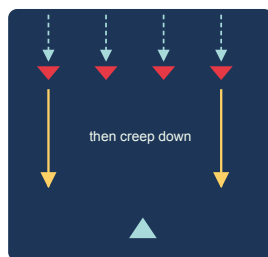
```
public kill(): void {  
    this.scene.tweens.killTweensOf(this);  
    this.disableBody(true, true);  
}
```

- a reused object brings back **everything** you do not reset
- tweens, velocities, angles, tints, fields
- "only goes wrong the second time" = this

11 / 20

Waves

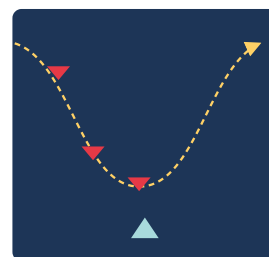
Three wave patterns



row
tweens, each with a longer delay



stream
a timer with repeat: size - 1



swoop (advanced)
a Spline path; a tween moves 0 to 1 along it

- row: tweens with `delay: i * ROW_STAGGER`
- stream: `addEvent({ delay, repeat: size - 1, ... })`
- count the wave down: `enemiesLeft`, one `enemyGone()`

12 / 20

Hit, flash, safe

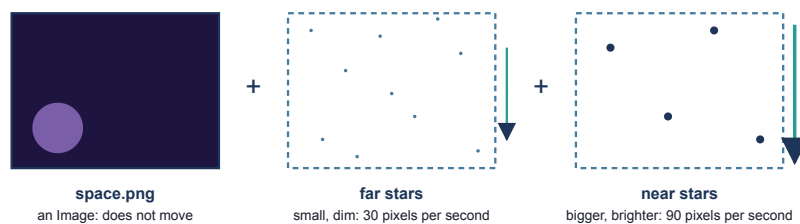
```
this.scene.tweens.add({
  targets: this,
  alpha: 0.15,
  duration: FLASH_TIME,
  yoyo: true,
  repeat: INVULNERABLE_TIME / (FLASH_TIME * 2) - 1,
  onComplete: () => {
    this.setAlpha(1);
    this.invulnerable = false;
  },
});
```

- a process callback: `() => !this.ship.isInvulnerable()`

13 / 20

A parallax starfield

A starfield in three layers - a picture and two TileSprites



Each TileSprite repeats a 256 x 256 star picture; changing `tilePositionY` slides the picture inside it.

`TileSprite` + `tilePositionY`; textures drawn with `Graphics` + `generateTexture`

14 / 20

Try it: Space Shooter Deluxe

- three levels, a boss each
- enemies shoot back; power-ups
- HUD scene, music, high scores
- open `levels.ts` - make level 1 easier



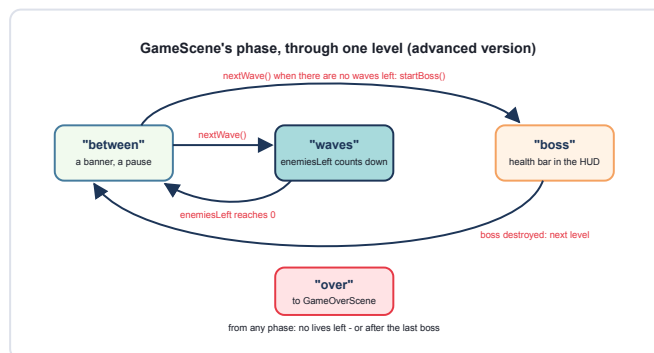
15 / 20

Levels as data, and a phase

```
export type WavePattern = "row" | "stream" | "swoop";

export interface WaveData {
  pattern: WavePattern;
  size: number;           // how many ships
  speed: number;          // pixels per second (swoop: ms)
}

type Phase = "waves" | "between" | "boss" | "over";
```



16 / 20

Along a curve

```
// WaveSpawner.swoop()
const path = new Phaser.Curves.Spline(points);
enemy?.followPath(path, duration, i * SWOOP_GAP);

// Enemy.followPath(): a tween moves pathProgress from 0 to 1
this.scene tweens.add({
  targets: this,
  pathProgress: 1,
  duration: duration,
  delay: delay,
});

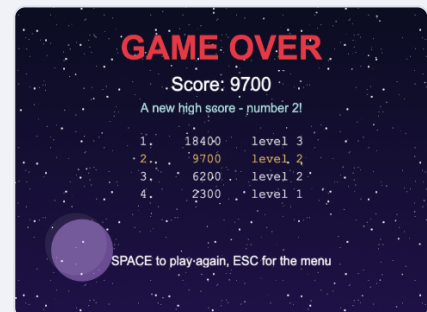
// Enemy.preUpdate(): every frame, go to that point on the curve
this.path.getPointAt(this.pathProgress, this.point);
this.setPosition(this.point.x, this.point.y);
```

A tween can move **any** number.

17 / 20

Boss, power-ups, HUD

- boss: an entry tween, then `yoyo: true, repeat: -1`; a hit flash with `setTint(0xffffffff).setTintMode(Phaser.TintModes.FILL)` (no `setTintFill` in Phaser 4)
- power-ups: "active until" `times`; `Record<PowerUpKind, string>`; narrowing in `collect()`
- HUD: `this.events.emit(SCORE_CHANGED, this.score)` - and `off()` on `SHUTDOWN`



18 / 20

Summary

- `time.addEvent` spawns; physics groups give members the same motion
- pool what you make **many of, often**: `classType` , `maxSize` , `get()` , `enableBody` / `disableBody`
- reset **everything** on reuse; `killAndHide` leaves the body on
- waves: staggered tweens, repeating timers, paths - counted down to zero
- fire rate and power-ups = stored times
- grow with data, phases, small classes, and a HUD that listens

19 / 20

Challenges

1. **Pointer control** - the basket follows the mouse or a finger (catcher)
2. **Combo scoring** - 10, 20, 30... for fruit in a row (catcher)
3. **Zig-zag enemies** - a third wave pattern (intermediate)
4. **Shield power-up** - takes the next hit (advanced)
5. **Smart bomb** - B clears the screen, three a game (advanced)
6. **Second player** - A / D / W, two scores (intermediate)

Next: **Chapter 15 - Platformer**

20 / 20

Overview

The second genre chapter where movement *is* the game. Students meet the same platformer three times, each version a complete game: a single screen of static platforms and coins; a scrolling Tiled level with a hero that feels good to control, patrolling enemies and hazards; and a three-level game with moving and one-way platforms, ladders, swooping bats, checkpoints, lives, a HUD scene and a level select.

The Phaser ideas are **Arcade gravity and** `body.blocked.down` , **acceleration, drag and max velocity, tile properties** (`setCollisionByProperty` , `tile.properties`), `checkCollision` **and process callbacks** for one-way collisions, and **immovable bodies that carry riders**. The design ideas are **game feel** (variable jump, coyote time, jump buffering), **small state machines** (hero, bat), **interfaces** for enemies, **passing a function** instead of an object, and **data-driven levels**. Almost everything else is revision of Chapters 2-11, used for real.

Prerequisites

- Chapters 7-11 in particular: sprite sheet animations and the hero state machine (7), colliders, overlaps, groups and process callbacks (8), acceleration, drag and gravity (9), tilemaps and cameras (10), Tiled maps and object layers (11)
- Chapters 4 and 6 for the advanced version's HUD scene and registry events; Chapter 6 for `localStorage`
- Java interfaces (for `Enemy`) and, ideally, a one-method interface or lambda in Java (for `LadderFinder`)

Learning outcomes

Students can:

1. make a hero that runs and jumps with Arcade Physics, jumping only when `body.blocked.down`
2. explain and implement a variable jump height, coyote time and jump buffering, and tune movement constants by play-testing
3. build a level from a Tiled map using tile properties for collision and hazards, and place game objects from the Objects layer
4. decide between stomp and hurt in an overlap callback, and use a process callback to make tile hazards fair
5. make one-way platforms with `checkCollision`, and platforms that carry the hero
6. structure a multi-level game: one data-driven game scene, a HUD scene fed by the registry, checkpoints and lives, and progress saved between sessions

Suggested session plan (2 x 2-hour labs)

Session 1 - the simple and intermediate versions

Time	Activity
0:00 - 0:10	Play <code>ch15_platformer_simple</code> . Ask: "what does it feel like?" (stiff; every jump the same). Keep the answers for later
0:10 - 0:30	Slides 3-5: gravity, the Arcade Sprite and <code>declare body</code> , <code>blocked.down</code> ; then in the code, the animation state machine and collider vs overlap. Live-code the jump (see demos)
0:30 - 0:50	Challenges 1 and 2
0:50 - 1:05	Play <code>ch15_platformer_intermediate</code> . Compare with the list from 0:00
1:05 - 1:25	Slides 7-9: the Tiled map and its properties, game feel (acceleration and drag in the code, then the jump cut, coyote time, buffering). The coyote demo
1:25 - 1:45	Slides 10-11: slimes, stomp or hurt, fair hazards
1:45 - 2:00	Start challenge 4 (wall jump)

Session 2 - the advanced version

Time	Activity
0:00 - 0:10	Play <code>ch15_platformer_advanced</code> , all three levels (unlock them by editing <code>localStorage</code> in the dev tools, or play level 1 fast)
0:10 - 0:40	Slides 13-17: data-driven levels, one-way and moving platforms, ladders and <code>LadderFinder</code> , bats, the <code>Enemy</code> interface, checkpoints and the HUD
0:40 - 1:00	Finish challenge 4
1:00 - 2:00	Challenges 3, 5 and 6 - or a free "make your own level" exercise in Tiled

A single 3-hour lab can cover the simple and intermediate versions with challenges 1, 2 and 4, and leave the advanced version for reading at home.

Key points to stress

- `blocked.down` **is set by collisions**, and only by a collider or the world bounds. It is the question "am I standing on something?", asked of the physics engine rather than worked out from positions
- **a jump is a velocity; gravity does the rest.** The height is $\text{speed}^2 / (2 \times \text{gravity})$: students who want "a jump 3 tiles high" can work out the speed rather than guessing
- **game feel is numbers.** Acceleration, drag, jump speed, gravity, jump cut, coyote time and buffer are all constants at the top of `Hero.ts` . Encourage students to change them one at a time and play: this is exactly what game designers do
- **remember *when*, not just *whether*** - coyote time and buffering both come from storing a timestamp. The same idea gives invulnerability after respawning (`safeUntil`) and the wall-jump key lock in challenge 4
- **tiles say what they are.** No tile numbers in the code: properties set once in the tileset apply to every tile in every level
- **overlap with a tilemap layer includes empty tiles** - the process callback is not optional
- **the scene decides what a touch means**, not the hero or the enemy. That keeps `Slime` , `Bat` and `Hero` independent - and in the advanced version, the `Enemy` interface keeps the scene independent of which enemies exist
- **the data drives the levels.** Adding a level means a map and a line in `levels.ts`

Common problems and errors

What students see	Cause	Fix
<code>TS2531 [ERROR]:</code> Object is possibly 'null'. on every <code>this.body...</code> line, and <code>Property</code> 'acceleration' does not exist on type 'Body StaticBody'.	the <code>declare body:</code> <code>Phaser.Physics.Arcade.Body;</code> line is missing from a class extending <code>Phaser.Physics.Arcade.Sprite</code>	add it (and discuss why it is safe: the constructor gives the sprite a dynamic body)
<code>TS4114 [ERROR]:</code> This member must have an 'override' modifier because it overrides a member in the base class 'Sprite'.	<code>preUpdate</code> without <code>override</code>	add <code>protected override</code>
The hero never animates, or is stuck on one frame	<code>super.preUpdate(time, delta)</code> missing from <code>preUpdate()</code> , or <code>play(key)</code> without <code>true</code> called every frame	call <code>super.preUpdate</code> ; use <code>play(key, true)</code>
The hero falls through the level	no collider with the ground layer, or <code>setCollisionByProperty</code> missing, or the property not set on the tiles in Tiled	check all three; <code>debug: true</code> shows the bodies
The hero cannot jump from a platform	the platform is an <code>overlap</code> , or a group made with <code>this.add.group()</code> without bodies - so <code>blocked.down</code> is never set	use <code>this.physics.add.collider</code>

What students see	Cause	Fix
Tilemap has no tileset "platform". Its tilesets are ... (with the list of names) in the console, then (in these projects) Error: The map has no tileset called "platform"	the first argument of <code>addTilesetImage</code> must be the tileset's name <i>inside the map</i>, not the image key	use the name shown in Tiled's Tilesets panel
Invalid Tilemap Layer ID: ground then Valid tilelayer names: ... in the console	a layer name typed with the wrong case	layer names are case-sensitive and must match Tiled exactly
TS2322 [ERROR]: Type 'TilemapLayer TilemapGPULayer' is not assignable to type 'TilemapLayer'.	<code>createLayer</code> can return either kind of layer	as <code>Phaser.Tilemaps.TilemapLayer</code>, with a comment (the projects' <code>createLayer</code> helper)
TS2345 [ERROR]: Argument of type 'Body StaticBody Tile GameObjectWithBody' is not assignable to parameter of type 'Sprite'.	passing a collider/overlap callback's argument straight to a method	cast it: <code>coin as Phaser.Physics.Arcade.Sprite</code> (Chapter 8)
Touching the air next to spikes kills the hero, or the hero dies as soon as the level starts	overlap with a tilemap layer calls back for empty tiles too	check <code>tile.properties.hazard</code> (or <code>tile.index !== -1</code>) in a process callback
Every touch of a slime squashes it - even walking into it	the stomp test has no "falling" check, or no margin	<code>velocity.y > 0</code> and <code>bottom <= top + margin</code>

What students see	Cause	Fix
The hero is hurt twice by one slime, or by a slime that was just squashed	the enemy's body is still enabled during its squash tween	<code>body.enable = false</code> at the start of <code>squash()</code>
The hero slides off a moving platform	the platform is moved by a tween (or by setting <code>x</code>)	move it by velocity, as <code>MovingPlatform</code> does
Static objects (the springboard in challenge 2) are hit in the wrong place after <code>setOrigin</code> or <code>setScale</code>	static bodies do not follow changes to their game object	call <code>refreshBody()</code>

Suggested demos and live-coding

- **Build the jump live** in the simple version: start with no jump; add `setVelocityY(-JUMP_SPEED)` on SPACE with no floor check (infinite flying - students love it); add the `blocked.down` check; then print `JUMP_SPEED2 / (2 x 900)` and measure the height with `debug: true`
- **Feel, side by side.** Run the simple and intermediate versions next to each other. Then in the intermediate version set `COYOTE_TIME` and `JUMP_BUFFER` to 0 and ask students to run off ledges and jump just before landing. Put them back. Most will not believe how much difference 100 ms makes until they try it
- **The empty-tile surprise.** Replace the hazard process callback with `undefined`: the hero dies the moment it goes near anything. Ask why before explaining
- **Tween vs velocity platform.** In `MovingPlatform`, swap the velocity code for a yoyo tween and `body.setDirectControl(true)`, and throttle the CPU in the browser's dev tools: the hero slides on the tween platform, and not on the velocity one
- **A new level in five minutes.** Open `tiled/level1.tmx` in Tiled, add a row of spikes and a coin, save, copy to `public/assets/maps/`, rebuild and play. Nothing in the code changes

Discussion questions

- Why does the hero need a smaller body than its picture? What would feel unfair if it were bigger? If it were much smaller?
- Coyote time lets the player jump when the hero is not on the ground. Is that cheating? Who is it for? (It makes the game do what the player *meant*.)
- The slime checks for edges by looking at the tilemap; the hero asks a function for ladders. Which design is better? What would you have to change to put a slime on a moving platform?
- `heroMeetsEnemy` lives in the scene, not in `Hero` or `Enemy`. Why? Where else could it go?
- Levels are data. What else in this game could be data rather than code? (Enemy speeds, medal times, the hero's constants - then a "moon level" with low gravity is data too.)

Extension ideas

- a level editor mode: click to place tiles and objects, then print the level as JSON
- slopes (Arcade Physics has none - discuss why Matter.js would handle them, Chapter 9)
- a crumbling platform that falls a moment after the hero lands on it
- parallax with several layers moving at different speeds, or a `tileSprite` background
- a speed-run ghost: record the hero's positions each frame and replay them as a translucent sprite on the next attempt

Assessment ideas

- **Practical:** "Add a fourth level" - a map (in Tiled) and one line in `levels.ts`. Checks Tiled skills, tile properties, object types and the data-driven design, with very little code
- **Practical:** add a new hazard tile (water that slows the hero instead of hurting it), using a new tile property
- **Code reading:** show `jump()` from the intermediate hero with the two `-1000` lines deleted; ask what the player would see and why
- **Short answer:** explain the difference between `collider` and `overlap`, and give one use of each from this chapter; explain what a process callback is for, with an example

from this chapter

Challenge solutions

See [solutions.md](#). Every challenge has a runnable solution in `solutions/`.

Chapter 15 - Platformer - Solutions

Each solution is a complete project in `solutions/`, made from the chapter project the challenge starts from. Every change to the code is marked with a `// CHALLENGE n` comment. Two solutions (4 and 6) also change the Tiled maps; JSON cannot hold comments, so those changes are described below.

1. Double jump

Project: `solutions/ch15_challenge_1_double_jump` (from `ch15_platformer_simple`)

Goal: a counter that is used up in the air and refilled on landing - the first taste of the hero remembering something between frames.

`src/objects/Hero.ts`

```
const AIR_JUMP_SPEED = 420; // CHALLENGE 1: the jump in the air is a little weaker
const AIR_JUMPS = 1; // CHALLENGE 1: how many extra jumps the hero gets in the air
...
// CHALLENGE 1: standing on something gives the air jumps back
if (this.body.blocked.down) {
  this.airJumpsLeft = AIR_JUMPS;
}

if (jumpPressed && this.body.blocked.down) {
  this.setVelocityY(-JUMP_SPEED); // negative y is UP
  this.scene.sound.play(JUMP_SOUND);
} else if (jumpPressed && this.airJumpsLeft > 0) {
  // CHALLENGE 1: a second jump, in the air - it REPLACES the speed, so it works the same
  // whether the hero is still rising or already falling
  this.airJumpsLeft = this.airJumpsLeft - 1;
  this.setVelocityY(-AIR_JUMP_SPEED);
  this.scene.sound.play(JUMP_SOUND, { rate: 1.3 }); // a higher-pitched jump sound
}
```

Tested with the harness, logging the vertical speed every frame while pressing jump three times: -520 (the ground jump), later -420 (the air jump), and nothing for the third press; the counter is back to 1 after landing.

Look for:

- the refill happens on the *floor*, not in the jump - otherwise the first jump would use the air jump up too
 - **setting** the speed rather than **adding** to it. `velocity.y - 420` while falling fast gives a feeble second jump; while rising it gives a rocket. Setting it makes the second jump the same every time
 - `AIR_JUMPS` as a constant: a triple jump is then a one-character change. Some students use a boolean `hasDoubleJumped` ; that is fine for exactly one extra jump
 - walking off a ledge also allows one air jump here. Either answer is fine if the student can say which they chose
-
-

2. Springboard

Project: `solutions/ch15_challenge_2_springboard` (from `ch15_platformer_simple`)

Goal: a texture drawn in code, a static body with a collider callback, and the hero being told to move rather than moving itself.

```
src/scenes/GameScene.ts
```



```

if (!this.textures.exists(SPRING_KEY)) {
    // make.graphics(..., false) draws off-screen: the Graphics is only used
    // to make a texture
    const g = this.make.graphics({}, false);
    ...
    g.generateTexture(SPRING_KEY, SPRING_WIDTH, SPRING_HEIGHT);
    g.destroy();
}

// origin (0.5, 1): (x, y) is the bottom middle - here, the top of the
// ground strip
const spring = this.physics.add.staticImage(SPRING_X, 536,
SPRING_KEY).setOrigin(0.5, 1);
spring.refreshBody(); // a static body does not follow changes to the image
// by itself

this.physics.add.collider(this.hero, spring, () => {
    // only when landing on TOP of it, not walking into its side
    if (this.hero.body.touching.down) {
        this.hero.launch(SPRING_SPEED);
        this.sound.play(JUMP_SOUND, { rate: 0.7 }); // a deeper "boing"
        this.tweens.add({ targets: spring, scaleY: 0.5, duration: 80, yoyo: true
});
    }
});

```

src/objects/Hero.ts

```

// CHALLENGE 2: thrown upwards by a springboard - a jump the hero did not
// choose
public launch(speed: number): void {
    this.setVelocityY(-speed);
}

```

The speed comes from the jump-height formula in the chapter: to rise about 430 pixels (from the ground at 536 to above the top platform at 144) with gravity 900 needs `sqrt(2 x 900 x 430)`, about 880. The spring sits at x = 570, in the gap between two platforms, so the hero is not stopped by a platform on the way up. Tested with the harness: dropped on the spring, the hero's feet rose to y = 93, above the top platform; steering left in the air it landed on the top platform and collected its coins.

Look for:

- `this.textures.exists` before generating: R restarts the scene, and the texture already exists the second time
 - `refreshBody()` after `setOrigin` - without it the static body stays where the image was, and the hero bounces off thin air. A classic, and worth showing with `debug: true`
 - `touching.down` (the hero's bottom touched something) so walking into the side does not launch. Because the spring is only 20 pixels high, Arcade often lifts a hero walking into it on top of it - which then launches it. That is how most games behave; accept it
 - a `launch()` method on `Hero` (ask, don't reach in - Chapter 2) is better than the scene setting the hero's velocity itself, though both work
 - a `processCallback` returning `touching.down` is a good alternative
-
-

3. Coinrecord

Project: `solutions/ch15_challenge_3_coin_record` (from `ch15_platformer_advanced`)

Goal: structured data in `localStorage`, checked on the way back in, and one class that owns it.

`src/Progress.ts`

```

// CHALLENGE 3: keep this result if it beats the record; true if it did
public static saveCoins(index: number, coins: number, total: number):
boolean {
    const records = Progress.loadCoinRecords();
    const old = records[index];
    if (old !== null && old !== undefined && old.coins >= coins) {
        return false;
    }
    records[index] = { coins, total };
    try {
        localStorage.setItem(COINS_KEY, JSON.stringify(records));
    } catch {
        // storage switched off: the record is simply not kept
    }
    return true;
}

// CHALLENGE 3: an array with a record (or null) for each level. Everything
read back from
// storage is checked: it might be missing, or not what we wrote
private static loadCoinRecords(): (CoinRecord | null)[] {
    try {
        const parsed: unknown = JSON.parse(localStorage.getItem(COINS_KEY) ?? "
[]");
        if (!Array.isArray(parsed)) {
            return [];
        }
        return parsed.map((item) => Progress.isCoinRecord(item) ? item : null);
    } catch {
        return []; // not valid JSON, or no storage
    }
}

```

src/scenes/MenuScene.ts

```

// CHALLENGE 3: a finished level shows its coin record
const record = Progress.coinRecord(index);
const coins = record === null ? "" : `    ${record.coins}/${record.total}`;
const label = open ? `${index + 1}    ${level.name}${coins}` : `${index + 1}
locked`;

```

`ResultScene` calls `Progress.saveCoins(r.level, r.coins, r.totalCoins)` when a level is completed and shows "New coin record!" if it returns `true`. The total is saved with the record because the menu has not loaded any map and cannot count the coins itself. Tested with the harness: a result of 5 coins showed "New coin record!", a later result of 3 did not, the menu showed "1 Green Hills 5/21", and a corrupted value (`{bad json}`) in storage gave a menu with no records rather than a crash.

Look for:

- the record saved only for a *completed* level (a game over should not count - the challenge says "finished")
 - `unknown` and a type guard (`item is CoinRecord`) for what comes back from `JSON.parse`, not `as CoinRecord[]` - the Chapter 6 lesson
 - `try / catch` around `JSON.parse` : it throws on bad JSON
 - the record stored by level *index* - fine while levels are never reordered; the level `key` would be more robust, and a good discussion point
-
-

4. Wall jump

Project: `solutions/ch15_challenge_4_wall_jump` (from `ch15_platformer_intermediate`)

Goal: reading `blocked.left / right` in the air, and briefly taking control away from the player - a real game-feel technique.

`src/objects/Hero.ts`

```

// CHALLENGE 4: in the air, pressed against a wall: slide down it slowly,
and jump off it
private wallJump(time: number): void {
  if (this.body.blocked.down) {
    return;
  }
  const onLeftWall = this.body.blocked.left && this.cursors.left.isDown;
  const onRightWall = this.body.blocked.right && this.cursors.right.isDown;
  if (!onLeftWall && !onRightWall) {
    return;
  }

  // sliding: never fall faster than WALL_SLIDE_SPEED while holding on
  if (this.body.velocity.y > WALL_SLIDE_SPEED) {
    this.setVelocityY(WALL_SLIDE_SPEED);
  }

  // jump pressed (just now, or a moment ago - the same buffer as a normal
  jump)
  if (time - this.jumpPressedAt <= JUMP_BUFFER) {
    this.jumpPressedAt = -1000;
    this.wallJumpDirection = onLeftWall ? 1 : -1; // away from the wall
    this.setVelocity(this.wallJumpDirection * WALL_JUMP_ACROSS, -
WALL_JUMP_UP);
    this.keysIgnoredUntil = time + WALL_JUMP_LOCK;
    this.setFlipX(this.wallJumpDirection < 0);
    this.scene.sound.play(JUMP_SOUND);
  }
}

```

src/objects/Hero.ts

```

// CHALLENGE 4: straight after a wall jump, keep going away from the wall
whatever the keys
// say - otherwise a player still holding the arrow towards the wall pulls
the hero back
if (time < this.keysIgnoredUntil) {
  this.setAccelerationX(this.wallJumpDirection * RUN_ACCELERATION);
  return;
}

```

The map. Two stone columns (tile 5, so `gid 6`) were added to the Ground layer of both copies of `level.tmj`, at columns 5 and 8 from row 4 down to row 13, making a shaft two tiles wide just right of the start. Rows 14 and 15 are left open so the hero can walk in underneath. Four coins (points of type `coin`) sit at the top of the shaft, at columns 6 and 7, rows 4 and 5: out of reach of a normal jump, which reaches about row 10. In Tiled: select the Ground layer, stamp the stone tile, then add the points on the Objects layer; save, and copy the file to `public/assets/maps/`.

Tested with the harness, with a small script in the page standing in for the player (hold towards a wall; when `blocked` against it in the air, press jump and switch direction): four wall jumps reached the top of the shaft and collected all four coins.

Look for:

- "pressing into the wall" in the test. Without it the hero sticks to walls it merely brushes
- the key lock. Without it, the player is still holding the arrow towards the wall when the jump happens, acceleration pulls the hero straight back, and it climbs a single wall - which feels like a bug. Many students discover this the hard way; it is the most useful thing in the challenge
- re-using `jumpPressedAt` gives wall jumps the same buffering as normal ones for free
- the max velocity (220) caps the push away from the wall, so `WALL_JUMP_ACROSS` above 220 does nothing - a good question to ask a student who set it to 400

5. Medal times

Project: `solutions/ch15_challenge_5_medal_times` (from `ch15_platformer_advanced`)

Goal: extend the level data, keep a clock in the HUD without flooding the registry with events, and compare "better" with a ranking.

`src/config/levels.ts`

```
// CHALLENGE 5: the times, in seconds, to beat for each medal
export interface MedalTimes {
  gold: number;
  silver: number;
  bronze: number;
}
```

src/config/medals.ts

```
export type Medal = "gold" | "silver" | "bronze" | "none";

// better medals have bigger numbers, so two medals can be compared with >
// and <
export const MEDAL_RANK: Record<Medal, number> = { none: 0, bronze: 1,
silver: 2, gold: 3 };
```

src/scenes/HudScene.ts

```
// CHALLENGE 5: the clock changes every frame, so it is worked out here
// rather than sent as an
// event. Every scene's clock is the game's clock, so this.time.now here is
// the same as in
// GameScene
override update(): void {
  const start: number = this.registry.get(START_TIME);
  const end: number = this.registry.get(END_TIME);
  const now = end > 0 ? end : this.time.now;
  this.clockText.setText(((now - start) / 1000).toFixed(1));
}
```

GameScene puts `START_TIME` in the registry when the level starts and sets `END_TIME` in `endLevel()`, so the clock stops the moment the flag is touched (the HUD would otherwise tick on through the fade-out). `ResultScene` works out the medal with `medalFor(r.seconds, times)`, shows it in its colour with the three target times, and saves it with `Progress.saveMedal()`, which keeps only a better rank. The menu draws a coloured disc with the medal's initial beside each level.

Tested with the harness: a 5-second run of Green Hills (after teleporting to the flag) showed "GOLD medal - a new best!" and a G disc on the menu; a later result of 50 seconds showed "BRONZE medal" without "a new best", and storage still held `["gold"]`.

Look for:

- the times in `LevelInfo`, not in the result scene: data about a level lives with the level
 - no `changedata` event every frame for the clock - the HUD reading two numbers in its own `update()` is the right tool here, and a good contrast with the event-driven coins and lives
 - `MEDAL_RANK` (or an array index) to compare medals; string comparison (`"gold" > "bronze"`) happens to work alphabetically for some pairs and not others
 - the medal times are guesses; ask how they would set them properly (play-testing: gold about the designer's best, bronze easy)
-
-

6. A new enemy

Project: `solutions/ch15_challenge_6_new_enemy` (from `ch15_platformer_advanced`)

Goal: add a class to an existing design through its interface, and change the interface when the design needs it.

`src/objects/Enemy.ts`

```
export interface Enemy {
  body: Phaser.Physics.Arcade.Body;
  isSquashed(): boolean;
  squash(): void;
  canBeStomped(): boolean; // CHALLENGE 6: false for an enemy that hurts
  even from above
}
```

`src/objects/Ghost.ts`


```
protected override preUpdate(time: number, delta: number): void {
    super.preUpdate(time, delta);

    const distance = Phaser.Math.Distance.BetweenPoints(this, this.target);
    if (distance < NOTICE_DISTANCE) {
        // aim at the hero's middle, not its feet (its origin is at the feet)
        this.scene.physics.moveTo(this, this.target.x, this.target.y - 24,
SPEED);
    } else {
        this.setVelocity(0, 0);
    }
    this.setFlipX(this.body.velocity.x > 0);
}
```

src/scenes/GameScene.ts

```
// CHALLENGE 6: only an enemy that CAN be stomped is squashed; the rest hurt
from any side
if (falling && feetOnTop && enemy.canBeStomped()) {
    enemy.squash();
}
```

Adding `canBeStomped()` to the interface makes the compiler list every class that must now have it: `Slime` and `Bat` each gain a three-line method returning `true`. `ghost.png` was copied from the asset library into `public/assets/spritesheets/`, loaded in `PreloadScene`, and given an animation. Ghosts are made from points of type `ghost`, with no collider (they drift through walls), and join the one enemy overlap.

The maps. Points of type `ghost` were added to the Objects layer of both copies of each map: level 1 at column 62, row 8; level 2 at column 25, row 13; level 3 at column 45, row 8 and column 88, row 7.

Tested with the harness: a ghost 150 pixels away drifted towards the hero; dropping the hero on top of the ghost cost a life, and dropping it on a slime still squashed the slime.

Look for:

- the interface changed, rather than `if (enemy instanceof Ghost)` in the scene. The `instanceof` answer works, but the scene then has to know every kind of enemy - exactly what the interface was there to avoid. A great discussion
- `Ghost.squash()` is an empty method the scene never calls. Ask whether that is a smell: splitting the interface (`Enemy` and `Stompable` extends `Enemy`) is the cleaner design,

and some students will find it

- a slow speed - a ghost as fast as the hero cannot be escaped
- `moveTo` every frame steers the ghost continuously; compare the bat, which aims once and misses

Chapter 15 - Platformer - Slides

Chapter 15

Platformer

Run, jump, and do not fall down the hole



1 / 20

Today

- gravity, `body.blocked.down` and a jump
- **game feel**: acceleration, a variable jump, coyote time, jump buffering
- a level from **Tiled** whose tiles say what they are
- enemies to stomp, hazards that are fair
- one-way and moving platforms, ladders
- lives, checkpoints, a HUD and a level select

2 / 20

Gravity

```
physics: {  
  default: "arcade",  
  arcade: {  
    gravity: { x: 0, y: 900 },  
    debug: false, // true draws every body's outline - try it  
  },  
},
```

Every dynamic body falls - unless something holds it up

3 / 20

The hero

```
export class Hero extends Phaser.Physics.Arcade.Sprite {  
  declare body: Phaser.Physics.Arcade.Body;  
  
  constructor(scene: Phaser.Scene, x: number, y: number) {  
    super(scene, x, y, HERO_KEY, 0);  
    scene.add.existing(this); // draw it, and call its preUpdate() every frame  
    scene.physics.add.existing(this); // give it a dynamic Arcade Physics body  
  }  
}
```

- `declare body` - narrows `Body | StaticBody | null` to the body it really has
- the body is **smaller than the picture** - fairer

4 / 20

Standing, and jumping

```
if (jumpPressed && this.body.blocked.down) {  
    this.setVelocityY(-JUMP_SPEED); // negative y is UP  
    this.scene.sound.play(JUMP_SOUND);  
}
```

- `blocked.down` - a collider stopped the body moving down: **standing on something**
- a jump is a velocity; gravity bends it into an arc

5 / 20

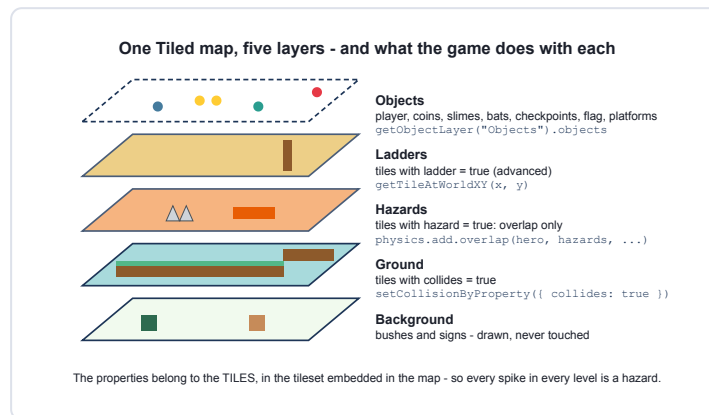
Try it: the simple platformer

- play `ch15_platformer_simple`
- how does the jump *feel*?
- set `debug: true` in `main.ts` and look at the bodies



6 / 20

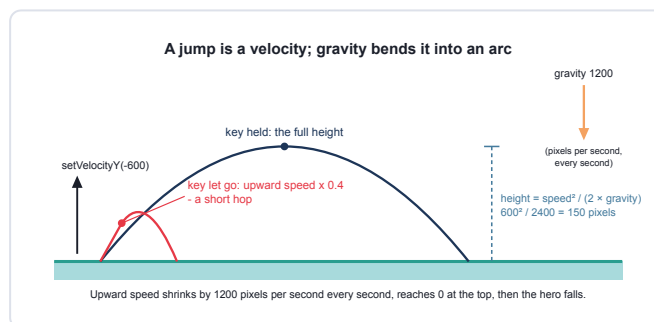
A level from Tiled



```
ground.setCollisionByProperty({ collides: true });
```

7 / 20

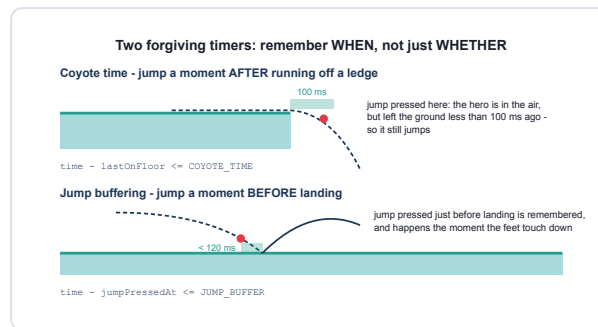
Feel 1: a jump you control



```
if (released && this.body.velocity.y < 0) {
    this.setVelocityY(this.body.velocity.y * JUMP_CUT);
}
```

8 / 20

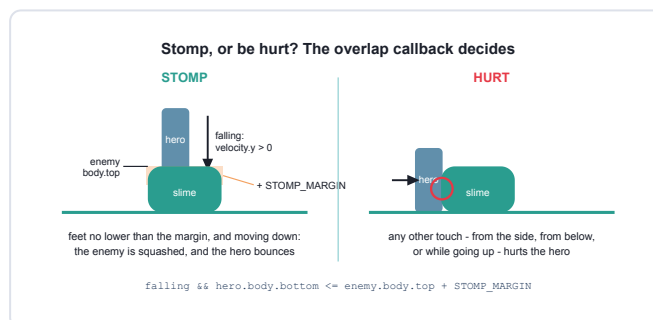
Feel 2: remember WHEN



```
const recentlyOnFloor = time - this.lastOnFloor <= COYOTE_TIME;
const recentlyPressed = time - this.jumpPressedAt <= JUMP_BUFFER;
```

9 / 20

Stomp, or be hurt?



`squash()` switches the enemy's body off **at once**

10 / 20

Fair hazards

```
this.physics.add.overlap(this.hero, hazards, () => this.hurtHero(), (_hero, tile) => {  
    return this.isDeadly(tile as Phaser.Tilemaps.Tile);  
});  
  
private isDeadly(tile: Phaser.Tilemaps.Tile): boolean {  
    return tile.properties.hazard === true && this.hero.body.bottom > tile.pixelY + HAZARD_MARGIN;  
}
```

Overlapping a tile layer calls back for **empty tiles too**

11 / 20

Try it: the intermediate platformer

- play `ch15_platformer_intermediate`
- set `COYOTE_TIME` and `JUMP_BUFFER` to 0 - play - put them back
- open `tilted/level.tmj` in Tiled



12 / 20

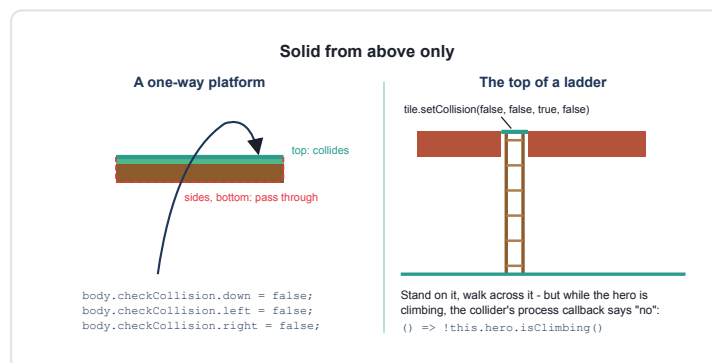
One scene, every level

```
export const LEVELS: LevelInfo[] = [
  { key: "level1", file: "assets/maps/level1.tmj", name: "Green Hills", skyTint: 0xffffffff },
  { key: "level2", file: "assets/maps/level2.tmj", name: "Deep Caves", skyTint: 0x7080a0 },
  { key: "level3", file: "assets/maps/level3.tmj", name: "Lava Castle", skyTint: 0xff9070 },
];
```

A new level = a map + one line. **Data-driven**

13 / 20

Solid from above only



Moving platforms move by **velocity**: the engine carries the rider

14 / 20

Ladders: pass a function

```
export type LadderFinder = (x: number, y: number) => number | null;  
  
this.hero = new Hero(this, start.x!, start.y!, (x, y) => this.findLadder(x, y));
```

- the hero does not know about tilemaps
- "climb" state: gravity off, arrows move up and down

15 / 20

Enemies share an interface

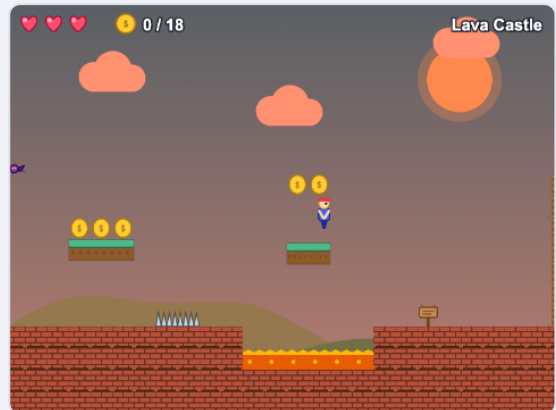
```
export interface Enemy {  
  body: Phaser.Physics.Arcade.Body;  
  isSquashed(): boolean;  
  squash(): void;  
}
```

class Bat extends Phaser.Physics.Arcade.Sprite implements Enemy - hover, swoop, return

16 / 20

Lives, checkpoints, HUD

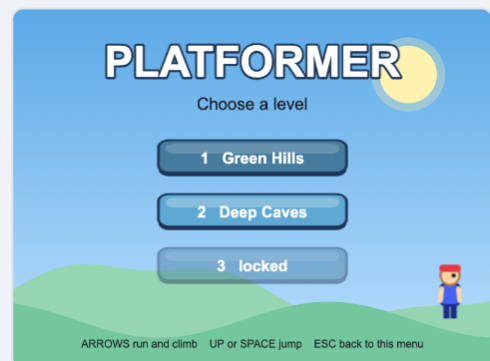
- being hurt costs a life; come back at the last **checkpoint**, flashing
- `HudScene` on top, redrawn on the registry's "changedata" event
- `Progress` keeps unlocked levels in `localStorage`



17 / 20

Try it: the advanced platformer

- play all three levels of `ch15_platformer_advanced`
- climb a ladder; ride both kinds of moving platform
- find `makeLadderTopsSolid()` - why is the collider switched off while climbing?



18 / 20

Summary

- gravity + `blocked.down` + an upward velocity = a jump
- acceleration, drag, jump cut, coyote time, buffering = **feel**
- tiles with properties; objects from the Objects layer
- stomp or hurt in the overlap callback; fair hazards in a process callback
- `checkCollision` for one-way; velocity for moving platforms
- one data-driven scene, a HUD scene, lives and checkpoints

19 / 20

Challenges

1. **Double jump** - once more, in the air
2. **Springboard** - drawn with `Graphics`, it throws the hero up
3. **Coin record** - best coins per level, on the level select
4. **Wall jump** - slide down walls, jump off them
5. **Medal times** - gold, silver, bronze, and a clock in the HUD
6. **A new enemy** - ghosts that cannot be stomped

20 / 20